Mystery-Zuordnung (I1-ID: 8m82qyt068i0)

Für die Methode `public static int mystery(int a[]) { ... }` stehen anstelle der Pünktchen `...` eine Reihe Codeschnipsel zur Auswahl. Ihre Aufgabe ist es, diesen Schnipseln Eigenschaften zuzuordnen. Dabei sind beliebige Kombinationen möglich, z.B. Code, der alle oder der keine der Eigenschaften hat oder auch Eigenschaften, die von keinem Code/von allen Codes erfüllt werden usw. — nichts wird ausgeschlossen.

Der Fall, dass Parameterwert `null` übergeben wird ist ausgeschlossen.

Jede richtige Zuordnung bringt Teilpunkte, jede falsche Abzug. Innerhalb dieser Aufgabe wird verrechnet und ggf. auf 0 aufgerundet.

Code A
```
int n = a.length;
if (n == 0)
    return 0;
int[] b = new int[n-1];
for (int i = 0; i < n-1; i++)
    b[i] = a[i];
return mystery(b) + 1;
```

Code B
```
int n = a.length;
if (n == 0)
    return 0;
int[] b = new int[n-1];
for (int i = 1; i < n; i++)
    b[i-1] = a[i];
return mystery(b) + a[0];
```

Code C
```
int n = a.length;
int e = 1;
for (int i = 0; i < n; i++)
    e += a[i];
return e-1;
```

Code D
```
int n = a.length;
int e = 0;
for (int i = 1; i < n; i++)
    e += a[i-1];
return e + a[n-1];
```

Code **Eigenschaft 1** enthält einen Syntaxfehler

Eigenschaft 2 kann abhängig vom Parameterwert Laufzeitfehler verursachen

Eigenschaft 3 berechnet die Summe der im Feld gespeicherten Werte

Eigenschaft 4 liefert den Wert `a.length`

Für jede richtige Zuordnung gibt es Teilpunkte, für falsche Zuordnungen Abzüge. Negative Punktsumme wird auf 0 aufgerundet.

Detaillierte Testergebnisse für Testdurchlauf 1

1. *ZU BESTÄTIGENDE ERKLÄRUNGEN* [ID: 184402]

Während der tatsächlichen Klausur besteht ebenfalls die Möglichkeit, mit einer Aufsichtsperson zu sprechen, um Hilfestellung zu erhalten wenn eine Formulierung bei den Aufgaben missverständlich erscheint o.ä.

- Die Aufgaben müssen selbst gelöst werden, d.h. dabei ist fremde Hilfe verboten.
- Während der Probeklausur erreichen Sie die Aufsicht über den Meetings-Tab (der Raum heißt "Probeklausuraufsicht" und ist der unterste in der linken Spalte auf diesem Tab)
- Programmierlösungen müssen in den Klausureditor eingefügt werden mittels Klick auf das Büroklammersymbol, Hochladen und Speicherung durch Wechsel zu anderer Aufgabe. Sie sind selbst verantwortlich für das Speichern Ihrer Eingaben.
- Die Arbeit an der Klausur ist **spätestens 1min vor Timer-Ablauf** zu beenden und nochmals zu speichern. Andernfalls riskieren Sie Datenverlust bei Timerablauf, ohne Möglichkeit der Rekonstruktion.

◉ Ich habe die Erklärung gelesen und verstanden. Ich erkläre, dass ich nur zugelassene Hilfsmittel verwende. Mir ist bewusst, dass die Arbeit mit Software zur Plagiatsprävention überprüft werden kann.

2. Einige Java-Grundkenntnisse (I1-ID: v969emj0ajh0) [ID: 184403]

Aufgabenteile sind unabhängig zu beantworten und werden unabhängig bewertet.

1. Von klein nach groß ordnen (0.25P für jede richtige Position)

   Ordnen Sie VON OBEN NACH UNTEN die Werte vom kleinsten bis zum größten, d.h. **wenn X kleiner als Y ist, so muss X weiter oben stehen als Y**. Achten Sie auch auf Minuszeichen!

   Integer.MIN_VALUE (am kleinsten)

   - Integer.MAX_VALUE

   Double.MIN_VALUE

   Float.MIN_VALUE

   - Short.MIN_VALUE

   2 Milliarden

   Integer.MAX_VALUE

   7.8e9 (am größten)

2. Intervalltreffer (0.5P für jede korrekte Angabe)

   Füllen Sie die Lücken, so dass das Programm mit ganzen Zahlen A und B (A ist die kleinere) als Kommandozeilenparametern aufgerufen Folgendes leistet: Nach Einlesen von 10 Werten von der Standardeingabe wird die Anzahl der eingelesenen Werte ausgegeben, die zwischen A und B (einschließlich) liegen.

   (0.5P für jede korrekte Angabe)

```
class Intervalltreffer {
    public static void main(String[] args) {
        int


A = Integer.parseInt(args[0])




;
        int


B = Integer.parseInt(args[1])




;
        int


n = 0




;
        for (


int i = 0; i < 10; i++




) {
            int


z = StdIn.readInt()




;
            if (


z >= A && B >= z




)
```

```
n++




;
        }


System.out.println(n)




;
    }
}
```

3. Was kommt raus? (1P für die richtige Antwort).

   Beim Compilieren eines Java-Programms entsteht Bytecode .

3. Schleifen-Fehler (I1-ID: z8g7z8p0ufg0) [ID: 184404]

Die ungeraden natürlichen Zahlen sollen nacheinander aufsummiert werden bis die Summe größer als 24 ist. Untenstehende Klasse dafür ist jedoch sehr ungeschickt und dadurch sogar fehlerhaft programmiert.

```
 1:  public class Teilsumme {
 2:      public static void main(String[] args) {
 3:          int s = 0, n = 0;
 4:          boolean istGerade = true;
 5:          while ( s <= 24) {
 6:              if (istGerade = false)
 7:                  s += n;
 8:              n++;
 9:              istGerade = !istGerade;
10:          }
11:          System.out.println(s);
12:      }
13:  }
```

Erzeugt die Klasse

- einen Compilezeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-): -
- eine Endlosschleife? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-): 6
- eine RuntimeException ("Programm stürzt ab")? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-): -

Hinweis: Sie müssen für jede der drei Fragen die richtige Auswahl treffen, um volle Punktzahl zu erhalten.

4. Elementare Daten (I1-ID: 93rdw3918rg0c) [ID: 184405]

```
public static void a() {
    char a = 'a';
    a++;
    int b = (int) a;
    b++;
    System.out.print( (char) ('A' + b - a));
}
public static boolean getTrue() {
    System.out.print("T");
    return true;
}
public static boolean getFalse() {
    System.out.print("F");
    return true;
}
public static void q(int i) {
    int j = 43;
    System.out.print("G");
    if ( i / j > 0) System.out.print("Q");
    else System.out.print("R");
}
public static void d() {
    double dd = Double.MIN_VALUE;
    dd *= 2;
    dd -= Double.MIN_VALUE;
    dd /= 2;
    System.out.print(dd);
}
```

Welche Ausgaben bewirken diese Aufrufe, nacheinander in der `main`-Methode der gleichen Klasse ausgeführt?:

- `a();` B
- `boolean val = getTrue() || getFalse();` T
- `val = getTrue() && !getTrue();`
- `q(42);` GR
- `d();` 0.0

5. Integer-Arithmetik (I1-ID: zzfd4k81gdg0) [ID: 184406]

Sei `c` eine `int`-Variable mit dem Wert `Integer.MAX_VALUE` (also 2147483647). Welche Textausgabe erzeugen diese Anweisungen?:

```
System.out.println(2147483646-c); //


-1




System.out.println((-1)*c);       //


-2147483647




System.out.println(c+2);          //


-2147483647




System.out.println(c*2-1);        //


-3
```

6. Lokale Variablen (I1-ID: akldnih0djh0) [ID: 184408]

```
class LokaleVariablen {
    public static void d1(int n) {
        n = n + n;
    }
    public static void d2(int[] f, int n) {
        f[n] = f[n] + f[n];
    }
    public static void main(String[] args) {
        int n = 2;
        int[] f = { n, n, n};
        for (int i = 0; i < 3; i++) {
            d1(n);
            d2(f,n);
        }
        System.out.println(n);
        System.out.println(f[n]);
    }
}
```

Wie lautet die erste Ausgabezeile? 2

Wie lautet die zweite Ausgabezeile? 16

7. Mystery-Rekursionen (I1-ID: cyub8r30gvh0) [ID: 184409]

Teile A und B sind unabhängig voneinander zu beantworten und werden auch unabhängig voneinander bewertet.

Teil A

Betrachten Sie folgende Java-Funktion.

```
1: public static int mystery(int a, int b) {
2:     if (a == 0) return b;
3:     if (b == 0) return a;
4:
5:     return mystery(a - b, b);
6: }
```

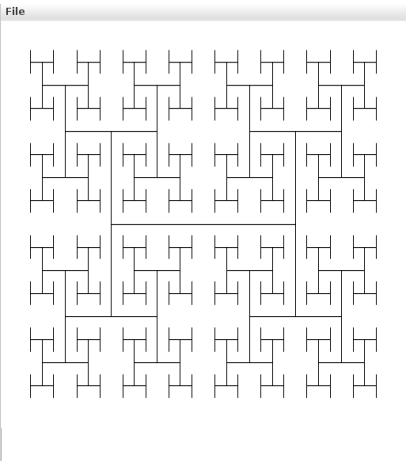
1. Was bewirkt der Aufruf `mystery(49,7)` ? Ergebnis 7 wird geliefert
2. Was bewirkt der Aufruf `mystery(7,49)` ? java.lang.StackOverflowError
3. Welcher Code muss in Zeile 4 eingefügt werden, damit jeder Aufruf mystery(a,b) mit a, b ≥0 den gleichen `int` -Wert liefert wie wie mystery(b,a)? if (a < b) return mystery(b, a);

Teil B

Der Aufruf `StdDraw.line(x0,y0,x1,y1)` zeichnet die Strecke zwischen Punkten `(x0,y0)` und `(x1,y1)` einer abstrakten Zeichenebene.

Dabei sind (x,y)-Koordinaten in üblicher Lage und der Punkt (0,0) links unten. Die Methode wird in nachfolgender Funktion benutzt.

```
 1: public static void draw(int n, double x, double y, double size) {
 2:     if (n == 0) return;
 3:     double x0 = x - size/2, x1 = x + size/2;
 4:     double y0 = y - size/2, y1 = y + size/2;
 5:     StdDraw.line( x0, y0, x0,  y1);
 6:     StdDraw.line( x1, y0, x1,  y1);
 7:     StdDraw.line( x0,  y, x1,  y);
 8:     // zeichne rekursiv 4 kleinere H-Baeume der Ordnung n-1
 9:     draw(n-1, x1, y1, size/2);
10:     draw(n-1, x0, y0, size/2);
11:     draw(n-1, x0, y1, size/2);
12:     draw(n-1, x1, y0, size/2);
13: }
```



Der „H-Baum" der Ordnung 4 auf dem Bild entsteht durch den Aufruf `draw(4, 0.5, 0.5, 0.5)` .

Dazu werden rekursiv an geeigneter Position vier H-Bäume (Teilbäume) nächstkleinerer Ordnung und Größe gezeichnet.

In welcher Reihenfolge werden die Teilbäume der Ordnung 3 fertig? oben rechts, unten links, oben links, unten rechts

8. Felder: Fehler oder ...? (I1-ID: 9je70sx0ejh0) [ID: 184410]

Erzeugt untenstehende Klasse

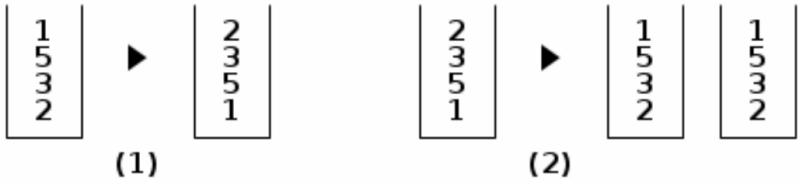
- einen Compilezeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-): -
- einen Laufzeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-): 6
- eine Ausgabe? Falls ja, so tragen Sie die Ausgabe ein, ansonsten einen Bindestrich (-): -

```
1:  class FehlerOderWas {
2:      public static void main(String[] args) {
3:          int[] a = { 2, 3, 5, 7, 11};
4:          int[] b = new int[6];
5:          b = a;
6:          a[0] = b[5];
7:          System.out.println(8+a[0] % 2);
8:      }
9:  }
```

9. Einen Stack kopieren (I1-ID: 36i3ih81hvh0) [ID: 184411]

Betrachten Sie folgende API zum Speichern von `int`-Werten (alle Methoden/Konstruktoren `public`) und die Grafik zur Illustration der Idee:

public class Stack	
Stack()	einen neuen Stack erzeugen
boolean empty()	testen, ob dieser Stack leer ist
void push(int i)	i in diesem Stack speichern
int pop()	letztgespeicherten Wert liefern und entfernen



Programmieren Sie eine weitere Instanzmethode:

```
public Stack copy() {
  // Rumpf (Ihre Aufgabe)
}
```

Spezifikation: Aufruf liefert Stack gleichen Inhalts wie dieser. Original und Kopie sind unabhängig benutzbar (Speichern beim Original ändert nicht die Kopie usw.).

Idee: (1) alle Werte aus diesem Stack entfernen und anderswo speichern, (2) von dort die Werte wieder entfernen und in diesem und im neuem Stack speichern.

Stack h = new Stack(); Stack c = new Stack();

while (!this.empty()) {

h.push(this.pop()); }

while (!h.empty()) {

int i = h.pop();

this.push(i); c.push(i);}

return c;

10. OOP-Verständnisfragen (I1-ID: 0kails31hvh0) [ID: 184412]

Java bietet mehrere für die Datenkapselung notwendige Mechanismen, zum Beispiel Instanzmethoden

Konstruktoraufrufe lösen eine Reihe von Aktionen aus, zum Beispiel Instanzvariablen setzen

Welche Erklärung trifft auf den Begriff "Überschreiben" zu?

eine von der Basisklasse geerbte Methode neu definieren

Welche dieser Situationen umschreibt man mit dem Begriff "Dynamische Bindung"?

bei Aufruf einer Instanzmethode steht der auszuführende Code erst zur Laufzeit fest

11. Laufzeitbetrachtungen (I1-ID: qgrk5ck0kvh0) [ID: 184413]

1. Was ist die Kernoperation bei der Methode `a(int n, boolean f)` ? Geben Sie die Zeilennummer an. 8

```
 1:         public static int a(int n, boolean f) {
 2:             if (n < 0)
 3:                 return 0;
 4:             int s = 0;
 5:             for (int i = 1; i <= n; i++)
 6:                 for (int j = 0; j < i; j++)
 7:                     if ( f )
 8:                         s += 1;
 9:             return s;
10:         }
```

- Geben Sie die worst-case-Anzahl der Kernoperationen von `a(int n, boolean f)` in Abhängigkeit vom Argument `n` in Tilde-Notation an. ~0.5n²
- Ergänzen Sie die Methode `b(int n, boolean f)` so, dass sie genau das gleiche leistet wie `a(int n, boolean f)` , dabei aber eine Laufzeit konstanter Wachstumsordnung hat.

```
public static int b(int n, boolean f) {
    if (n < 0 || !f)
        return 0;
    return


n*(n+1)/2;







    }
```

12. Klasse Punkt (I1-ID: esidz5x0kvh1) [ID: 184416]

Ein Punkt der Ebene ist durch seine Koordinaten (zwei `double` -Werte) definiert. Schreiben Sie eine Java-Klasse für unveränderliche Punkt-Objekte, die folgende API realisiert:

public class Punkt	
public Punkt(double x, double y)	erzeugt neues Punkt-Objekt
public static int anzahl()	liefert die Anzahl erzeugter Punkt-Objekte
public double getX()	liefert die x-Koordinate dieses Punktes
public boolean equals(Punkt other)	testet, ob dieser Punkt und other die gleiche Position haben
public String toString()	String-Umwandlung

Fügen Sie einen Test-Client hinzu, der eine ganze Zahl N von der Kommandozeile übernimmt und genau folgendes leistet: Koordinaten von N Punkten aus der Standardeingabe lesen, die Punkte zeilenweise ausgeben (keinen weiteren Text) und testen, ob darunter ein Paar von Punkten mit gleichen Koordinaten vorkommt und das Ergebnis des Tests (nur true oder false auf einer neuen Zeile) ausgeben.

Beispiel: der Aufruf `echo 0.0 0.1 1.0 1.1 0.0 0.1 | java Punkt 3` erzeugt exakt die Ausgabe

```
Punkt(0.0,0.1)
Punkt(1.0,1.1)
Punkt(0.0,0.1)
true
```

Gehen Sie davon aus, dass ein korrekter Aufruf erfolgt, insbesondere N>0 ist. Dies soll nicht getestet werden.

- Verwenden Sie exakt die angegebenen Bezeichner und das vorgegebene Ausgabeformat.
- Verwenden Sie zum Einlesen der Koordinaten die Bibliothek `StdIn` .
- Achten Sie auf Codeeinrückung.
- Machen Sie die Unveränderlichkeit der Objekte am Code deutlich.
- Machen Sie keine unnötige Methodenaufrufe und definieren Sie keine nicht geforderten Methoden.

```
1  public class Punkt {
2      private final double x;
3      private final double y;
4
5      private static int anzahl = 0;
6
7      public Punkt(double x, double y){
8          this.x = x;
9          this.y = y;
10
11         anzahl++;
12     }
13
14     public static int anzahl(){
15         return anzahl;
16     }
17
18     public double getX(){
19         return this.x;
20     }
21
22     public boolean equals(Punkt other){
23         return this.x == other.x && this.y == other.y;
24     }
25
26     public String toString(){
27         return "Punkt(" + this.x + "," + this.y + ")";
28     }
29
30     public static void main(String[] args){
31         int N = Integer.parseInt(args[0]);
32         Punkt[] p = new Punkt[N];
33
34         for(int i = 0; i < N; i++){
35             p[i] = new Punkt(StdIn.readDouble(), StdIn.readDouble());
```

	0:0
	0:0

13. Ein weiteres Wort in eine Suchsel-Lösung eintragen (I1-ID: 85reico093k0) [ID: 184417]

"Suchsels" bestehen aus einer $N \times N$ - Anordnung von Buchstaben und einer Liste von Suchwörtern, die darin von links nach rechts oder von oben nach unten zu lesen sind. Hier ein Beispiel (mit Zeilen- und Spaltennummern):

```
Lösung                          Rätsel

   0 1 2 3                         0 1 2 3
0: F                            0: F B B X
1: E N T E                      1: E N T E
2: I                            2: I U E K
3: N O T E                      3: N O T E
```

Die Lösung besteht aus einem zweidimensionalen `char` - Array, das die Buchstaben der Suchwörter an den richtigen Stellen enthält, ansonsten nur Leerzeichen. Im Rätsel sind die Leerzeichen zufällig durch Buchstaben ersetzt. In die Lösung passen aber noch weitere Wörter, z.B. `FILZ` (ab Position (0,0) waagerecht) und `INFO` (ab (0,1) senkrecht).

Aufgabe

Programmieren Sie eine statische Methode

```
public static boolean insertLR(String w, int x, int y, char[][] s)
```

die (1) prüft, ob `w` ab Position `(x,y)` von links nach rechts in die in `s` gespeicherte Lösung eingetragen werden kann, ohne bereits vorhandene Buchstaben zu überschreiben (Leerzeichen können überschrieben werden). Ist das der Fall, so wird (2) das Wort "eingetragen" (also der Inhalt von `s` entsprechend geändert). Im Erfolgsfall wird `true` zurückgeliefert, ansonsten `false`.

Hinweise

1. Sie müssen unter anderem mittels `w.length()` überprüfen, ob `s` groß genug ist, um das Wort ab Position `(x,y)` aufzunehmen.
2. Auf die Zeichen von `w` können Sie mit `w.charAt(int pos)` zugreifen (siehe Skript Seite 168f.).
3. Tragen Sie die komplette Methode in untenstehenden "Klausureditor" ein. Sie brauchen sie nicht in eine Klasse einbetten.

```
1  public static boolean insertLR(String w, int x, int y, char[][] s) {
2      if(x + w.length() > s[y].length){
3          return false;
4      }
5
6      for(int i = 0, i < w.length(); i++){
7          if(s[y][x + i] != ' ' && s[y][x + i] != w.charAt(i)){
8              return false;
9          }
10     }
11
12     for(int i = 0; i < w.length(); i++){
13         s[y][x + i] = w.charAt(i);
14     }
15
16     return true;
17 }
```

Info I, Klausur Wintersemester 23/24

1. Allgemeine Fragen

- Speicherbereiche nach Größe sortieren. Lösung: Register, Prozessorcache, Festplatte
- Typischen Ablauf eines Algorithmus beschreiben. Lösung: Initialisiere, Iteriere, Terminiere
- Java-Codeelemente nach Umfang/Größe sortieren. Lösung: Methode, Klasse, Bibliothek
- Prozess beim Schreiben von Programmen beschreiben. Lösung: Editiere, Compiliere, Teste
- `java Intervallsumme A B` soll die Summe der natürlichen Zahlen zwischen `A` und `B` ausgeben. (Es muss kein eigener Code geschrieben werden. Stattdessen sollen die Zeilen des Programms per Dropdown-Menü ausgewählt werden.)
  Lösung:

```java
class Intervallsumme {
    public static void main(String[] args) {
        int s = 0;
        int min = Integer.parseInt(args[0]); int max = Integer.parseInt(args[1]
        int n = min;
        while (max >= n) {
            s += n; n++; // alternativ: n += 1; s += n;
        }
        System.out.println(s);
    }
}
```

- Beim Compilieren eines Java-Programms entsteht _____. Lösung: Bytecode

2. Vermischtes zu Klassentypen

Kommentar der/des Prüfungsverantwortlichen:

Lösung

1. überladen
2. `int[] i = {1.500,000}` ist fehlerhaft, denn `1.500` ist ein Gleitpunktliteral.
3. 4 mal: `a` durchläuft der Reihe nach die Werte 10, 9, 8, 7. Dann erst hat `run` den Wert `false`, d.h. die Abbruchbedingung ist erreicht.
4. a.length ist 2, a[0].length ist 3. Deswegen verursacht int c = a[0][a.length]; KEINEN Fehler und boolean b hat in beiden Zeilen den Wert true. Die einizige korrekte Aussage ist: Die Zeile a = new int[6]; verursacht einen Fehler (einen Compilezeitfehler, weil hierbei an a eine 1D-Referenz zugewiesen werden soll, a aber als 2D-Referenz deklariert ist).
5. Jeder Konstruktoraufruf erzeugt ein neues Objekt mit eigener Adresse. Der Vergleich bezieht sich auf die Referenzen (Adressen) der Objekte, nicht ihren Wert (hier: allein der Wert des `honor`-Attributs). Also lautet die Ausgabe `false` (1P).
6. Richtige Antwort (2P): Die Klasse kann nicht compiliert werden, denn es gibt keinen Konstruktor mit passenden Argumenttypen.
7. Ergänzte Klasse

```
1: public class C {
2:     private int a;
3:     public C(int a) {
4:         this.a=a;     // initialisiere Attribut mit uebergebenem Parame
5:     }
6:     private void f() {
7:         a *= 2;
8:         System.out.println("a=" + a);
9:     }
10:    public static void main(String[] args) {
11:        C c=new C(5); // erzeuge ein Objekt der Klasse C mit Attributwert
12:                      // ... verwenden Sie dafuer den Bezeichner c
13:        c.f();        // Methode f() aufrufen
14:    }
15: }
```

8. Erzeugte Ausgabe `a=10`

Aufgaben zu 5. und 6. (Bild: Altklausurensammlung):

1. Objekte verstehen:

```
public class Ninja {
    private int honor;
    public Ninja( int s) {
        honor = s;
    }
}
```

Zwei `Ninja`-Objekte werden instanziiert:

```
Ninja a = new Ninja(20), b = new Ninja(20);
```

Welchen Wert hat nun der Ausdruck `a == b`?

2. Fehler oder ...?

```
public class SillyTestClass {
    private int x, y;
    public SillyTestClass(int x, int y) {
        System.out.println(y);
    }
    public SillyTestClass(String string1, String string2) {
        System.out.println(string1 + string2);
    }
    public static void main (String[] args) {
        SillyTestClass app = new SillyTestClass(2, "Tests");
    }
}
```

Welche Aussage charakterisiert diesen Code am besten?

3. Fehler oder ...?

Bestmögliche Lösung:

Erzeugt die untenstehende Klasse

- einen Compilezeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-): -
- einen Laufzeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-): 6
- eine Ausgabe? Falls ja, so tragen Sie die Ausgabe ein, ansonsten einen Bindestrich (-): -

Für volle Punktzahl müssen Sie an jeder Stelle eine Auswahl treffen!

```
1:  class FehlerOderWas {
2:      public static void main(String[] args) {
3:          int[] a = { 2, 3, 5, 7, 11};
4:          int[] b = new int[6];
5:          b = a;
6:          a[0] = b[5];
7:          System.out.println(5+a[0] % 2);
8:      }
9:  }
```

Kommentar der/des Prüfungsverantwortlichen:

Lösungsweg:

- ArrayIndexOutOfBoundsException: 5 (Zugriff jenseits der Arraygrenzen) in Zeile 6

Erklärung: Der Code ist syntaktisch korrekt, darum kein Compilezeitfehler. Durch Zeile 5 verweist b auf die gleiche Speicheradresse wie a, also auf ein Array der Länge 5. Der Zugriff b[5] dagegen erfordert mindestens ein Array der Länge 6 (0-basierte Zählung). Da dieser Laufzeitfehler vor der Ausgabeanweisung auftritt, wird diese nicht mehr ausgeführt.

4. Der Datentyp char (Bild: Aufgabensammlung)

char-Merkwürdigkeiten (I1-ID: 8w1aucc1tng0)

In folgendem Quelltext gibt es mindestens eine Zeile, die der Compiler nicht akzeptiert. Streichen Sie alle betroffenen Zeilen. Welche Textausgaben erzeugen dann die verbleibenden Zeilen?

```
1: char c = 'a';  System.out.println(c);
2: c = c+1; System.out.println(c);
3: System.out.println('c'+1);
4: System.out.println((char)('c'+1));
```

(Die falsche Zeile soll hier mit "falsch" markiert werden)
Lösung: 1. a, 2. falsch, 3. 100, 4. d

5. Zahlensysteme

- Dezimalzahl 2020 ins Binärsystem umwandeln. Lösung: 11111100100
- Hexadezimalzahl 2020 ins Dezimalsystem umwandeln. Lösung: 8224
- Oktalzahl 2020 ins Hexadezimalsystem umwandeln. Lösung: 410

6. Mystery-Rekursion, Schiefe H-Bäume

Teil A

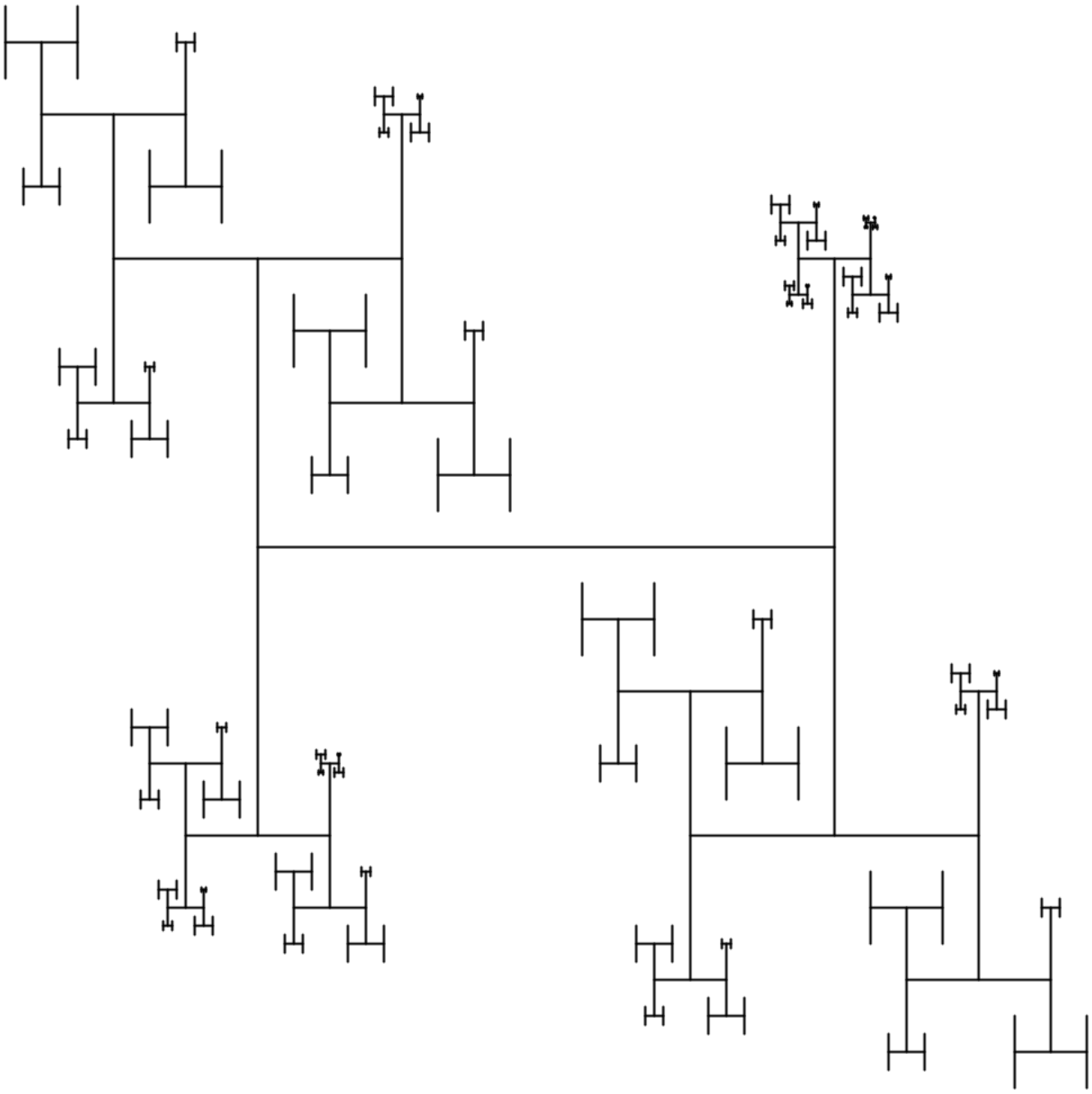
```
1: public static int mystery(int n){
2:     if(n == 0){
3:         return 0;
4:     }
5:
6:     return mystery(n-1) + 2*n-1;
7: }
```

1. Was bewirkt der Aufruf `mystery(5)`? Lösung: Ergebnis 25 wird geliefert
2. Was bewirkt der Aufruf `mystery(-5)`? Lösung: java.lang.StackOverflowError
3. Welcher Code muss in Zeile 5 eingefügt werden, damit `mystery(-n) == -mystery(n)` für alle n gilt? Lösung: `if(n < 0) return mystery(n+1) + 2*n+1;`

Teil B

```
1:  public static void draw(int n, double x, double y, double size) {
2:      if (n == 0) return;
3:      double x0 = x - size/2, x1 = x + size/2;
4:      double y0 = y - size/2, y1 = y + size/2;
5:      StdDraw.line( x0, y0, x0, y1);
6:      StdDraw.line( x1, y0, x1, y1);
7:      StdDraw.line( x0, y, x1, y);
8:
9:      // zeichne rekursiv 4 kleinere H-Baeume der Ordnung n-1
10:     draw(n-1, x0, y1, 0.5 * size);
11:     draw(n-1, x0, y0, a * size);
12:     draw(n-1, x1, y1, b * size);
13:     draw(n-1, x1, y0, 0.5 * size);
14: }
```

Was muss in Zeile 8 eingefügt werden, damit der Aufruf `draw(4, 0.5, 0.5, 0.5)` das folgende Bild eines schiefen H-Baumes ergibt?



Lösung: `double a = 0.25, b = 0.125;`

7. Lokale Variablen (Bild: Altklausurensammlung)

7. 3. Statische Methoden (l1-ID: 0f1dmv607ih0)

```
public class StatischeMethoden {
        public static void q1(int n) {
                n = n * n;
         }
        public static void q2(int[] f, int i) {
                f[i] = f[i] * f[i];
        }
        public static void main(String[] args) {
                int n = 2;
                 int[] f = { n, n, n};
                for (int i = 0; i < 3; i++) {
                        q1(n); q2(f,n);
                }
                System.out.println(n);
                System.out.println(f[n]);
         }
}
```

Wie lautet die erste Ausgabezeile?

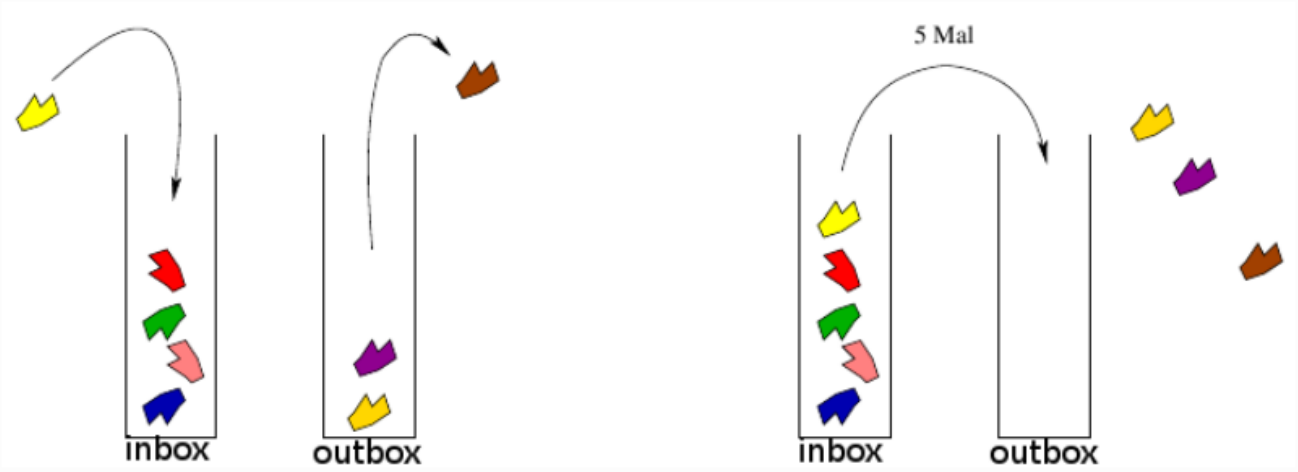
2

Wie lautet die zweite Ausgabezeile?

256

8. Queue aus zwei Stacks (Bild: Aufgabensammlung)

Queue aus zwei Stacks bauen (I1-ID: a4g40y31x4h0)



Mit zwei Stacks kann man eine Queue implementieren:

- `enqueue(...)` entspricht Einspeichern in `inbox`
- `dequeue()` entspricht Ausspeichern aus `outbox`.

Unter Verwendung dieser API

`public class Stack`	
`boolean empty()`	testet, ob dieser Stack leer ist
`void push(String s)`	speichert `s` in diesem Stack
`String pop()`	liefert zuletzt gespeicherten String aus diesem Stack und entfernt ihn

sähe die entsprechende Klasse z.B. so aus:

```
public class ZSQueue { // Zwei-Stack-Queue
    private Stack inbox, outbox;
    public ZSQueue() { ... }
    public boolean empty() { ... }        // Leerheitstest
    public void enqueue(String s) { ... } // einspeichern
    public String dequeue() {             // ausspeichern und liefern
        // Rumpf (Ihre Aufgabe)
    }
}
```

Programmieren Sie den Rumpf der Methode `dequeue()`:

- ist die Queue leer, ist Ausspeichern nicht möglich!
- teste ob `outbox` leer ist und fülle in diesem Fall den kompletten Inhalt von `inbox` um in `outbox`, dann ein Element aus `outbox` ausspeichern und liefern.

(Es müssen wieder die Zeilen des Programms per Dropdown-Menü ausgewählt werden; daher die fehlende Einrückung)
Lösung:

```
if(outbox.empty())
while(!inbox.empty())
outbox.push(inbox.pop());
if(!outbox.empty())
return outbox.pop();
throw new RuntimeException("Queue ist leer");
```

9. Objektorientierung

- Wodurch wird Kapselung bei der objektorientierten Programmierung erreicht?
  Durch _____ Instanzvariablen und _____ Methoden.
  Lösung: 1. private, 2. public
- Welche der Konzepte (A) Vererbung, (B) Kapselung, (C) Überladen kann man anhand folgender Situationsbeschreibung illustrieren?
  Jedes Tier kann einen Laut von sich geben (ein Geräusch machen). Jeder Hund ist ein Tier und kann einen Laut von sich geben, nämlich: Wau Wau! Jede Katze ist ein Tier und kann einen Laut von sich geben, nämlich: Miau!
  Lösung: (A) ist richtig, (B) und (C) sind falsch

10. Sortieralgorithmen

Ihre Antwort:

Betrachten Sie folgendes Code für einen Sortieralgorithmus, der ein Array f der Länge N sortiert.

```
public static void sort(int[] f){
    int min, tmp;
    for(int i = 0; i < f.length; i++){
        min = i;
        for(int j = i; j < f.length; j++){
            if(f[min] > f[j])
                min = j;
        }
        tmp = f[i];
        f[i] = f[min];
        f[min] = tmp;
    }
}
```

Die algorithmische Idee sollte Ihnen bekannt vorkommen. Was können Sie darüber aussagen?

Dieser Algorithmus ist Vergleichsbasiert ✓ .

Wieviel Speicherplatz benötigt der Algorithmus (zusätzlich zum Array f)? Das hängt nicht von N ab. ✓ .

Die Laufzeit des Algorithmus im besten Fall ist O(N^2) ✓ .

Die Laufzeit des Algorithmus im schlechtesten Fall ist O(N^2) ✓ .

Kommentar der/des Prüfungsverantwortlichen:

Es ist ein vergleichsbasiertes Verfahren: die Bedingung in der if-Anweisung ist ein Vergleich zweier Array-Werte.

Zusätzlich zu f werden die Variablen min, tmp, i, j benutzt: konstant viel zusätzlicher Speicher.

Dem Code ist anzusehen, dass die äußere Schleife N mal iteriert, die innere 1, 2, ..., N mal - je nach Wert von i. Die Gaussche Summe führt zur Schrittzahl O(N^2).

[Das ist eine bekannte Eigenschaft von Selection-Sort.]

11. Ist ein bestimmtes Wort im Suchsel-Rätsel enthalten?

Ist ein bestimmtes Wort im Suchsel-Rätsel enthalten? (I1-ID: 7ub80vo093k0) - Full score: 18

Description

"Suchsels" bestehen aus einer \(N\times N\)-Anordnung von Buchstaben und einer Liste von Suchwörtern, die darin von links nach rechts oder von oben nach unten zu lesen sind. Hier ein Beispiel (mit Zeilen- und Spaltennummern):

```
Lösung                          Rätsel

   0 1 2 3                         0 1 2 3
0: F                            0: F I L Z
1: E N T E                      1: E N T E
2: I                            2: I F E K
3: N O T E                      3: N O T E
```

Die Lösung besteht aus einem zweidimensionalen `char`-Array, das die Buchstaben der Suchwörter an den richtigen Stellen enthält, ansonsten nur Leerzeichen. Das Rätsel besteht ebenfalls aus einem zweidimensionalen `char`-Array, nur sind die Leerzeichen zufällig durch Buchstaben ersetzt. Im Beispiel sind dadurch weitere sinnvolle Wörter entstanden: `FILZ` (ab Position (0,0) von links nach rechts) und `INFO` (ab (0,1) von oben nach unten).

Aufgabe

Programmieren Sie zwei statische Methoden

```
public static boolean containsLR(String w, int x, int y, char[][] r)
public static boolean containsLR(String w, char[][] r)
```

- Hierbei speichert `r` die Buchstaben des Rätsels und `w` ist das zu prüfende Wort.
- Die erste Methode prüft, ob `w` *ab Position* `(x,y)` von links nach rechts in `r` zu finden ist (im Beispiel würde damit FILZ gefunden werden, nicht aber INFO).
- Die zweite Methode ruft die erste Methode mehrmals auf, um zu prüfen, ob `w` *irgendwo* in `r` von links nach rechts zu finden ist.

Hinweise

1. Behandeln Sie bei der ersten Methode am besten zuerst den Fall, dass `w` zu lang ist, um ab Position `(x,y)` in `r` zu passen (welcher Wert muss dann geliefert werden?).
2. Auf die Zeichen von `w` können Sie mit `w.charAt(int pos)` zugreifen (siehe Skript Seite 168f.).
3. Programmieren Sie die beiden Methoden in untenstehendem "Klausureditor". Bitte nicht in eine Klasse einbetten.

Score: 18

```java
public static boolean containsLR(String w, int x, int y, char[][] r){
    if(y + w.length() > r[x].length){
        return false;
    }

    for(int i = 0; i < w.length(); i++){
        if(w.charAt(i) != r[x][y + i]){
            return false;
        }
    }

    return true;
}

public static boolean containsLR(String w, char[][] r){
    for(int i = 0; i < r.length; i++){
        for(int j = 0; j < r[i].length; j++){
            if(containsLR(w, i, j, r)){
                return true;
            }
        }
    }

    return false;
}
```

12. Klasse Gerade

Klasse Gerade (I1-ID: 0hl56v31lvh2) - Full score: 12

Description

Eine **Gerade** \(y = m\cdot x + n\) in der Ebene soll allein durch `m` und `n` gegeben sein. Schreiben Sie eine Java-Klasse für *unveränderliche* `Gerade`-Objekte, die folgende API realisiert:

```
| public class Gerade                |
|------------------------------------+-------------------------------------------------------------
| public Gerade(double m, double n)| erzeugt neues Gerade-Objekt
| public static int anzahl()       | liefert die Anzahl erzeugter Gerade-Objekte
| public double getM()             | liefert m dieser Gerade
| public double getN()             | liefert n dieser Gerade
| public String toString()         | liefert String-Darstellung wie im Beispiel unten
```

Beispiel: Der String zur Geraden \(y = 2\cdot x + 3\) lautet `y = 2*x + 3`.

Score: 12

```java
public class Gerade {
    private final double m;
    private final double n;
    private static int anzahl = 0;

    public Gerade(double m, double n){
        this.m = m;
        this.n = n;

        anzahl++;
    }

    public static int anzahl(){
        return anzahl;
    }

    public double getM(){
        return this.m;
    }

    public double getN(){
        return this.n;
    }

    public String toString(){
        return "y = " + this.m + "*x + " + this.n;
    }
}
```

Wiederholungsklausur Informatik I
Sommersemester 2010
Prof. Dr. C. Damm
7. April 2010
von 08:00+x bis 09:30+x Uhr

Vorname:

Nachname:

Matrikelnummer:

Es sind **keine Hilfsmittel** (Skript, Taschenrechner etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten **Blätter** für Ihre Antworten. Schreiben Sie mit blauem/schwarzem Kugelschreiber, Füller etc.. Bleistift ist nicht erlaubt. Unlesbare Antworten können nicht gewertet werden. Algorithmen sind in Java-Code zu notieren, sofern keine Alternativen genannt sind.

Viel Erfolg!

	Max. Punkte	Erreichte Punkte
Aufgabe 1 (Allgemeine Kenntnisfragen)	21	
Aufgabe 2 (Elementare und strukturierte Daten)	15	
Aufgabe 3 (Suchbäume und Traversierungen)	17	
Aufgabe 4 (Algorithmen und Datenstrukturen)	14	
Aufgabe 5 (Sortieren)	9	
Aufgabe 6 (Fehlersuche und Programmverstehen)	14	
Summe	90	

Note:

Name: MatNr.:

Aufgabe 1 (Allgemeine Kenntnisfragen [21 Punkte])
3P pro Aufgabenteil. Antworten Sie kurz und prägnant.

1. Seien f, g Funktionen, die natürliche Zahlen auf natürliche abbilden. Was bedeutet $f(n) \in \mathcal{O}(g(n))$?

2. Die Tiefe eines Baums ist das Maximum über die Anzahlen von Kanten auf den Wegen von der Wurzel bis zu einem Blatt. Wieviele *Knoten* (nicht nur Blätter) besitzt ein Binärbaum der Tiefe n mindestens und wieviele höchstens?

3. Die Klasse `A` soll so mit dem ganzzahligen Klassenattribut `z` ausgestattet werden, dass sie nur innerhalb der Klasse genutzt werden kann. Schreiben Sie die Definition in die Box.

4. Wir betrachten die Methode **static int** zauberFormel(**int** z) {**int** x = 2; **return** x∗z;} und den Aufruf a = zauberFormel(7); in irgendeiner anderen Methode. Beschreiben Sie den Inhalt des Aktivierungssatzes, den dieser Aufruf erzeugt.

5. Welche Ausgabe liefert folgende Anweisung?
System.out.println((3.2==3+1/5)+"AND("+(3/2!=1.0)+"OR"+((**double**)3/2==1.5)+")");

6. Geben Sie in $\mathcal{O}$-Notation eine möglichst gute obere Schranke für die Anzahl der Multiplikationen Zahl×Zahl an, um zwei $n \times n$-Matrizen zu multiplizieren. Begründen Sie kurz.

7. Was bedeutet (a) Überschreiben und (b) Überladen einer Methode?

Name: MatNr.:

Aufgabe 2 (Elementare und strukturierte Daten [15 Punkte])

1. (6P) Welche ganze Zahl stellt das 4-Byte-Bitmuster `0x8000000F` in Zweierkomplementdarstellung dar? Als Ergebnis geben Sie bitte eine geeignete Kombination von Zweierpotenzen an.

2. (5P) Begründen Sie: (1) Jeder `int`-Wert ist ohne Informationsverlust als `double`-Wert darstellbar. (2) Nicht jeder `int`-Wert ist ohne Informationsverlust als `float`-Wert darstellbar.

3. (4P) Der Speicher, der einer geladenen Applikation zur Verfügung steht, enthält die Bereiche Stack, Heap und Method Area. In welchem dieser Bereiche werden strukturierte Datenobjekte angelegt? Was wissen Sie über deren Lebenszeit?

Name: MatNr.:

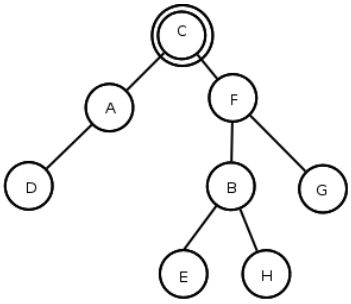
Aufgabe 3 (Suchbäume und Traversierungen [17 Punkte])

1. (3P) Zeichnen Sie den binären Suchbaum, der durch Einfügen der folgenden Werte (in dieser Reihenfolge) entsteht: 5, 2, 1, 3, 7, 9, 4, 6, 8

2. (5P) Geben Sie eine Einfügereihenfolge der Zahlen 1 bis 7 an, bei der der entstehende Suchbaum maximale Tiefe hat und begründen Sie die Maximalität.
   Geben Sie auch eine Einfügereihenfolge für die Zahlen 1 bis 7 an, bei der minimale Tiefe erreicht wird.

3. (5P) Welche Tiefe hat ein binärer Suchbaum mit n Einträgen mindestens und wieviele höchstens? Geben Sie die Werte möglichst genau an und begründen Sie Ihre Antwort.

4. (4P) Im unten angegebenen Binärbaum sind die Knoten mit Schlüsseln A, B, .., H markiert. Der Baum soll in Postorder traversiert und dabei die Schlüssel der besuchten Knoten ausgegeben werden. Geben Sie die entstehende Reihenfolge der Schlüssel an.



Name: MatNr.:

Aufgabe 4 (Algorithmen und Datenstrukturen [14 Punkte])

1. (6P) Im folgenden Programmfragment wird ein anfangs leerer Stack `s` gesteuert, der Zahlen speichern kann: `push` speichert eine Zahl `pop` entfernt die zuletzt gespeicherte Zahl und liefert diese auch als Rückgabewert.

   1. Notieren Sie in der Tabelle, welchen Inhalt der Stack nach den einzelnen Anweisungen hat. Die ersten beiden Spalten sind vorgegeben.

   2. Geben Sie an, welche Ausgabe das Programmstück erzeugt.

```
s.push(12);
s.push(6);
s.push(s.pop()+s.pop());      // auf Auswertungsreihenfolge achten!
s.push(8);
s.push(8);
s.push(64);
s.push(s.pop()/s.pop());      // ..
s.push(1/(s.pop()/s.pop()));  // ..
s.push(3);
s.push(s.pop()*s.pop());      // ..
s.push(s.pop()+s.pop());      // ..
System.out.println(s.pop());
```

	6										
12	12										

Ausgabe:

2. (8P) Kreuzen Sie bei den folgenden Aussagen „Richtig“ oder „Falsch“ an und geben Sie eine stichwortartige Begründung. Für jede richtige Antwort gibt es einen Punkt und einen weiteren für die passende Begründung.

Aussage	Richtig	Falsch	Begründung
Binäre Suche arbeitet auf manchen `int`-Arrays inkorrekt.			
Binäre Suche hat auf sortierten Eingaben im schlechtesten Fall logarithmische Laufzeit.			
Der Aufwand für das Sortieren von N Zahlen mit MergeSort ist im schlechtesten Fall $\Omega(N)$.			
Quicksort hat immer logarithmische Laufzeit.			

Name: MatNr.:

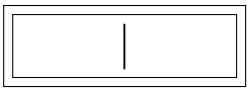
Aufgabe 5 (Sortieren [9 Punkte])
Wir betrachten folgenden elementaren Sortieralgorithmus

```
30    public static void selectionSort(int[] feld) {
31     int maxpos, tmp;
32     for (int i=feld.length; i>1; i--) {
33       // bestimme Position des Maximums im Intervall [0,...,i-1]
34       maxpos = 0; // Position des Maximums
35       for (int j=0; j<i; j++)
36         if ( feld[j] > feld[maxpos])
37             maxpos = j;
38        // tausche feld[i-1] und feld[maxpos]:
39        tmp = feld[i-1]; feld[i-1] = feld[maxpos]; feld[maxpos] = tmp;
40     }
41    }
```

1. (4P) Wenden Sie den Algorithmus an auf
   `int[] test = {1, 6, 9, 8, 2, 3, 4, 5, 7};`
   und ergänzen Sie die Tabelle um die Feldinhalte *nach* Iterationen 1, 2 und 3 der *äußeren* `for`*-Schleife* (Zeilen 32-40).

Start	1	6	9	8	2	3	4	5	7
nach 1. Iteration									
nach 2. Iteration									
nach 3. Iteration									

2. (5P) Geben Sie (mit Begründung) eine möglichst gute asymptotische Abschätzung der Laufzeit des Algorithmus auf Eingaben der Länge n an



Name: MatNr.:

Aufgabe 6 (Fehlersuche und Programmverstehen [14 Punkte])

1. (8P) Das folgende Codefragment enthält Fehler in den angegebenen Zeilen. Unterstreichen Sie jeweils den fehlerhaften Code und geben Sie für jeden Fehler eine kurze Beschreibung an. Geben Sie in der Tabelle an, ob es sich um einen Compilezeit- oder Laufzeitfehler handelt und schlagen Sie eine möglichst einfache Korrektur vor, um die im Kommentar angegebenen Wirkungsweise zu erreichen.

```
class List {
  Element ListHead;
  public int zeroCnt(int x) {
  // Zaehlt in Liste die Anzahl der Nullen vor erstem Auftreten von x.
  // Falls x nicht vorhanden, so wird 0 zurueckgegeben.
    int cnt;
    Element p = ListHead;
    while (p.dat != x && p != null)        // Fehlerzeile 1
      if (p.dat = 0)                       // Fehlerzeile 2
        cnt++;                             // Fehlerzeile 3(a)
      else
        p = p.next;
    return cnt;                            // Fehlerzeile 3(b)
  }
  private class Element {
    int dat;
    Element next;
  }
  public static void main(String[] args) {
    List lst  = new List;                  // Fehlerzeile 4
        :
    // Aufbau von lst. Dieser Code ist fehlerfrei.
        :
    lst.zeroCnt(3);
  }
}
```

Fehlerzeile	Fehlerart/-beschreibung	Korrektur
1		
2		
3a und 3b		
4		

2. (6P) Welche Ausgabe erzeugt der Aufruf folgenden Programms? Zur Hilfestellung ist ein Ausschnitt aus der ASCII-Tabelle angegeben.

```
public class ausgabe {
  public static void main(String[] argv) {
    int c = 'H';
      while ( c > 'A') {
        if (c == 'E') {
          System.out.println();
          System.out.println(c);
          }
        else
          System.out.print((char) c);
        --c;
        }
      System.out.println();
  }
}
```

Oct	Dec	Hex	Char
100	64	40	@
101	65	41	A
102	66	42	B
103	67	43	C
104	68	44	D
105	69	45	E
106	70	46	F
107	71	47	G
110	72	48	H
111	73	49	I
112	74	4A	J
113	75	4B	K

Name: MatNr.:

Klausur Informatik I
Sommersemester 2013
Prof. Dr. C. Damm
12. April 2013
von 08:00+x bis 09:30+x Uhr

Vorname:

Nachname:

Matrikelnummer:

Es sind **keine Hilfsmittel** (Skript, Taschenrechner etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten **Blätter** für Ihre Antworten. Schreiben Sie mit blauem/schwarzem Kugelschreiber, Füller etc.. Bleistift ist nicht erlaubt. **Es wird nur das gewertet, was ohne Mühe lesbar ist**. Algorithmen sind in Java-Code zu notieren, sofern keine Alternativen genannt sind.

Zum **Bestehen** der Klausur sind **45** Punkte ausreichend.

Viel Erfolg!

	Max. Punkte	Erreichte Punkte
Aufgabe 1 (Elementare Daten)	18	
Aufgabe 2 (Programmierparadigmen I)	16	
Aufgabe 3 (Strukturierte Daten)	20	
Aufgabe 4 (Programmierparadigmen II)	17	
Aufgabe 5 (Algorithmen)	19	
Summe	90	

Note:

Name: MatNr.:

Aufgabe 1 (Elementare Daten [18 Punkte])

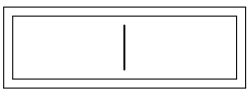
1. [6 Punkte] `a` und `b` seien `boolean-Werte`. Weisen Sie mit der Wertetafel nach, dass der Ausdruck `(!(a && b) && (a || b)) || (a && b) || !(a || b)` den Wert `true` hat (kürzen Sie bitte `true`/`false` mit 1/0 ab — der besseren Lesbarkeit wegen).

2. [6 Punkte] Sei `a` eine `int`-Variable mit dem Wert `Integer.MAX_VALUE` (also 2147483647). Welche Textausgabe erzeugen die vier Anweisungen?

```
System.out.println(a);
System.out.println(a+1);
System.out.println(2-a);
System.out.println(2*a);
```

3. [6 Punkte] Welche Textausgabe erzeugt folgendes Fragment?:

```
int a = 2, d = 4;
double b = 80e-1;
System.out.println( b / a );
int c = ( int ) 8.2;
System.out.println( c / a );
System.out.println( 1.0 * a / (double) d );
System.out.println( (double) a / d );
System.out.println( (double) ( a / d ) );
```



Name: MatNr.:

Aufgabe 2 (Programmierparadigmen I [16 Punkte])

1. [4 Punkte] Ein Jahr ist ein Schaltjahr, wenn es entweder durch 4 aber nicht durch 100 teilbar ist oder aber, wenn es durch 400 teilbar ist. Sei `j` eine `int`-Variable mit Wert ≥ 0. Geben Sie einen `boolean`-Ausdruck an, der genau dann den Wert `true` hat, wenn Jahr `j` ein Schaltjahr ist.

2. [5 Punkte] (A) Welche Textausgabe erzeugt das untenstehende Fragment bei der Initialisierung `int a = 14;`? (B) Formulieren Sie die Zeilen zwischen `START` und `STOP` mittels `for`-Schleife semantisch äquivalent um (d.h. unabhängig von der Initialisierung soll Ihr Fragment die gleiche Textausgabe erzeugen wie das untenstehende):

```
int a = ...; // beliebige Initialisierung mit int-Wert
  // START
{ int i = a;
    do {
        System.out.print(i);
        i-=3;
    } while ( i>=5);
} // STOP
System.out.println();
```

3. [3 Punkte]

   (A) Ergänzen Sie den Typ der Methode.

```
public static ..... square(int i) {
  System.out.println(i);
  i = i*i;}
```

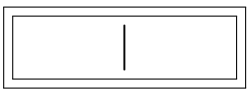
   (B) Wieviele Zeilen Textausgabe produziert die folgende Anweisung?

```
for (int i = 0; i < 5; i++) {
  System.out.println(i);
  square(i);}
```

   Begründen Sie stichpunktartig.

4. [4 Punkte] Welche Textausgabe erzeugt der Aufruf `strange(5);` ?

```
public static void strange(int n){
    if (n <=2) return;
    System.out.print(n+" ");
    strange(n-2);
    strange(n-3);
    System.out.print(n+" "); }
```



Name: MatNr.:

Aufgabe 3 (Strukturierte Daten [20 Punkte])

1. [4 Punkte] Erzeugt untenstehendes Fragment (A) einen Compilezeitfehler? Falls ja, was ist das Problem? (B) einen Laufzeitfehler? Falls ja, was ist das Problem? (C) eine Textausgabe? Falls ja, so geben Sie die Ausgabe an.

```
int[] a = {2,3,5,7,11}; int[] b = new int[6];
b = a;
a[0] = b[5];
if (a[0] % 2 == 0) System.out.println("gerade");
if (a[0] % 2 != 0) System.out.println("ungerade");
```

   Jeweils mit Ja oder Nein antworten, bei 'Ja' Fehler bzw. Ausgabe nennen:

   (A) .....;

   (B) .....;

   (C) .....;

2. [4 Punkte] `static int[][] copy2d(int[][] a)` liefert eine Referenz auf eine Inhaltskopie des übergebenen Feldes. Ergänzen Sie den Quelltext an den gekennzeichneten Stellen:

```
public static int[][] copy2d(int[][] a) {

      int n = ................;   // <-----

      int m = ................;   // <-----
      int[][] b = new int[n][m];
      for (int i = 0; i < n; i++)
          for (int j = 0; j < m; j++)
              b[i][j] = a[i][j];

      return  ................;}  // <-----
```

3. [5 Punkte] Welche Textausgabe erzeugt das folgende Codefragment?

```
int[] d = new int[10];
for (int i = 0; i < 10; i++) d[i] = 9 - i;
for (int i = 0; i < 10; i++) d[i] = d[d[i]];
for (int i = 0; i < 10; i++) System.out.print(d[i]);
System.out.println();
```

4. [3 Punkte] Welche Textausgabe erzeugt das folgende Codefragment? Begründen Sie stichpunktartig.

```
int[] a = {1, 2, 3};
int[] b = {1, 2, 3};
System.out.println(a == b);
```

5. [4 Punkte] Ein `Zeit`-Objekt speichert eine Tageszeitangabe in Stunden und Minuten: Ist $0 \leq$ `hh` $\leq 23, 0 \leq$ `mm` ≤ 59, so liefert `jetzt = new Zeit(hh,mm);` die Referenz auf ein `Zeit`-Objekt, dass der Zeitangabe `hh:mm` entspricht. Die Methode `public void in(int h, int m)` soll „die Zeit vorstellen“ um `h` Stunden und `m` Minuten ($\mathtt{h}, \mathtt{m} \geq 0$, z.B. 12:30 um 1:20 vorgestellt ergibt 13:50). Korrigieren Sie die beiden logischen Fehler im Methodenrumpf:

```
public class Zeit {
 int hh, mm;
 public Zeit(int h, int m) { hh = h; mm = m;}
 public void in(int h, int m) {
   mm = mm + m % 60; // ZEILE 1 ENTHAELT EINEN LOGISCHEN FEHLER
   hh = hh + h % 24; // ZEILE 2 ENTHAELT EINEN LOGISCHEN FEHLER
   }
}
```

korrigierte Zeile 1:

korrigierte Zeile 2:

Name: MatNr.:

Aufgabe 4 (Programmierparadigmen II [17 Punkte])

1. [10 Punkte] Welche Textausgaben erzeugt der Aufruf `java Tier`?

```
public class Tier {
    String name, bezeichnung, laut, bewegung;
    boolean geschlecht; // true = weiblich, false = maennlich
    void laut_geben() {System.out.println(this.laut);}
    void bewegen(char a, char b) {
        System.out.print(name+" "+bewegung+" von "+a+" nach "+b + ": ");
        this.laut_geben();}
    public Tier (String nam, String bez, boolean g, String l, String bew) {
        this.name = nam; this.bezeichnung = bez;
        this.geschlecht = g; this.laut = l; this.bewegung = bew;}
    public static void main(String[] args) {
        Haustier h1 =
            new Haustier("Garfield","Kater",false,"Schnurr","schleicht");
        Haustier h2 =
            new Kuh("Lisa","Muh","geht");
        WildesTier w =
            new WildesTier("Kaa","Schlange",true,"Zisch!","schlängelt sich");
        h1.ansprache(" komm mal her!"); h1.bewegen('A','B'); // Zeilen 1, 2
        Kuh k = (Kuh) h2;
        k.ansprache("Brav "); k.bewegen('A','B');           // Zeilen 3, 4
        w.ansprache(" tu mir nichts!"); w.bewegen('B','A');} // Zeilen 5, 6
}
class Haustier extends Tier {
    void ansprache(String r) {
        String s;
        if (geschlecht) s = "Liebe "; else s = "Lieber ";
        s += this.bezeichnung; s+= " "; s += this.name;
        System.out.println(s + r);}
    public Haustier(String nam, String bez, boolean g, String l, String bew){
        super(nam,bez,g,l,bew);}
}
class Kuh extends Haustier {
    void ansprache(String s) {
        System.out.println(s + this.name + ". " + s + "!");}
    public Kuh(String nam, String l, String bew) {
        super(nam,"Kuh",true,l,bew);}
}
class WildesTier extends Tier{
    void ansprache(String r) {
        String s;
        if (geschlecht) s = "Liebe "; else s = "Lieber ";
        s += this.bezeichnung; System.out.println(s + r);}
    public WildesTier(String nam, String bez, boolean g, String l, String bew) {
        super(nam,bez,g,l,bew);}
}
```

Zeile 1:
Zeile 2:
Zeile 3:
Zeile 4:
Zeile 5:
Zeile 6:

2. [7 Punkte] Quelltexte der Klassen `Angestellter` und `Chef`:

```
public class Angestellter {
 public int alter;
 public String name;
 public Angestellter(String name, int alter) {
  this.name = name;
  this.alter = alter;}
 public String toString() {
  return name + " (" + alter + ")";}
    public static void main(String[] args) {              //
        String s, abt;                                    //
        Angestellter a = new Angestellter("Schmidt",30);  //
        Chef b = new Chef("Meier",37,"Buchhaltung");      //
          ...   // Ergaenzungen                           //
    }                                                     //
}
class Chef extends Angestellter {
 public String abteilung;
 public Chef(String name, int alter, String abteilung) {
  super(name,alter);
  this.abteilung = abteilung;}
 public String toString() {
  String s = super.toString();
  return s + " leitet " + abteilung; }
}
```

Mit welchen Fragmenten folgender Tabelle kann obige `main`-Methode ergänzt werden? Kennzeichnen Sie diese in der rechten Tabellenspalte mit "OK", die anderen mit "Compilezeit-" bzw. "Laufzeit-Fehler", je nachdem welche Probleme sie verursachen. [Die Reihenfolge der Fragmente ist irrelevant.]

Fragment	Eignung
`s = a.name;`	
`abt = a.abteilung;`	
`abt = ((Chef) a).abteilung;`	
`b = a;`	
`a = b;` `abt = a.abteilung;`	
`a = b;` `abt = ((Chef) a).abteilung;`	
`System.out.println(((Angestellter) a).toString());`	
`System.out.println(((Angestellter) b).toString());`	

Name: MatNr.:

Aufgabe 5 (Algorithmen [19 Punkte])

1. [6 Punkte] Geben Sie die Laufzeitkomplexitäten der folgenden drei Funktionen in $\mathcal{O}$-Notation an (jeweils in Abhängigkeit vom Argumentwert).

```
static int fct1(int m) {      return m;    }
static int fct2(int m) {
  int i = 0;
  while ( m > 0) {
    m = m/2;
    i++;       }
  return i;    }
static int fct3(int m) {
  int s = 0;
  for (int i = 0; i < m; i++)
  s = s + i;
  return s;    }
```

`fct1:`

`fct2:`

`fct3:`

2. [4 Punkte] Die Zahlenfolge t_n ist definiert durch die Anfangsbedingungen $t_0 = 0, t_1 = 1, t_2 = 2$ und die Rekursion $t_n = t_{n-3} + t_{n-2} + t_{n-1}$ für $n > 2$. Formulieren Sie eine iterative Java-Methode `static int t(int n)` zur Berechnung von t_n. Vervollständigen Sie dazu nur die gekennzeichneten Zeilen:

```
static int t(int n) {
    if (n <= 2) return n;
    int a = 0;
    int vvl = 0, vl = 1, l = 2;

    for (int i = 0; i < ........; i++) { // <---

        a =                             // <---

        vvl =                           // <---

        vl =                            // <---

        l =                             // <---
    }
    return a;
}
```

3. [3 Punkte] In `int[] f = {4, 6, 13, 18, 23, 100, 143, 899};` soll binär gesucht werden. Geben Sie in der Reihenfolge des Auftretens an, mit welchen Werten der Suchwert `q`=13 dabei verglichen wird.

```
static int binarysearch(int[] f, int q) {
  int a = 0, e = f.length-1, m;
  while ( a <= e) {
    m = (a+e)/2;
    if ( f[m] == q) return m;
    if (q < f[m])
      e = m-1;
    else a = m+1;}
  return -1;}
```

4. [6 Punkte] Ein Sortieralgorithmus:

```
public static void sort(int[] feld) {
    int pos = 0, tmp, n = feld.length;
    for (int i = 0; i < n; i++) {
        pos = i;
        for (int j = i+1; j < n; j++)
            if ( feld[j] < feld[pos])
                pos = j;
        tmp = feld[i]; feld[i] = feld[pos]; feld[pos] = tmp;
    }
}
```

Ergänzen Sie die Feldinhalte nach Iterationen der äußeren `for`-Schleife:

Startbelegung von `feld`	6	1	9	8	3	2	4	5	7
nach 1. Iteration									
nach 2. Iteration									
nach 3. Iteration									

Probeklausur Informatik I
Wintersemester 2008/09
Prof. Dr. Carsten Damm
20. Januar 2009
von 16:00+x bis 17:30+x Uhr
(Bearbeitungszeit: 90 Minuten)

Vorname:

Nachname:

Matrikelnummer:

Es sind **keine Hilfsmittel** (Skript, Taschenrechner etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten **Blätter** für Ihre Antworten. Schreiben Sie mit blauem/schwarzem Kugelschreiber, Füller etc..

Bleistift ist nicht erlaubt. Korrekturfelder | | **nicht beschriften**!

Zum **Bestehen** der Klausur sind **45** Punkte ausreichend.

Viel Erfolg!

	Max. Punkte	Erreichte Punkte
Aufgabe 1 (Auswahlaufgabe)	5+0	
Aufgabe 2 (Java: eine einfache Klasse)	11+0	
Aufgabe 3 (Datenstrukturen und Graphen)	13+0	
Aufgabe 4 (Zahlendarstellung)	14+0	
Aufgabe 5 (Algorithmen-Entwurf und Analyse)	16+2	
Aufgabe 6 (Sortieren und Schleifeninvariante)	15+0	
Aufgabe 7 (Rekursion und Iteration)	16+2	
Summe	90+4	

Note:

Name: MatNr.:

Aufgabe 1 (Auswahlaufgabe [5+0 Punkte])

Nur jeweils eine Antwort ist richtig. Falsche Antworten werden innerhalb dieser Aufgabe verrechnet (Bsp.: a) und b) richtig, aber c), d) und e) falsch ergibt 0P).

Kreisen Sie die richtige Nummer ein. Mehr Kreise (oder unleserlich) ergeben 0P für die Teilaufgabe.

a) Überladen einer Methode

1. ist die Zuordnung mehrerer Semantiken zum gleichen Methodennamen (Auswahl erfolgt durch Aufrufkontext)
2. ist die Programmierung unübersichtlich vieler verschiedener Funktionalitäten innerhalb einer Methode
3. geschieht zum Beispiel bei der Neudefinition der Methode `Object.toString()` in einer abgeleiteten Klasse

b) imperative Programmierung ist gekennzeichnet durch

1. Kommandozeilensteuerung
2. Befehlsfolgen zur Zustandsänderung
3. Zerlegen von Problemen in Teilprobleme

c) Rekursion ist

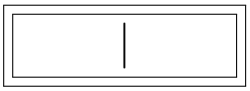
1. das Rückverfolgen von Methodenaufrufen
2. das Aufrufen einer Methode in deren eigenem Methodenrumpf
3. das Rückführen einer Problemlösung auf zwei kleinere Probleme

d) Aus welchen drei Kontrollstrukturen lassen sich alle Algorithmen aufbauen?

1. Eingabe, Verarbeitung und Ausgabe
2. for-Schleife, while-Schleife und do-while-Schleife
3. Komposition, Alternative und while-Schleife

e) Algorithmen sind stets

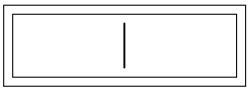
1. in endlicher Zeit abarbeitbar
2. endlich beschreibbar
3. in asymptotischer Form analysierbar



Name: MatNr.:

Aufgabe 2 (Java: eine einfache Klasse [11+0 Punkte])
Geben Sie eine einfache Klasse `Modul` an, die man zur Verwaltung der Module in einem Bachelorstudiengang benutzen kann. Jedes Modul zeichnet sich durch seinen Namen, z.B. “Info I” oder “C-Kurs”, sowie seine Leistungspunktezahl aus. Außerdem will man jederzeit die Anzahl angebotener Module ermitteln können, sowie die Gesamtanzahl der Leistungspunkte in diesen. *Benutzen Sie in Ihrem Quelltext die Bezeichner* `name`, `punkte`, `anzahl` *und* `gesamtPunkte` *für diese Dinge.* Die folgenden Operationen sollen möglich sein:

- Erzeugen einer Instanz dieser Klasse unter Angabe des Namens,
- Erzeugen einer Instanz dieser Klasse unter Angabe des Namens und der Leistungspunktezahl,
- Setzen und Abfragen der Punktzahl jeder einzelnen Instanz
- Es soll möglich sein, zu jeder Zeit abzufragen, wieviele Instanzen der Klasse bereits erzeugt wurden, und wieviele Leistungspunkte diese zusammen haben.



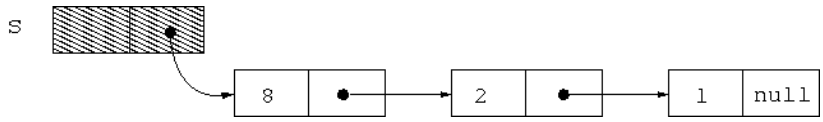
Name: MatNr.:

Aufgabe 3 (Datenstrukturen und Graphen [13+0 Punkte])

a) (3P) Auf einen anfangs leeren Stack werden `push/pop`-Operationen angewendet. Tragen Sie den Zustand des Stacks nach den angegebenen Aufrufen ein (erste Zeile ist komplett vorgegeben). Bitte schreiben Sie deutlich!

`push(1);push(2);push(8);`	1	2	8									
`push(5);push(3);pop();`												
`push(1);pop();pop();`												
`pop();push(7);push(1);`												

b) (6P)

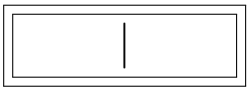


Dies skizziert die Realisierung eines Stacks durch eine verkettete Liste mit Startknoten `s` (Zustand wie erste Tabellenzeile oben). Mögliche Befehle:

- *n* `= neu(`*i*`)` — Erzeugen von neuem Knoten namens *n* mit Inhalt *i*
- *x* `= inhalt(`*m*`)` — kopiere in *m* gespeicherte Zahl nach *x*
- `nachfolger(`*o*`) = `*p* — der Nachfolger von *o* ist nun *p*
  (oder auch: *k* `= nachfolger(`*z*`)` — Nachfolger von *z* in *k* speichern)

Geben Sie je eine Befehlsfolge an, um (1) bzw. (2) zu realisieren:
(1) `x = top(); pop();` (2) `push(5);`
Auf Reihenfolge achten! Auf Variablendeklarationen, Leerheitstest o.ä. brauchen Sie nicht achten!

c) (4P) Welche Tiefe hat ein binärer Baum mit n Blättern mindestens? Geben Sie den Wert möglichst genau an und begründen Sie Ihre Antwort.



Name: MatNr.:

Aufgabe 4 (Zahlendarstellung [14+0 Punkte])

a) (3P) Der hypothetische Datentyp `gint` stellt ganze Zahlen bekanntlich mit dem Zweierkomplement der Länge 4 dar (4-Bit-Zweierkomplement). Ergänzen Sie die folgenden Konstantendefinitionen der Wrapper-Klasse `Ginteger`:

```
public static final gint MAX_VALUE = ........;

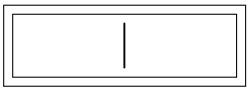
public static final gint MIN_VALUE = ........;
```

b) (5P) Wir betrachten dazu noch den Datentyp `glint`: ganze Zahlen werden hier mit der 8-Bit-Zweierkomplement dargestellt. Ermitteln Sie die Bitmuster für 40 und -81.

c) (6P) Ergänzen Sie diese Tabelle von `glint`-Werten.

Bitte deutlich schreiben, Nebenrechnung oder Probe nur auf Blatt rechts!

dezimal	binär (mit Zweierkompl.)	oktal	hexadezimal
13			
	$(10001011)_{\overline{2}}$		
		$(357)_8$	
			$(1E)_{16}$



Name: MatNr.:

Aufgabe 5 (Algorithmen-Entwurf und Analyse [16+2 Punkte])
Gegeben ist eine nur aus "+" oder "-" bestehende endliche Zeichenfolge, z.B. "`--+++--+++++-+-+-+-+--+`"
gespeichert in einem Feld `z[0]...z[k-1]`.

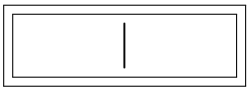
a) (8P + (2 Zusatzpunkte für (asymptotisch) optimalen Algorithmus)) Geben Sie einen Algorithmus an, der berechnet, wie lang die längste Teilfolge ist, die nur aus "+"-Zeichen besteht (im Beispiel: 5).

Der Algorithmus soll entweder in Java- oder Pseudocode- oder als Programmablaufplan oder als klar formulierter Text beschrieben werden. |

b) (5P) Geben Sie eine möglichst gute Abschätzung Ihres Algorithmus in $\mathcal{O}$-Notation an (mit kurzer Begründung). |

c) (3P) Ein Algorithmus zur Lösung von a) liest beginnend bei Index 0 fortlaufend die Feldeinträge (und stellt interne Berechnungen beliebiger Art an). Das Lesen wird aber sofort unterbrochen, wenn die Antwort feststeht. Der Algorithmus kann die Feldlänge im vorhinein ermitteln.

Geben Sie Zeichenketten (1) und (2) der Länge 8 wie folgt an: (1) Es muss bis zum Ende gelesen werden. (2) Es muss *nicht* bis zum Ende gelesen werden und das Resultat ist 4. Begründen Sie jeweils kurz. |



Name: MatNr.:

Aufgabe 6 (Sortieren und Schleifeninvariante [15+0 Punkte])
Wir betrachten folgenden elementaren Sortieralgorithmus

```
30    public static void selectionSort(int[] feld) {
31        int maxpos, tmp;
32        for (int i=feld.length; i>1; i--) {
33            // bestimme Position des Maximums im Intervall [0,...,i-1]
34            maxpos = 0; // Position des Maximums
35            for (int j=0; j<i; j++)
36                if ( feld[j] > feld[maxpos])
37                    maxpos = j;
38            // tausche feld[i-1] und feld[maxpos]:
39            tmp = feld[i-1]; feld[i-1] = feld[maxpos]; feld[maxpos] = tmp;
40        }
41    }
```

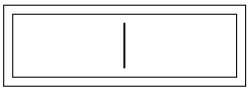
a) (4P) Wenden Sie den Algorithmus an auf
`int[] test = {1, 6, 9, 8, 2, 3, 4, 5, 7};`
und ergänzen Sie die Tabelle um die Feldinhalte *nach* Iterationen 1, 2 und 3 der *äußeren* `for`*-Schleife* (Zeilen 32-40).

Start	1	6	9	8	2	3	4	5	7
nach 1. Iteration									
nach 2. Iteration									
nach 3. Iteration									

b) (5P) Geben Sie (mit Begründung) eine möglichst gute asymptotische Abschätzung der Laufzeit des Algorithmus auf Eingaben der Länge n an

c) (6P) Es gilt folgende Aussage: Nach Ausführung der inneren Schleife (Zeilen 35–37) gilt `feld[maxpos]` enthält das Maximum der Einträge im Teilfeld `feld[0..i-1]`.

Begründen Sie damit, dass das Feld nach `feld.length` − 1 Iterationen der *äußeren Schleife* (Zeilen 32–40) sortiert ist. Das formale Konzept der Schleifeninvariante können, müssen Sie aber nicht benutzen.



Name: MatNr.:

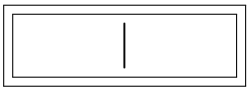
Aufgabe 7 (Rekursion und Iteration [16+2 Punkte])
Pseudocode ist ausreichend für diese Aufgaben. Die Zahlenfolge $a_0, a_1, a_2, \ldots$ ist definiert durch: $a_0 = 0, a_1 = 1$ und $a_n = -2a_{n-1} + 3a_{n-2}$ für $n > 2$.

a) (6P) Formulieren Sie einen *rekursiven* Algorithmus, der bei Eingabe n (natürliche Zahl) den Wert a_n zurückgibt (nicht druckt!). Zeichnen Sie den Aufrufbaum (Rekursionsbaum) bei Aufruf mit der Eingabe $n = 5$.

b) (6P) Formulieren Sie einen *iterativen*[2] Algorithmus, der bei Eingabe n (natürliche Zahl) den Wert a_n zurückgibt (nicht druckt!).

c) (4P + 2 Zusatzpunkte für Beweis der Formel) Für alle n gilt $a_n = \frac{1-(-3)^n}{4}$. Formulieren Sie aufgrund dessen einen Algorithmus, der bei Eingabe n (natürliche Zahl), den Wert a_n zurückgibt (nicht druckt!). Potenzieren darf dabei *nicht* benutzt werden.

[2]Zwei Möglichkeiten: (1) Feld anlegen, das nach und nach mit den Werten a_0, a_1, a_2, ... gefüllt wird oder (2) lediglich die letzten Werte speichern, daraus den nächsten berechnen und geeignet umspeichern usw.



Name: MatNr.:

Wiederholungsklausur Informatik I
Sommersemester 2007
Prof. Dr. Carsten Damm
19. April 2007, 12:00 - 13:30 Uhr
Bearbeitungszeit: 90 Minuten

Vorname:

Nachname:

Matrikelnummer:

Bei der Klausur sind **keine Hilfsmittel** (Skripten, Taschenrechner etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten **Blätter** für Ihre Antworten. Schreiben Sie auf **jedes Blatt** oben Ihren Namen und Ihre Matrikelnummer. Schreiben Sie mit blauem/schwarzem Kugelschreiber, Füller etc.; Bleistift ist nicht erlaubt.

Zum **Bestehen** der Klausur sind **45** Punkte hinreichend.

	Punkte	Schätzung für
Aufgabe 1 (Reg. Ausdrücke, Endl. Automaten, Zahldarstellung)	20	10
Aufgabe 2 (Aufwand und Komplexität + 3 Bonuspunkte)	15	10
Aufgabe 3 (Objektorientierung & Programmierung)	10	5
Aufgabe 4 (Java)	20	10
Aufgabe 5 (Sortierverfahren)	10	5
Aufgabe 6 (Algorithmen und Datenstrukturen)	15	8
Summe	90	48

Note:

Aufgabe 1 (Reg. Ausdrücke, Endl. Automaten, Zahldarstellung [20 Punkte])

a) Geben Sie einen regulären Ausdruck über dem Alphabet $\{a, f, g, s, v, z\}$ an, der das übliche Verhalten von Passagieren in einem Flughafen beschreibt.

Die Aktionen sind: Gepäckwagen nehmen: g, Gepäckwagen zurückgeben: z, Flughafen verlassen: v, Gepäckwagen schieben: s, Gepäckstück am Schalter abgeben: a, mit Flugzeug abreisen: f.

Beispiel $gsssaaaszf$: Passagier schiebt Gepäckwagen mehrfach, gibt drei Gepäckstücke am Schalter ab, gibt den Gepäckwagen ab und fliegt ab.

Vorgaben: Eingang, Ausgang und Schalter sind voneinander entfernt. Wenn Gepäckstücke am Schalter nicht abgegeben werden (Reisepaß abgelaufen), wird der Gepäckwagen zurückgegeben und der Flughafen verlassen. (6 P)

Lösung

$gss^*aa^*ss^*zf \mid gss^*zv \;=\; gss^*(aa^*ss^*zf \mid zv)$

b) Entwerfen Sie einen endlichen Automaten der die folgende Sprache L über dem Alphabet $\{1, 2, ., E\}$ akzeptiert (Zustandsdiagramm genügt).

$$L = (\{1,2\}\{1,2\}^*.\{1,2\}^*)(E\{1,2\}\{1,2\}^* \cup \{\varepsilon\})$$

Erläuterung: L besteht aus den Gleitkommaliteralen, die mit einer Ziffer beginnen, einen Dezimalpunkt enthalten und außer diesem und einem eventuellen Exponentenzeichen nur die Ziffern 1 und 2 (z.B. 12.12 oder 12.E2, aber nicht 123.123 oder .11E12 oder 12.E-1): (6 P)

Lösung

Siehe Automat auf Beiblatt 1

c) Gegeben sind die Zahlen 128 und 35. Ihre Summe ist 163.

1. Stellen Sie beide als 8-Bit Binärzahlen dar. (3 P)

   Lösung

   128: 10000000 (1.5 P)
   35: 00100011 (1.5 P)

2. Addieren Sie beide Zahlen im Binärformat und verifizieren Sie das Ergebnis indem Sie es in das Dezimalformat zurückwandeln. (2 P)

   Lösung

	128:	10000000	
	35:	00100011	
Add:	163:	10100011	= 128 + 32 + 2 + 1

3. Stellen Sie -35 als Zweierkomplement dar und berechnen Sie $128 - 35$. (3 P)

Lösung

Zweierkomplement von 35: 35 ist		00100011
Einerkomplement:		11011100
(Zweierkomplement) +1:		11011101
Addieren:	10000000	
+	11011101	
Summe:	01011101	$= 128 - 35 = 93$

Aufgabe 2 (Aufwand und Komplexität [15 Punkte] + 3 Bonuspunkte)

a) Berechnen Sie mit Hilfe des Mastertheorems eine asymptotische obere Schranke für eine Funktion T mit folgenden Eigenschaften (2 P)

$$T(1) = 1$$
$$T(n) = 49T(n/7) + 3n^2$$

Lösung

$T(n) = 49T(n/7) + 3n^2$
$\Leftrightarrow \; a = 49, \; b = 7, \; k = 2$

Offensichtlich ist $a = b^k \; (49 = 7^2)$

Damit folgt nach dem Master-Theorem:

$T(n) \in O(n^2 \log n)$.

b) Bestimmen Sie für folgende Programmfragmente wie oft *genau* die Methode `tuWas()` aufgerufen wird. K und M sind dabei beliebige natürliche Zahlen (je 2 P):

1. ```
   for (int i=0;i<K;i++)
     for (int j=i,j>=0;j--)
       tuWas();
   ```

   Lösung

   $K * (K + 1)/2$

2. ```
   for (int i=0;i<K;i++)
     for (int j=0,j<K;j++)
       tuWas();
   ```

   Lösung

   K^2

3. ```
   for (int i=1;i<K;i++)
     for (int j=0,j<M;j++)
       tuWas();
   ```

   Lösung

   $(K - 1) * M$

c) Für natürliche Zahlen a, m bezeichnet $a \bmod m$ den Rest von a bei Division durch m. [Erinnerung: `%` steht für mod.]
Die Methode `modexp` liefert bei positiven Argumentwerten a, e und m den Wert $a^e \bmod m$ (also den Rest von a^e bei Division durch m).
   ```

```
public class modexp {
  // Voraussetzung: a,e,m > 0
  public static int modexp(int a, int e, int m) {
    int r = 1;
    while ( e > 0) {
      if ( (e % 2) == 1)
      r = ( r * a) % m;
      e = e/2;
      a = a * a % m;
    }
    return r;
  }
}
```

1. Berechnen Sie $10^{13} \bmod 7$. Sie können obigen Algorithmus verwenden. (4 P)

   Lösung

   (3P Berechnungsweg, 1P Ergebnis)

   1. Möglichkeit:
   $10^{13} = 10 \cdot (10^6)^2 = 3 \cdot (3^6)^2 \bmod 7$
   $= 3 \cdot ((3^3)^2)^2$
   $= 3 \cdot ((3 \cdot 3^2)^2)^2 \qquad (3^2 = 9 \bmod 7 = 2)$
   $= 3 \cdot ((3 \cdot 2)^2)^2$
   $= 3 \cdot (36)^2 \qquad (36 \bmod 7 = 1)$
   $= 3 \cdot 1^2$
   $= 3 \bmod 7$

   2. Möglichkeit: gemäß angegebenem Algorithmus

   - $r = 1, a = 10,\ e = 13\ > 0$ ungerade
     $\Rightarrow r = 10 \bmod 7\ = 3,\ e = 6,\ a = 100 \bmod 7\ = 2$
   - $r = 3, a = 2,\ e = 6\ > 0$ gerade
     $\Rightarrow e = 3,\ a = 4 \bmod 7\ = 4$
   - $r = 3, a = 4,\ e = 3\ > 0$ ungerade
     $\Rightarrow r = 4 \cdot 3 =\ 12\ =\ 5 \bmod 7,\ e = 1,\ a = 4 \cdot 4 = 16\ =\ 2 \bmod 7$
   - $r = 5, a = 2,\ e = 1\ > 0$ ungerade
     $\Rightarrow r = 2 \cdot 5 =\ 10\ =\ 3 \bmod 7,\ e = 0,\ a = 4\ \Rightarrow$ return 3

2. Wie viele Iterationen erfolgen höchstens, falls $e = 1000000$? Wie viele mindestens? Begründen Sie die Antwort. (3 P)

   Lösung

   e wird in jeder Iteration halbiert (ganzzahlig).
   Wie oft muss 1000000 halbiert werden, bis der Wert 0 erreicht wird ist?

   Abschätzung: $1000000 = 1000000\ < 10241024 = 2^{20}$

$\Rightarrow$ spätestens nach 20 Iterationen ist der Wert 0 erreicht.
(19 mal genügt nicht, denn $2^{19} = 512.1024 \ < 1000000$)

3. *Bonus **(3 Bonuspunkte)**: Beweisen Sie mit einer Schleifeninvariante, dass der Algorithmus korrekt ist. Sie können benutzen, dass $(x \bmod m) * (y \bmod m) = (x * y \bmod m)$ ist.

Lösung

Korrektheit: Sei $z = a^e \bmod m$

($e > 0$ und $z = r \cdot a^e \bmod m$) ist die Schleifeninvariante.

Sie gilt offenbar in der ersten Iteration.

Sei ($e > 0$ und $z = r \cdot a^e \bmod m$ vor einer Iteration

1. Fall:
e ist gerade. Dann bleibt der Wert von r erhalten, e wird halbiert, a wird quadriert.
Wegen $r \cdot a^e = r \cdot (a^2)^{\frac{e}{2}} \bmod m$ gilt die Gleichung auch nach der Iteration.

2. Fall:
e ist ungerade.
Wegen $r \cdot a^e = r \cdot a \cdot (a^2)^{\frac{e-1}{2}} \bmod m$ gilt die Gleichung auch nach der Iteration.

Aufgabe 3 (Objektorientierung & Programmierung [10 Punkte])

a) Betrachten Sie folgende Java-Klassen: (2 P)

```
class A {
  public static void foo (int x, long y) {
    System.out.println(x+y);
  }
  public static void foo (long x, int y) {
    System.out.println(x*y);
  }
}

class B {
  private byte x = 3;
  public void foo (int x, int y) {
    A.foo(this.x+y, x);
  }
  public static void main (String[] args) {
    B objB = new B();
    objB.foo(1,2);
  }
}
```

Geben Sie im obigen Beispiel alle Situationen an, in denen Methoden überladen oder überschrieben werden und Attribute überdeckt werden.

Lösung

Die Methode `foo` in der Klasse A ist überladen.
Es werden im Beispiel keine Methoden überschrieben.
In der Methode `foo` in der Klasse B überdeckt der Parameter `x` vom Typ `int` das Instanzattribut `x` der Klasse B vom Typ `byte`.

b) Erläutern Sie folgende Begriffe und geben Sie je ein Beispiel: (4 P)

- Implizite Typumwandlung
- Explizite Typumwandlung

Lösung Bei einer impliziten Typumwandlung wird ein Ausdruck von Typ `A` an einer Stelle verwendet, wo eigentlich ein Ausdruck vom Typ `B` erwartet wird (z.B. Zuweisung an Variable vom Typ `B`). Der Ausdruck wird intern in einen vom Typ `B` verwandelt. Implizite Typumwandlungen sind in der Regel nicht verlustbehaftet und erfolgen in Richtung „größerer“ Typen (z.B. von `int` nach `long`). (1 P)

```
class A {                    (1 P)
  int x;
```

```
  long y;
  // Implizite Typumwandlung int -> long
  y = x*1000;
  .
  .
}
```

Bei einer expliten Typumwandlung (*cast*) wird ein Ausdruck von Typ `A` explizit in einen Ausdruck eines „kleineren“ Typs `B` umgewandelt. Dieses ist in der Regel verlustbehaftet und muss explizit durch einen in Klammern vorangestellten Typ angezeigt werden. (1 P)

```
class A {                        (1 P)
  int x;
  long y;
  // Explizite Typumwandlung long -> int
  x = (int) y;
  .
  .
}
```

c) Kreuzen Sie bei den folgenden Aussagen „Richtig“ oder „Falsch“ an. Pro korrekter Auswahl gibt es 1 Punkt. (4 P)

Für jede falsche Antwort wird 1 Punkt abgezogen! Jede nicht gegebene Antwort wird aber nur als 0 Punkt gewertet. Sollten Sie eine negative Punktzahl erreichen, wird diese Aufgabe mit 0 Punkt gewertet.

Aussage	**Richtig**	**Falsch**
Die `goto`-Anweisung ist in Java erlaubt.		
Es gibt Java-Programme, die nicht terminieren.		
Auf jedes Java-Objekt darf nur eine Referenzvariable zeigen.		
Auf Maschine A kompilierte Java-Programme laufen grundsätzlich nicht auf der Java-Virtual-Machine der Maschine B.		

Lösung

Aussage	**Richtig**	**Falsch**
Die `goto`-Anweisung ist in Java erlaubt		X
Es gibt Java-Programme, die nicht terminieren.	X	
Auf jedes Java-Objekt darf nur eine Referenzvariable zeigen.		X
Auf Maschine A kompilierte Java-Programme laufen grundsätzlich nicht auf der Java-Virtual-Machine der Maschine B.		X

Aufgabe 4 (Java [20 Punkte])

a) Geben Sie eine einfache Klasse `Person` an, mit der Personendaten abgespeichert werden können. Jede Person zeichnet sich durch ihren Namen aus, sowie ihr Gewicht. Folgende Operationen sollen möglich sein: (10 P)

- Erzeugen einer Instanz unter Angabe des Namens,
- Erzeugen einer Instanz unter Angabe von Name und Gewicht,
- Setzen und Abfragen des Gewichtes jeder einzelnen Instanz
- Es soll möglich sein, zu jeder Zeit abzufragen, wieviele Instanzen der Klasse bereits erzeugt wurden, und wieviele diese zusammen wiegen.

Lösung

```
public class Person {            // Punktezusammensetzung:
  String name;                                    // 0.5 P
  long gewicht;      // oder int oder sonstwas    // 0.5 P
  static int anzahl;                     // attr: 0.5 P,
                                        // static: 0.5 P
  static long gesamtgewicht;            // attr: 0.5 P,
                                        // static: 0.5 P
  public Person(String t){                           // 1 P
    name = t;
    anzahl++;
  }

  public Person(String t, long p) {                  // 1 P
    name = t;
    anzahl++;
    gewicht = p;
    gesamtgewicht = gesamtgewicht + p;
  }

  public void setGewicht(long p) {                  // 1 P
    gesamtgewicht = gesamtgewicht - gewicht + p;
    gewicht = p;
  }

  public long getGewicht() {                        // 1 P
    return gewicht;
  }

  public static int wieviele() {          // Meth: 1 P,
    return anzahl;                       // static 0.5 P
  }

  public static long gesamtgewicht() {       // Meth: 1 P,
    return gesamtgewicht;                 // static 0.5 P
  }
}
```

b) Für natürlicher Zahlen n, k ($n \geq k \geq 0$) gibt der Binomialkoeffizient $\binom{n}{k}$ an, wieviele Möglichkeiten es gibt, aus n Elementen k auszuwählen. Hieraus ergibt sich zwangsläufig, dass in den beiden Fällen $n = k$ und $k = 0$ das Ergebnis jeweils 1 ist. Für die anderen Fälle kann bei der Berechnung die folgende Eigenschaft des Binomialkoeffizient ausgenutzt werden:

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

Geben Sie eine rekursive Java-Methode `binom` an, die für zwei nicht-negative Integer-Argumente n und k, mit $n \geq k \geq 0$, den Binomialkoeffizienten berechnet. (2 P)
Hinweis: In der Funktion soll nicht getestet werden, ob die Voraussetzung $n \geq k \geq 0$ erfüllt ist.

Lösung

```
static int binom (int n, int k)
{
    if (n == k || k == 0) {
        return 1;
    } else {
        return binom (n - 1, k) + binom (n - 1, k - 1);
    }
}
```

c) In folgendem Java-Codefragment sind einige Fehler enthalten. Die Zeilen des Aufretens sind durch Kommentare markiert. Lokalisieren Sie die Fehler näher, indem Sie die genauen Fehlerstellen im Programmtext markieren. Geben Sie für jeden Fehler eine kurze Fehlerbeschreibung an, die eine mögliche Meldung des Java-Systems widerspiegelt. Machen Sie insbesondere eine Aussage dazu, ob der Fehler zur Übersetzungszeit oder zur Laufzeit auftritt. Schlagen Sie vor, wie der Fehler möglichst einfach im Sinne der beabsichtigten Wirkungsweise des Programmes korrigiert werden kann. (8 P)

```
class List {
  Element ListHead;

  public int zeroCnt(int x) {

  // Zaehlt die  Anzahl der Nullen vor dem ersten Auftreten von x.
  // Falls x nicht vohanden ist, wird 0 zurueckgegeben.

    int cnt;
    Element p = ListHead;
    while (p.dat != x && p != null)        // Fehlerzeile 1
      if (p.dat = 0)                       // Fehlerzeile 2
```

```
      cnt++;                              // Fehlerzeile 3(a)
    else
      p = p.next;
  return cnt;                             // Fehlerzeile 3(b)
}

private class Element {
  int dat;
  Element next;
}

public static void main(String[] args) {
  List lst  = new List;                   // Fehlerzeile 4
      :
  // Aufbau von lst. Dieser Code ist fehlerfrei.
      :
  lst.zeroCnt(3);
}
}
```

Lösung

```
class List {
  Element ListHead;

  public int zeroCnt(int x) {

  // Zaehlt die  Anzahl der Nullen vor dem ersten Auftreten von x.
  // Falls x nicht vohanden ist, wird 0 zurueckgegeben.

    int cnt;
    Element p = ListHead;
    while (p.dat != x && p != null)       // Fehlerzeile 1
      if (p.dat = 0)                      // Fehlerzeile 2
        cnt++;                            // Fehlerzeile 3(a)
      else
        p = p.next;
    return cnt;                           // Fehlerzeile 3(b)
  }

  private class Element {
    int dat;
    Element next;
  }

  public static void main(String[] args) {
    List lst  = new List;                 // Fehlerzeile 4
        :
    // Aufbau von lst. Dieser Code ist fehlerfrei.
```

```
        :
    lst.zeroCnt(3);
  }
}
```

– Fehlerzeile 1: **(2 P)**
Fehlerart: *Laufzeitfehler. Zugriff auf* `null`*-Referenz.*

Korrekturvorschlag: *Vertauschen der Bedingungsreihenfolge (weil* `&&` *abhängig von Reihenfolge). Also:* `(p != null && p.dat != x)`

– Fehlerzeile 2: **(2 P)**
Fehlerart: *Compilierfehler, genauer Typfehler. Die Zuweisung liefert ein Integerresultat, wo ein Boolescher Wert erwartet wird.*

Korrekturvorschlag: *Zuweisungsoperator = durch Vergleichsoperator == ersetzen.*

– Fehlerzeile 3(a) und 3(b): **(2 P)**
Fehlerart: *Compilierfehler, genauer statischer Initialisierungsfehler. Die lokale Methodenvariable* `cnt` *ist (möglicherweise) uninitialisiert.*

Korrekturvorschlag: `cnt`*-Deklaration um Initialisierung ergänzen, also:* `int cnt = 0;`

– Fehlerzeile 4: **(2 P)**
Fehlerart: *Compilierfehler, genauer Syntaxfehler. Bei Konstruktoraufruf wird (leere) Parameterliste erwartet.*

Korrekturvorschlag: *Fehlerzeile 4:* `List lst = new List();`

Aufgabe 5 (Sortierverfahren [10 Punkte])

a) Beschreiben Sie stichpunktartig die Merkmale eines *Teile und Herrsche*-Algorithmus (Divide&Conquer). (3 P)

Lösung

Voraussetzung Problem lässt sich in unabhängige Teilprobleme zerlegen, die separat gelöst werden können und deren Lösung zu Lösung des Ausgangsproblems kombiniert werden kann.

Teile das Problem in Teilprobleme auf.

Beherrsche große Teilprobleme durch rekursives Aufteilen und kleine Teilprobleme durch direktes Lösen.

Kombiniere Lösungen der Teilprobleme zu Lösung des Ausgangsproblems.

(Wohlwollend korrigieren. Wer sich allein auf Mergesort bezieht (und dies korrekt beschreibt) bekommt auch die volle Punktzahl.)

b) Betrachten Sie folgende Implementierung der Methode `mergeSort` aus dem Vorlesungsskript:

```
public void mergeSort() {
  mergeSort(0,feld.length-1);
}
private void mergeSort(int von, int bis) {
  if (von == bis) return;
  int mitte = (von + bis) / 2; // ganzz. Division
  mergeSort(von, mitte);
  mergeSort(mitte+1, bis);
  vereinige(von,mitte,bis);
}
```

Implementieren Sie dazu die Methode `vereinige`. (7 P)

Lösung

```
private void vereinige(int von, int mitte, int bis) {
  int[] tmp = new int[bis-von+1]; // Hilfsfeld
  int i = von,j = mitte+1, k;

  // fuelle tmp sukzessive auf, benutze dazu den folgenden ..
  for (k=0; i<=mitte && j <= bis;k++) {   //.. Wechselschalter:
    tmp[k]=(feld[i] <= feld[j] ? feld[i++] : feld[j++]);
  }   // i oder j ist am Ende angekommen ..
  while ( i <= mitte) // .. => Rest noch kopieren:
    tmp[k++] = feld[i++];
    // falls j < bis, macht nichts: sind alle groesser
```

```
    // tmp[0..k-1] zurueckkopieren:
  for (int t = 0; t<k; t++)
    feld[von + t] = tmp[t];
}
```

Aufgabe 6 (Algorithmen und Datenstrukturen [15 Punkte])

a) Im folgenden Programmfragment wird mit Hilfe von `push` und `pop` ein anfangs leerer Stack gesteuert. `push` operiert wie üblich, aber `pop` soll etwas anders wirken als in der Vorlesung: Der Aufruf entfernt das zuletzt hinzugefügte Element und liefert es auch als Rückgabewert[1]. Notieren Sie in einer Tabelle, welchen Inhalt der Stack nach den einzelnen Anweisungen hat. Geben Sie auch an, welche Ausgabe das Programmstück erzeugt. (5 P)

```
push(12);
push(6);
push(pop()+pop());
push(8);
push(8);
push(64);
push(pop()/pop());      // auf Auswertungsreihenfolge achten!
push(1/(pop()/pop()));  // ..
push(3);
push(pop()*pop());      // ..
push(pop()+pop());      // ..
System.out.println(pop());
```

Lösung

					64					
				8	8	8		3		
	6		8	8	8	8	1	1	3	
12	12	18	18	18	18	18	18	18	18	21

b) Kreuzen Sie bei den folgenden Aussagen „Richtig“ oder „Falsch“ an. Pro korrekter Auswahl gibt es 1 Punkt[2]. (4 P)

Aussage	**Richtig**	**Falsch**
Binäre Suche arbeitet auf manchen `int`-Arrays inkorrekt.		
Binäre Suche hat auch im schlechtesten Fall logarithmische Laufzeit.		
Der Aufwand für Merge-Sort ist immer $O(N)$		
Ein Stack arbeitet nach dem FIFO-Prinzip		

Lösung

Aussage	**Richtig**	**Falsch**
Binäre Suche arbeitet auf manchen `int`-Arrays inkorrekt.	X	
Binäre Suche hat auch im schlechtesten Fall logarithmische Laufzeit.	X	
Der Aufwand für Merge-Sort ist immer $O(N)$		X
Ein Stack arbeitet nach dem FIFO-Prinzip		X

[1] In der Vorlesung war für Letzteres der Aufruf `top()` zuständig.

[2] **Für jede falsche Antwort wird 1 Punkt abgezogen!** Jede nicht gegebene Antwort wird aber nur als 0 Punkt gewertet. Sollten Sie eine negative Punktzahl erreichen, wird diese Aufgabe mit 0 Punkt gewertet.

c) Ein Algorithmus heißt

- *deterministisch*, falls er zu jeder Eingabe und jedem Zustand der Berechnung einen eindeutig bestimmten Folgezustand definiert,
- *terminierend*, falls er bei jeder Eingabe nach endlich vielen Schritten einen Endzustand erreicht (d.h. abbricht) und
- *determiniert*, falls jede Berechnung des Algorithmus bei gleichen Eingaben das gleiche Ergebnis (d.h. die gleiche Ausgabe) liefert[3].

Geben Sie für die beiden folgenden Algorithmen an, welche der drei genannten Eigenschaften sie haben und welche nicht und begründen Sie die Antwort stichpunktartig. (6 P)

Algorithmus 1:

1. Wähle eine beliebige Zahl zwischen -10 und 20.
2. Multipliziere diese Zahl mit sich selbst.
3. Gib das Ergebnis aus, falls es größer als 10 ist. Andernfalls gehe zu 2.

Algorithmus 2:

```
public float rechne (float a) {
  float p;
  if (a>10) {
    p = Math.random(); // erzeugt Zufallszahl zwischen 0 und 1
    if (a>20) {
      a = a-p;
    }
    a = a+p;
  }
  return a;
}
```

Lösung

Algorithmus 1: (3 P)
nicht-deterministisch (ich rate 3 du rätst 4), nicht-terminierend (Endlosschleife z.B. bei 0), nicht-determiniert (wenn er hält, dann zwar immer mit der gleichen Ausgabe — aber er hält eben nicht immer)

Algorithmus 2: (3 P)
nicht-deterministisch (da konkrete Berechnung vom Zufall abhängig), terminierend (da keine Schleife), nicht-determiniert (z.B. $a = 11$ und $a = 21$ liefern unterschiedliche Ergebnisse (sofern $p \neq 0$)).

[3] man sollte vielleicht besser sagen: das Ergebnis des Algorithmus ist determiniert

Achtung: Auf die Begründung kommt es an! Bei Algorithmus 2 wäre z.B. auch die Antwort „deterministisch“ korrekt, falls zusätzlich bemerkt wurde, dass Math.random() bei gleichem Startwert des Pseudozufallsgenerators gleiche Ergebnisse liefert.

Wiederholungsklausur Informatik I
Sommersemester 2007
Prof. Dr. Carsten Damm
19. April 2007, 12:00 - 13:30 Uhr
Bearbeitungszeit: 90 Minuten

Vorname: Tiger

Nachname: Scott

Matrikelnummer: 0815

Bei der Klausur sind **keine Hilfsmittel** (Skripten, Taschenrechner etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten **Blätter** für Ihre Antworten. Schreiben Sie auf **jedes Blatt** oben Ihren Namen und Ihre Matrikelnummer. Schreiben Sie mit blauem/schwarzem Kugelschreiber, Füller etc.; Bleistift ist nicht erlaubt.

Zum **Bestehen** der Klausur sind **45** Punkte hinreichend.

	Punkte	erreicht
Aufgabe 1 (Reg. Ausdrücke, Endl. Automaten, Zahldarstellung)	20	
Aufgabe 2 (Aufwand und Komplexität + 3 Bonuspunkte)	15	
Aufgabe 3 (Objektorientierung & Programmierung)	10	
Aufgabe 4 (Java)	20	
Aufgabe 5 (Sortierverfahren)	10	
Aufgabe 6 (Algorithmen und Datenstrukturen)	15	
Summe	90	

Note:

Name: Tiger Scott MatNr.: 0815

Aufgabe 1 (Reg. Ausdrücke, Endl. Automaten, Zahldarstellung [20 Punkte])

a) Geben Sie einen regulären Ausdruck über dem Alphabet $\{a, f, g, s, v, z\}$ an, der das übliche Verhalten von Passagieren in einem Flughafen beschreibt.

Die Aktionen sind: Gepäckwagen nehmen: g, Gepäckwagen zurückgeben: z, Flughafen verlassen: v, Gepäckwagen schieben: s, Gepäckstück am Schalter abgeben: a, mit Flugzeug abreisen: f.

Beispiel $gsssaaaszf$: Passagier schiebt Gepäckwagen mehrfach, gibt drei Gepäckstücke am Schalter ab, gibt den Gepäckwagen ab und fliegt ab.

Vorgaben: Eingang, Ausgang und Schalter sind voneinander entfernt. Wenn Gepäckstücke am Schalter nicht abgegeben werden (Reisepaß abgelaufen), wird der Gepäckwagen zurückgegeben und der Flughafen verlassen. (6 P)

b) Entwerfen Sie einen endlichen Automaten der die folgende Sprache L über dem Alphabet $\{1, 2, ., E\}$ akzeptiert (Zustandsdiagramm genügt).

$$L = (\{1,2\}\{1,2\}^*.\{1,2\}^*)(E\{1,2\}\{1,2\}^* \cup \{\varepsilon\})$$

Erläuterung: L besteht aus den Gleitkommaliteralen, die mit einer Ziffer beginnen, einen Dezimalpunkt enthalten und außer diesem und einem eventuellen Exponentenzeichen nur die Ziffern 1 und 2 (z.B. 12.12 oder 12.E2, aber nicht 123.123 oder .11E12 oder 12.E-1): (6 P)

c) Gegeben sind die Zahlen 128 und 35. Ihre Summe ist 163.

1. Stellen Sie beide als 8-Bit Binärzahlen dar. (3 P)
2. Addieren Sie beide Zahlen im Binärformat und verifizieren Sie das Ergebnis indem Sie es in das Dezimalformat zurückwandeln. (2 P)
3. Stellen Sie -35 als Zweierkomplement dar und berechnen Sie $128 - 35$. (3 P)

Name: Tiger Scott MatNr.: 0815

Aufgabe 2 (Aufwand und Komplexität [15 Punkte] + 3 Bonuspunkte)

a) Berechnen Sie mit Hilfe des Mastertheorems eine asymptotische obere Schranke für eine Funktion T mit folgenden Eigenschaften (2 P)

$$T(1) = 1$$
$$T(n) = 49T(n/7) + 3n^2$$

b) Bestimmen Sie für folgende Programmfragmente wie oft *genau* die Methode `tuWas()` aufgerufen wird. K und M sind dabei beliebige natürliche Zahlen (je 2 P):

1. ```
   for (int i=0;i<K;i++)
     for (int j=i,j>=0;j--)
       tuWas();
   ```
2. ```
   for (int i=0;i<K;i++)
     for (int j=0,j<K;j++)
       tuWas();
   ```
3. ```
   for (int i=1;i<K;i++)
     for (int j=0,j<M;j++)
       tuWas();
   ```

c) Für natürliche Zahlen a, m bezeichnet $a \bmod m$ den Rest von a bei Division durch m. [Erinnerung: `%` steht für mod .]
Die Methode `modexp` liefert bei positiven Argumentwerten a, e und m den Wert $a^e \bmod m$ (also den Rest von a^e bei Division durch m).

```
public class modexp {
  // Voraussetzung: a,e,m > 0
  public static int modexp(int a, int e, int m) {
    int r = 1;
    while ( e > 0) {
      if ( (e % 2) == 1)
      r = ( r * a) % m;
      e = e/2;
      a = a * a % m;
    }
    return r;
  }
}
```

1. Berechnen Sie $10^{13} \bmod 7$. Sie können obigen Algorithmus verwenden. (4 P)
2. Wie viele Iterationen erfolgen höchstens, falls $e = 1000000$? Wie viele mindestens? Begründen Sie die Antwort. (3 P)
3. *Bonus **(3 Bonuspunkte)**: Beweisen Sie mit einer Schleifeninvariante, dass der Algorithmus korrekt ist. Sie können benutzen, dass $(x \bmod m) * (y \bmod m) = (x * y \bmod m)$ ist.
```

Name: Tiger Scott MatNr.: 0815

Aufgabe 3 (Objektorientierung & Programmierung [10 Punkte])

a) Betrachten Sie folgende Java-Klassen: (2 P)

```
class A {
  public static void foo (int x, long y) {
    System.out.println(x+y);
  }
  public static void foo (long x, int y) {
    System.out.println(x*y);
  }
}

class B {
  private byte x = 3;
  public void foo (int x, int y) {
    A.foo(this.x+y, x);
  }
  public static void main (String[] args) {
    B objB = new B();
    objB.foo(1,2);
  }
}
```

Geben Sie im obigen Beispiel alle Situationen an, in denen Methoden überladen oder überschrieben werden und Attribute überdeckt werden.

b) Erläutern Sie folgende Begriffe und geben Sie je ein Beispiel: (4 P)

- Implizite Typumwandlung
- Explizite Typumwandlung

c) Kreuzen Sie bei den folgenden Aussagen „Richtig“ oder „Falsch“ an. Pro korrekter Auswahl gibt es 1 Punkt. (4 P)

Für jede falsche Antwort wird 1 Punkt abgezogen! Jede nicht gegebene Antwort wird aber nur als 0 Punkt gewertet. Sollten Sie eine negative Punktzahl erreichen, wird diese Aufgabe mit 0 Punkt gewertet.

Aussage	**Richtig**	**Falsch**
Die `goto`-Anweisung ist in Java erlaubt.		
Es gibt Java-Programme, die nicht terminieren.		
Auf jedes Java-Objekt darf nur eine Referenzvariable zeigen.		
Auf Maschine A kompilierte Java-Programme laufen grundsätzlich nicht auf der Java-Virtual-Machine der Maschine B.		

Name: Tiger Scott MatNr.: 0815

Aufgabe 4 (Java [20 Punkte])

a) Geben Sie eine einfache Klasse `Person` an, mit der Personendaten abgespeichert werden können. Jede Person zeichnet sich durch ihren Namen aus, sowie ihr Gewicht. Folgende Operationen sollen möglich sein: (10 P)

- Erzeugen einer Instanz unter Angabe des Namens,
- Erzeugen einer Instanz unter Angabe von Name und Gewicht,
- Setzen und Abfragen des Gewichtes jeder einzelnen Instanz
- Es soll möglich sein, zu jeder Zeit abzufragen, wieviele Instanzen der Klasse bereits erzeugt wurden, und wieviele diese zusammen wiegen.

b) Für natürlicher Zahlen n, k ($n \geq k \geq 0$) gibt der Binomialkoeffizient $\binom{n}{k}$ an, wieviele Möglichkeiten es gibt, aus n Elementen k auszuwählen. Hieraus ergibt sich zwangsläufig, dass in den beiden Fällen $n = k$ und $k = 0$ das Ergebnis jeweils 1 ist. Für die anderen Fälle kann bei der Berechnung die folgende Eigenschaft des Binomialkoeffizient ausgenutzt werden:

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

Geben Sie eine rekursive Java-Methode `binom` an, die für zwei nicht-negative Integer-Argumente n und k, mit $n \geq k \geq 0$, den Binomialkoeffizienten berechnet. (2 P)
Hinweis: In der Funktion soll nicht getestet werden, ob die Voraussetzung $n \geq k \geq 0$ erfüllt ist.

c) In folgendem Java-Codefragment sind einige Fehler enthalten. Die Zeilen des Aufretens sind durch Kommentare markiert. Lokalisieren Sie die Fehler näher, indem Sie die genauen Fehlerstellen im Programmtext markieren. Geben Sie für jeden Fehler eine kurze Fehlerbeschreibung an, die eine mögliche Meldung des Java-Systems widerspiegelt. Machen Sie insbesondere eine Aussage dazu, ob der Fehler zur Übersetzungszeit oder zur Laufzeit auftritt. Schlagen Sie vor, wie der Fehler möglichst einfach im Sinne der beabsichtigten Wirkungsweise des Programmes korrigiert werden kann. (8 P)

```
class List {
  Element ListHead;

  public int zeroCnt(int x) {

  // Zaehlt die  Anzahl der Nullen vor dem ersten Auftreten von x.
  // Falls x nicht vohanden ist, wird 0 zurueckgegeben.
```

```
    int cnt;
    Element p = ListHead;
    while (p.dat != x && p != null)        // Fehlerzeile 1
      if (p.dat = 0)                       // Fehlerzeile 2
        cnt++;                             // Fehlerzeile 3(a)
      else
        p = p.next;
    return cnt;                            // Fehlerzeile 3(b)
  }

  private class Element {
    int dat;
    Element next;
  }

  public static void main(String[] args) {
    List lst  = new List;                 // Fehlerzeile 4
        :
    // Aufbau von lst. Dieser Code ist fehlerfrei.
        :
    lst.zeroCnt(3);
  }
}
```

Name: Tiger Scott MatNr.: 0815

Name: Tiger Scott MatNr.: 0815

Aufgabe 5 (Sortierverfahren [10 Punkte])

a) Beschreiben Sie stichpunktartig die Merkmale eines *Teile und Herrsche*-Algorithmus (Divide&Conquer). (3 P)

b) Betrachten Sie folgende Implementierung der Methode `mergeSort` aus dem Vorlesungsskript:

```
public void mergeSort() {
  mergeSort(0,feld.length-1);
}
private void mergeSort(int von, int bis) {
  if (von == bis) return;
  int mitte = (von + bis) / 2; // ganzz. Division
  mergeSort(von, mitte);
  mergeSort(mitte+1, bis);
  vereinige(von,mitte,bis);
}
```

Implementieren Sie dazu die Methode `vereinige`. (7 P)

Name: Tiger Scott MatNr.: 0815

Aufgabe 6 (Algorithmen und Datenstrukturen [15 Punkte])

a) Im folgenden Programmfragment wird mit Hilfe von `push` und `pop` ein anfangs leerer Stack gesteuert. `push` operiert wie üblich, aber `pop` soll etwas anders wirken als in der Vorlesung: Der Aufruf entfernt das zuletzt hinzugefügte Element und liefert es auch als Rückgabewert[1]. Notieren Sie in einer Tabelle, welchen Inhalt der Stack nach den einzelnen Anweisungen hat.

Geben Sie auch an, welche Ausgabe das Programmstück erzeugt. (5 P)

```
push(12);
push(6);
push(pop()+pop());
push(8);
push(8);
push(64);
push(pop()/pop());      // auf Auswertungsreihenfolge achten!
push(1/(pop()/pop()));  // ..
push(3);
push(pop()*pop());      // ..
push(pop()+pop());      // ..
System.out.println(pop());
```

b) Kreuzen Sie bei den folgenden Aussagen „Richtig“ oder „Falsch“ an. Pro korrekter Auswahl gibt es 1 Punkt[2]. (4 P)

Aussage	**Richtig**	**Falsch**
Binäre Suche arbeitet auf manchen `int`-Arrays inkorrekt.		
Binäre Suche hat auch im schlechtesten Fall logarithmische Laufzeit.		
Der Aufwand für Merge-Sort ist immer $O(N)$		
Ein Stack arbeitet nach dem FIFO-Prinzip		

c) Ein Algorithmus heißt

- *deterministisch*, falls er zu jeder Eingabe und jedem Zustand der Berechnung einen eindeutig bestimmten Folgezustand definiert,
- *terminierend*, falls er bei jeder Eingabe nach endlich vielen Schritten einen Endzustand erreicht (d.h. abbricht) und
- *determiniert*, falls jede Berechnung des Algorithmus bei gleichen Eingaben das gleiche Ergebnis (d.h. die gleiche Ausgabe) liefert[3].

[1] In der Vorlesung war für Letzteres der Aufruf `top()` zuständig.

[2] **Für jede falsche Antwort wird 1 Punkt abgezogen!** Jede nicht gegebene Antwort wird aber nur als 0 Punkt gewertet. Sollten Sie eine negative Punktzahl erreichen, wird diese Aufgabe mit 0 Punkt gewertet.

[3] man sollte vielleicht besser sagen: das Ergebnis des Algorithmus ist determiniert

Geben Sie für die beiden folgenden Algorithmen an, welche der drei genannten Eigenschaften sie haben und welche nicht und begründen Sie die Antwort stichpunktartig. (6 P)

Algorithmus 1:

1. Wähle eine beliebige Zahl zwischen -10 und 20.
2. Multipliziere diese Zahl mit sich selbst.
3. Gib das Ergebnis aus, falls es größer als 10 ist. Andernfalls gehe zu 2.

Algorithmus 2:

```
public float rechne (float a) {
  float p;
  if (a>10) {
    p = Math.random(); // erzeugt Zufallszahl zwischen 0 und 1
    if (a>20) {
      a = a-p;
    }
    a = a+p;
  }
  return a;
}
```

Name: Tiger Scott MatNr.: 0815

Name: Tiger Scott MatNr.: 0815

Name: Tiger Scott MatNr.: 0815

Probeklausur (überarbeitet)

Bearbeitungszeit 90 Minuten

Name	**Probeklausur**
Vorname	**.**
Matrikelnummer	**.**
Datum	**22.01.2008**

Bei der Klausur sind **keine Hilfsmittel** (Skripten, Taschenrechner, etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten Blätter für Ihre Antworten. Schreiben Sie mit blauem oder schwarzem Kugelschreiber, Füller, Bleistift, etc. ist nicht erlaubt.

1 Abstrakte Beschreibung	14	
2 Logik	10	
3 Zahlen	14	
4 Algorithmen	22	
5 Wachstum von Funktionen	16	
6 Abstrakte Datentype	10	
7 Java	04	
Summe	**90**	

Note	

1 Abstrakte Beschreibung (1/2)

1. Gegeben ist der folgenden Java-Quelltext der Methoden `fac(n)` zur Berechnung der Fakultät von n.

```
1  public int fac(int n) {
2    int f = 1;
3    while (n > 1) {
4      f = f * n;
5      n = n - 1;
6    }
7    return f;
8  }
```

   Zeichnen Sie das Flussdiagramm der Methoden `fac`. (4 Punkte)

1 Abstrakte Beschreibung (2/2)

2. Geben Sie über dem Alphabet der Ziffern von 0 bis 9 eine EBNF (erweiterte Backus-Naur-Form) Beschreibung für die Sprache aller natürlichen Zahlen an, die durch 4 teilbar sind. (4 Punkte)

3. Es sind reguläre Audrücke definiert für das leere Wort (ε), die Konkatenation (Hintereinanderstellen), die Alternative ($|$) und beliebig viele Vorkommen ($*$).

   Für reguläre Ausdrücke gilt Folgendes.

   - $*$ bindet stärker als die Konkatenation, Beispiel. $rs* = r(s*)$.
   - Die Konkatenation bindet stärker als $|$. Beispiel. $(rs|ab) = ((rs)|(ab))$.

   Der reguläre Ausdruck $r?$ soll kein oder ein Vorkommen, der reguläre Ausdruck $r+$ soll beliebig viele, aber mindestens ein, Vorkommen von r beschreiben.

   Geben Sie Definitonen von $r?$ und $r+$ mit den bereits definierten regulären Ausdrücken an. (6 Punkte)

2 Logik

Folgende Gesetze (Rechenregeln) für Operatoren auf Wahrheitswerten können als bekannt vorausgesetzt werden.

Neutralitäts-	Extremal-	Komplementär	Kommutativ-	Idempotenz-
$x \vee 0 = x$	$x \vee 1 = 1$	$x \vee \neg x = 1$	$x \vee y = y \vee x$	$x \vee x = x$
$x \wedge 1 = x$	$x \wedge 0 = 0$	$x \wedge \neg x = 0$	$x \wedge y = y \wedge x$	$x \wedge x = x$
				$\neg\neg x = x$

Assoziativ-	Distributiv-
$x \vee (y \vee z) = (x \vee y) \vee z$	$x \vee (y \wedge z) = (x \vee y) \wedge (x \vee z)$
$x \wedge (y \wedge z) = (x \wedge y) \wedge z$	$x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$

Hinweis. Der $\neg$ Operator bindet stärker als die Operatoren $\vee$, $\wedge$, d.h. $\neg\ x \vee y = (\neg x) \vee y$ und $\neg\ x \wedge y = (\neg x) \wedge y$. Die Operatoren $\vee$, $\wedge$ binden gleichstark, deshalb sind hier Klammern, die die richtige Auswertungsreihenfolge bestimmen, zwingend erforderlich (Beispiel, Distributivgesetz).

1. Zeigen Sie, mit Hilfe einer ausführlichen Wahrheitswertetafel, dass folgendes De-Morgansche-Gesetz gilt. (4 Punkte)

$$\neg(x \vee y) = \neg x \wedge \neg y$$

2. Betrachten Sie die Absorptionsgesetze.

$$x \vee (x \wedge y) = x$$
$$x \wedge (x \vee y) = x$$

   a) Zeigen Sie, durch Anwendung der passenden bekannten Gesetze, dass aus der Gültigkeit eines Absorptionsgesetze das andere folgt. Geben Sie bei jedem Schritt das angewendete Gesetz an. (3 Punkte)

   b) Zeigen Sie die Gültigkeit eines Absorptionsgesetzes durch Anwendung der passenden bekannten Gesetze. Geben Sie bei jedem Schritt das angewendete Gesetz an. (3 Punkte)

      Hinweis. Beginnen Sie, indem Sie $\wedge\neg x$ auf beide Seiten der Gleichung anwenden.

3 Zahlen

1. Rechnen Sie von einem Zahlensytem in das andere um. Der Rechenweg ist Teil der Lösung.

   a) $(377)_8 = (?)_{10}$ (2 Punkte)

   b) $(3200)_{10} = (?)_{16}$ (2 Punkte)

   c) $(1001011001)_2 = (?)_8$ (2 Punkte)

   d) $(E2A0)_{16} = (?)_2$ (2 Punkte)

2. Betrachten Sie vorzeichenbehaftete Dualzahlen der Länge n Bit, wobei für die Darstellung der negativen Zahlen das Basiskomplement verwendet wird.

   a) Erläutern Sie, wie zwei solche vorzeichenbehafteten Dualzahlen addiert werden. (4 Punkt)

   b) Rechnen Sie exemplarisch $\texttt{01110}_2 - \texttt{00111}_2$ mit $n = 5$. (2 Punkt)

4 Algorithmen (1/2)

1. Behauptung. Die Anzahl der Teile der Länge 1 oder 2, in die ein Faden der positiven ganzzahligen Länge n zerschnitten werden kann, wird durch folgende rekursive Formel bestimmt.

$$T(1) = 1$$
$$T(2) = 2$$
$$T(n) = T(n-1) + T(n-2)$$

   a) Zeigen Sie mit vollständiger Induktion die Korrektheit der Behauptung. | (6 Punkte)

   b) Geben Sie in Pseudo-/Java-Code einen Algorithmus zur iterativen Berechung der Anzahl der Teile der Länge 1 oder 2, in die ein Faden der positiven ganzzahligen Länge n zerschnitten werden kann, an. (6 Punkte)

4 Algorithmen (2/2)

2. Betrachten sie den Java-Programmtext der Funktion `f`.

```
1  public boolean f(int x) {
2      boolean b = true;
3      if ( x < 1 )
4          b = false;
5      else if ( x > 1 ) {
6          int t = 2;
7          while ( t*t <= x ) {
8              if ( b && ((x%t) == 0) )
9                  b = false;
10             t++;
11         }
12     }
13     return b;
14 }
```

a) Beschreiben Sie in maximal zwei Sätzen die Funktion der Methode `f`. (4 Punkte)

Hinweis. Welche Eigenschaft der übergebenen Zahl kann aus dem Rückgabewert der Methode gefolgert werden.

b) Definieren Sie für die Schleife Vorbedingung, Nachbedingung und Schleifeninvariante. Beweisen Sie mit deren Hilfe die Korrektheit Ihrer Aussage über die Funktion der Methode `f`. (6 Punkte)

Hinweis. Berücksichtigen Sie auch die Sonderfälle.

5 Wachstum von Funktionen

1. Sei $f(n) \in \mathcal{O}(g(n))$ und $a > 0$.

   Beweisen Sie $f(n) \in \mathcal{O}(a \cdot g(n))$. (4 Punkte)

2. Die folgende Tabelle definiert die Primitiven *Zuweisung*, *Sequenz*, *Alternative* und *Schleife* zur Zerlegung von Methoden. Berechnungen und Vergleiche ohne Zuweisung bleiben unberücksichtigt. Weiterhin ordnet die Tabelle jeder Primitive P eine Kostenmaß $T(P)$ zu.

Primitive	*Beispiel für P*	*Kosten T(P)*
Zuweisung	`a = b + c;`	1
Sequenz	P_1; P_2; ⋮ P_m;	$T(P_1) + T(P_2) + \ldots + T(P_m)$

Alternative	`if (<bedingung>)`
	  P_1;
	`else`
	  P_2;
		$\max(T(P_1), T(P_2))$

Schleife	`for(i=0; i<k; i++)` P_1;	$k \cdot T(P_1)$

   Hinweis. Die Rückgabe eines Funktionswertes (`return`) ist eine Zuweisung.

   Betrachten Sie folgende Methode `armstrong` zum Test ob die übergebene Zahl eine Armstrong-Zahl ist.

```
1  public boolean armstrong(int x) {
2      int summe = 0;
3      int zahl = x;
4      if ( zahl > 0 ) {
5          do {
6              int ziffer = zahl % 10;
7              summe = summe + ziffer*ziffer*ziffer;
8              zahl = zahl/10;
9          } while (zahl > 0);
10     }
11     return (x == summe);
12 }
```

   a) Geben sie, basierend auf obigem Kostenmaß, eine Laufzeitfunktion $T_a(n)$ für die Methode `armstrong` an. (6 Punkte)

   b) Geben Sie, mit Herleitung, die asymptotische Laufzeit der Methode `armstrong` in $\mathcal{O}$-Notation an. (6 Punkte)

6 Abstrakte Datentypen

Ihnen steht eine Klasse `CharStack`, die einen Kellerspeicher für Zeichen implementiert, mit folgenden Methoden zur Verfügung.

Methode	*Funktion*
`void create`	Konstruktor, erzeugt einen leeren Stapel
`boolean empty`	Liefert *true*, wenn der Stapel leer ist, sonst *false*.
`void push(char ch)`	Legt das Zeichen `ch` oben auf den Stapel.
`char pop()`	Liefert das oberste Zeichen zurück und entfernt es vom Stapel, ist der Stapel leer wird `null` zurückgeliefert.

Schreiben Sie in Pseudo-/Java-Code eine Methode, die, mit Hilfe der Klasse `CharStack`, einen übergebenen String darauf testet, ob er ein Palindrom ist. (10 Punkte)

Hinweis. Ein Palindrom ist ein Wort bzw. eine Zeichenkette, die vorwärts und rückwärtsgelesen, genau gleich ist. Beispiele. `LAGERREGAL` oder `p#?& &!& &?#p`.

7 Java

Betrachten Sie folgendes Java-Programmfragment.

```
1  for ( i=0; i < n; i++) {
2      if ( a[i] == x)
3          break;
4      else
5          a[i] = 0;
6  }
```

Schreiben Sie das Java-Programmfragment so um, dass anstelle der `for`– eine `do-while`–Schleife und keine `break`–Anweisung benutzt wird. (4 Punkte)

Hinweis. Auch ein leeres Feld ($n = 0$) muß berücksichtigt werden.

Klausur Informatik I
Wintersemester 2010
Prof. Dr. C. Damm
25. Februar 2011
von 10:00+x bis 11:30+x Uhr

Vorname:

Nachname:

Matrikelnummer:

Es sind **keine Hilfsmittel** (Skript, Taschenrechner etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten **Blätter** für Ihre Antworten. Schreiben Sie mit blauem/schwarzem Kugelschreiber, Füller etc.. Bleistift ist nicht erlaubt. Unlesbare Antworten werden nicht gewertet. Algorithmen sind in Java-Code zu notieren, sofern keine Alternativen genannt sind.

Viel Erfolg!

	Max. Punkte	Erreichte Punkte
Aufgabe 1 (Auswahlaufgaben)	15	
Aufgabe 2 (Allgemeine Kenntnisfragen)	15	
Aufgabe 3 (Java: eine einfache Java-Klasse)	15	
Aufgabe 4 (Ausdrücke, rekursives Ablaufprotokoll)	15	
Aufgabe 5 (Datenstrukturen und Traversierungen)	15	
Aufgabe 6 (Sortieren und Suchen)	15	
Summe	90	

Note:

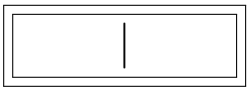
Name: MatNr.:

Aufgabe 1 (Auswahlaufgaben [15 Punkte])

In jeder Teilaufgabe ist nur eine Antwort richtig. Kreuzen (**X**) Sie an.
Achtung: Falsche (oder Mehrfach-)Antworten zu Teilaufgaben werden in der Aufgabe verrechnet: Bsp.: 5× richtig, 6× falsch/mehrfach angekreuzt, 4× ohne Antwort (+5P-6P+0P) ergibt 0P für diese Aufgabe.

- `class A {void tuWas(){..}..} class B extends A {void tuWas(){..}..}`.
  Was ist richtig?
  1. `tuWas` wurde erweitert.
  2. `tuWas` wurde überladen.
  3. `tuWas` wurde überschrieben.
- Eine Klasse kann mehrere Konstruktordefinitionen enthalten, ...
  1. ... wenn sich ihre Parametertyplisten unterscheiden.
  2. ... wenn sich ihre Namen unterscheiden.
  3. ... falls sie `static` definiert sind.
- `!a || !b` ist gleichbedeutend mit
  1. weder `a` noch `b`
  2. aus `a` folgt `b`
  3. aus `a` folgt ¬`b`
- Rekursion ist
  1. das Rückverfolgen von Methodenaufrufen
  2. das Aufrufen einer Methode in deren eigenem Methodenrumpf
  3. das Rückführen einer Problemlösung auf zwei kleinere Probleme
- Ein Baum ist ein Graph
  1. der eine Wurzel hat, aber keine Schlingen
  2. der zusammenhängend ist und keine Kreise hat
  3. in dem alle Wege einfach sind
- Datentypen legen fest:
  1. Daten und Typ von Methoden
  2. Wertemenge, Operationen und Abbildung Wert ↦ „Bitmuster“
  3. die Liste der zuweisungskompatiblen Typen zu einem Datum
- Ein Bezeichner ist ...
  1. ... eine Variable,
  2. ... ein für einen Programmbestandteil verwendbarer Name,
  3. ... ein Softwarewerkzeug, das Programmteile markiert.

- Dynamisches Binden bedeutet, dass ...
  1. ... die zugrundeliegende Datenstruktur bei Bedarf wachsen oder schrumpfen kann.
  2. ... der Compiler je nach Problemstellung schnell und flexibel reagiert.
  3. ... der auszuführende Code erst zur Laufzeit zugeordnet wird.
- Traversieren eines Baumes bedeutet ...
  1. ... zuerst die Wurzel, dann die Teilbäume zu bearbeiten.
  2. ... die Knoten eines Baumes systematisch zu besuchen.
  3. ... die Blätter des Baumes aufzuzählen.
- Ein Informationssystem besteht aus ...
  1. ... einer Menge von Daten, die nach einem Schlüssel durchsucht werden können.
  2. ... Informationen, Nachrichten und einer Decodierungsvorschrift.
  3. ... Syntax, Semantik und Schnittstelle dafür.
- Der Gültigkeitsbereich eines Bezeichners ist ...
  1. ... zur Compilezeit global.
  2. ... der Aktivierungssatz, in dem seine Definition gilt.
  3. ... statisch, d.h. aus dem Quelltext ablesbar.
- Ein Stack ...
  1. ... ist eine einfach verkettete Liste.
  2. ... arbeitet nach dem Prinzip last in, first out.
  3. ... hat im schlechtesten Fall logarithmische Zugriffszeit.
- Objekte ...
  1. ... sind die Elemente einer Klasse.
  2. ... sind die kleinsten Bestandteile eines Programms.
  3. ... haben Zustand und Verhalten.
- Quicksort ...
  1. ... benötigt auf Feldern der Länge n höchstens $\mathcal{O}(n \log n)$ Schritte.
  2. ... benötigt auf Feldern der Länge n höchstens $\mathcal{O}(n^2)$ Schritte.
  3. ... wird durch dynamisches Programmieren implementiert.
- Laufzeitfehler sind Fehler ...
  1. ... die zur Compilezeit nicht bemerkt werden.
  2. ... die überlange Laufzeiten verursachen.
  3. ... die im laufenden Betrieb übersehen werden.



Name: MatNr.:

Aufgabe 2 (Allgemeine Kenntnisfragen [15 Punkte])
3P pro Aufgabenteil. Antworten Sie kurz und prägnant.

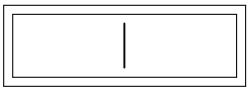
1. Seien f, g Funktionen, die natürliche Zahlen auf natürliche abbilden. Was bedeutet $f(n) \in \mathcal{O}(g(n))$?

2. Objekte der Klasse `B` sollen das ganzzahlige Instanzattribut `z` mit Initialwert 1 haben. Es soll in jeder Klasse gelesen und geändert werden können. Wie muss die Definition von `z` lauten?

3. Nennen Sie drei Programmierparadigmen.

4. Nennen Sie drei Arten von Fehlern, die beim Programmieren auftreten.

5. Was versteht man unter einer funktionalen Methode? Welche Anweisung **muss** jede funktionale Java-Methode haben? Welche sollte sie nicht haben?



Name: MatNr.:

Aufgabe 3 (Java: eine einfache Java-Klasse [15 Punkte])
1. (11P) Geben Sie eine einfache Klasse `Auto` an, zur Verwaltung von Autos. Jedes Auto hat einen Typ, z.B. "VW Golf" oder "Mercedes 600" (benutzen Sie hierfür ein Attribut `typ`), und einen Preis (Attribut `preis`). Außerdem werden noch `anzahl` und `gesamtpreis` benötigt (*bitte verwenden Sie diese Bezeichnungen*). Folgenden Operationen sollen möglich sein:

- Erzeugen einer Instanz dieser Klasse unter Angabe nur des Typs,
- Erzeugen einer Instanz dieser Klasse unter Angabe von Typ und Preis,
- Setzen und Abfragen des Preises jeder einzelnen Instanz.
- Es soll jederzeit möglich sein, die Anzahl erzeugter Instanzen der Klasse und ihren Gesamtpreis (Summe der Preise) abzufragen.

Bezeichner `typ`, `preis`, `anzahl` und `gesamtpreis` benutzen!!

2. (4P) Stellen Sie sich vor, die Klasse `Auto` soll getestet werden. Schreiben Sie dafür 4 Anweisungen (keine vollständige Methode oder Klasse!) die genau Folgendes leisten:
1. `Auto`-Objekt `a1` mit Typ- und Preisangabe erzeugen,
2. `Auto`-Objekt `a2` nur mit Typ-Angabe erzeugen,
3. nachträglich für `a2` den Preis setzen,
4. den Gesamtpreis beider Autos als Text ausgeben.

Nocheinmal: Nur die 4 Anweisungen untereinander schreiben, sonst nichts!

Name: MatNr.:

Aufgabe 4 (Ausdrücke, rekursives Ablaufprotokoll [15 Punkte])

1. (5P) Wie lautet der folgende Ausdruck in Postorder-Notation?

```
(a+b)*c+d/(e-f)
```

2. (2P) Welchen Wert liefert der folgende Postorder-Ausdruck, wenn die `int`-Variablen darin alle den Wert 2 haben?

```
abc/+d*ef-+
```

3. (4P) In einem Java-Quelltext befinden sich folgende Definitionen: `int i = 1, j = 2, k = 3; boolean b = false, c = true, d = false;` Welche der folgenden Zuweisungen sind erlaubt, welche nicht? Geben Sie bei den erlaubten den zugewiesenen Wert an, bei den unerlaubten die Fehlerstelle (Zeichen nennen oder im Text markieren):

```
i = 2 * i + k;          // Wert:                    oder
                        // Fehlerstelle:

i = (i > j && i > k);   // Wert:                    oder
                        // Fehlerstelle:

b = (i > j && i > k);   // Wert:                    oder
                        // Fehlerstelle:

c = ( b || d + 2);      // Wert:                    oder
                        // Fehlerstelle:
```

4. (4P) Betrachten Sie folgende Java-Methode.

```
static int strange( int k) {
    if (k <= 1)
        return k;
    return k - strange(k - 2);
}
```

Notieren Sie das rekursive Ablaufprotokoll des Aufrufs `strange(7);` und geben Sie jedes Zwischenergebnis und den berechneten Wert an.

Name: MatNr.:

Aufgabe 5 (Datenstrukturen und Traversierungen [15 Punkte])

a) (3P) Auf einen anfangs leeren Stack werden `push/pop`-Operationen angewendet. Tragen Sie den Zustand des Stacks nach den angegebenen Aufrufen ein (erste Zeile ist komplett vorgegeben). Bitte schreiben Sie deutlich!

`push(1);push(2);push(8);`	1	2	8									
`push(5);push(3);pop();`												
`push(1);pop();pop();`												
`pop();push(7);push(1);`												

b) (6P) Unten sehen Sie den Quelltext der Java-Klasse `IntNode` und einen Teil der Klasse `IntListe`, die eine Liste von `int`-Werten als doppelt verkettete Liste mit Ankerknoten realisiert. Ergänzen Sie dort den Rumpf der Methode `void prepend(int x)`, die den Wert `x` am Anfang in die Liste einspeichert (d.h. der neue Knoten soll *Nachfolger* des Ankerknotens werden). Dafür genügen 3 Anweisungen.

```
class IntNode{
    int data;
    IntNode next, prev;
    IntNode (int x, IntNode next, IntNode prev) {
        data = x; this.prev = prev; this.next = next;}
}
```

```
class IntListe {
    IntNode list;
    IntListe(){
        list = new IntNode(0,null,null); // Ankerknoten
        list.next = list.prev = list;
    }
    void prepend(int x) // Knoten am Anfang einfuegen
    {



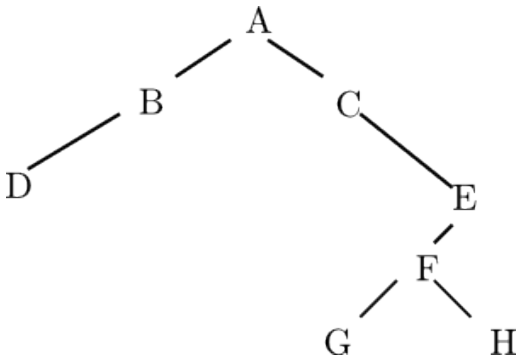






    }
}
```

c) (6P) Geben Sie die Reihenfolge an, in der die Knoten des untenstehenden Baumes mit Wurzel A besucht werden, bei den genannten Traversierungsreihenfolgen:



Preorder-Traversierung:								
Inorder-Traversierung:								
Postorder-Traversierung:								

Name: MatNr.:

Aufgabe 6 (Sortieren und Suchen [15 Punkte])

1. (3P) Die Java-Klasse `DiesUndDas` soll eine Klassenmethode `maxPos` enthalten, die in jeder beliebigen Klasse genutzt werden kann. Angewendet auf ein Feld von `int`-Werten und einen Grenzwert `g` soll sie den Index des Maximums aller Werte im Feld liefern, mit Index zwischen 0 und g.
Wie lautet der Kopf der Methode?

2. (3P) In einer Methode der Java-Klasse `Gugel` (im gleichen Verzeichnis, aber keine Erweiterungsklasse von `DiesUndDas`) soll die Methode aus Aufgabenteil 1. auf das `int`-Feld `bla` und Index `hier` angewendet werden, das Ergebnis in `mf` abgespeichert werden. Schreiben Sie diese Codezeile.

3. (6P) Wie kann man `maxBis` nutzen, um ein Feld von `int`-Werten zu sortieren? Beschreiben Sie kurz und prägnant die Idee (z.B. in Pseudocode, Java oder strukturierter Prosa ...).
Nehmen wir an, dass `maxPos` $\mathcal{O}(n-g)$ Schritte braucht bei Feldlänge n und Grenzparameter g. Geben Sie die Laufzeit für Ihr Verfahren in $\mathcal{O}$-Notation an (mit Begründung).

4. (3P) Nachfolgend der Rumpf einer Methode für die binäre Suche nach dem Eintrag `q` im `int`-Feld `f`:

```
int a = 0, e = f.length-1, m;
while ( a <= e) {
    m = (a+e)/2;
    if ( f[m] == q)
        return m;
    if (q < f[m])
        e = m-1;
    else
        a = m+1;
}
return -1;
```

Die Methode wird benutzt, um in der folgenden Liste nach dem Wert 12 zu suchen. Mit welchen Werten wird 12 dabei verglichen (in der Reihenfolge des Auftretens angeben)?:
3, 7, 12, 19, 22, 101, 142, 900

Name: MatNr.:

Probeklausur Informatik I
Wintersemester 2006/2007
Prof. Dr. Carsten Damm
08 und 12 Februar 2006
Bearbeitungszeit: 90 Minuten

Vorname:

Nachname:

Matrikelnummer:

Bei der Klausur sind **keine Hilfsmittel** (Skripten, Taschenrechner etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten **Blätter** für Ihre Antworten. Schreiben Sie auf **jedes Blatt** oben Ihren Namen und Ihre Matrikelnummer. Schreiben Sie mit blauem/schwarzem Kugelschreiber, Füller etc.; Bleistift ist nicht erlaubt.

Zum **Bestehen** der Klausur sind **45** Punkte hinreichend.

☐ meine Note soll mit Matrikelnummer so bald wie möglich auf der Vorlesungs-Webseite veröffentlicht werden.

☐ meine Note soll nicht veröffentlicht werden; ich erfahre sie dann aus Munopag/Wopag (bzw. beim Prüfungsamt der Informatik).

	Max. P.	Schätzung
Aufgabe 1 (Binäre Zahlendarstellung)	11	8
Aufgabe 2 (Grammatik)	14	9
Aufgabe 3 (Divide and Conquer / Aufwand und Komplexität)	20	13
Aufgabe 4 (Java: eine einfache Klasse)	14	7
Aufgabe 5 (Sortierverfahren, Schleifeninvariante und Java)	21	14
Aufgabe 6 (Fibonacci-Funktion und Algorithmen-Entwurf)	10	6
Summe	90	57

Note:

Aufgabe 1 (Binäre Zahlendarstellung [11 Punkte])
Gegeben sind die Zahlen „122“ und „89“

a) Stellen Sie beide als Binärzahlen sowie als Hexadezimalen dar. (3 P)

Lösung

122:	01111010	= Hex 7A
89:	01011001	= Hex 59

b) Addieren Sie beide Zahlen im Binärformat und verifieren Sie das Ergebnis indem Sie es in das Dezimalformat zurückwandeln. (4 P)

Lösung

122:	01111010		
89:	01011001		
Add: 211:	11010011	$= 128 + 64 + 16 + 2 + 1$	

c) Stellen Sie „-89“ als Zweierkomplement dar und berechnen Sie „$122 - 89$“. (4 P)

Lösung

Zweierkomplement von 89: 89 ist		01011001	
	Einerkomplement:	10100110	
	$+1$:	10100111	ist das Zweierkomplement
Addieren:	01111010		
$+$	10100111		
Summe:	00100001	$= 122 - 89 = 33$	

Aufgabe 2 (Grammatik [14 Punkte])

a) Geben Sie eine Grammatik in EBNF (*erweiterte Backus-Naur-Form*) für die Menge aller Palindrome aus dem Alphabet $\{a, b\}$ an (Palindrome sind Ausdrücke, die sowohl vorwärts als auch rückwärts dasselbe Wort ergeben, z.B. *abba*, *bbabb*). (5 P)

Lösung

Die Grammatik wird definiert über:

$G = \{N, T, P, S\}$ oder $G = \{V, \Sigma, P, S\}$
wobei
V = Variablenmenge (N = Nichtterminalsymbol-Menge) = $\{S\}$
Σ = Terminalalphabet (T = Terminalsymbol-Menge) = $\{\epsilon, a, b\}$
P = Produktionsregelnmenge
S = Startsymbol

Für P gilt:

$S := a|b|aSa|bSb|\epsilon$ (EBNF-Darstellung)

b) Leiten Sie mit den Produktionsregeln Ihrer Grammatik das Wort *aababaa* aus dem Startsymbol ab. (4 P)

Lösung

$S \rightarrow aSa \rightarrow\rightarrow aaSaa \rightarrow aabSbaa \rightarrow aababaa$

c) Geben Sie einen deterministischen endlichen Automaten an der alle Palindrome aus dem Alphabet $\{a, b\}$ mit einer Länge von mindestens 1 bis maximal 4 Zeichen erkennt. (5 P)

Lösung

Siehe Automat auf Beiblatt 1

Aufgabe 3 (Divide and Conquer / Aufwand und Komplexität [20 Punkte])
Gegeben sei ein Divide-and-Conquer-Algorithmus, der auf eine Folge von n Zahlen wie folgt angewendet wird:

- falls $n = 1$, gib die Zahl zurück
- sonst:
  1. Löse dasselbe Problem für die aus den ersten 1 ... $\frac{n}{2}$ Zahlen bestehende Teilfolge, erhalte einen Wert x_1,
  2. Löse dasselbe Problem für die aus den restlichen $(\frac{n}{2} + 1)$ ... n Zahlen bestehende Teilfolge, erhalte einen Wert x_2,
  3. Löse dasselbe Problem für die aus den mittleren $(\frac{n}{4} + 1)$ ... $\frac{3}{4}n$ Zahlen bestehende Teilfolge, erhalte einen Wert x_3,
  4. Löse dasselbe Problem für die aus den $(\frac{3}{4}n + 1)$ ...n sowie 1 ... $\frac{1}{4}n$ Zahlen bestehende Teilfolge, erhalte einen Wert x_4,
  5. mache eine Nachverarbeitung über alle n Zahlen, die n^2 Schritte benötigt, erhalte einen Wert y,
  6. Gib $max(x_1, x_2, x_3, x_4, y)$ zurück.

Tipp: Schätzen Sie die Kostenfunktion sinnvoll nach oben hin ab, um Theoreme aus der Vorlesung benutzen zu können!

a) Geben Sie die Rekurrenzgleichung für $T(n)$ an. (6 P)

Lösung

$T(n) = 4T(\frac{n}{2}) + n^2 + 5$ für $n > 1$
$T(1) = 1$

Anmerkung: einige Lösungen zielten auf $T(n) = 3T(\frac{n}{2}) + 2T(\frac{n}{4}) + n^2 + 5$.
In Punkt 4) wird explizit gesagt, dass es sich um *eine* Teilfolge handelt, die aus den beiden Fragmenten zusammengesetzt ist.

b) Welche asymptotische Laufzeit hat der Algorithmus (mit Begründung). (5 P)

Lösung

Die Konstante „+5“ kann man direkt in $T(n) = 4T(\frac{n}{2}) + O(n^2)$ mit einbeziehen.
Master-Theorem mit $a = 4$, $b = 2$ und $k = 2$, also $a = b^k$ und damit Lösung $O(n^k \log n)$, also in diesem Fall $O(n^2 \log n)$.

c) Jemand optimiert den Algorithmus nun, und stellt fest, dass Schritt (4) weggelassen werden kann, und Schritt (5) in $O(n \log n)$ erledigt werden kann. Geben Sie wieder die Rekurrenzgleichung an. (4 P)

Lösung

$T_2(n) = 3T_2(\frac{n}{2}) + O(n \log n) + 4$ für $n > 1$
$T_2(1) = 1$

d) Geben Sie eine obere Schranke für die asymptotische Laufzeit des optimierten Algorithmus an? (5 P)

Lösung

Wieder fällt die Konstante „+4“ in das $O(n \log n)$

Zuerst einmal: da es eine Optimierung des ersten Algorithmus ist, gilt auf jeden Fall $T_2(n) = O(n^2 \log n)$.

Master-Theorem mit $a = 3$, $b = 2$, k kann als beliebige Konstante $k > 1$ angenommen werden ($\log n = O(n^k)$ für alle $k > 1$). Also $a > b^k$ und damit $T(n) = O(n^{log_b(a)}) = O(n^{log_2(3)}) \approx O(n^{1.58})$.

Aufgabe 4 (Java: eine einfache Klasse [14 Punkte])
Geben Sie eine einfache Klasse `Person` an, mit der Personen abgespeichert werden können. Jede Person zeichnet sich durch ihren Namen aus, sowie ihr Gewicht. Die folgenden Operationen sollen möglich sein:

- Erzeugen einer Instanz dieser Klasse unter Angabe des Namens,
- Erzeugen einer Instanz dieser Klasse unter Angabe des Namens und des Gewichtes,
- Setzen und Abfragen des Gewichtes jeder einzelnen Instanz
- Es soll möglich sein, zu jeder Zeit abzufragen, wieviele Instanzen der Klasse bereits erzeugt wurden, und wieviele diese zusammen wiegen.

Lösung

```
public class Person {
  String name;
  long gewicht;      // oder int oder sonstwas
  static int anzahl;
  static long gesamtgewicht;

  public Person(String t){
    name = t;
    anzahl++;
  }

  public Person(String t, long p) {
    name = t;
    anzahl++;
    gewicht = p;
    gesamtgewicht = gesamtgewicht + p;
  }

  public void setGewicht(long p) {
    gesamtgewicht = gesamtgewicht - gewicht + p;
    gewicht = p;
  }

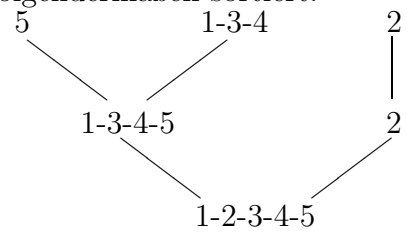
  public void getGewicht(long p) {
    return gewicht;
  }

  public static int wieviele() {
    return anzahl;
  }
```

```
  public static long gesamtgewicht() {
    return gesamtgewicht;
  }
}
```

Aufgabe 5 (Sortierverfahren, Schleifeninvariante und Java [21 Punkte])

a) Betrachten Sie folgende Variante von MergeSort:
Anstatt eine Zahlenfolge auf ihre einzelnen Elemente herunter zu brechen und diese dann mittels Reißverschlussverfahren zusammenzufügen, werden bereits sortierte Teilfolgen gesucht und dann vereint. Z. B. Wird 5,1,3,4,2 folgendermaßen sortiert:



Angenommen, eine Funktion `static public void merge (int[] a, int l, int m, int r)` , die auf dem Feld `a[]` die sortierten Bereiche von l bis m und $m+1$ bis r zusammenfügt, sei bereits implementiert. Implementieren Sie diese MergeSort-Variante in Java. (6 P)

(Hinweis: Es ist weder eine Main-Funktion, noch eine Klasse notwendig zu schreiben, beschränken Sie sich lediglich auf die Funktion selbst. Die Funktion muss nicht effektiv sein, sondern sollte möglichst einfach geschrieben werden.)

Lösung

```
public static void sort (int[] a){
  int l,m,r;

  //1. Schritt: linke Folge suchen
  l=0;
  m=l+1;
  while (m!=a.length&&a[m-1]<=a[m]) m++;

  if (m!=a.length){
    //2. Schritt: rechte Folge bestimmen
    r=m+1;
    while (r!=a.length&&a[r-1]<=a[r]) r++;

    //3. Schritt: merge mit korrekten Parametern
    merge(a,l,m-1,r-1);
    sort(a);
  }
}
```

b) Gegeben ist das folgende Programm:

```
public class Mystery {
  public static int compute(int n) {
```

```
    int i = 1;
    int sq = 1;

    while (i<n)
    {
      sq = sq + 2*i + 1;
      i = i + 1;
    }
    return sq;
  }
}
```

1. Was berechnet der Aufruf `Mystery.compute(n)` ? (3 P)
   Hinweis: probieren Sie es für n = 1, ... , 5 aus

   Lösung

   Mystery berechnet n^2 : $1^2 = 1$, $(i+1)^2 = i^2 + 2i + 1$ (Binomische Formel)

2. Geben Sie für die Schleife die Vorbedingung, Schleifeninvariante und Nachbedingung an - beweisen Sie damit induktiv Ihre Aussage in Aufgabenteil 1. (6 P)

   Lösung

   - Vorbedingung und Schleifeninvariante: $sq = i^2$
   - Schleifenbedingung: $i < n$
   - Nachbedingung: $sq = n^2$
   - Beweis, dass Vorbedingung von der Initialisierung garantiert wird: $1 = 1^2$
   - Beweis der Schleifeninvariante (induktiv):
     Am Anfang des Schleifenkörpers gilt (Aussage für i): $sq = i^2$.
     Dann wird sq um $2i + 1$ erhöht, und dann wird noch i um 1 erhöht.
     Damit gilt jetzt wieder $sq = i^2 + 2i + 1 = (i+1)^2$
   - Beweis der Nachbedingung (= Schleifeninvariante + negierte Schleifenbedingung):
     $sq = i^2$ und $i = n$, daraus folgt $sq = n^2$

c) Kreuzen Sie bei den folgenden Aussagen „Richtig“ oder „Falsch“ an. Pro korrekter Auswahl gibt es 1P. (6 P)

Für jede falsche Antwort wird ein Punkt abgezogen! Jede nicht gegebene Antwort wird aber nur als 0P gewertet. Sollten Sie eine negative Punktzahl erreichen, wird diese Aufgabe mit 0P gewertet.

Geben Sie keine zusätzlichen Erläuterungen oder Einschränkungen zu Ihrer Auswahl an!

Aussage	**Richtig**	**Falsch**
Java		
Eine Klasse, die keine Methoden besitzt, ist ein Interface.		
Eine Klasse darf keine 2 Methoden mit gleicher Signatur enthalten.		
`class A extends B implements C,D {...}` ist korrekter Java-Code		
Sortieren und Suchen		
Mit n Vergleichen kann man prüfen, ob n+1 Zahlen geordnet sind.		
Binärsuche ist ein Divide & Conquer-Verfahren.		
Eine Queue arbeitet nach dem LILO-Prinzip. (LILO = *Last In Last Out*)		

Lösung

Aussage	**Richtig**	**Falsch**
Java		
Eine Klasse, die keine Methoden besitzt, ist ein Interface.		*X*
Eine Klasse darf keine 2 Methoden mit gleicher Signatur enthalten.		*X*
`class A extends B implements C,D {...}` ist korrekter Java-Code	*X*	
Sortieren und Suchen		
Mit n Vergleichen kann man prüfen, ob n+1 Zahlen geordnet sind.	*X*	
Binärsuche ist ein Divide & Conquer-Verfahren.	*X*	
Eine Queue arbeitet nach dem LILO-Prinzip. (LILO = *Last In Last Out*)	*X*	

Aufgabe 6 (Fibonacci-Funktion und Algorithmen-Entwurf [10 Punkte])
Gegeben sei die Fibonacci-Funktion mit

- $fib(0) := fib(1) := 1$
- $fib(n+1) := fib(n) + fib(n-1)\ \forall n \in \mathrm{N},\ n > 1$

a) Berechnen Sie $fib(5)$. (2 P)

Lösung

$fib(5) = fib(4) + fib(3) = fib(3) + fib(2) + fib(2) + fib(1) = fib(2) + fib(1) + fib(1) + fib(0) + fib(1) + fib(0) + fib(1) = fib(1) + fib(0) + fib(1) + fib(1) + fib(0) + fib(1) + fib(0) + fib(1) = 8$

b) Wie lautet der größte gemeinsame Teiler (ggT) von $fib(4)$ und $fib(5)$? (3 P)

Lösung

$fib(5) = 8, fib(4) = 5 \Rightarrow ggT(5,4) = 1 \Rightarrow fib(4)$ und $fib(5)$ sind teilerfremd.

Gegeben ist eine nur aus „+“ und „−“ bestehende endliche Zeichenfolge, z.B. „− − + + + − − + + + + + − + − + − + − + − − +“
(Sie können annehmen, dass die Zeichenfolge als Array z[0]...z[k] gegeben ist).

c) Geben Sie einen Algorithmus an, der ausgibt, wie lang die längste Teilsequenz ist, die nur aus „+“-Zeichen besteht (im Beispiel 5). (5 P)
Sie können dies entweder in Java, als Flussdiagramm, oder als Pseudocode tun.

Lösung

Initialisierung: $i = 0,\ n = 0,\ max = 0$
Schritt für Schritt nach rechts durchlaufen.

- Falls die Zahl an Stelle i ein „+“ ist, $n = n + 1$,
- Falls die Zahl an Stelle i ein „−“ ist,
  falls $n > max$, setze $max = n$
  setze $n = 0$.

P R O B E K L A U S U R Info I

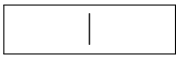
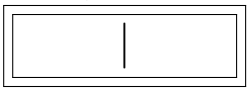
Wintersemester 2008/09
Prof. Dr. Carsten Damm
20. Januar 2009
von 16:00+x bis 17:30+x Uhr
(Bearbeitungszeit: 90 Minuten)

Vorname:

Nachname:

Matrikelnummer:

Es sind **keine Hilfsmittel** (Skript, Taschenrechner etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten **Blätter** für Ihre Antworten. Schreiben Sie mit blauem/schwarzem Kugelschreiber, Füller etc..

Bleistift ist nicht erlaubt. Korrekturfelder  

nicht beschriften!

Zum **Bestehen** der Klausur sind **45** Punkte ausreichend.

Viel Erfolg!

	Max. Punkte	Schätzung für "4"
Aufgabe 1 (Auswahlaufgabe)	5+0	5
Aufgabe 2 (Java: eine einfache Klasse)	11+0	6
Aufgabe 3 (Datenstrukturen und Graphen)	13+0	6
Aufgabe 4 (Zahlendarstellung)	14+0	6
Aufgabe 5 (Algorithmen-Entwurf und Analyse)	16+2	7
Aufgabe 6 (Sortieren und Schleifeninvariante)	15+0	5
Aufgabe 7 (Rekursion und Iteration)	16+2	8
Summe	90+4	43

M U S T E R K L A U S U R Info I

Aufgabe 1 (Auswahlaufgabe [5+0 Punkte])

Nur jeweils eine Antwort ist richtig. Falsche Antworten werden innerhalb dieser Aufgabe verrechnet (Bsp.: a) und b) richtig, aber c), d) und e) falsch ergibt 0P).

Kreisen Sie die richtige Nummer ein. Mehr Kreise (oder unleserlich) ergeben 0P für die Teilaufgabe.

a) Überladen einer Methode

1. ist die Zuordnung mehrerer Semantiken zum gleichen Methodennamen (Auswahl erfolgt durch Aufrufkontext)
2. ist die Programmierung unübersichtlich vieler verschiedener Funktionalitäten innerhalb einer Methode
3. geschieht zum Beispiel bei der Neudefinition der Methode `Object.toString()` in einer abgeleiteten Klasse

Lösung 1.

b) imperative Programmierung ist gekennzeichnet durch

1. Kommandozeilensteuerung
2. Befehlsfolgen zur Zustandsänderung
3. Zerlegen von Problemen in Teilprobleme

Lösung 2.

c) Rekursion ist

1. das Rückverfolgen von Methodenaufrufen
2. das Aufrufen einer Methode in deren eigenem Methodenrumpf
3. das Rückführen einer Problemlösung auf zwei kleinere Probleme

Lösung 2.

d) Aus welchen drei Kontrollstrukturen lassen sich alle Algorithmen aufbauen?

1. Eingabe, Verarbeitung und Ausgabe
2. for-Schleife, while-Schleife und do-while-Schleife
3. Komposition, Alternative und while-Schleife

Lösung 3.

e) Algorithmen sind stets

1. in endlicher Zeit abarbeitbar
2. endlich beschreibbar
3. in asymptotischer Form analysierbar

Lösung 2.

Aufgabe 2 (Java: eine einfache Klasse [11+0 Punkte])
Geben Sie eine einfache Klasse `Modul` an, die man zur Verwaltung der Module in einem Bachelorstudiengang benutzen kann. Jedes Modul zeichnet sich durch seinen Namen, z.B. "Info I" oder "C-Kurs", sowie seine Leistungspunktezahl aus. Außerdem will man jederzeit die Anzahl angebotener Module ermitteln können, sowie die Gesamtanzahl der Leistungspunkte in diesen. *Benutzen Sie in Ihrem Quelltext die Bezeichner* ***`name`****,* ***`punkte`****,* ***`anzahl`*** *und* ***`gesamtPunkte`*** *für diese Dinge.* Die folgenden Operationen sollen möglich sein:

- Erzeugen einer Instanz dieser Klasse unter Angabe des Namens,
- Erzeugen einer Instanz dieser Klasse unter Angabe des Namens und der Leistungspunktezahl,
- Setzen und Abfragen der Punktzahl jeder einzelnen Instanz
- Es soll möglich sein, zu jeder Zeit abzufragen, wieviele Instanzen der Klasse bereits erzeugt wurden, und wieviele Leistungspunkte diese zusammen haben.

Lösung

```
public class Modul {                          // Punktzusammensetzung:
  String name;                                     // 0.5 Punkte
  long punkte;  // oder int oder sonstwas          // 0.5 Punkte
  static int anzahl = 0;                           // attr: 0.5 P,
                                                   // static: 0.5 P
  static long gesamtpunkte;                        // attr: 0.5 P,
                                                     static: 0.5 P
  public Modul(String t) { name = t; anzahl++; } // 1 Punkt
  public Modul(String t,long p) {                  // 1 Punkt
    name = t; anzahl++;
    punkte = p; gesamtpunkte = gesamtpunkte+p; }
  public void setPunkte(long p) {                  // 2 Punkte
    gesamtpunkte = gesamtpunkte - punkte + p;// vergessen/falsch=>nur 1P
    punkte = p; }
  public long getPunkte(long p) {                  // 1 Punkt
    return punkte; }
  public static int wieviele() { return anzahl;}
                                          // Meth: 1 P, static 0.5P
  public static long insgesamt() { return gesamtpunkte;}
                                          // Meth: 1P, static: 0.5P.
}
```

Aufgabe 3 (Datenstrukturen und Graphen [13+0 Punkte])

a) (3P) Auf einen anfangs leeren Stack werden `push`/`pop`-Operationen angewendet. Tragen Sie den Zustand des Stacks nach den angegebenen Aufrufen ein (erste Zeile ist komplett vorgegeben). Bitte schreiben Sie deutlich!

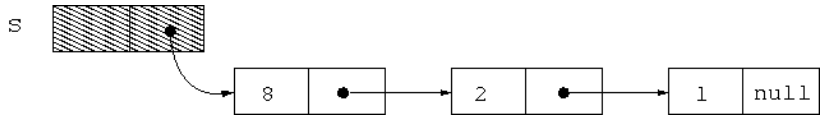
`push(1);push(2);push(8);`	1	2	8									
`push(5);push(3);pop();`												
`push(1);pop();pop();`												
`pop();push(7);push(1);`												

Lösung

`push(1);push(2);push(1);`	1	2	8									
`push(5);push(3);pop();`	1	2	8	5								
`push(1);pop();pop();`	1	2	8									
`pop();push(7);push(1);`	1	2	7	1								

Jede Abweichung gibt einen Punkt Abzug.

b) (6P)



Dies skizziert die Realisierung eines Stacks durch eine verkettete Liste mit Startknoten `s` (Zustand wie erste Tabellenzeile oben). Mögliche Befehle:

- *n* `= neu(`*i*`)` — Erzeugen von neuem Knoten namens *n* mit Inhalt *i*
- *x* `= inhalt(`*m*`)` — kopiere in *m* gespeicherte Zahl nach *x*
- `nachfolger(`*o*`) = `*p* — der Nachfolger von *o* ist nun *p*
  (oder auch: *k* `= nachfolger(`*z*`)` — Nachfolger von *z* in *k* speichern)

Geben Sie je eine Befehlsfolge an, um (1) bzw. (2) zu realisieren:
(1) `x = top(); pop();` (2) `push(5);`
Auf Reihenfolge achten! Auf Variablendeklarationen, Leerheitstest o.ä. brauchen Sie nicht achten!

Lösung

(1) `x = inhalt(nachfolger(s)); nachfolger(nachfolger(s)) = nachfolger(s);`

(2) `n = neu(5); nachfolger(n) = nachfolger(s); nachfolger(s) = n;`

Syntax nicht wichtig, solange das Gemeinte erkennbar ist. Je 3 Punkte. Für gut erkennbare Skizze (mit Reihenfolgekennzeichnung) können je nach Korrektheit auch Punkte gegeben werden — aber nicht die volle Punktzahl.

c) (4P) Welche Tiefe hat ein binärer Baum mit n Blättern mindestens? Geben Sie den Wert möglichst genau an und begründen Sie Ihre Antwort.

Lösung Ein binärer Baum hat in Tiefe 0 die Wurzel, in Tiefe 1 höchstens 2 Knoten, in Tiefe 2 höchstens 4 Knoten, ..., in Tiefe d höchstens 2^d Knoten. Die Tiefe eines solchen Baumes ist demnach die kleinste Zahl d mit $n \leq 2^d$. Anders ausgedrückt: $d = \lceil \log_2 n \rceil$.

$\log$ anstelle $\log_2$ anstreichen — kein Abzug

$\lfloor\ \rfloor$ anstelle $\lceil\ \rceil$, oder ganz ohne Klammer — 1 Punkt Abzug

$\log$, aber keine Begründung — 2 Punkte Abzug

Kein $\log$ — 2–3 Punkte Abzug (je nach Qualität der Begründung), sofern die Antwort nicht definitiv fehlerhaft ist (aber eben nicht genau), sonst 4 Punkte Abzug

Aufgabe 4 (Zahlendarstellung [14+0 Punkte])

a) (3P) Der hypothetische Datentyp `gint` stellt ganze Zahlen bekanntlich mit dem Zweierkomplement der Länge 4 dar (4-Bit-Zweierkomplement). Ergänzen Sie die folgenden Konstantendefinitionen der Wrapper-Klasse `Ginteger`:

```
public static final gint MAX_VALUE = ........;

public static final gint MIN_VALUE = ........;
```

Lösung 7 bzw. -8

(jeweils 1.5 P)

Richtiges Bitmuster, aber keine Zahl: 0.5P

b) (5P) Wir betrachten dazu noch den Datentyp `glint`: ganze Zahlen werden hier mit der 8-Bit-Zweierkomplement dargestellt. Ermitteln Sie die Bitmuster für 40 und -81.

Lösung (2P) $40 = (00101000)_{\overline{2}}$

(3P) $81 = (01010001)_{\overline{2}} \overset{\text{Komplement+1}}{\Rightarrow} -81 = (10101111)_{\overline{2}}$

[Bezeichnung unwichtig, Hauptsache Lösung+Rechenweg stimmen]

c) (6P) Ergänzen Sie diese Tabelle von `glint`-Werten.

Bitte deutlich schreiben, Nebenrechnung oder Probe nur auf Blatt rechts!

dezimal	binär (mit Zweierkompl.)	oktal	hexadezimal
13			
	$(10001011)_{\overline{2}}$		
		$(357)_8$	
			$(1E)_{16}$

Lösung

dezimal	binär (mit Zweierkompl.)	oktal	hexadezimal
13	$(00001101)_{\overline{2}}$	$(015)_8$	$(0D)_{16}$
-117	$(10001011)_{\overline{2}}$	$(213)_8$	$(8B)_{16}$
-15	$(11101111)_{\overline{2}}$	$(357)_8$	$(EF)_{16}$
30	$(00011110)_{\overline{2}}$	$(036)_8$	$(1E)_{16}$

pro richtigem Eintrag 1/2 Punkt

(Dinge wie z.B. $(15)_8$ anstelle $(015)_8$ sind *noch* OK)

Aufgabe 5 (Algorithmen-Entwurf und Analyse [16+2 Punkte])
Gegeben ist eine nur aus "+" oder "-" bestehende endliche Zeichenfolge, z.B. "`--+++--+++++-+-+-+-+--+`"
gespeichert in einem Feld `z[0]...z[k-1]`.

a) (8P + (2 Zusatzpunkte für (asymptotisch) optimalen Algorithmus)) Geben Sie einen Algorithmus an, der berechnet, wie lang die längste Teilfolge ist, die nur aus "+"-Zeichen besteht (im Beispiel: 5).

Der Algorithmus soll entweder in Java- oder Pseudocode- oder als Programmablaufplan oder als klar formulierter Text beschrieben werden.

Lösung optimal:

Initialisierung: $i = 0$, $n = 0$, $\max = 0$.
Schritt für Schritt nach rechts durchlaufen.

- Falls Zeichen an Stelle i ein "+" ist, $n = n+1$,
- Falls Zeichen an Stelle i ein "-" ist,
  falls $n > \max$, setze $\max = n$,
  setze $n = 0$.

nicht optimal: Schritt für Schritt nach rechts durchlaufen.

- wenn aktuelles Zeichen ='+', dann zähle durch while-Schleife, wieviele '+' danach kommen, Ergebnis + 1 in Hilfsfeld festhalten

Anschließend Maximum der Einträge in Hilfsfeld ermitteln

b) (5P) Geben Sie eine möglichst gute Abschätzung Ihres Algorithmus in $\mathcal{O}$-Notation an (mit kurzer Begründung).

Lösung [muss nicht so detailliert sein wie hier]

optimal in jeder Iteration der äußeren Schleife: höchstens 3 Vergleiche (Zeichen, Vergleich mit max, Abbruchbed.), höchstens eine arithmetische Operation, höchstens 2 Zuweisungen: d.h. jede Iteration kostet nur konstante Zeit. Da k Iterationen folgt Laufzeit $\mathcal{O}(k)$

nicht optimal erste äußere Schleife: k Iterationen, innere Schleifen bis zu $k-1$ Iterationen (man kann es auch komplizierter durch arithmetische Reihe angeben), in jeder inneren Iteration nur konstant viele Elementaroperationen, folglich: Laufzeit für erste äußere Schleife kann nur durch $\mathcal{O}(k^2)$ abgeschätzt werden, Maximumbestimmung $\mathcal{O}(k)$ (wenn man es richtig macht), d.h. insgesamt $\mathcal{O}(k)$

c) (3P) Ein Algorithmus zur Lösung von a) liest beginnend bei Index 0 fortlaufend die Feldeinträge (und stellt interne Berechnungen beliebiger Art an). Das Lesen wird aber sofort unterbrochen, wenn die Antwort feststeht. Der Algorithmus kann die Feldlänge im vorhinein ermitteln.

Geben Sie Zeichenketten (1) und (2) der Länge 8 wie folgt an: (1) Es muss bis zum Ende gelesen werden. (2) Es muss *nicht* bis zum Ende gelesen werden und das Resultat ist 4. Begründen Sie jeweils kurz.

Lösung

(1) ++++++++ — hört er vorher auf, könnte ja noch ein + kommen, würde falsche Antwort geben

(2) ++++---- — nach 4 + eine Unterbrechung: da Länge bekannt, kann nichts Längeres kommen, also Antwort 4 steht fest (auch -++++-+- etc. wäre richtig)

je 0.5 + 1 Punkte für Beispiel + Begründung

Aufgabe 6 (Sortieren und Schleifeninvariante [15+0 Punkte])
Wir betrachten folgenden elementaren Sortieralgorithmus

```
30    public static void selectionSort(int[] feld) {
31        int maxpos, tmp;
32        for (int i=feld.length; i>1; i--) {
33            // bestimme Position des Maximums im Intervall [0,...,i-1]
34            maxpos = 0; // Position des Maximums
35            for (int j=0; j<i; j++)
36                if ( feld[j] > feld[maxpos])
37                    maxpos = j;
38            // tausche feld[i-1] und feld[maxpos]:
39            tmp = feld[i-1]; feld[i-1] = feld[maxpos]; feld[maxpos] = tmp;
40        }
41    }
```

a) (4P) Wenden Sie den Algorithmus an auf
`int[] test = {1, 6, 9, 8, 2, 3, 4, 5, 7};`
und ergänzen Sie die Tabelle um die Feldinhalte *nach* Iterationen 1, 2 und 3 der *äußeren* `for`*-Schleife* (Zeilen 32-40).

Start	1	6	9	8	2	3	4	5	7
nach 1. Iteration									
nach 2. Iteration									
nach 3. Iteration									

Lösung

167823459
167523489
164523789

b) (5P) Geben Sie (mit Begründung) eine möglichst gute asymptotische Abschätzung der Laufzeit des Algorithmus auf Eingaben der Länge n an

Lösung Äußere Schleife besteht im Wesentlichen aus konstant vielen Zuweisungen und einer inneren Schleife der Länge i (Laufvariable der äußeren Schleife), die wiederum nur aus Vergleichen und Zuweisungen besteht. Deswegen ist deren Laufzeit proportional zu i und insgesamt erhalten wir die Laufzeitschranke $\mathcal{O}(n + (n-1) + \ldots + 2) = \mathcal{O}(n^2)$.

[Ich denke, die Formel für die arithemtische Reihe ist hinlänglich bekannt und braucht hier nicht bewiesen werden.]

c) (6P) Es gilt folgende Aussage: Nach Ausführung der inneren Schleife (Zeilen 35–37) gilt `feld[maxpos]` enthält das Maximum der Einträge im Teilfeld `feld[0..i-1]`.

Begründen Sie damit, dass das Feld nach `feld.length` − 1 Iterationen der *äußeren Schleife* (Zeilen 32–40) sortiert ist. Das formale Konzept der Schleifeninvariante können, müssen Sie aber nicht benutzen.

Lösung

[großzügig korrigieren]

informal:

Zeile 39 sorgt dafür, dass das Maximum in `feld[0..i-1]` mit `feld[i-1]` die Positionen tauscht. Nach der ersten Iteration der äußeren Schleife haben wir also das Feldmaximum am Ende und stehen dann vor der gleichen Situation wie vorher — jedoch eingeschränkt auf das Teilfeld `feld[0..feld.length-2]`. Dessen Maximum rückt nun an dessen Ende, usw. Dies wird `feld.length-1` mal durchgeführt — wir erhalten schließlich ein Feld, bei dem die `feld.length-1` größten Einträge sortiert am Ende stehen und der Verbleibende in Position `0`. Da die letzten Einträge alle nicht kleiner sind, ist das ganze Feld sortiert.

formal $\mathcal{I}$ = „das Teilfeld T =`feld[i..feld.length-1]` ist aufsteigend sortiert und jeder Wert in `feld[0..i-1]` ist kleiner gleich jedem Wert in T“

$\mathcal{I}$ ist wahr vor der ersten Iteration ($i =$ `feld.length`), denn in diesem Fall haben wir ein leeres Feld T: nirgends ist die Ordnung verletzt und es gibt auch keine Einträge in T, die kleiner als ein Eintrag das Anfangsstücks sein könnten.

Sei $\mathcal{I}$ wahr vor Iteration $2 \leq i \leq$ `feld.length`. Das heißt `feld`$[$`i..feld.length` $- 1]$ ist aufsteigend sortiert und alle Einträge am Anfang sind kleiner gleich `feld[i]`. Da durch die innere Schleife das Maximum im Anfangsstück bestimmt wird und in Zeile 39 den Platz mit `feld[i-1]` tauscht, gilt wieder $\mathcal{I}$.

Nach der letzen Iteration gilt $\mathcal{I}$ also auch.

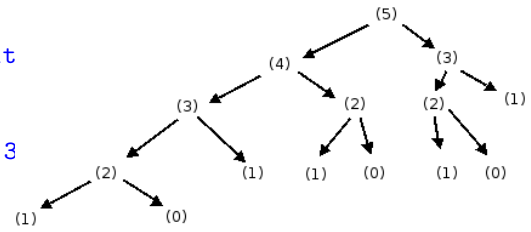
Die letzte Iteration beginnt mit `i==2` und endet mit `i==1`. $\mathcal{I}$ bedeutet somit: die letzten `feld.length-1` Einträge sind aufsteigend sortiert und der erste Eintrag (Index `0`) ist kleiner gleich allen Einträgen im Endstück. Somit ist das Feld aufsteigend sortiert.

Aufgabe 7 (Rekursion und Iteration [16+2 Punkte])
Pseudocode ist ausreichend für diese Aufgaben. Die Zahlenfolge $a_0, a_1, a_2, \ldots$ ist definiert durch: $a_0 = 0, a_1 = 1$ und $a_n = -2a_{n-1} + 3a_{n-2}$ für $n > 2$.

a) (6P) Formulieren Sie einen *rekursiven* Algorithmus, der bei Eingabe n (natürliche Zahl) den Wert a_n zurückgibt (nicht druckt!). Zeichnen Sie den Aufrufbaum (Rekursionsbaum) bei Aufruf mit der Eingabe $n = 5$.

Lösung

```
3       private static int aRek(int
4           if ( n == 0) return 0;
5           if ( n == 1) return 1;
6           return -2*aRek(n-1) + 3
7       }
```



4P für Alg, 2P für Baum

b) (6P) Formulieren Sie einen *iterativen*[1] Algorithmus, der bei Eingabe n (natürliche Zahl) den Wert a_n zurückgibt (nicht druckt!).

Lösung

```
8       private static int aIte1(int n) {
9           int[] a = new int[n+1];
10          a[0] = 0;
11          if (n > 0) a[1] = 1;
12          if (n > 1) {
13              for (int i = 2; i <= n; i++)
14                  a[i]=-2*a[i-1] + 3*a[i-2];
15          }
16          return a[n];
17      }

18      private static int aIte2(int n) {
19          int akt = 0, vor=1, vorvor=0;
20          if ( n == 0) return 0;
21          if ( n == 1) return 1;
22          for (int i = 0; i <= n-2; i++) {
23              akt = -2*vor + 3*vorvor;
24              vorvor = vor;
25              vor = akt;
26          }
27          return akt;
28      }
```

[1] Zwei Möglichkeiten: (1) Feld anlegen, das nach und nach mit den Werten a_0, a_1, a_2, ... gefüllt wird oder (2) lediglich die letzten Werte speichern, daraus den nächsten berechnen und geeignet umspeichern usw.

c) (4P + 2 Zusatzpunkte für Beweis der Formel) Für alle n gilt $a_n = \frac{1-(-3)^n}{4}$. Formulieren Sie aufgrund dessen einen Algorithmus, der bei Eingabe n (natürliche Zahl), den Wert a_n zurückgibt (nicht druckt!). Potenzieren darf dabei *nicht* benutzt werden.

Lösung

Die Aussage gilt offenbar für $n = 0$ und $n = 1$. Gelte die Aussage für $n \geq 1$. Dann folgt die Aussage für $n + 1$ aus der Gleichung

$$\frac{1-(-3)^{n+1}}{4} = -2\frac{1-(-3)^n}{4} + 3\frac{1-(-3)^{n-1}}{4},$$

die wiederum äquivalent ist zu der wahren Gleichung

$$1-(-3)^{n+1} = 1 + 2(-3)^n - 3(-3)^{n-1}.$$

```
29      private static int aImm(int n) {
30          int p = 1;
31          for (int i = 0; i < n; i++)
32              p*=(-3);
33          return (1-p)/4;
34      }
```

Klausur Informatik I
Wintersemester 2009/10
Prof. Dr. C. Damm
24. Februar 2010
von 10:00+x bis 11:30+x Uhr

Vorname:

Nachname:

Matrikelnummer:

Es sind **keine Hilfsmittel** (Skript, Taschenrechner etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten **Blätter** für Ihre Antworten. Die Blätter **nicht trennen!** Schreiben Sie mit blauem/schwarzem Kugelschreiber. Füller oder Bleistift sind nicht erlaubt.

Viel Erfolg!

	Max. Punkte	Erreichte Punkte
Aufgabe 1 (Allgemeine Kenntnisfragen)	15+0	
Aufgabe 2 (Elementare und strukturierte Daten)	15+0	
Aufgabe 3 (Datenstrukturen und OO-Konzepte)	15+0	
Aufgabe 4 (Sortieren und Suchen)	15+0	
Aufgabe 5 (Algorithmenanalyse)	15+0	
Aufgabe 6 (Iteration und Rekursion)	15+2	
Summe	90+2	

Name: MatNr.:

Aufgabe 1 (Allgemeine Kenntnisfragen [15+0 Punkte])
3P pro Aufgabenteil. Antworten Sie kurz und prägnant.

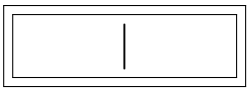
1. Nennen Sie die Voraussetzung zur Anwendbarkeit der binären Suche auf Feldern. Geben Sie in $\mathcal{O}$-Notation die worst-case-Komplexität an (Feldlänge n).

2. Was bedeutet partielle Korrektheit eines Algorithmus in Bezug auf die Problemspezifikation (V, N)?

3. Nennen und erläutern sie das Prinzip, nach dem bei Java die Parameterübergabe bei Methodenaufrufen erfolgt.

4. Welche Elemente einer Java-Klassendefinition werden durch das Schlüsselwort `static` gekennzeichnet? Was bedeutet `private`?

5. Beschreiben Sie das Teile-und-Herrsche-Prinzip und nennen Sie einen Beispielalgorithmus dafür.



Name: MatNr.:

Aufgabe 2 (Elementare und strukturierte Daten [15+0 Punkte])

1. (5P) Stellen Sie den Dezimalbruch 18.625 binär dar.

2. (2P) Sei $b_{N-1}b_{N-2}...b_1b_0 \in \{0,1\}^N$ die N-Bit-Zweierkomplement-Darstellung der ganzen Zahl z mit $-2^{N-1} < z < 2^{N-1}$. Wie erhält man daraus die N-Bit-Zweierkomplement-Darstellung von $-z$?

3. (3P) Wieviele Bytes werden durch folgenden Konstruktoraufruf in Java angefordert? Es genügt eine Zweierpotenz 2^n sowie eine kurze Begründung als Antwort. **int** [][][][] feld = **new int** [16][16][16][16];

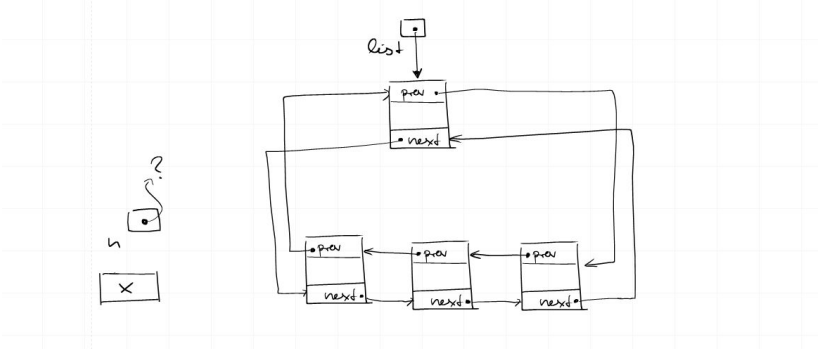
4. (5P) Die `main`-Methode soll einen „Countdown“ erzeugen, d.h. nacheinander „4, 3, 2, 1, Start!“ ausgeben, ist aber fehlerhaft. Beschreiben Sie den Fehler und geben Sie an, an welcher Stelle er auf dem Weg vom Programmcode bis zur Ausführung auftritt. Korrigieren Sie durch *zusätzliche* Programmzeilen (*nachfolgend* angeben, Einfügestelle kennzeichnen).

```
public static void main(String[] argv) {
    int[] x;
    for (int i = 0; i < 4; ++i)
        System.out.println(x[i]);
    System.out.println("Start!");
}
```

Name: MatNr.:

Aufgabe 3 (Datenstrukturen und OO-Konzepte [15+0 Punkte])

1. (6P) Ein Stack für `int`-Werte kann konkret mittels doppelt-verketteter Liste mit Ankerknoten realisiert werden. Dazu eine Zeichnung und Quellcode-Ausschnitte aus Knoten- und Listenklasse:




```
class IntNode{
    int wert;
    IntNode next;
    IntNode prev;
    public IntNode (int x, IntNode next, IntNode prev) {
        wert = x;
        this.prev = prev;
        this.next = next;}
}
```

```
class IntListe {
    IntNode list;
      ...
    void prepend(int x) // Knoten "links" einfuegen
      ...
    void append(int x) // Knoten "rechts" einfuegen
      ...
    void insert(int x)//vor erstem Knoten mit Wert <= x einspeichern
      ...
    int removeHead() // Knoten "links" entfernen (davor Wert liefern)
      ...
    int removeTail() // Knoten "rechts" entfernen (davor Wert liefern)
      ...
}
```

Nennen Sie ein Methodenpaar,das Stack-artiges „Ein-“ bzw. „Ausspeichern“ realisiert. Programmieren Sie davon die Einspeicher-Methode in Java.

2. (4P) Auf eine anfangs leere Queue werden Operationen angewendet. Tragen Sie den Zustand nach den angegebenen Aufrufen so ein, dass das Ende der Queue jeweils ganz links steht (erste Zeile ist komplett vorgegeben). *Deutlich schreiben!*

`enqueue(1);enqueue(2);enqueue(8);`	8	2	1									
`enqueue(5);enqueue(3);dequeue();`												
`enqueue(1);dequeue();dequeue();`												
`dequeue();enqueue(7);enqueue(1);`												

3. (5P) Welche Zeile im folgenden Java-Code verursacht einen Compilezeit-Fehler? Wenn diese Zeile gestrichen wird, welche Ausgabe erzeugt der Programmaufruf `java AB`? Begründen Sie ihre Antworten.

```
1  class A {void hallo(){System.out.print("A");}}
2  class B extends A{String s = "b";void hallo(){System.out.print("B");}}
3  class AB {
4      public static void main(String[] argv) {
5          B y = new B();
6          A x = y;
7          y = x;
8          x.hallo();
9          y.hallo();
10     }
11 }
```

Name: MatNr.:

Aufgabe 4 (Sortieren und Suchen [15+0 Punkte])

1. (7P) Sortieren Sie folgende Zahlenfolge aufsteigend mit Quicksort. Dazu das jeweils letzte Element der aktuellen Teilfolge als Pivot markieren (z.B. einkreisen), ein Schritt des Verfahrens durchführen und das Ergebnis in die nächste Zeile der Tabelle eintragen. Es müssen nicht alle Zeilen der Tabelle benutzt werden.

4	8	6	3	2	7	1	9	5

2. (6P) Zeichnen Sie den binären Suchbaum S, der durch Einfügen von 15, 5, 16, 3, 12, 10, 13, 6, 7 (in dieser Reihenfolge) entsteht. Zeichnen Sie dann den binären Suchbaum, der aus S durch Löschen der 5 entsteht.

3. (2P) Wahr oder falsch?: „Bei Preorder-Traversierung werden die Knoten eines Binärbaumes in umgekehrter Reihenfolge abgearbeitet wie bei Postorder-Traversierung.“ Begründen Sie oder geben Sie ein kleines Gegenbeispiel (mit Angabe und Benennung der Reihenfolgen).

Name: MatNr.:

Aufgabe 5 (Algorithmenanalyse [15+0 Punkte])

1. (6P) Seien m und n natürliche Zahlen. Bestimmen Sie für folgende Programmteile, die GENAUE Anzahl von `tuWas()`-Aufrufen. Der Rechenweg ist Teil der Lösung.
   Geben Sie anschließend in $\mathcal{O}$-Notation eine möglichst gute obere Schranke für diese Anzahl an.

```
13        // Teil A
14        for (int i = 0; i < m; ++i)
15            for (int j = i; j >= 0; --j)
16                tuWas();
```

```
20        // Teil B
21        for (int i = 1; i < m; ++i)
22            for (int j = 0; j <= n; ++j)
23                tuWas();
```

2. (3P) Gegeben sei eine sortierte Folge der Länge n und der Suchschlüssel q. „Ternäre Suche“ hat folgende Rekursionsidee:

   - Ist $n = 1$, so teste, ob q gleich dem einzigen Folgenglied (bei Erfolg liefere dessen Index, sonst -1).
   - Anderenfalls teile in 3 Teilfolgen T_0, T_1, T_2 annähernd gleicher Länge[1], bestimme durch Vergleiche mit dem jeweils letzten Element der 2 Teilfolgen T_0, T_1 in welcher der Suchschlüssel sein müsste und suche in dieser rekursiv weiter.

   Die Anzahl der Vergleiche mit q sei $C(n)$ im schlechtesten Fall. Stellen Sie eine Rekursionsgleichung für $C(n)$ und die Randbedingung $C(1) =?$ auf.

[1] d.h. keine ist länger als $\lfloor n/3 \rfloor + 2$.

3. (6P) Sei n eine beliebige Zweierpotenz. Beweisen Sie, dass für $T(n) = 2n^2 \log_2 n + n^2$ die Rekursionsgleichung $T(n) = 4T(n/2) + 2n^2$ und die Randbedingung $T(1) = 1$ gilt.

Name: MatNr.:

Aufgabe 6 (Iteration und Rekursion [15+2 Punkte])

1. (4P) Die Folge g_n ist rekursiv definiert durch

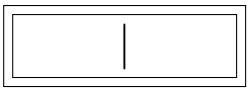
$$g_0 = 0, g_1 = 1, g_n = g_{n-1} + 2g_{n-2}.$$

   Programmieren Sie eine rekursive Methode `int g(int n)`, die zu Argumentwert n das Folgenglied g_n zurückliefert.

2. (5P) Zeichnen Sie den Aufrufbaum zur rekursiven Berechnung von g_5. Stellen Sie damit eine Rekursionsgleichung auf für die Anzahl a_n der Aufrufe, die die rekursive Berechnung von g_n erfordert.

3. (6P) Beschreiben Sie informal eine Idee zur iterativen Berechnung von g_n und programmieren Sie diese.

4. (+2P) Beschreiben Sie informal eine weitere Idee zur iterativen Berechnung von g_n.



Name: MatNr.:

Probeklausur Informatik I

Erklärungen

- bei der E-Klausur werden die Aufgaben ILIAS-typisch auf einzelnen Seiten angezeigt (gut lesbar)
- außerdem gibt es bei der E-Klausur diese Möglichkeiten:
  - Vergrößerung des Texts mit Strg-Mausrad
  - Eingabefenster für Freitexte an Ecke rechts-unten aufziehbar
  - Antwortoptionen (nachfolgend in blauer Schrift) werden in Drop-Down-Menüs angezeigt
- bei der Klausur wird ein leeres Blatt für Notizen bereitgelegt, Notizen werden nicht bewertet
- im hier vorliegenden Layout sind Texte mit sehr langen Zeilen abgeschnitten, hier die Ergänzungen (fehlender/unlesbarer Text kursiv)

  Aufgabe 2.2 ... leistet: Es *wird die Summe der ganzen* Zahlen ...

  Aufgabe 12 ... zutreffenden A *ntworten aus (jeweils 1.5P für* die ...

  Aufgabe 13.1 ... weitere as *ymptotische Analyse* geeignet?

  Aufgabe 13.2 ... sehen *typischerweise so aus:*

  Aufgabe 14 ... vorkomm *t*

  ... Fol *gendes leisten:*

  ... Bereichsbedingung erf *üllt ist, dann mit zweiter Schleife* in einem Hilfsfeld ...

  ... Feld anl *egen und in Schleife die* Komponenten ...
- zu Aufgaben 14/15 gehören Freitextfelder, die hier aber nicht zu sehen sind
- Punkte

Aufgabe	erreichbar
1	4
2	5
3	6
4	5
5	6
6	4
7	6
8	6
9	6
10	6
11	2
12	3
13	6
14	12
15	13
Summe	90

Liste der Antworten

1. Allgemeine Grundkenntnisse (I1-ID: rjyb60x0u5h0)

1. Geben Sie die richtige Antwortkombination an (1P für die richtige/0P für jede falsche Kombination):
   - (A) Spezifikationen bestehen aus einer präzisen Beschreibung der erlaubten Eingaben.
   - (B) Algorithmen sind endliche Vorschriften, die Rechenabläufe schrittweise beschreiben.
   - (C) Compiler übersetzen Quellcode erst in Assembler, dann in Unix-Code.
   - (D) Das Java-Programm ist auf dem System lauffähig, wenn darauf der entsprechende Compiler installiert ist.

2. Geben Sie die richtige Antwortkombination an (1P für die richtige/0P für jede falsche Kombination):
   - (A) Bei der Entwicklung eines Programms muss nach jedem Compilieren editiert werden.
   - (B) Interaktive Programme können Daten von der Standard-Eingabe erhalten.
   - (C) Am CMD-Flag ist erkennbar, ob die Speicherzelle eine Zahl oder einen Befehl enthält.
   - (D) Eine *Pipe* funktioniert im wesentlichen wie eine Queue.

3. Geben Sie die richtige Antwortkombination an (1P für die richtige/0P für jede falsche Kombination):
   - (A) Durch Eingabeumleitung kann man gespeicherte Daten an ein interaktives Kommando leiten.
   - (B) Informationsverarbeitung zerfällt in die Schritte Codierung, Datenverarbeitung und Decodierung.
   - (C) Mit 4 Hexadezimalziffern kann man mehr als 130 000 Werte darstellen.
   - (D) Der Bruch 4/5=0.8 kann exakt als endlicher Dualbruch dargestellt werden.

4. Geben Sie die richtige Antwortkombination an (1P für die richtige/0P für jede falsche Kombination):
   - (A) Ganzzahlarithmetik ist normalerweise schneller als Gleitkomma-Arithmetik.
   - (B) Strukturierte Programmierung ist die Programmierung mit strukturierten Daten.
   - (C) Rekursion benötigt mehr Speicher auf dem Stack als Iteration.
   - (D) Die JVM interpretiert jeden Bytecodebefehl und prüft dessen Syntax.

2. Einige Java-Grundkenntnisse (I1-ID: u1j7g25165h0)

1. Ordnen Sie Begriffe zeitlich (0.25P für jede korrekte Position)

2. Füllen Sie die Lücken, so dass das Programm mit zwei beliebigen Zahlwerten A und B als Kommandozeilenparametern aufgerufen Folgendes leistet: Es Zahlen ausgeben, die innerhalb des *abgeschlossenen* Intervalls [A,B] liegen. (0.25P für jede korrekte Angabe)

```
public class Intervallsumme {
    public static void main(String[] args) {
            ,
            min =                   (args[0]),
            max =                   (args[1]);
        int        ;
        while (          ) {
             ;
             ;
        }

    }
}
```

3. Wieviele Bytes beansprucht eine Variable?
   - `int`:
   - `char`:
   - `float`:
   - `boolean`:

   (0.25P für jede richtige Antwort)

3. Grundlegende Datentypen (I1-ID: ksu13v0075h0)

1. Geben Sie die richtige Antwortkombination an (1P für die richtige/0P für jede falsche Kombination):
   - (A) Das Leerzeichen ist ein druckbares Zeichen.
   - (B) iso8859-1 ist eine Teilmenge von ASCII.
   - (C) `'C'++` liefert das gleiche Ergebnis wie `'C'+1`.
   - (D) `"C" + 1` ist *kein* erlaubter Java-Ausdruck.

2. Geben Sie die richtige Antwortkombination an (1P für die richtige/0P für jede falsche Kombination):
   - (A) `int` ist ein Ganzzahldatentyp, `char` nicht.
   - (B) `int x = Integer.MAX_VALUE + 1;` ist eine erlaubte Anweisung.
   - (C) Der Wert von `(( 3 < 7 ) && ((17 + 4) % (12 - 5)) != 0)` ist `false`.
   - (D) Der Wert von `( 3 + 1/3 == 3 - 1/3)` ist `false`.

3. Geben Sie die zutreffenden Antworten an:

- (A) `String s = "Hallo", System.out.println(s.length);`

  Ausgabe?

- (B) `System.out.println(10e-1f);`

  Ausgabe?

- (C) `2 * (short) 0xCafe`

  Typ des Ausdrucks?

- (D) `double y = Double.MIN_VALUE; System.out.println(y/2);`

  Ausgabe?

- (E) `int x = Integer.MAX_VALUE, y = Integer.MAX_VALUE++, z = x+y;`

  Wert von `z`?

- (F) `System.out.println( (double) (1/2));`

  Ausgabe?

4. Geben Sie die richtige Antwortkombination an (1P für die richtige/0P für jede falsche Kombination):
   - (A) `int[] z = {1}; System.out.println(z);` produziert die Ausgabe 1.
   - (B) `length` ist ein `final`-Attribut von Feldern.
   - (C) Durch `int[] a = {1, 2, 3}, b = new int[];` werden zwei neue Felder angelegt.
   - (D) Durch `int[][][][] x = new char[32][32][32][16];` wird etwa 1Mb Heap-Speicher reserviert.

4. Integer-Arithmetik (I1-ID: 5wa3il81gdg0)

Sei `d` eine `int`-Variable mit dem Wert `Integer.MAX_VALUE` (also 2147483647). Welche Textausgabe erzeugen diese Anweisungen?:

```
1:  System.out.println(-d);     //
2:  System.out.println(d-1);    //
3:  System.out.println(1+2*d);  //
4:  System.out.println(-(2*d)); //
```

Achten Sie darauf, dass Ihre Antworten NUR Dezimalziffern enthalten (sowie bei Bedarf Vorzeichen)!

5. Iteration in Feldern (I1-ID: t75bm8e005h0)

```
int[] p = { 1, 4, 0, 3, 6, 2, 7, 5 };
int[] c = { 0, 0, 0, 0, 0, 0, 0, 0 };
int[] q = { 0, 0, 0, 0, 0, 0, 0, 0 };
for (int i = 0; i < 8; i++)
    c[p[i]] = c[p[i]] + 1;
for (int i = 0; i < 8; i++)
    q[p[i]] = i;
```

Tragen Sie die Werte der Feldkomponenten *nach* Ausführung des Java-Fragments ein:

Index	0	1	2	3	4	5	6	7
`c`								
`q`								

6. Felder: Fehler oder...? (I1-ID: o2vav4a1gdg0)

Erzeugt untenstehende Klasse

- einen Compilezeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-):
- einen Laufzeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-):
- eine Ausgabe? Falls ja, so tragen Sie Ausgabe ein, ansonsten einen Bindestrich (-):

```
1: class FehlerOderKeiner {
2:     public static void main(String[] args) {
3:         int[] a = {2,3,5,7,11}; int[] b = { a[2], a[2]};
4:         b = a;
5:         b[1] = b[0];
6:         System.out.println(a[0] != a[1]);
7:     }
8: }
```

7. Rekursion (I1-ID: qft7dic1y4h0)

Betrachten Sie folgende Klasse:

```
 1: class Myst {
 2:     static void ery(int n) {
 3:         if (n == 0)
 4:             return;
 5:         ery( n-1);
 6:         A();
 7:         yre( n-1);
 8:     }
 9:     static void yre(int n) {
10:         if (n == 0)
11:             return;
12:         ery( n-1);
13:         B();
14:         yre( n-1);
15:     }
16:     static void A() {
17:         System.out.print("A");
18:     }
19:     static void B() {
20:         System.out.print("B");
21:     }
```

```
22:     public static void main(String[] args) {
23:         int n = Integer.parseInt(args[0]);
24:         ery(n);
25:         System.out.println();
26:     }
27: }
```

1. Geben Sie die ersten 8 Zeichen der Ausgabe zum Aufruf `java Myst 4` an:

2. Aus wievielen Zeichen besteht der Ausgabestring beim Aufruf `java Myst 7` (ohne den abschließenden Zeilenwechsel durch Zeile 25)?

3. Wird Zeile 14 durch `yre( n-2);` ersetzt, so führt beispielsweise der Aufruf `java Myst 4` zu einem Fehler.
   Es genügt, ein einzelnes Zeichen in einer *anderen* Zeile zu ändern, um diesen Fehler nicht mehr hervorzurufen. Um welche Zeile handelt es sich?

4. Werde Zeile 12 durch `ery( n-2);` und zusätzlich Zeile 14 durch `yre( n-2)` ersetzt.
   Geben Sie das maximale Argument `n` an, bei dem der Aufruf `java Myst n` *nicht* zu einem Fehler führt:

8. Stack und Queue in Aktion (I1-ID: 8303049035h0)

1. Auf einem Stack werden *in gemischter Reihenfolge* 10 Push- und 10 Pop-Operationen ausgeführt. Es ist bekannt, dass die Push-Operationen die Zahler
   Reihenfolge einspeichern und dass bei jedem Pop der ausgespeicherte Wert ausgegeben wird. Welche Ausgabelisten sind möglich?

   Wählen Sie aus den beiden Blöcken jeweils die zutreffende Antwort aus (jeweils 2P für die richtige Kombination/0P für jede andere).

   - (A) 1 2 3 4 5 6 9 8 7 0
   - (B) 0 4 6 5 3 8 1 7 2 9

   - (C) 1 4 7 9 8 6 5 3 0 2
   - (D) 2 1 4 3 6 5 8 7 9 0

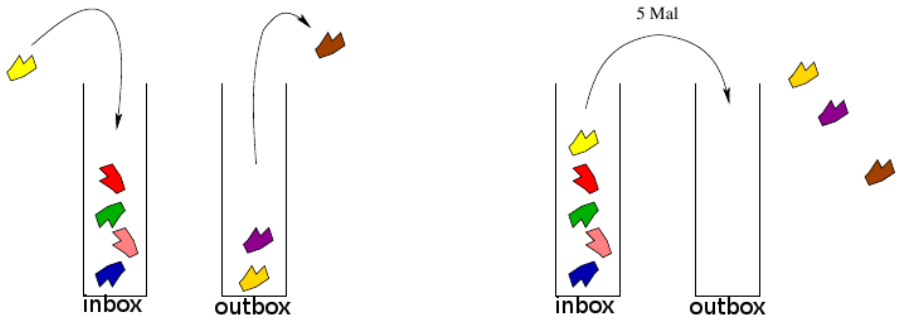
2. Auf einer Queue werden *in gemischter Reihenfolge* 10 Enqueue- und 10 Dequeue-Operationen ausgeführt. Es ist bekannt, dass die Enqueue-Operation
   dieser Reihenfolge einspeichern und dass bei jedem Dequeue der ausgespeicherte Wert ausgegeben wird. Welche Ausgabelisten sind möglich?

   Wählen Sie aus den beiden Blöcken jeweils die zutreffende Antwort aus (jeweils 1P für die richtige Kombination/0P für jede andere).

   - (E) 4 6 8 7 5 3 2 9 0 1
   - (F) 2 5 6 7 4 8 9 3 1 0

   - (G) 0 1 2 3 4 5 6 7 8 9
   - (H) 4 3 2 1 0 5 6 7 8 9

9. Queue aus zwei Stacks bauen (I1-ID: a4g40y31x4h0)



Mit zwei Stacks kann man eine Queue implementieren:

- `enqueue(...)` entspricht Einspeichern in `inbox`
- `dequeue()` entspricht Ausspeichern aus `outbox`.

Unter Verwendung dieser API

`public class Stack`	
`boolean empty()`	testet, ob dieser Stack leer ist
`void push(String s)`	speichert `s` in diesem Stack
`String pop()`	liefert zuletzt gespeicherten String aus diesem Stack und entfernt ihn

sähe die entsprechende Klasse z.B. so aus:

```
public class ZSQueue { // Zwei-Stack-Queue
    private Stack inbox, outbox;
    public ZSQueue() { ... }
    public boolean empty() { ... }        // Leerheitstest
    public void enqueue(String s) { ... } // einspeichern
    public String dequeue() {             // ausspeichern und liefern
        // Rumpf (Ihre Aufgabe)
    }
}
```

Stellen Sie die Zeilen für den Rumpf der Methode `dequeue()` richtiger Reihenfolge zusammen:

- ist die Queue leer, ist Ausspeichern nicht möglich!
- teste ob `outbox` leer ist und fülle in diesem Fall den kompletten Inhalt von `inbox` um in `outbox`, dann ein Element aus `outbox` ausspeichern und liefern.

10. Vermischtes zu Klassentypen (I1-ID: t5k8w9h0frg0)

1. Objekte verstehen:

```
public class Spieler {
    private int score;
    public Spieler( int s) {
        this.score = s;
    }
}
```

Zwei `Spieler`-Objekte werden instanziiert:

```
Spieler a = new Spieler(50), b = new Spieler(50);
```

Welchen Wert hat nun der Ausdruck a == b?

2. Fremden Code deuten:

```
1:  public class FirstBpp extends hepno.users.Frame {
2:      private hepno.users.Wring wrags;
3:      public FirstBpp() {
4:          wrags = new hepno.users.Wring();
5:      }
6:      public static void main(String[] args) {
7:          FirstBpp app = new FirstBpp();
8:      }
9:  }
```

Was bewirkt Zeile 2?:

3. Statische Methoden und Instanzmethoden:

```
public class Klasse {
    private final int zahl = 10;
    double a() {
        System.out.println(zahl);
        return c();
    }
    void b() {
        System.out.println(this);
    }
    double c() {
        double r = Math.random();
        System.out.println(r);
        return r;
    }
    void d() {
        a();
        a();
    }
    int e() {
        return zahl;
    }
}
```

Welche der obigen Methoden ist ihrer Natur nach eher eine statische Methode (und sollte daher mit `static` versehen werden)?

4. Fehlersuche:

```
public class Test {
    private int testCount;
    public static int getCount(){
        return testCount;
    }
    public Test(){
        testCount++;
    }
}
```

Obiger Code verursacht einen Compilezeitfehler. Was ist das Problem?

5. Tracing:
Hinweis: Beachten Sie, dass man über Objektreferenzen auch auf Klassenvariablen zugreifen kann.

```
public class Tset {
    static int tsetCount = 0;
    public Tset(){
        tsetCount++;
    }
    public static void main(String [] args){
        Tset t1 = new Tset();
        System.out.print(t1.tsetCount + ", ");
        Tset t2 = new Tset();
        System.out.print(t1.tsetCount + " "+ t2.tsetCount + ", ");
```

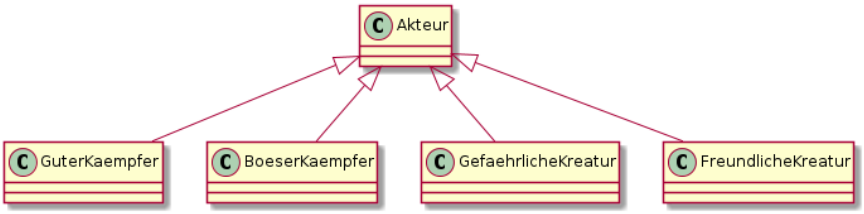
```
            Tset t3 = new Tset();
            System.out.print(t1.tsetCount+ " "+ t2.tsetCount+ " "+ t3.tsetCount);
            System.out.println();
        }
    }
```

Welche Ausgabe produziert der Code?

6. Begriffsverständnis: Wählen Sie aus dem Menü die einzige Aussage aus, die NICHT korrekt ist.

11. OOP-Entwurf (I1-ID: lf9g4np0qtg0)

In einer Spiele-Software gibt es folgende Hierarchie von Java-Klassen, die die verschiedenen Sorten von Akteuren modelliert:



Die (hier nicht gezeigte) Klasse `Spiel` benutzt die zugreifbaren Methoden obiger Klassen, um den Ablauf zu gestalten.

Wie ist (unter Erhalt der gezeigten Klassenbeziehungen) folgende Zusatzanforderung am besten zu modellieren?
Alle Kaempfer-Objekte aber keine Kreatur-Objekte sollen mit einer Methode benutzeWaffe() ausgestattet werden.

- Die Methode `benutzeWaffe()` nur in `GuterKaempfer` und in `BoeserKaempfer` wie benötigt implementieren.
- `benutzeWaffe()` mit leerem Rumpf in `Akteur` implementieren, in `GuterKaempfer` und `BoeserKaempfer` geeignet überschreiben.
- `benutzeWaffe()` in `Akteur` implementieren, in `GefaerlicheKreatur` und `FreundlicheKreatur` mit `private` kennzeichnen.
- Interface `Kaempfer` mit `benutzeWaffe()` definieren und durch `GuterKaempfer` und `BoeserKaempfer` implementieren lassen.
- Weitere Klasse `Kaempfer` schreiben, die `benutzeWaffe()` implementiert, `GuterKaempfer`, `BoeserKaempfer` davon erben lassen.

12. OOP-Verständnisfragen (I1-ID: fnkdcz9075h0)

Sei p eine Referenzvariable vom Typ X. Was kann alles der dynamische Typ von p sein? Wählen Sie aus den beiden Blöcken unten jeweils die zutreffenden Ar die richtige Kombination/0P für jede andere).

- (A) eine Erweiterungsklasse von X (falls X eine Klasse ist)
- (B) ein elementarer Datentyp (falls X die zugehörige Wrapper-Klasse ist)
- (C) eine Klasse, die X implementiert (falls X ein Interface ist)

- (D) eine Klasse, von der X direkt oder indirekt erbt (falls X eine Klasse ist)
- (E) ein Interface, das X als abstrakte Klasse erweitert (falls X eine Klasse ist)
- (F) eine Erweiterungsklasse einer Klasse, die X implementiert (falls X ein Interface ist)

13. Analyse von Algorithmen (I1-ID: 7o9glw3115h0)

1. Betrachten Sie folgenden Ausschnitt aus einem größeren Programm:

```
1: for (int i = 0; i < N; i++)
2:     for (int j = i+1; j < N; j++)
3:         for (int k = j+1; k < N; k++)
4:             verarbeite(daten);
```

Die Ausführungszahl $V(N)$ von Zeile 4 bei Problemgröße N soll untersucht werden. Welcher der folgenden Ausdrücke ist am besten für die weitere as geeignet?

2. Eine Implementation wird zahlreichen Verdopplungstests unterzogen, mit zufällig generierten Eingabedaten. Die Ergebnisse — alle sehr ähnlich — sehe

Problemgröße N	1024	2048	4096	8192	16384	32768	65536	131072	262144	524288	1048576	2097152	4194304	8388608
Zeit (s)	0.00	0.01	0.01	0.02	0.03	0.06	0.14	0.30	0.65	1.24	2.46	4.98	10.02	19.94

Außerdem kann man nachweisen, dass die Ausführungszahl der Kernoperationen bei spezieller Struktur der Eingabe linear mit der Problemgröße N wä

Welche dieser Hypothesen über das Laufzeitverhalten erscheint dann am plausibelsten?

3. Welche dieser Aussagen trifft NICHT zu?

14. Eine Bibliothek für Permutationen (I1-ID: l120mlc005h0)

Definition

1. Wir nennen ein `int`-Feld `p` der Länge `n` eine *Permutation*, wenn `p[0]`, ..., `p[n-1]` im Bereich $0, 1, \ldots, n-1$ liegen und jeder Wert genau einmal vorkomm
2. Die Permutation `q` heißt *Inverse* der Permutation `p`, wenn für alle `i` der Wert von `q[p[i]]` gleich `i` ist.

Aufgabe

Schreiben Sie eine Klasse `Permutation` mit den *statischen* Methoden `check(..)` und `invers(..)`, die aus jeder Clientklasse aufgerufen werden können und Folg

- `check(p)` überprüft, ob `p` eine Permutation ist und liefert den Wahrheitswert (**Hinweis**: z.B. zuerst in einer Schleife prüfen, ob die Bereichsbedingung erfü in einem Hilfsfeld `c` die Vorkommen der Werte `p[0]`, ..., `p[n-1]` erfassen)
- `invers(p)` ermittelt und liefert die Inverse von `p`. (Dabei ist bereits gesichert, dass `p` wirklich eine Permutation ist - nicht separat prüfen! **Hinweis**: Feld anl Komponenten so setzen, wie es Definition 2. vorschreibt.)

0 Zeichen zugelassen, Anzahl der eingegebenen Zeichen: **0**

15. Eine einfache Klasse (I1-ID: jr90w0703vg0)

Programmieren Sie eine einfache Klasse `Auto` mit der Angaben zu Autos abgespeichert werden können.
Jedes Auto zeichnet sich durch seinen Typ aus (z.B. "Mercedes") und durch seinen Preis (z.B. 30000).
Die folgenden Operationen sollen durch Methoden/Konstruktoren ermöglicht werden (*wenn nicht anders angegeben*: aufrufbar aus beliebigen Clientklassen):

- Erzeugen einer `Auto`-Instanz unter Angabe des Typs (Typ soll nach Erzeugen des Objekts nicht mehr änderbar sein)
- Erzeugen einer `Auto`-Instanz unter Angabe des Typs und des Preises (Typ soll nach Erzeugen des Objekts nicht mehr änderbar sein)
- Setzen des Preises einer Instanz (*nur von `Auto`-Methoden aus*),
- Abfragen des Preises einer Instanz
- Abfragen der Anzahl der erzeugten Instanzen und der Summe der Preise

VERWENDEN SIE FÜR DIE DATEN BITTE DIESE BEZEICHNER:
`typ`, `preis`, `anzahl`, `gesamt`

0 Zeichen zugelassen, Anzahl der eingegebenen Zeichen: **0**

Wiederholungsklausur Informatik I
Sommersemester 2008
Prof. Dr. Carsten Damm
16. April 2008
von 08:15+x bis 09:45+x Uhr
(Bearbeitungszeit: 90 Minuten)

Vorname:

Nachname:

Matrikelnummer:

Es sind **keine Hilfsmittel** (Skript, Taschenrechner etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten **Blätter** für Ihre Antworten. Schreiben Sie mit blauem/schwarzem Kugelschreiber, Füller etc..

Bleistift ist nicht erlaubt. Korrekturfelder | | **nicht beschriften**!

Zum **Bestehen** der Klausur sind **45** Punkte ausreichend.

Viel Erfolg!

	Max. Punkte	Erreichte Punkte
Aufgabe 1 (Auswahlaufgabe)	5+0	
Aufgabe 2 (Java: eine einfache Klasse)	11+0	
Aufgabe 3 (Datenstrukturen und Graphen)	13+0	
Aufgabe 4 (Zahlendarstellung)	14+0	
Aufgabe 5 (Algorithmen-Entwurf und Analyse)	16+2	
Aufgabe 6 (Sortieren und Schleifeninvariante)	15+0	
Aufgabe 7 (Rekursion und Iteration)	16+2	
Summe	90+4	

Note:

Name: MatNr.:

Aufgabe 1 (Auswahlaufgabe [5+0 Punkte])

Nur jeweils eine Antwort ist richtig. Falsche Antworten werden innerhalb dieser Aufgabe verrechnet (Bsp.: a) und b) richtig, aber c), d) und e) falsch ergibt 0P).

Kreisen Sie die richtige Nummer ein. Mehr Kreise (oder unleserlich) ergeben 0P für die Teilaufgabe.

a) Überladen einer Methode

1. ist die Zuordnung mehrerer Semantiken zum gleichen Methodennamen (Auswahl erfolgt durch Aufrufkontext)
2. ist die Programmierung unübersichtlich vieler verschiedener Funktionalitäten innerhalb einer Methode
3. geschieht zum Beispiel bei der Neudefinition der Methode `Object.toString()` in einer abgeleiteten Klasse

b) imperative Programmierung ist gekennzeichnet durch

1. Kommandozeilensteuerung
2. Befehlsfolgen zur Zustandsänderung
3. Zerlegen von Problemen in Teilprobleme

c) Rekursion ist

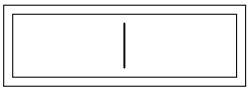
1. das Rückverfolgen von Methodenaufrufen
2. das Aufrufen einer Methode in deren eigenem Methodenrumpf
3. das Rückführen einer Problemlösung auf zwei kleinere Probleme

d) Aus welchen drei Kontrollstrukturen lassen sich alle Algorithmen aufbauen?

1. Eingabe, Verarbeitung und Ausgabe
2. for-Schleife, while-Schleife und do-while-Schleife
3. Komposition, Alternative und while-Schleife

e) Algorithmen sind stets

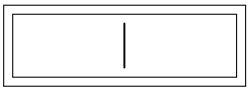
1. in endlicher Zeit abarbeitbar
2. endlich beschreibbar
3. in asymptotischer Form analysierbar



Name: MatNr.:

Aufgabe 2 (Java: eine einfache Klasse [11+0 Punkte])
Geben Sie eine einfache Klasse `Modul` an, die man zur Verwaltung der Module in einem Bachelorstudiengang benutzen kann. Jedes Modul zeichnet sich durch seinen Namen, z.B. “Info I” oder “C-Kurs”, sowie seine Leistungspunktezahl aus. Außerdem will man jederzeit die Anzahl angebotener Module ermitteln können, sowie die Gesamtanzahl der Leistungspunkte in diesen. *Benutzen Sie in Ihrem Quelltext die Bezeichner* `name`, `punkte`, `anzahl` *und* `gesamtPunkte` *für diese Dinge.* Die folgenden Operationen sollen möglich sein:

- Erzeugen einer Instanz dieser Klasse unter Angabe des Namens,
- Erzeugen einer Instanz dieser Klasse unter Angabe des Namens und der Leistungspunktezahl,
- Setzen und Abfragen der Punktzahl jeder einzelnen Instanz
- Es soll möglich sein, zu jeder Zeit abzufragen, wieviele Instanzen der Klasse bereits erzeugt wurden, und wieviele Leistungspunkte diese zusammen haben.



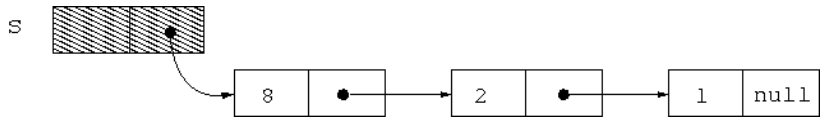
Name: MatNr.:

Aufgabe 3 (Datenstrukturen und Graphen [13+0 Punkte])

a) (3P) Auf einen anfangs leeren Stack werden `push/pop`-Operationen angewendet. Tragen Sie den Zustand des Stacks nach den angegebenen Aufrufen ein (erste Zeile ist komplett vorgegeben). Bitte schreiben Sie deutlich!

`push(1);push(2);push(8);`	1	2	8									
`push(5);push(3);pop();`												
`push(1);pop();pop();`												
`pop();push(7);push(1);`												

b) (6P)

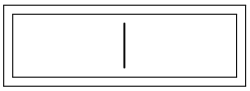


Dies skizziert die Realisierung eines Stacks durch eine verkettete Liste mit Startknoten `s` (Zustand wie erste Tabellenzeile oben). Mögliche Befehle:

- *n* `= neu(`*i*`)` — Erzeugen von neuem Knoten namens *n* mit Inhalt *i*
- *x* `= inhalt(`*m*`)` — kopiere in *m* gespeicherte Zahl nach *x*
- `nachfolger(`*o*`) = `*p* — der Nachfolger von *o* ist nun *p*
  (oder auch: *k* `= nachfolger(`*z*`)` — Nachfolger von *z* in *k* speichern)

Geben Sie je eine Befehlsfolge an, um (1) bzw. (2) zu realisieren:
(1) `x = top(); pop();` (2) `push(5);`
Auf Reihenfolge achten! Keine Variablendeklarationen! Nicht testen, ob Stack leer — das können Sie einfach voraussetzen!

c) (4P) Welche Tiefe hat ein binärer Baum mit n Blättern mindestens? Geben Sie den Wert möglichst genau an und begründen Sie Ihre Antwort.



Name: MatNr.:

Aufgabe 4 (Zahlendarstellung [14+0 Punkte])

a) (3P) Der hypothetische Datentyp `gint` stellt ganze Zahlen bekanntlich mit dem Zweierkomplement der Länge 4 dar (4-Bit-Zweierkomplement). Ergänzen Sie die folgenden Konstantendefinitionen der Wrapper-Klasse `Ginteger`:

```
public static final gint MAX_VALUE = ........;

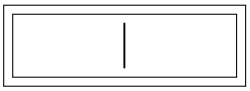
public static final gint MIN_VALUE = ........;
```

b) (5P) Wir betrachten dazu noch den Datentyp `glint`: ganze Zahlen werden hier mit der 8-Bit-Zweierkomplement dargestellt. Ermitteln Sie die Bitmuster für 40 und -81.

c) (6P) Ergänzen Sie diese Tabelle von `glint`-Werten.

Bitte deutlich schreiben, Nebenrechnung oder Probe nur auf Blatt rechts!

dezimal	binär (mit Zweierkompl.)	oktal	hexadezimal
13			
	$(10001011)_{\overline{2}}$		
		$(357)_8$	
			$(1E)_{16}$



Name: MatNr.:

Aufgabe 5 (Algorithmen-Entwurf und Analyse [16+2 Punkte])
Gegeben ist eine nur aus "+" oder "-" bestehende endliche Zeichenfolge, z.B. "`--+++--+++++-+-+-+-+--+`"
gespeichert in einem Feld `z[0]...z[k-1]`.

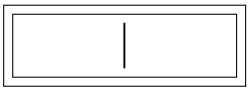
a) (8P + (2 Zusatzpunkte für (asymptotisch) optimalen Algorithmus)) Geben Sie einen Algorithmus an, der berechnet, wie lang die längste Teilfolge ist, die nur aus "+"-Zeichen besteht (im Beispiel: 5).

Der Algorithmus soll entweder in Java- oder Pseudocode- oder als Programmablaufplan oder als klar formulierter Text beschrieben werden. |

b) (5P) Geben Sie eine möglichst gute Abschätzung Ihres Algorithmus in $\mathcal{O}$-Notation an (mit kurzer Begründung). |

c) (3P) Ein Algorithmus zur Lösung von a) liest beginnend bei Index 0 fortlaufend die Feldeinträge (und stellt interne Berechnungen beliebiger Art an). Das Lesen wird aber sofort unterbrochen, wenn die Antwort feststeht. Der Algorithmus kann die Feldlänge im vorhinein ermitteln.

Geben Sie Zeichenketten (1) und (2) der Länge 8 wie folgt an: (1) Es muss bis zum Ende gelesen werden. (2) Es muss *nicht* bis zum Ende gelesen werden und das Resultat ist 4. Begründen Sie jeweils kurz. |



Name: MatNr.:

Aufgabe 6 (Sortieren und Schleifeninvariante [15+0 Punkte])
Wir betrachten folgenden elementaren Sortieralgorithmus

```
30    public static void selectionSort(int[] feld) {
31        int maxpos, tmp;
32        for (int i=feld.length; i>1; i--) {
33            // bestimme Position des Maximums im Intervall [0,...,i-1]
34            maxpos = 0; // Position des Maximums
35            for (int j=0; j<i; j++)
36                if ( feld[j] > feld[maxpos])
37                    maxpos = j;
38            // tausche feld[i-1] und feld[maxpos]:
39            tmp = feld[i-1]; feld[i-1] = feld[maxpos]; feld[maxpos] = tmp;
40        }
41    }
```

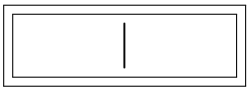
a) (4P) Wenden Sie den Algorithmus an auf
`int[] test = {1, 6, 9, 8, 2, 3, 4, 5, 7};`
und ergänzen Sie die Tabelle um die Feldinhalte *nach* Iterationen 1, 2 und 3 der *äußeren* `for`*-Schleife* (Zeilen 32-40).

Start	1	6	9	8	2	3	4	5	7
nach 1. Iteration									
nach 2. Iteration									
nach 3. Iteration									

b) (5P) Geben Sie (mit Begründung) eine möglichst gute asymptotische Abschätzung der Laufzeit des Algorithmus auf Eingaben der Länge n an

c) (6P) Es gilt folgende Aussage: Nach Ausführung der inneren Schleife (Zeilen 35–37) gilt `feld[maxpos]` enthält das Maximum der Einträge im Teilfeld `feld[0..i-1]`.

Begründen Sie damit, dass das Feld nach `feld.length` − 1 Iterationen der *äußeren Schleife* (Zeilen 32–40) sortiert ist. Das formale Konzept der Schleifeninvariante können, müssen Sie aber nicht benutzen.



Name: MatNr.:

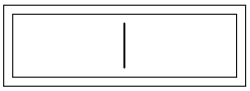
Aufgabe 7 (Rekursion und Iteration [16+2 Punkte])
Pseudocode ist ausreichend für diese Aufgaben. Die Zahlenfolge $a_0, a_1, a_2, \ldots$ ist definiert durch: $a_0 = 0, a_1 = 1$ und $a_n = -2a_{n-1} + 3a_{n-2}$ für $n > 2$.

a) (6P) Formulieren Sie einen *rekursiven* Algorithmus, der bei Eingabe n (natürliche Zahl) den Wert a_n zurückgibt (nicht druckt!). Zeichnen Sie den Aufrufbaum (Rekursionsbaum) bei Aufruf mit der Eingabe $n = 5$.

b) (6P) Formulieren Sie einen *iterativen*[2] Algorithmus, der bei Eingabe n (natürliche Zahl) den Wert a_n zurückgibt (nicht druckt!).

c) (4P + 2 Zusatzpunkte für Beweis der Formel) Für alle n gilt $a_n = \frac{1-(-3)^n}{4}$. Formulieren Sie aufgrund dessen einen Algorithmus, der bei Eingabe n (natürliche Zahl), den Wert a_n zurückgibt (nicht druckt!). Potenzieren darf dabei *nicht* benutzt werden.

[2]Zwei Möglichkeiten: (1) Feld anlegen, das nach und nach mit den Werten a_0, a_1, a_2, ... gefüllt wird oder (2) lediglich die letzten Werte speichern, daraus den nächsten berechnen und geeignet umspeichern usw.



Name: MatNr.:

1. Allgemeine Grundkenntnisse (I1-ID: mq4g9vq04ih0)

1. Wählen Sie die richtige Antwortkombination aus (1P, bei einer Abweichung 0.5P, sonst 0P):
   - (A) Eine Spezifikation ist eine präzise Beschreibung der erlaubten Ausgaben.
   - (B) Algorithmen beschreiben Handlungsvorschriften (z.B. zur Berechnung eines Werts).
   - (C) Programmentwicklung heißt immer wieder testen, dann compilieren, dann editieren.
   - (D) Compiler laden Maschinencode zusammen mit Klassenbibliotheken in das Laufzeitsystem.
2. Geben Sie die richtige Antwortkombination an (1P, bei einer Abweichung 0.5P, sonst 0P):
   - (A) Der Befehlszähler im von Neumann-Prozessormodell speichert arithmetische Befehle.
   - (B) Interaktive Programme können auf Daten von der Standard-Eingabe reagieren.
   - (C) Die Pipe der bash-Shell ist der Stack der noch nicht verarbeiteten Eingaben.
   - (D) Speicherstellen ist nicht anzusehen, ob sie Zahlen oder Befehle speichern.
3. Geben Sie die richtige Antwortkombination an (1P, bei einer Abweichung 0.5P, sonst 0P):
   - (A) Durch Ausgabeumleitung kann man erzeugte Daten in einer Datei abspeichern.
   - (B) Decodieren bedeutet, eine Nachricht als Information zu interpretieren bzw. zu verstehen.
   - (C) Durch N Hexadezimalziffern werden 4mal soviel Informationen dargestellt wie mit N Binärziffern.
   - (D) Der ASCII-Standard wurde zur Lösung des Jahr-2000-Problems durch Unicode ersetzt.
4. Geben Sie die richtige Antwortkombination an (1P, bei einer Abweichung 0.5P, sonst 0P):
   - (A) Strukturierte Daten sind zerlegbar in kleinere Bestandteile.
   - (B) Gleitkomma-Arithmetik ist schneller als Ganzzahl-Arithmetik.
   - (C) Die JVM interpretiert Bytecodebefehle ohne ihre Syntax zu prüfen.
   - (D) Imperative Programmierung heißt, Daten durch Befehle zu ersetzen.

Loading Web-Font TeX/Main/Regular

2. Einige Java-Grundkenntnisse (I1-ID: ia4a8tq09ih0)

1. Von klein nach groß ordnen (0.25P für jede korrekte Angabe)

   Was kann ineinander enthalten bzw. Teil voneinander sein?
   Ordnen Sie von OBEN nach UNTEN die Begriffe vom "kleinsten" bis zum "größten", d.h. **wenn X Teil von Y sein kann, so muss X *weiter oben* stehen als Y**:

   `Anweisung; Ausdruck; Datei; Java-Archiv; Klasse; Klassenpfad; Literal; Methode`

2. Zahlen summieren (0.5P für jede korrekte Angabe)

   Füllen Sie die Lücken, so dass das Programm mit Zahlen A und B als Kommandozeilenparametern aufgerufen Folgendes leistet:
   Es wird die Summe der *ganzen* Zahlen ausgeben, die zwischen A und B (einschließlich) liegen.

```
class Intervallsumme {
    public static void main(String[] args) {
        ;

        int ;
        while ( ) {

        }
        ;
    }
}
```

Loading Web-Font TeX/Main/Regular

3. Grundlegende Datentypen (I1-ID: lw64f0a15ih0)

1. Wählen Sie die richtige Antwortkombination aus (1P, bei einer Abweichung 0.5P, sonst 0P):
   - (A) Das Tabulatorzeichen ist ein Steuerzeichen.
   - (B) Unicode ist eine Teilmenge von iso8859-1.
   - (C) `"C" + '1'` hat den gleichen Wert wie `'C'+1`.
   - (D) `"C" + 1` ist *kein* erlaubter Java-Ausdruck.
2. Wählen Sie die richtige Antwortkombination aus (1P, bei einer Abweichung 0.5P, sonst 0P):
   - (A) `int` ist ein Ganzzahldatentyp, `char` nicht.
   - (B) `double x = Integer.MAX_VALUE + 1;` ist eine erlaubte Anweisung.
   - (C) Der Wert von `(7 % (5 - 2) == 0) || (7 % (5 + 2) == 0)` ist true.
   - (D) Der Wert von (`1000 + 11/12 == 1000 - 11/12)` ist `false`.
3. Geben Sie die zutreffenden Antworten an (entweder Ausgabewert angeben oder Fehlerart (z.B. Syntaxfehler))(0.5P für jede richtige Angabe):
   - (A) `String s = "Hallo", System.out.println(s.length);`

     Ausgabe?
   - (B) `System.out.println(10e-1f);`

     Ausgabe?
   - (C) `2 * (short) 0xC0CA C01A`

     Typ des Ausdrucks?
   - (D) `double y = Double.MIN_VALUE; System.out.println(y/2);`

     Ausgabe?
   - (E) `int x = Integer.MAX_VALUE, y = Integer.MAX_VALUE--, z = x-y;`

     Wert von `z`?
   - (F) `System.out.println( (double) (1/2));`

     Ausgabe?
4. Wählen Sie die richtige Antwortkombination aus (2P, bei einer Abweichung 1P, sonst 0P):
   - (A) `int[] z = {1}; System.out.println(z.length());` produziert die Ausgabe 1.
   - (B) `int[] z = {1}; System.out.println(z[1]); produziert nicht die Ausgabe 1.`
   - (C) Durch `int[][][][] x = new char[32][32][32][16];` wird 1Mb Stack-Speicher reserviert.
   - (D) Durch `int[] a = {1, 2, 3}, b = new int[3];` werden zwei neue Felder angelegt.

Loading Web-Font TeX/Main/Regular

4. Integer-Arithmetik (I1-ID: zzfd4k81gdg0)

Sei `c` eine `int`-Variable mit dem Wert `Integer.MAX_VALUE` (also 2147483647). Welche Textausgabe erzeugen diese Anweisungen?:

```
System.out.println(2147483646-c); //
System.out.println((-1)*c);       //
System.out.println(c+2);          //
System.out.println(c*2-1);        //
```

Loading Web-Font TeX/Main/Regular

5. Iteration in Feldern (I1-ID: 435dkh80w5h0)

```
int[] p = { 1, 0, 7, 3, 5, 4, 6, 2 };
int[] c = { 0, 0, 0, 0, 0, 0, 0, 0 };
int[] q = { 0, 0, 0, 0, 0, 0, 0, 0 };
for (int i = 0; i < 8; i++)
    c[i] = p[p[i]];
for (int i = 0; i < 8; i++)
    q[p[i]] = i;
```

Tragen Sie die Werte der Feldkomponenten *nach* Ausführung des Java-Fragments ein:

Index	0	1	2	3	4	5	6	7
c	0	1	2	3	4	5	6	7
q								

Loading Web-Font TeX/Main/Regular

6. Felder: Fehler oder …? (I1-ID: xb2c64a1gdg0)

Erzeugt untenstehende Klasse

- einen Compilezeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-):
- einen Laufzeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-):
- eine Ausgabe? Falls ja, so tragen Sie die Ausgabe ein, ansonsten einen Bindestrich (-):

```
 1:  class FehlerOderWie {
 2:      public static void main(String[] args) {
 3:          int[] a = {2,3,5,7,11};
 4:          int[] b = new int[2];
 5:          b = a;
 6:          b[4] = -4;
 7:          if (a[4] >= 1) System.out.println(1);
 8:          else System.out.println(0);
 9:      }
10:  }
```

Loading Web-Font TeX/Main/Regular

7. Statische Methoden (I1-ID: 0f1dmv607ih0)

```
public class StatischeMethoden {
    public static void q1(int n) {
        n = n * n;
    }
    public static void q2(int[] f, int i) {
        f[i] = f[i] * f[i];
    }
    public static void main(String[] args) {
        int n = 2;
        int[] f = { n, n, n};
        for (int i = 0; i < 3; i++) {
            q1(n);
            q2(f,n);
        }
        System.out.println(n);
        System.out.println(f[n]);
    }
}
```

Wie lautet die erste Ausgabezeile?

Wie lautet die zweite Ausgabezeile?

Loading Web-Font TeX/Main/Regular

8. Rekursion (I1-ID: 4mqjzge16ih0)

```
public static void strange(int n) {
    if ( n <= 2) return;
    System.out.print("(");
    strange(n-2);
    strange(n-3);
    System.out.print(")");
}
```

Welche Ausgabe bewirkt der Aufruf `strange(7)`?

Tipp: Rekursionsbaum benutzen

Loading Web-Font TeX/Main/Regular

9. Binäre Suche protokollieren (I1-ID: i5k29e615ih0)

Betrachten Sie folgende Implementierung der binären Suche in einem aufsteigend sortierten Feld:

```
public static int binarysearch(int[] f, int q) {
    int lo = 0, hi = f.length - 1;
    while (lo<=hi)
    {
        int m=lo+(hi-lo)/2;
        if (q<f[m]) {
            System.out.print("L");
            hi=m-1;
        }
        else if (q>f[m]) {
            System.out.print("R");
            lo=m+1;
        }
        else {
            System.out.print("!");
            return m;
        }
    }
    System.out.print("-");
    return -1;
}
```

Hier ein Ausschnitt aus dem Unit-Test.

```
int[] g = { 1, 1, 3, 5, 6, 7, 8, 9, 9, 9, 10, 11, 11, 12, 13, 15};
int s = g[4];
System.out.println(binarysearch(g,s));
```

Welche Ausgabe wird erzeugt?

Loading Web-Font TeX/Main/Regular

10. Stack und Queue in Aktion (I1-ID: vh5gi9b1aih0)

1. Auf einem Stack werden *in gemischter Reihenfolge* 10 Push- und 10 Pop-Operationen ausgeführt. Es ist bekannt, dass die Push-Operationen die Zahlen 0, 1, 2, …, 9 in dieser Reihenfolge einspeichern und dass bei jedem Pop der ausgespeicherte Wert ausgegeben wird. Welche Ausgabelisten sind möglich?
   - (A) 1 2 3 4 5 6 9 8 7 0
   - (B) 0 4 6 5 3 8 1 7 2 9
   - (C) 1 4 7 9 8 6 5 3 0 2
   - (D) 2 1 4 3 6 5 8 7 9 0

   (1P für jede korrekte Angabe)
2. Auf einer Queue werden *in gemischter Reihenfolge* 10 Enqueue- und 10 Dequeue-Operationen ausgeführt. Es ist bekannt, dass die Enqueue-Operationen die Zahlen 0, 1, 2, …, 9 in dieser Reihenfolge einspeichern und dass bei jedem Dequeue der ausgespeicherte Wert ausgegeben wird. Welche Ausgabelisten sind möglich?
   - (E) 4 6 8 7 5 3 2 9 0 1
   - (F) 2 5 6 7 4 8 9 3 1 0
   - (G) 0 1 2 3 4 5 6 7 8 9
   - (H) 4 3 2 1 0 5 6 7 8 9

   (0.25P für jede korrekte Angabe)

Loading Web-Font TeX/Main/Regular

11. Einen Stack kopieren (I1-ID: kw3g7f70y6h0)

Betrachten Sie folgende API von Stacks zum Speichern von `int`-Werten (alle Methoden/Konstruktoren `public`):

`public class Stack`	
`Stack()`	erzeugt einen neuen Stack
`boolean empty()`	testet, ob dieser Stack leer ist
`void push(int i)`	speichert `i` in diesem Stack
`int pop()`	in diesem Stack zuletzt gespeicherter Wert wird *geliefert* und *entfernt*

Stellen Sie die Zeilen für den Rumpf einer weiteren Instanzmethode

```
public Stack copy() {
  // Rumpf (Ihre Aufgabe)
}
```

in richtiger Reihenfolge zusammen, so dass ihr Aufruf einen Stack mit gleichem Inhalt wie dieser liefert (d.h. es soll dann möglich sein, Original und Kopie unabhängig zu benutzen: Einspeichern beim Original ändert nicht den Zustand der Kopie usw.).

Idee: (1) alle Werte aus diesem Stack zunächst aus- und in einen neuen Stack `h` einspeichern, dann (2) von dort alle Werte aus- und sowohl in diesen Stack als auch in neuen Stack `c` einspeichern, schließlich Referenz der Kopie liefern.

```
Stack h = new Stack(); Stack c = new Stack();

while (!this.empty()) {

h.push( this.pop()); }

while (!h.empty()) {

int i = h.pop();

this.push(i); c.push(i);}

return c;
```

Loading Web-Font TeX/Main/Regular

12. OOP-Verständnisfragen (I1-ID: c6j5qfl08ih0)

Frage 1

Wodurch wird Kapselung bei der objektorientierten Programmierung erreicht?

Antwortmöglichkeiten

Durch `public/private/protected` Instanzvariablen und `public/private/protected` Methoden.

Frage 2

Welche der Konzepte

(A) Polymorphie, (B) Vererbung,(C) Kapselung, (D) Überladen

kann man anhand folgender Situationsbeschreibung illustrieren?

- Jedes *Tier* kann einen Laut von sich geben (ein Geräusch machen).
- Jeder *Hund* ist ein *Tier* und kann einen Laut von sich geben, nämlich: Wau Wau!
- Jede *Katze* ist ein *Tier* und kann einen Laut von sich geben, nämlich: Miau!

Loading Web-Font TeX/Main/Regular

13. Vermischtes zu Klassentypen (I1-ID: sy9i2nb0bih0)

1. Objekte verstehen:

   ```
   public class Ninja {
       private int honor;
       public Ninja( int s) {
           honor = s;
       }
   }
   ```

   Zwei `Ninja`-Objekte werden instanziiert:

   ```
   Ninja a = new Ninja(20), b = new Ninja(20);
   ```

   Welchen Wert hat nun der Ausdruck `a == b`?

2. Fehler oder …?

   ```
   public class SillyTestClass {
       private int x, y;
       public SillyTestClass(int x, int y) {
           System.out.println(y);
       }
       public SillyTestClass(String string1, String string2) {
           System.out.println(string1 + string2);
       }
       public static void main (String[] args) {
           SillyTestClass app = new SillyTestClass(2, "Tests");
       }
   }
   ```

   Welche Aussage charakterisiert diesen Code am besten?

3. Begriffsverständnis:

   In dem Menü stehen einige Aussagen zu Klassenelementen, darunter *eine einzige falsche.*
   Wählen Sie genau die NICHT ZUTREFFENDE Aussage aus
   `In einer Klasse darf es Methoden geben\, die den gleichen Namen und gleiche Argumenttyplisten haben aber unterschiedliche Ergebnistypen.`

Loading Web-Font TeX/Main/Regular

14. Eine Bibliothek für Permutationen (I1-ID: 8834zrd08ih0)

Permutationen entsprechen `int`-Feldern einer Länge n, die die Werte 0,1,...,n-1 speichern (in beliebiger Reihenfolge).

Definition

1. Das *Quadrat* der Permutation `p`, ist die Permutation `q` mit: `q[i]` ist gleich `p[p[i]]` für alle Indexwerte `i`.
2. Die *Unordnung* der Permutation `p` ist die Anzahl der Indexpaare `(i,j)` mit `i<j` und `p[i]>p[j]`. Zum Beispiel hat (2,0,1) Unordnung 2.

Aufgabe

Schreiben Sie eine Klasse `Permutation` mit den *statischen* Methoden `quadrat(..)` und `unordnung(..)`, die aus jeder Clientklasse aufgerufen werden können und Folgendes leisten:

1. `quadrat(p)` ermittelt und liefert das Quadrat der Permutation `p`
2. `unordnung(p)` ermittelt und liefert die Unordnung von

`p`.

Es ist jeweils gesichert, dass `p` wirklich eine Permutation ist - nicht separat prüfen!

15. Eine einfache Klasse (I1-ID: 3g3j2i70x5h0)

Programmieren Sie eine einfache Klasse `Wein` mit der Angaben zu Weinflaschen abgespeichert werden können.
Jeder Wein zeichnet sich durch seine unveränderliche Sorte aus (z.B. "2001er Goldtropfen"), und durch seinen Preis (z.B. 13,50€).
Die folgenden Operationen sollen durch Methoden/Konstruktoren ermöglicht werden (*wenn nicht anders angegeben*: aufrufbar aus beliebigen Clientklassen):

- Erzeugen einer `Wein`-Instanz unter Angabe der Sorte
- Erzeugen einer `Wein`-Instanz unter Angabe der Sorte und des Preises
- Setzen des Preises einer Instanz (*nur von `Wein`-Methoden aus*),
- Abfragen des Preises einer Instanz
- Abfragen der Anzahl der erzeugten Instanzen und der Summe ihrer Preise.

VERWENDEN SIE FÜR DIE DATEN BITTE DIESE BEZEICHNER:
`sorte`, `preis`, `anzahl`, `gesamt`

Zum Einrücken bitte unbedingt die Tabulatortaste anstelle Leerzeichen benutzen (siehe unten).

Loading Web-Font TeX/Main/Regular

Wiederholungsklausur Informatik I
Sommersemester 2011
Prof. Dr. C. Damm
08. April 2011
von 08:00+x bis 09:30+x Uhr

Vorname:

Nachname:

Matrikelnummer:

Es sind **keine Hilfsmittel** (Skript, Taschenrechner etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten **Blätter** für Ihre Antworten. Schreiben Sie mit blauem/schwarzem Kugelschreiber, Füller etc.. Bleistift ist nicht erlaubt. Unlesbare Antworten werden nicht gewertet. Algorithmen sind in Java-Code zu notieren, sofern keine Alternativen genannt sind.

Viel Erfolg!

	Max. Punkte	Erreichte Punkte
Aufgabe 1 (Auswahlaufgaben)	15	
Aufgabe 2 (Elementare und strukturierte Daten)	15	
Aufgabe 3 (Allgemeine Kenntnisfragen)	15	
Aufgabe 4 (Rekursion und Iteration)	15	
Aufgabe 5 (Algorithmen-Entwurf und Analyse)	15+2	
Aufgabe 6 (Vererbung und dynamische Bindung)	15	
Summe	90+2	

Note:

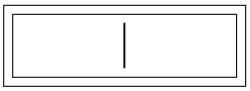
Name: MatNr.:

Aufgabe 1 (Auswahlaufgaben [15 Punkte])

Jeweils nur eine Antwort ankreuzen (**X**). Mehrere Kreuze ergeben 0P für die Teilaufgabe.

- `class A {void tuWas(){..}.. {void tuWas(int x){..}}`
  Was ist richtig?
  1. `tuWas` wurde erweitert.
  2. `tuWas` wurde überladen.
  3. `tuWas` wurde überschrieben.
- Klassen ...
  1. ... sind Mengen von Objekten.
  2. ... sind ausführbare Quellcodeeinheiten.
  3. ... geben die Struktur von Objekten vor.
- `a && !b` ist gleichbedeutend mit ...
  1. weder `a` noch `b`
  2. aus `a` folgt `b`
  3. es gilt nicht: aus `a` folgt `b`
- Eine Schleifeninvariante ist ...
  1. ... eine Aussage.
  2. ... eine Spezifikation.
  3. ... immer eindeutig bestimmt.
- Der Garbage-Collector ...
  1. ... entfernt fehlerhaften Code.
  2. ... gibt Speicherplatz frei.
  3. ... läuft parallel zum Compiler.
- Ein vollständiger binärer Baum ...
  1. ... hat noch alle Blätter.
  2. hat eine Wurzel.
  3. ... hat Tiefe 2^k.
- Mergesort ist ...
  1. ... ein in-place-Verfahren.
  2. ... dynamisch programmiert.
  3. ... asymptotisch Laufzeit-optimal.
- Die von-Neumann-Architektur ...
  1. ... sieht Programm und Daten im gleichen Speicher vor.
  2. ... ermöglicht Programmierung ohne bedingte Sprünge.
  3. ... liegt dem EVA-Prinzip zugrunde.

- Der Aktivierungssatz ...
  1. ... beschreibt die Struktur des Laufzeitstapels.
  2. ... wurde von Leibniz bewiesen.
  3. ... hat für jeden Aufruf einer Methode die gleiche Struktur.
- Durch Stellenwertsysteme ...
  1. ... wird Rechnen mit Zahlen auf das Rechnen mit Ziffern zurückgeführt.
  2. ... ist die Syntax jeder Stelle vorgeschrieben.
  3. ... sind auch nicht druckbare Zeichen im Unicode.
- Aufrufe von funktionalen Methoden ...
  1. ... können überall da stehen, wo ein Ausdruck erlaubt ist.
  2. ... dürfen nicht `null` liefern
  3. ... sind nur außerhalb von Konstruktoren erlaubt.
- Eine Queue ...
  1. ... ist eine doppelt verkettete Liste.
  2. ... hat im schlechtesten Fall logarithmische Zugriffszeit.
  3. ... verwirklicht auch das Prinzip last in, last out.
- Mit `final` kann man ...
  1. ... Klassen ohne Konstruktor kennzeichnen.
  2. ... symbolische Konstanten definieren.
  3. ... nur Methoden markieren, die `return` enthalten.
- Binäre Suche ...
  1. ... benötigt auf Feldern der Länge n mindestens $\mathcal{O}(n \log n)$ Schritte.
  2. ... ist das schnellste Sortierverfahren.
  3. ... ist nur unter bestimmten Bedingungen auf Felder anwendbar.
- Compilezeitfehler sind Fehler ...
  1. ... die die Compilezeit verschieben.
  2. ... die den Compiler zum Absturz bringen.
  3. ... in einem Quelltext.



Name: MatNr.:

Aufgabe 2 (Elementare und strukturierte Daten [15 Punkte])

a) (3P) Der hypothetische Datentyp `gint` stellt ganze Zahlen mit 6-Bit-Zweierkomplement dar. Geben Sie folgende Werte dezimal an:

- größter darstellbarer `gint`-Wert
- kleinster darstellbarer `gint`-Wert.

b) (3P) Wandeln Sie den Dezimalbruch $(0.203125)_{10}$ in einen Binärbruch um.

c) (6P) Ergänzen Sie diese Tabelle von `byte`-Werten (= „8-Bit-Ganzzahlen“).

Bitte deutlich schreiben, Nebenrechnung nur unten oder auf Blatt rechts!

dezimal	binär (mit Zweierkompl.)	hexadezimal
13		
	$(00101011)_{\overline{2}}$	
		$(1E)_{16}$

d) (3P) Betrachten Sie folgende Definition:

```
int[] f = new int[13];
```

Die Anfangsadresse von f sei `0x1200A011`. Welche Adresse hat `f[7]`? Geben Sie den Rechenweg an.

Name: MatNr.:

Aufgabe 3 (Allgemeine Kenntnisfragen [15 Punkte])
3P pro Aufgabenteil. Antworten Sie kurz und prägnant.

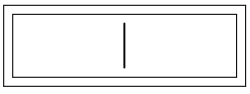
1. `static int bla(int x){int z = 21; return x+1;}` wird in `main` mit `int z = bla(7);` aufgerufen. Was wird im Aktivierungssatz gespeichert?

2. Welche Elemente einer Java-Klassendefinition werden durch das Schlüsselwort `static` gekennzeichnet? Was bedeutet `private`?

3. Nennen Sie drei Funktionseinheiten der von Neumann-Rechnerarchitektur.

4. Wodurch ist imperative Programmierung gekennzeichnet?

5. Die Tiefe eines Baums ist die maximale Kantenzahl längs eines Wurzel↦Blatt-Weges. Wieviele Knoten hat ein Tiefe-n-Binärbaum mindestens, wieviel höchstens?



Name: MatNr.:

Aufgabe 4 (Rekursion und Iteration [15 Punkte])

1.(5P) Die Java-Methode `static String rev(String s)` liefert den „Umkehrstring“ von `s` (enthält die Zeichen von `s` in umgekehrter Reihenfolge, Beispiel `rev("Maus")` liefert `"suaM"`).
Programmieren Sie die Methode rekursiv mit der Rekursionsidee

$$\texttt{rev}(\texttt{s}) = \begin{cases} \texttt{s} & \text{, falls} |\texttt{s}| \leq 1 \\ \texttt{rev}(\texttt{s}_{1..n-1}) + \texttt{s}_0 & \text{sonst.} \end{cases}$$

Dabei stehen $|\texttt{s}|$ für die Länge und $\texttt{s}_0$ für das Anfangszeichen von `s`, $\texttt{s}_{1..n-1}$ für den String ohne Anfangszeichen und + für die Verkettung (z.B. $ab{+}a = aba$).
- Formulieren Sie nur die Methode, keine Klasse!
- Benutzen Sie `length`, `charAt`, `substring` (s.u.) und den Operator `+`:

```
// Laenge von s = Anzahl der Zeichen darin (Bsp. "Maus" hat Länge 4)
int len = s.length();

// erstes Zeichen von s:
char c = s.charAt(0);

// alle Zeichen von s vom einschliesslich zweiten bis letzten:
String rest = s.substring(1);
```

2. (5P) Schreiben Sie eine iterative Methode `REV`, die das Gleiche leistet wie `rev`.
- Formulieren Sie nur die Methode, keine Klasse!
- Benutzen Sie `length` und `charAt` (s.o.) und den Operator `+`:

3. (5P) Betrachten Sie folgende Methode

```
static int quasi(int b, int e) {
  int p = 1;
  if (e > 0) {
    p = quasi(b,e/2) - 1;
    p = p*p + 1;
    if ( e % 2 != 0)
        p *= b;
    }
  return p;
}
```

Welchen Wert liefert der Aufruf `quasi(2,5);` ? Die richtige Lösung zählt nur, wenn auch der Rechenweg angegeben wird (sonst 0P).

Name: MatNr.:

Aufgabe 5 (Algorithmen-Entwurf und Analyse [15+2 Punkte])
Gegeben ist eine Zeichenfolge, z.B. "`cwNHOpoQMMNIsQnUaZbAsG`"
gespeichert in einem Feld `z[0]...z[k-1]`.

a) (6P + (2 Zusatzpunkte für asymptotisch optimalen Algorithmus)) Geben Sie einen Algorithmus an, der berechnet, wie lang die längste Teilfolge ist, die nur aus Großbuchstaben besteht (im Beispiel würde 5 berechnet werden, denn die Folge hat keine längere Großbuchstabenfolge als `QMMNI`). Geben Sie den Algorithmus in "strukturierter Prosa"[2] an (kein Java!).

[2] knapp & übersichtlich, Beispiel: Berechnung des Mittelwerts eingegebener Zahlen

1. Initialisierung: Setze Anzahl n und Summe s auf 0.
2. Ende erreicht?: Ist keine Zahl mehr zu lesen, weiter bei 4.
3. Einlesen und Verarbeiten einer Zahl:
   - Lies nächste Eingabezahl z.
   - Erhöhe a um 1 und s um z.
   - Weiter bei 2.
4. Berechnung und Beendigung: Falls $a > 0$, liefere Mittelwert s/a zurück, sonst erzeuge Fehlermeldung

b) (5P) Geben Sie eine möglichst gute Abschätzung Ihres Algorithmus in $\mathcal{O}$-Notation an (mit kurzer Begründung).

c) (4P) Algorithmus `clever` zur Lösung von a) ermittelt zunächst die Feldlänge und liest dann beginnend bei Index 0 fortlaufend die Feldeinträge (und stellt interne Berechnungen beliebiger Art an). Das Lesen wird aber sofort unterbrochen, wenn die Antwort feststeht.

Geben Sie Zeichenketten (1), (2) und (3) der Länge 8 wie folgt an: (1) `clever` liest bis zum Ende. (2) `clever` liest nicht bis zum Ende und das Resultat ist 4. (3) `clever` liest nicht bis zum Ende und das Resultat ist 2.

Begründen Sie jeweils kurz.

Name: MatNr.:

Aufgabe 6 (Vererbung und dynamische Bindung [15 Punkte])

1. (8P) Geben Sie unten die beim Aufruf `java PersonTest` entstehenden Ausgaben an (Nummerierung: siehe Kommentar der Zeilen 7–11).

```
public class PersonTest {
    public static void main(String args[]) {
        Person jemand = new Person("Tom", "Muster", false, "Dr.");
        Heiliger martin = new Heiliger("Martin", "", false);
        Heiliger anna = new Heiliger("Anna", "", true);
        Herrscher kw = new Herrscher("Kalle", "Wirsch", false);
        System.out.println(jemand.bezeichnung()); // [1]
        System.out.println(jemand.anrede()); // [2]
        System.out.println(martin.bezeichnung()); // [3]
        System.out.println(anna.anrede()); // [4]
        System.out.println(kw.anrede()); // [5]
    }
}
class Person {
    private String vorname, nachname, titel;
    private boolean weiblich;
    public Person(String v, String n, boolean w, String t) {
        vorname = v; nachname = n; titel = t; weiblich = w;
    }
    public String vorname() { return vorname; }
    public String nachname() { return nachname; }
    public String titel() { return titel; }
    public boolean weiblich() { return weiblich; }
    public String name() { return vorname + " " + nachname; }
    public String bezeichnung() { return titel() + " " + name(); }
    public String anrede() {
        if (weiblich)
            return "Sehr geehrte Frau " + bezeichnung();
        else
            return "Sehr geehrter Herr " + bezeichnung();
    }
}
class Heiliger extends Person {
    public Heiliger(String v, String n, boolean w) {
        super(v, n, w, "St.");
    }
    public String bezeichnung() {
        if (weiblich())
            return "Heilige " + titel() + " " + vorname();
        else
            return "Heiliger " + titel() + " " + vorname();
    }
}
class Herrscher extends Person {
    public Herrscher(String v, String n, boolean w) {
        super(v, n, w, "");
    }
    public String titel() {return "Hoheit";}
}
```

Ausgabe 1:
Ausgabe 2:
Ausgabe 3:
Ausgabe 4:
Ausgabe 5:

2. (7P) Vervollständigen Sie anhand der Klassen `Angestellter` und `Chef` (im gleichen Verzeichnis) die nachfolgende Tabelle mit den Einträgen "korrekt" oder "Compilezeitfehler" oder "Laufzeitfehler", je nachdem, ob die Anweisung(en) in der linken Spalte unproblematisch sind oder welche Art Fehler verursacht wird.

```
public class Angestellter {
 protected int alter;
 protected String name;
 public Angestellter(String name, int alter) {
  this.name = name;
  this.alter = alter;}
 public String toString() {
  return name + " (" + alter + ")";}
class Chef extends Angestellter {
 protected String abteilung;
 public Chef(String name, int alter, String abteilung) {
  super(name,alter);
  this.abteilung = abteilung;}
 public String toString() {
  String s = super.toString();
  return s + " leitet " + abteilung; }
}
```

Anweisung(en)	Eintrag

`String s, abt;`
`Angestellter a=new Angestellter("Berthold",30);`
`Chef b=new Chef("Steinke",37,"Schadensregulierung");`	

`s = a.name;`	
`abt = a.abteilung;`	
`abt = ((Chef) a).abteilung;`	
`b = a;`	
`a = b;`	
`a = b;` `abt = a.abteilung;`	
`a = b;` `abt = ((Chef) a).abteilung;`	
`System.out.println(((Angestellter) a).toString());`	
`System.out.println(((Angestellter) b).toString());`	

Name: MatNr.:

Probeklausur Informatik I

Erklärungen

- bei der E-Klausur werden die Aufgaben ILIAS-typisch auf einzelnen Seiten angezeigt (gut lesbar)
- außerdem gibt es bei der E-Klausur diese Möglichkeiten:
  - Vergrößerung des Texts mit Strg-Mausrad
  - Eingabefenster für Freitexte an Ecke rechts-unten aufziehbar
  - Antwortoptionen (nachfolgend in blauer Schrift) werden in Drop-Down-Menüs angezeigt
- bei der Klausur wird ein leeres Blatt für Notizen bereitgelegt, Notizen werden nicht bewertet
- im hier vorliegenden Layout sind Texte mit sehr langen Zeilen abgeschnitten, hier die Ergänzungen (fehlender/unlesbarer Text kursiv)

  Aufgabe 2.2 ... leistet: Es *wird die Summe der ganzen* Zahlen ...

  Aufgabe 12 ... zutreffenden A *ntworten aus (jeweils 1.5P für* die ...

  Aufgabe 13.1 ... weitere as *ymptotische Analyse* geeignet?

  Aufgabe 13.2 ... sehen *typischerweise so aus:*

  Aufgabe 14 ... vorkomm *t*

  ... Fol *gendes leisten:*

  ... Bereichsbedingung erf *üllt ist, dann mit zweiter Schleife* in einem Hilfsfeld ...

  ... Feld anl *egen und in Schleife die* Komponenten ...
- zu Aufgaben 14/15 gehören Freitextfelder, die hier aber nicht zu sehen sind
- Punkte

Aufgabe	erreichbar
1	4
2	5
3	6
4	5
5	6
6	4
7	6
8	6
9	6
10	6
11	2
12	3
13	6
14	12
15	13
Summe	90

Liste der Antworten

1. Allgemeine Grundkenntnisse (I1-ID: rjyb60x0u5h0)

1. Geben Sie die richtige Antwortkombination an (1P für die richtige/0P für jede falsche Kombination):
   - (A) Spezifikationen bestehen aus einer präzisen Beschreibung der erlaubten Eingaben.
   - (B) Algorithmen sind endliche Vorschriften, die Rechenabläufe schrittweise beschreiben.
   - (C) Compiler übersetzen Quellcode erst in Assembler, dann in Unix-Code.
   - (D) Das Java-Programm ist auf dem System lauffähig, wenn darauf der entsprechende Compiler installiert ist.

2. Geben Sie die richtige Antwortkombination an (1P für die richtige/0P für jede falsche Kombination):
   - (A) Bei der Entwicklung eines Programms muss nach jedem Compilieren editiert werden.
   - (B) Interaktive Programme können Daten von der Standard-Eingabe erhalten.
   - (C) Am CMD-Flag ist erkennbar, ob die Speicherzelle eine Zahl oder einen Befehl enthält.
   - (D) Eine *Pipe* funktioniert im wesentlichen wie eine Queue.

3. Geben Sie die richtige Antwortkombination an (1P für die richtige/0P für jede falsche Kombination):
   - (A) Durch Eingabeumleitung kann man gespeicherte Daten an ein interaktives Kommando leiten.
   - (B) Informationsverarbeitung zerfällt in die Schritte Codierung, Datenverarbeitung und Decodierung.
   - (C) Mit 4 Hexadezimalziffern kann man mehr als 130 000 Werte darstellen.
   - (D) Der Bruch 4/5=0.8 kann exakt als endlicher Dualbruch dargestellt werden.

4. Geben Sie die richtige Antwortkombination an (1P für die richtige/0P für jede falsche Kombination):
   - (A) Ganzzahlarithmetik ist normalerweise schneller als Gleitkomma-Arithmetik.
   - (B) Strukturierte Programmierung ist die Programmierung mit strukturierten Daten.
   - (C) Rekursion benötigt mehr Speicher auf dem Stack als Iteration.
   - (D) Die JVM interpretiert jeden Bytecodebefehl und prüft dessen Syntax.

2. Einige Java-Grundkenntnisse (I1-ID: u1j7g25165h0)

1. Ordnen Sie Begriffe zeitlich (0.25P für jede korrekte Position)

2. Füllen Sie die Lücken, so dass das Programm mit zwei beliebigen Zahlwerten A und B als Kommandozeilenparametern aufgerufen Folgendes leistet: Es Zahlen ausgeben, die innerhalb des *abgeschlossenen* Intervalls [A,B] liegen. (0.25P für jede korrekte Angabe)

```
public class Intervallsumme {
    public static void main(String[] args) {
            ,
            min =                   (args[0]),
            max =                   (args[1]);
        int        ;
        while (          ) {
             ;
             ;
        }

    }
}
```

3. Wieviele Bytes beansprucht eine Variable?
   - `int`:
   - `char`:
   - `float`:
   - `boolean`:

   (0.25P für jede richtige Antwort)

3. Grundlegende Datentypen (I1-ID: ksu13v0075h0)

1. Geben Sie die richtige Antwortkombination an (1P für die richtige/0P für jede falsche Kombination):
   - (A) Das Leerzeichen ist ein druckbares Zeichen.
   - (B) iso8859-1 ist eine Teilmenge von ASCII.
   - (C) `'C'++` liefert das gleiche Ergebnis wie `'C'+1`.
   - (D) `"C" + 1` ist *kein* erlaubter Java-Ausdruck.

2. Geben Sie die richtige Antwortkombination an (1P für die richtige/0P für jede falsche Kombination):
   - (A) `int` ist ein Ganzzahldatentyp, `char` nicht.
   - (B) `int x = Integer.MAX_VALUE + 1;` ist eine erlaubte Anweisung.
   - (C) Der Wert von `(( 3 < 7 ) && ((17 + 4) % (12 - 5)) != 0)` ist `false`.
   - (D) Der Wert von `( 3 + 1/3 == 3 - 1/3)` ist `false`.

3. Geben Sie die zutreffenden Antworten an:

- (A) `String s = "Hallo", System.out.println(s.length);`

  Ausgabe?

- (B) `System.out.println(10e-1f);`

  Ausgabe?

- (C) `2 * (short) 0xCafe`

  Typ des Ausdrucks?

- (D) `double y = Double.MIN_VALUE; System.out.println(y/2);`

  Ausgabe?

- (E) `int x = Integer.MAX_VALUE, y = Integer.MAX_VALUE++, z = x+y;`

  Wert von `z`?

- (F) `System.out.println( (double) (1/2));`

  Ausgabe?

4. Geben Sie die richtige Antwortkombination an (1P für die richtige/0P für jede falsche Kombination):
   - (A) `int[] z = {1}; System.out.println(z);` produziert die Ausgabe 1.
   - (B) `length` ist ein `final`-Attribut von Feldern.
   - (C) Durch `int[] a = {1, 2, 3}, b = new int[];` werden zwei neue Felder angelegt.
   - (D) Durch `int[][][][] x = new char[32][32][32][16];` wird etwa 1Mb Heap-Speicher reserviert.

4. Integer-Arithmetik (I1-ID: 5wa3il81gdg0)

Sei `d` eine `int`-Variable mit dem Wert `Integer.MAX_VALUE` (also 2147483647). Welche Textausgabe erzeugen diese Anweisungen?:

```
1:  System.out.println(-d);     //
2:  System.out.println(d-1);    //
3:  System.out.println(1+2*d);  //
4:  System.out.println(-(2*d)); //
```

Achten Sie darauf, dass Ihre Antworten NUR Dezimalziffern enthalten (sowie bei Bedarf Vorzeichen)!

5. Iteration in Feldern (I1-ID: t75bm8e005h0)

```
int[] p = { 1, 4, 0, 3, 6, 2, 7, 5 };
int[] c = { 0, 0, 0, 0, 0, 0, 0, 0 };
int[] q = { 0, 0, 0, 0, 0, 0, 0, 0 };
for (int i = 0; i < 8; i++)
    c[p[i]] = c[p[i]] + 1;
for (int i = 0; i < 8; i++)
    q[p[i]] = i;
```

Tragen Sie die Werte der Feldkomponenten *nach* Ausführung des Java-Fragments ein:

Index	0	1	2	3	4	5	6	7
`c`								
`q`								

6. Felder: Fehler oder...? (I1-ID: o2vav4a1gdg0)

Erzeugt untenstehende Klasse

- einen Compilezeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-):
- einen Laufzeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-):
- eine Ausgabe? Falls ja, so tragen Sie Ausgabe ein, ansonsten einen Bindestrich (-):

```
1: class FehlerOderKeiner {
2:     public static void main(String[] args) {
3:         int[] a = {2,3,5,7,11}; int[] b = { a[2], a[2]};
4:         b = a;
5:         b[1] = b[0];
6:         System.out.println(a[0] != a[1]);
7:     }
8: }
```

7. Rekursion (I1-ID: qft7dic1y4h0)

Betrachten Sie folgende Klasse:

```
 1: class Myst {
 2:     static void ery(int n) {
 3:         if (n == 0)
 4:             return;
 5:         ery( n-1);
 6:         A();
 7:         yre( n-1);
 8:     }
 9:     static void yre(int n) {
10:         if (n == 0)
11:             return;
12:         ery( n-1);
13:         B();
14:         yre( n-1);
15:     }
16:     static void A() {
17:         System.out.print("A");
18:     }
19:     static void B() {
20:         System.out.print("B");
21:     }
```

```
22:     public static void main(String[] args) {
23:         int n = Integer.parseInt(args[0]);
24:         ery(n);
25:         System.out.println();
26:     }
27: }
```

1. Geben Sie die ersten 8 Zeichen der Ausgabe zum Aufruf `java Myst 4` an:

2. Aus wievielen Zeichen besteht der Ausgabestring beim Aufruf `java Myst 7` (ohne den abschließenden Zeilenwechsel durch Zeile 25)?

3. Wird Zeile 14 durch `yre( n-2);` ersetzt, so führt beispielsweise der Aufruf `java Myst 4` zu einem Fehler.
   Es genügt, ein einzelnes Zeichen in einer *anderen* Zeile zu ändern, um diesen Fehler nicht mehr hervorzurufen. Um welche Zeile handelt es sich?

4. Werde Zeile 12 durch `ery( n-2);` und zusätzlich Zeile 14 durch `yre( n-2)` ersetzt.
   Geben Sie das maximale Argument `n` an, bei dem der Aufruf `java Myst n` *nicht* zu einem Fehler führt:

8. Stack und Queue in Aktion (I1-ID: 8303049035h0)

1. Auf einem Stack werden *in gemischter Reihenfolge* 10 Push- und 10 Pop-Operationen ausgeführt. Es ist bekannt, dass die Push-Operationen die Zahler
   Reihenfolge einspeichern und dass bei jedem Pop der ausgespeicherte Wert ausgegeben wird. Welche Ausgabelisten sind möglich?

   Wählen Sie aus den beiden Blöcken jeweils die zutreffende Antwort aus (jeweils 2P für die richtige Kombination/0P für jede andere).

   - (A) 1 2 3 4 5 6 9 8 7 0
   - (B) 0 4 6 5 3 8 1 7 2 9

   - (C) 1 4 7 9 8 6 5 3 0 2
   - (D) 2 1 4 3 6 5 8 7 9 0

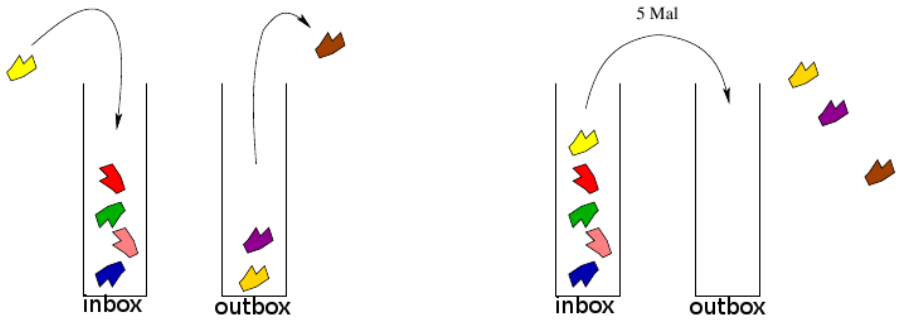
2. Auf einer Queue werden *in gemischter Reihenfolge* 10 Enqueue- und 10 Dequeue-Operationen ausgeführt. Es ist bekannt, dass die Enqueue-Operation
   dieser Reihenfolge einspeichern und dass bei jedem Dequeue der ausgespeicherte Wert ausgegeben wird. Welche Ausgabelisten sind möglich?

   Wählen Sie aus den beiden Blöcken jeweils die zutreffende Antwort aus (jeweils 1P für die richtige Kombination/0P für jede andere).

   - (E) 4 6 8 7 5 3 2 9 0 1
   - (F) 2 5 6 7 4 8 9 3 1 0

   - (G) 0 1 2 3 4 5 6 7 8 9
   - (H) 4 3 2 1 0 5 6 7 8 9

9. Queue aus zwei Stacks bauen (I1-ID: a4g40y31x4h0)



Mit zwei Stacks kann man eine Queue implementieren:

- `enqueue(...)` entspricht Einspeichern in `inbox`
- `dequeue()` entspricht Ausspeichern aus `outbox`.

Unter Verwendung dieser API

`public class Stack`	
`boolean empty()`	testet, ob dieser Stack leer ist
`void push(String s)`	speichert `s` in diesem Stack
`String pop()`	liefert zuletzt gespeicherten String aus diesem Stack und entfernt ihn

sähe die entsprechende Klasse z.B. so aus:

```
public class ZSQueue { // Zwei-Stack-Queue
    private Stack inbox, outbox;
    public ZSQueue() { ... }
    public boolean empty() { ... }        // Leerheitstest
    public void enqueue(String s) { ... } // einspeichern
    public String dequeue() {             // ausspeichern und liefern
        // Rumpf (Ihre Aufgabe)
    }
}
```

Stellen Sie die Zeilen für den Rumpf der Methode `dequeue()` richtiger Reihenfolge zusammen:

- ist die Queue leer, ist Ausspeichern nicht möglich!
- teste ob `outbox` leer ist und fülle in diesem Fall den kompletten Inhalt von `inbox` um in `outbox`, dann ein Element aus `outbox` ausspeichern und liefern.

10. Vermischtes zu Klassentypen (I1-ID: t5k8w9h0frg0)

1. Objekte verstehen:

```
public class Spieler {
    private int score;
    public Spieler( int s) {
        this.score = s;
    }
}
```

Zwei `Spieler`-Objekte werden instanziiert:

```
Spieler a = new Spieler(50), b = new Spieler(50);
```

Welchen Wert hat nun der Ausdruck a == b?

2. Fremden Code deuten:

```
1:  public class FirstBpp extends hepno.users.Frame {
2:      private hepno.users.Wring wrags;
3:      public FirstBpp() {
4:          wrags = new hepno.users.Wring();
5:      }
6:      public static void main(String[] args) {
7:          FirstBpp app = new FirstBpp();
8:      }
9:  }
```

Was bewirkt Zeile 2?:

3. Statische Methoden und Instanzmethoden:

```
public class Klasse {
    private final int zahl = 10;
    double a() {
        System.out.println(zahl);
        return c();
    }
    void b() {
        System.out.println(this);
    }
    double c() {
        double r = Math.random();
        System.out.println(r);
        return r;
    }
    void d() {
        a();
        a();
    }
    int e() {
        return zahl;
    }
}
```

Welche der obigen Methoden ist ihrer Natur nach eher eine statische Methode (und sollte daher mit `static` versehen werden)?

4. Fehlersuche:

```
public class Test {
    private int testCount;
    public static int getCount(){
        return testCount;
    }
    public Test(){
        testCount++;
    }
}
```

Obiger Code verursacht einen Compilezeitfehler. Was ist das Problem?

5. Tracing:
Hinweis: Beachten Sie, dass man über Objektreferenzen auch auf Klassenvariablen zugreifen kann.

```
public class Tset {
    static int tsetCount = 0;
    public Tset(){
        tsetCount++;
    }
    public static void main(String [] args){
        Tset t1 = new Tset();
        System.out.print(t1.tsetCount + ", ");
        Tset t2 = new Tset();
        System.out.print(t1.tsetCount + " "+ t2.tsetCount + ", ");
```

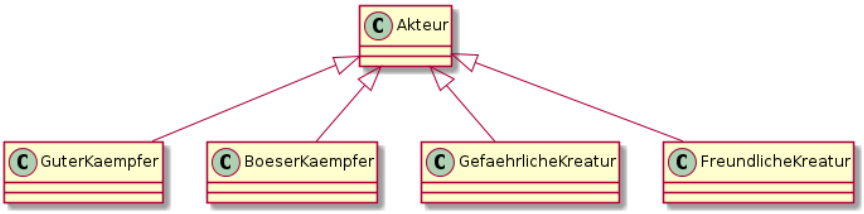
```
            Tset t3 = new Tset();
            System.out.print(t1.tsetCount+ " "+ t2.tsetCount+ " "+ t3.tsetCount);
            System.out.println();
        }
    }
```

Welche Ausgabe produziert der Code?

6. Begriffsverständnis: Wählen Sie aus dem Menü die einzige Aussage aus, die NICHT korrekt ist.

11. OOP-Entwurf (I1-ID: lf9g4np0qtg0)

In einer Spiele-Software gibt es folgende Hierarchie von Java-Klassen, die die verschiedenen Sorten von Akteuren modelliert:



Die (hier nicht gezeigte) Klasse `Spiel` benutzt die zugreifbaren Methoden obiger Klassen, um den Ablauf zu gestalten.

Wie ist (unter Erhalt der gezeigten Klassenbeziehungen) folgende Zusatzanforderung am besten zu modellieren?
Alle Kaempfer-Objekte aber keine Kreatur-Objekte sollen mit einer Methode benutzeWaffe() ausgestattet werden.

- Die Methode `benutzeWaffe()` nur in `GuterKaempfer` und in `BoeserKaempfer` wie benötigt implementieren.
- `benutzeWaffe()` mit leerem Rumpf in `Akteur` implementieren, in `GuterKaempfer` und `BoeserKaempfer` geeignet überschreiben.
- `benutzeWaffe()` in `Akteur` implementieren, in `GefaerlicheKreatur` und `FreundlicheKreatur` mit `private` kennzeichnen.
- Interface `Kaempfer` mit `benutzeWaffe()` definieren und durch `GuterKaempfer` und `BoeserKaempfer` implementieren lassen.
- Weitere Klasse `Kaempfer` schreiben, die `benutzeWaffe()` implementiert, `GuterKaempfer`, `BoeserKaempfer` davon erben lassen.

12. OOP-Verständnisfragen (I1-ID: fnkdcz9075h0)

Sei p eine Referenzvariable vom Typ X. Was kann alles der dynamische Typ von p sein? Wählen Sie aus den beiden Blöcken unten jeweils die zutreffenden Ar die richtige Kombination/0P für jede andere).

- (A) eine Erweiterungsklasse von X (falls X eine Klasse ist)
- (B) ein elementarer Datentyp (falls X die zugehörige Wrapper-Klasse ist)
- (C) eine Klasse, die X implementiert (falls X ein Interface ist)

- (D) eine Klasse, von der X direkt oder indirekt erbt (falls X eine Klasse ist)
- (E) ein Interface, das X als abstrakte Klasse erweitert (falls X eine Klasse ist)
- (F) eine Erweiterungsklasse einer Klasse, die X implementiert (falls X ein Interface ist)

13. Analyse von Algorithmen (I1-ID: 7o9glw3115h0)

1. Betrachten Sie folgenden Ausschnitt aus einem größeren Programm:

```
1: for (int i = 0; i < N; i++)
2:     for (int j = i+1; j < N; j++)
3:         for (int k = j+1; k < N; k++)
4:             verarbeite(daten);
```

Die Ausführungszahl $V(N)$ von Zeile 4 bei Problemgröße N soll untersucht werden. Welcher der folgenden Ausdrücke ist am besten für die weitere as geeignet?

2. Eine Implementation wird zahlreichen Verdopplungstests unterzogen, mit zufällig generierten Eingabedaten. Die Ergebnisse — alle sehr ähnlich — sehe

Problemgröße N	1024	2048	4096	8192	16384	32768	65536	131072	262144	524288	1048576	2097152	4194304	8388608
Zeit (s)	0.00	0.01	0.01	0.02	0.03	0.06	0.14	0.30	0.65	1.24	2.46	4.98	10.02	19.94

Außerdem kann man nachweisen, dass die Ausführungszahl der Kernoperationen bei spezieller Struktur der Eingabe linear mit der Problemgröße N wä

Welche dieser Hypothesen über das Laufzeitverhalten erscheint dann am plausibelsten?

3. Welche dieser Aussagen trifft NICHT zu?

14. Eine Bibliothek für Permutationen (I1-ID: l120mlc005h0)

Definition

1. Wir nennen ein `int`-Feld `p` der Länge `n` eine *Permutation*, wenn `p[0]`, ..., `p[n-1]` im Bereich $0, 1, \ldots, n-1$ liegen und jeder Wert genau einmal vorkomm
2. Die Permutation `q` heißt *Inverse* der Permutation `p`, wenn für alle `i` der Wert von `q[p[i]]` gleich `i` ist.

Aufgabe

Schreiben Sie eine Klasse `Permutation` mit den *statischen* Methoden `check(..)` und `invers(..)`, die aus jeder Clientklasse aufgerufen werden können und Folg

- `check(p)` überprüft, ob `p` eine Permutation ist und liefert den Wahrheitswert (**Hinweis**: z.B. zuerst in einer Schleife prüfen, ob die Bereichsbedingung erfü in einem Hilfsfeld `c` die Vorkommen der Werte `p[0]`, ..., `p[n-1]` erfassen)
- `invers(p)` ermittelt und liefert die Inverse von `p`. (Dabei ist bereits gesichert, dass `p` wirklich eine Permutation ist - nicht separat prüfen! **Hinweis**: Feld anl Komponenten so setzen, wie es Definition 2. vorschreibt.)

0 Zeichen zugelassen, Anzahl der eingegebenen Zeichen: **0**

15. Eine einfache Klasse (I1-ID: jr90w0703vg0)

Programmieren Sie eine einfache Klasse `Auto` mit der Angaben zu Autos abgespeichert werden können.
Jedes Auto zeichnet sich durch seinen Typ aus (z.B. "Mercedes") und durch seinen Preis (z.B. 30000).
Die folgenden Operationen sollen durch Methoden/Konstruktoren ermöglicht werden (*wenn nicht anders angegeben*: aufrufbar aus beliebigen Clientklassen):

- Erzeugen einer `Auto`-Instanz unter Angabe des Typs (Typ soll nach Erzeugen des Objekts nicht mehr änderbar sein)
- Erzeugen einer `Auto`-Instanz unter Angabe des Typs und des Preises (Typ soll nach Erzeugen des Objekts nicht mehr änderbar sein)
- Setzen des Preises einer Instanz (*nur von `Auto`-Methoden aus*),
- Abfragen des Preises einer Instanz
- Abfragen der Anzahl der erzeugten Instanzen und der Summe der Preise

VERWENDEN SIE FÜR DIE DATEN BITTE DIESE BEZEICHNER:
`typ`, `preis`, `anzahl`, `gesamt`

0 Zeichen zugelassen, Anzahl der eingegebenen Zeichen: **0**

1. Allgemeine Grundkenntnisse

1. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) Top-Down-Entwurf besteht darin, Details erst grob, dann feingliedrig zu sortieren.
   - (B) Algorithmen sind endliche Vorschriften, die Rechenabläufe schrittweise präzise beschreiben.
   - (C) Compiler übersetzen Quellcode erst in Assembler, dann in Unix-Code.
   - (D) Das Java-Programm ist auf dem System lauffähig, wenn darauf der entsprechende Compiler installiert ist.
2. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) Bei der Entwicklung eines Programms muss nach jedem Compilieren editiert werden.
   - (B) Interaktive Programme können Daten von der Standard-Eingabe erhalten.
   - (C) Am CMD-Flag ist erkennbar, ob die Speicherzelle eine Zahl oder einen Befehl enthält.
   - (D) Eine *Pipe* funktioniert im wesentlichen wie eine Queue.
3. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) Durch Eingabeumleitung kann man gespeicherte Daten an ein interaktives Kommando leiten.
   - (B) Informationsverarbeitung kann man zerlegen in die Schritte Codierung, Datenverarbeitung und Decodierung.
   - (C) Mit 7 Hexadezimalziffern kann man mehr als 1 Milliarde Werte darstellen.
   - (D) Der Bruch 4/5=0.8 kann exakt als `double`-Wert dargestellt werden.
4. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) Ganzzahlarithmetik ist normalerweise schneller als Gleitkomma-Arithmetik.
   - (B) Strukturierte Programmierung ist die Programmierung mit strukturierten Daten.
   - (C) Rekursion benötigt mehr Speicher auf dem Stack als Iteration.
   - (D) Die JVM interpretiert jeden Bytecodebefehl und prüft ihn auf syntaktische Korrektheit.

2. Einige Java-Grundkenntnisse

1. Ordnen Sie Begriffe zeitlich (0.25P für jede korrekte Position)
2. Füllen Sie die Lücken, so dass das Programm mit zwei beliebigen Zahlwerten A und B als Kommandozeilenparametern aufgerufen Folgendes leistet: Es wird die Summe der ganzen Zahlen ausgeben, die innerhalb des *abgeschlossenen* Intervalls [A,B] liegen. (0.25P für jede korrekte Angabe)

```
public class Intervallsumme {
    public static void main(String[] args) {
        ,
            min = (args[0]),
            max = (args[1]);
        int ;
        while ( ) {
            ;
            ;
        }

    }
}
```

3. Wieviele Bytes beansprucht eine Variable?
    - `int`:
    - `char`:
    - `float`:
    - `boolean`:

    (0.25P für jede richtige Antwort)

3. Grundlegende Datentypen

1. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) Das Leerzeichen ist ein druckbares Zeichen.
   - (B) iso8859-1 ist eine Teilmenge von ASCII.
   - (C) `'C'++` liefert das gleiche Ergebnis wie `'C'+1`.
   - (D) `"C" + 1` ist *kein* erlaubter Java-Ausdruck.
2. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) `int` ist ein Ganzzahldatentyp, `char` nicht.
   - (B) `int x = Integer.MAX_VALUE + 1;` ist eine erlaubte Anweisung.
   - (C) Der Wert von `( 3 < 7 ) && ((17 + 4) % (12 - 5)) != 0)` ist `false`.
   - (D) Der Wert von `( 3 + 1/3 == 3 - 1/3)` ist `false`.
3. Wählen Sie die zutreffenden Antworten aus:
   - (A) `String s = "Hallo"; System.out.println(s.length);`

     Ausgabe?
   - (B) `System.out.println(10e-1f);`

     Ausgabe?
   - (C) `2 * (short) 0xCafe`

     Typ des Ausdrucks?
   - (D) `double y = Double.MIN_VALUE; System.out.println(y/2);`

     Ausgabe?
   - (E) `int x = Integer.MAX_VALUE, y = Integer.MAX_VALUE++, z = x+y;`

     Wert von `z`?
   - (F) `System.out.println( (double) (1/2));`

     Ausgabe?
4. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) `int[] z = {1}; System.out.println(z);` produziert die Ausgabe 1.
   - (B) `length` ist ein `private`-Attribut von Feldern.
   - (C) Durch `int[] a = {1, 2, 3}, b = new int[];` werden zwei neue Felder angelegt.
   - (D) Durch `int[][][][] x = new char[32][32][32][16];` wird etwa 1Mb Heap-Speicher reserviert.

4. Integer-Arithmetik

Sei `d` eine `int`-Variable mit dem Wert `Integer.MAX_VALUE` (also 2147483647). Welche Textausgabe erzeugen diese Anweisungen?:

```
1:  System.out.println(-d);     //
2:  System.out.println(d-1);    //
3:  System.out.println(1+2*d);  //
4:  System.out.println(-(2*d)); //
```

Achten Sie darauf, dass Ihre Antworten NUR Dezimalziffern enthalten (sowie bei Bedarf Vorzeichen)!

5. Iteration in Feldern

```
int[] p = { 1, 4, 0, 3, 6, 2, 7, 5 };
int[] c = { 0, 0, 0, 0, 0, 0, 0, 0 };
int[] q = { 0, 0, 0, 0, 0, 0, 0, 0 };
for (int i = 0; i < 8; i++)
    c[p[i]] = c[p[i]] + 1;
for (int i = 0; i < 8; i++)
    q[p[i]] = i;
```

Tragen Sie die Werte der Feldkomponenten *nach* Ausführung des Java-Fragments ein:

Index	0	1	2	3	4	5	6	7
c								
q								

6. Felder: Fehler oder...?

Erzeugt untenstehende Klasse

- einen Compilezeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-):
- einen Laufzeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-):
- eine Ausgabe? Falls ja, so tragen Sie Ausgabe ein, ansonsten einen Bindestrich (-):

```
1: class FehlerOderKeiner {
2:     public static void main(String[] args) {
3:         int[] a = {2,3,5,7,11}; int[] b = { a[2], a[2]};
4:         b = a;
5:         b[1] = b[0];
6:         System.out.println(a[0] != a[1]);
7:     }
8: }
```

7. Rekursion

Betrachten Sie folgende Klasse:

```
 1: class Myst {
 2:     static void ery(int n) {
 3:         if (n == 0)
 4:             return;
 5:         ery( n-1);
 6:         A();
 7:         yre( n-1);
 8:     }
 9:     static void yre(int n) {
10:         if (n == 0)
11:             return;
12:         ery( n-1);
13:         B();
14:         yre( n-1);
15:     }
16:     static void A() {
17:         System.out.print("A");
18:     }
19:     static void B() {
20:         System.out.print("B");
21:     }
22:     public static void main(String[] args) {
23:         int n = Integer.parseInt(args[0]);
24:         ery(n);
25:         System.out.println();
26:     }
27: }
```

1. Geben Sie die ersten 8 Zeichen der Ausgabe zum Aufruf `java Myst 4` an:
2. Aus wievielen Zeichen besteht der Ausgabestring beim Aufruf `java Myst 7` (ohne den abschließenden Zeilenwechsel durch Zeile 25)?
3. Wird Zeile 14 durch `yre( n-2);` ersetzt, so führt beispielsweise der Aufruf `java Myst 4` zu einem Fehler.
   Es genügt, ein einzelnes Zeichen in einer *anderen* Zeile zu ändern, um diesen Fehler nicht mehr hervorzurufen. Um welche Zeile handelt es sich?
4. Werde Zeile 12 durch `ery( n-2);` und zusätzlich Zeile 14 durch `yre( n-2)` ersetzt.
   Geben Sie das maximale Argument `n` an, bei dem der Aufruf `java Myst n` *nicht* zu einem Fehler führt:

8. Stack und Queue in Aktion

1. Auf einem Stack werden *in gemischter Reihenfolge* 10 Push- und 10 Pop-Operationen ausgeführt. Es ist bekannt, dass die Push-Operationen die Zahlen 0, 1, 2, …, 9 in dieser Reihenfolge einspeichern und dass bei jedem Pop der ausgespeicherte Wert ausgegeben wird. Welche Ausgabelisten sind möglich?

   Wählen Sie aus den beiden Blöcken jeweils die zutreffende Antwort aus (jeweils 2P für die richtige Kombination/0P für jede andere).

   - (A) 1 2 3 4 5 6 9 8 7 0
   - (B) 0 4 6 5 3 8 1 7 2 9

   - (C) 1 4 7 9 8 6 5 3 0 2
   - (D) 2 1 4 3 6 5 8 7 9 0

2. Auf einer Queue werden *in gemischter Reihenfolge* 10 Enqueue- und 10 Dequeue-Operationen ausgeführt. Es ist bekannt, dass die Enqueue-Operationen die Zahlen 0, 1, 2, …, 9 in dieser Reihenfolge einspeichern und dass bei jedem Dequeue der ausgespeicherte Wert ausgegeben wird. Welche Ausgabelisten sind möglich?

   Wählen Sie aus den beiden Blöcken jeweils die zutreffende Antwort aus (jeweils 1P für die richtige Kombination/0P für jede andere).

   - (E) 4 6 8 7 5 3 2 9 0 1
   - (F) 2 5 6 7 4 8 9 3 1 0

   - (G) 0 1 2 3 4 5 6 7 8 9
   - (H) 4 3 2 1 0 5 6 7 8 9

9. Queue aus zwei Stacks bauen

Mit zwei Stacks kann man eine Queue implementieren:

- `enqueue(...)` entspricht Einspeichern in `inbox`
- `dequeue()` entspricht Ausspeichern aus `outbox`.

Unter Verwendung dieser API

`public class Stack`	
`boolean empty()`	testet, ob dieser Stack leer ist
`void push(String s)`	speichert `s` in diesem Stack
`String pop()`	liefert zuletzt gespeicherten String aus diesem Stack und entfernt ihn

sähe die entsprechende Klasse z.B. so aus:

```
public class ZSQueue { // Zwei-Stack-Queue
    private Stack inbox, outbox;
    public ZSQueue() { ... }
    public boolean empty() { ... }        // Leerheitstest
    public void enqueue(String s) { ... } // einspeichern
    public String dequeue() {             // ausspeichern und liefern
        // Rumpf (Ihre Aufgabe)
    }
}
```

Stellen Sie die Zeilen für den Rumpf der Methode `dequeue()` richtiger Reihenfolge zusammen:

- ist die Queue leer, ist Ausspeichern nicht möglich!
- teste ob `outbox` leer ist und fülle in diesem Fall den kompletten Inhalt von `inbox` um in `outbox`, dann ein Element aus `outbox` ausspeichern und liefern.

10. Vermischtes zu Klassentypen

1. Objekte verstehen:

```
public class Spieler {
    private int score;
    public Spieler( int s) {
        this.score = s;
    }
}
```

Zwei `Spieler`-Objekte werden instanziiert:

```
Spieler a = new Spieler(50), b = new Spieler(50);
```

Welchen Wert hat nun der Ausdruck a == b?

2. Fremden Code deuten:

```
1:  public class FirstBpp extends hepno.users.Frame {
2:      private hepno.users.Wring wrags;
3:      public FirstBpp() {
4:          wrags = new hepno.users.Wring();
5:      }
6:      public static void main(String[] args) {
7:          FirstBpp app = new FirstBpp();
8:      }
9:  }
```

Was bewirkt Zeile 2?:

3. Statische Methoden und Instanzmethoden:

```
public class Klasse {
    private final int zahl = 10;
    double a() {
        System.out.println(zahl);
        return c();
    }
    void b() {
        System.out.println(this);
    }
    double c() {
        double r = Math.random();
        System.out.println(r);
        return r;
    }
    void d() {
        a();
        a();
    }
    int e() {
        return zahl;
    }
}
```

Welche der obigen Methoden ist ihrer Natur nach eher eine statische Methode (und sollte daher mit `static` versehen werden)?

4. Fehlersuche:

```
public class Test {
    private int testCount;
    public static int getCount(){
        return testCount;
    }
    public Test(){
        testCount++;
    }
}
```

Obiger Code verursacht einen Compilezeitfehler. Was ist das Problem?

5. Tracing:
   Hinweis: Beachten Sie, dass man über Objektreferenzen auch auf Klassenvariablen zugreifen kann.

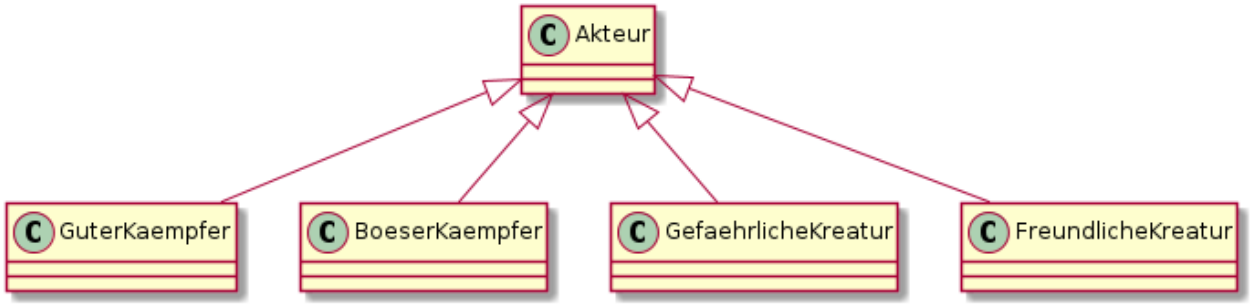
```
public class Tset {
    static int tsetCount = 0;
    public Tset(){
        tsetCount++;
    }
    public static void main(String [] args){
        Tset t1 = new Tset();
        System.out.print(t1.tsetCount + ", ");
        Tset t2 = new Tset();
        System.out.print(t1.tsetCount + " "+ t2.tsetCount + ", ");
        Tset t3 = new Tset();
        System.out.print(t1.tsetCount+ " "+ t2.tsetCount+ " "+ t3.tsetCount);
        System.out.println();
    }
}
```

   Welche Ausgabe produziert der Code?

6. Begriffsverständnis: Wählen Sie aus dem Menü die einzige Aussage aus, die NICHT korrekt ist.

11. OOP-Entwurf

In einer Spiele-Software gibt es folgende Hierarchie von Java-Klassen, die die verschiedenen Sorten von Akteuren modelliert:



Die (hier nicht gezeigte) Klasse `Spiel` benutzt die zugreifbaren Methoden obiger Klassen, um den Ablauf zu gestalten.

Wie wird folgende Zusatzanforderung korrekt modelliert?

Alle Kaempfer-Objekte aber keine Kreatur-Objekte sollen mit einer Methode benutzeWaffe() ausgestattet werden.

- `benutzeWaffe()` in `Akteur` implementieren, in `GefaerlicheKreatur` und `FreundlicheKreatur` mit `private` kennzeichnen.
- Interface `Kaempfer` mit `benutzeWaffe()` definieren und durch `GuterKaempfer` und `BoeserKaempfer` implementieren lassen.
- Die Methode `benutzeWaffe()` nur in `GuterKaempfer` und in `BoeserKaempfer` wie benötigt implementieren.
- Weitere Klasse `Kaempfer` schreiben, die `benutzeWaffe()` implementiert, `GuterKaempfer`, `BoeserKaempfer` davon erben lassen.
- `benutzeWaffe()` mit leerem Rumpf in `Akteur` implementieren, in `GuterKaempfer` und `BoeserKaempfer` geeignet überschreiben.

12. OOP-Verständnisfragen

Sei `p` eine Referenzvariable vom Typ `X`. Was kann alles der dynamische Typ von `p` sein? Wählen Sie aus den beiden Blöcken unten jeweils die zutreffenden Antworten aus (jeweils 1.5P für die richtige Kombination/0P für jede andere).

- (A) eine Erweiterungsklasse von `X` (falls `X` eine Klasse ist)
- (B) ein elementarer Datentyp (falls `X` die zugehörige Wrapper-Klasse ist)
- (C) eine Klasse, die `X` implementiert (falls `X` ein Interface ist)

- (D) eine Klasse, von der `X` direkt oder indirekt erbt (falls `X` eine Klasse ist)
- (E) ein Interface, das `X` als abstrakte Klasse erweitert (falls `X` eine Klasse ist)
- (F) eine Erweiterungsklasse einer Klasse, die `X` implementiert (falls `X` ein Interface ist)

13. Analyse von Algorithmen

1. Betrachten Sie folgenden Ausschnitt aus einem größeren Programm:

```
1: for (int i = 0; i < N; i++)
2:     for (int j = i+1; j < N; j++)
3:         for (int k = j+1; k < N; k++)
4:             verarbeite(daten);
```

   Die Ausführungszahl $V(N)$ von Zeile 4 bei Problemgröße N soll untersucht werden. Welcher der folgenden Ausdrücke ist am besten für die weitere asymptotische Analyse geeignet?

2. Eine Implementation wird zahlreichen Verdopplungstests unterzogen, mit zufällig generierten Eingabedaten. Die Ergebnisse — alle sehr ähnlich — sehen typischerweise so aus:

Problemgröße N	1024	2048	4096	8192	16384	32768	65536	131072
Zeit (s)	0.00	0.01	0.01	0.02	0.03	0.06	0.14	0.30

262144	524288	1048576	2097152	4194304	8388608
0.65	1.24	2.46	4.98	10.02	19.94

   Außerdem kann man nachweisen, dass die Ausführungszahl der Kernoperationen bei spezieller Struktur der Eingabe linear mit der Problemgröße N wächst.

   Welche dieser Hypothesen über das Laufzeitverhalten erscheint dann am plausibelsten?

3. Welche dieser Aussagen trifft NICHT zu?

14. Eine Bibliothek für Permutationen

Definition

1. Wir nennen ein `int`-Feld `p` der Länge `n` eine *Permutation*, wenn `p[0]`, …, `p[n-1]` im Bereich $0, 1, \ldots, n-1$ liegen und jeder Wert genau einmal vorkommt.
2. Die Permutation `q` heißt *Inverse* der Permutation `p`, wenn für alle `i` der Wert von `q[p[i]]` gleich `i` ist.

Aufgabe

Schreiben Sie eine Klasse `Permutation` mit den *statischen* Methoden `check(..)` und `invers(..)`, die aus jeder Clientklasse aufgerufen werden können und Folgendes leisten:

- `check(p)` überprüft, ob `p` eine Permutation ist und liefert das Ergebnis (**Hinweis**: z.B. zuerst in einer Schleife prüfen, ob die Bereichsbedingung erfüllt ist, dann mit zweiter Schleife in einem Hilfsfeld `c` die Vorkommen der Werte `p[0]`, …, `p[n-1]` erfassen)
- `invers(p)` ermittelt und liefert die Inverse von `p`. (Dabei ist bereits gesichert, dass `p` wirklich eine Permutation ist - nicht separat prüfen! **Hinweis**: Feld anlegen und in Schleife die Komponenten so setzen, wie es Definition 2. vorschreibt.)

15. Eine einfache Klasse

Programmieren Sie eine einfache Klasse `Auto` mit der Angaben zu Autos abgespeichert werden können. Jedes Auto zeichnet sich durch seinen Typ aus (z.B. "Mercedes") und durch seinen Preis (z.B. 30000). Die folgenden Operationen sollen durch Methoden/Konstruktoren ermöglicht werden (*wenn nicht anders angegeben*: aufrufbar aus beliebigen Clientklassen):

- Erzeugen einer `Auto`-Instanz unter Angabe des Typs, der Typ soll nach Erzeugen des Objekts nicht mehr änderbar sein
- Erzeugen einer `Auto`-Instanz unter Angabe des Typs und des Preises,
- Setzen des Preises einer Instanz (*nur von `Auto`-Methoden aus*),
- Abfragen des Preises einer Instanz
- Abfragen der Anzahl der erzeugten Instanzen und der Summe der Preise

VERWENDEN SIE FÜR DIE DATEN BITTE DIESE BEZEICHNER:
`typ`, `preis`, `anzahl`, `gesamt`

Probeklausur (überarbeitet)

Bearbeitungszeit 90 Minuten

Name	**Probeklausur**
Vorname	**.**
Matrikelnummer	**.**
Datum	**22.01.2008**

Bei der Klausur sind **keine Hilfsmittel** (Skripten, Taschenrechner, etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten Blätter für Ihre Antworten. Schreiben Sie mit blauem oder schwarzem Kugelschreiber, Füller, Bleistift, etc. ist nicht erlaubt.

1 Abstrakte Beschreibung	14	
2 Logik	10	
3 Zahlen	14	
4 Algorithmen	22	
5 Wachstum von Funktionen	16	
6 Abstrakte Datentype	10	
7 Java	04	
Summe	**90**	

Note	

1 Abstrakte Beschreibung (1/2)

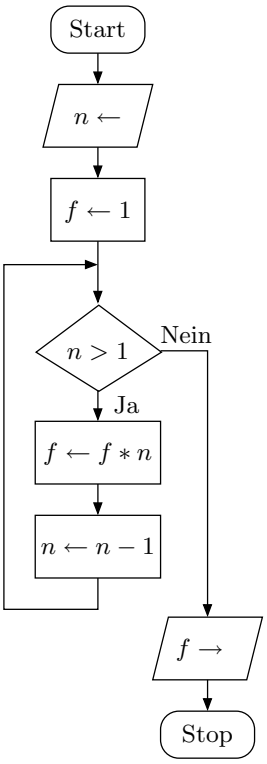
1. Gegeben ist der folgenden Java-Quelltext der Methoden `fac(n)` zur Berechnung der Fakultät von n.

```
1  public int fac(int n) {
2    int f = 1;
3    while (n > 1) {
4      f = f * n;
5      n = n - 1;
6    }
7    return f;
8  }
```

   Zeichnen Sie das Flussdiagramm der Methoden `fac`. (4 Punkte)

Lösung.

1. Flussdiagramm



1 Abstrakte Beschreibung (2/2)

2. Geben Sie über dem Alphabet der Ziffern von 0 bis 9 eine EBNF (erweiterte Backus-Naur-Form) Beschreibung für die Sprache aller natürlichen Zahlen an, die durch 4 teilbar sind. (4 Punkte)

3. Es sind reguläre Audrücke definiert für das leere Wort (ε), die Konkatenation (Hintereinanderstellen), die Alternative ($|$) und beliebig viele Vorkommen ($*$).

   Für reguläre Ausdrücke gilt Folgendes.

   - $*$ bindet stärker als die Konkatenation, Beispiel. $rs* = r(s*)$.
   - Die Konkatenation bindet stärker als $|$. Beispiel. $(rs|ab) = ((rs)|(ab))$.

   Der reguläre Ausdruck $r?$ soll kein oder ein Vorkommen, der reguläre Ausdruck $r+$ soll beliebig viele, aber mindestens ein, Vorkommen von r beschreiben.

   Geben Sie Definitonen von $r?$ und $r+$ mit den bereits definierten regulären Ausdrücken an. (6 Punkte)

Lösung.

2.
```
<S>  ::= "4" | "8"
         | <V0> "0" | <V0> "4" | <V0> "8" | <V1> "2" | <V1> "6"
<V0> ::= "2" | "4" | "6" | "8"
         | <T> "0" | <T> "2" | <T> "4" | <T> "6" | <T> "8"
<V1> ::= "1" | "3" | "5" | "7" | "9"
         | <T> "1" | <T> "3" | <T> "5" | <T> "7" | <T> "9"
<T> ::=  ("1" | ... | "9") { "0" | "1" | ... | "9"}
```

3. $r? = (\varepsilon \mid r)$

   $r+ = (r\ r*)$.

2 Logik

Folgende Gesetze (Rechenregeln) für Operatoren auf Wahrheitswerten können als bekannt vorausgesetzt werden.

Neutralitäts-	Extremal-	Komplementär-	Kommutativ-	Idempotenz-
$x \vee 0 = x$	$x \vee 1 = 1$	$x \vee \neg x = 1$	$x \vee y = y \vee x$	$x \vee x = x$
$x \wedge 1 = x$	$x \wedge 0 = 0$	$x \wedge \neg x = 0$	$x \wedge y = y \wedge x$	$x \wedge x = x$
				$\neg\neg x = x$

Assoziativ-	Distributiv-
$x \vee (y \vee z) = (x \vee y) \vee z$	$x \vee (y \wedge z) = (x \vee y) \wedge (x \vee z)$
$x \wedge (y \wedge z) = (x \wedge y) \wedge z$	$x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$

Hinweis. Der $\neg$ Operator bindet stärker als die Operatoren $\vee$, $\wedge$, d.h. $\neg\ x \vee y = (\neg x) \vee y$ und $\neg\ x \wedge y = (\neg x) \wedge y$. Die Operatoren $\vee$, $\wedge$ binden gleichstark, deshalb sind hier Klammern, die die richtige Auswertungsreihenfolge bestimmen, zwingend erforderlich (Beispiel, Distributivgesetz).

1. Zeigen Sie, mit Hilfe einer ausführlichen Wahrheitswertetafel, dass folgendes De-Morgansche-Gesetz gilt. (4 Punkte)

$$\neg(x \vee y) = \neg x \wedge \neg y$$

2. Betrachten Sie die Absorptionsgesetze.

$$x \vee (x \wedge y) = x$$
$$x \wedge (x \vee y) = x$$

   a) Zeigen Sie, durch Anwendung der passenden bekannten Gesetze, dass aus der Gültigkeit eines Absorptionsgesetze das andere folgt. Geben Sie bei jedem Schritt das angewendete Gesetz an. (3 Punkte)

   b) Zeigen Sie die Gültigkeit eines Absorptionsgesetzes durch Anwendung der passenden bekannten Gesetze. Geben Sie bei jedem Schritt das angewendete Gesetz an. (3 Punkte)

      Hinweis. Führen Sie eine Fallunterscheidung mit $y = x$ und $y = \neg x$ durch. |

Lösung.

1. De-Morgansches-Gesetz

x	y	$x \vee y$	$\neg(x \vee y)$	$\neg x$	$\neg y$	$\neg x \wedge \neg y$
0	0	0	**1**	1	1	**1**
0	1	1	**0**	1	0	**0**
1	0	1	**0**	0	1	**0**
1	1	1	**0**	0	0	**0**

2. a) Gilt ein Absorptionsgesetz, dann gilt auch das andere.

$$\begin{aligned} x \vee (x \wedge y) &= (x \vee x) \wedge (x \vee y) && \text{Distributiv-} \\ &= x \wedge (x \vee y) && \text{Idempotenz-} \end{aligned}$$

b) Fall $x = y$.

$$\begin{aligned} x \vee (x \wedge y) &= x \vee (x \wedge x) && y = x \\ &= x \vee x && \text{Idempotenz-} \\ &= x && \text{Idempotenz-} \end{aligned}$$

Fall $x = \neg y$.

$$\begin{aligned} x \vee (x \wedge y) &= x \vee (x \wedge \neg x) && y = \neg x \\ &= x \vee 0 && \text{Komplementär-} \\ &= x && \text{Neutralitäts-} \end{aligned}$$

Bemerkung. Es gibt einen *Exkurs, Äquivalenzumformung* als Anhang der Probeklausur.

3 Zahlen

1. Rechnen Sie von einem Zahlensytem in das andere um. Der Rechenweg ist Teil der Lösung.

   a) $(377)_8 = (?)_{10}$ (2 Punkte)

   b) $(3200)_{10} = (?)_{16}$ (2 Punkte)

   c) $(1001011001)_2 = (?)_8$ (2 Punkte)

   d) $(E2A0)_{16} = (?)_2$ (2 Punkte)

2. Betrachten Sie vorzeichenbehaftete Dualzahlen der Länge n Bit, wobei für die Darstellung der negativen Zahlen das Basiskomplement (= Zweierkomplement) verwendet wird.

   a) Erläutern Sie, wie zwei solche vorzeichenbehafteten Dualzahlen addiert werden. (4 Punkt)

   b) Rechnen Sie exemplarisch $\texttt{01110}_2 - \texttt{00111}_2$ mit $n = 5$. (2 Punkt)

Lösung.

1. a) Ausmultiplizieren.

$$377_8 = 3_{10} \cdot 8_{10}^2 + 7_{10} \cdot 8_{10} + 7_{10} = 255_{10}$$

b) Dividierende Umrechung.

$$\begin{aligned} 3200_{10}/16 = 200_{10} \quad & \text{Rest} \quad a_0 = 0 \\ 200/16 = 12 \quad & \text{Rest} \quad a_1 = 8 \\ 12/16 = 0 \quad & \text{Rest} \quad a_2 = 12_{10} = \mathrm{C}_{16} \end{aligned}$$

$$3200_{10} = \mathrm{C80}_{16}$$

c) Zerlegen

$$\begin{aligned} (1001011001)_2 &= (100)_2^3 \cdot (001)_2 + (100)_2^2 \cdot (001)_2 + (100)_2 \cdot (011)_2 + (001)_2 \\ &= (10)_8^3 \cdot (1)_8 + (10)_8^2 \cdot (1)_8 + (10)_8 \cdot (3)_8 + (1)_8 \\ &= (1131)_8 \end{aligned}$$

oder

$$(1001011001)_2 = 001|001|011|001 = 1|1|3|1 = (1131)_8$$

d) Zerlegen

$$\begin{aligned} (E2A0)_{16} &= E|2|A|0 = 14_{10}|2_{10}|10_{10}|0_{10} = 1110|0010|1010|0000 \\ &= (1110001010100000)_2 \end{aligned}$$

2. a) Die Addition erfolgt ebenso wir bei vorzeichenlosen Dualzahlen. Ein eventuell auftrettender Übertrag wird ignoriert, sodass das Ergebnis wieder eine Dualzahlen der Länge n ist.

b) Die Subtraktion wird auf Vorzeichenwechsel und Addition zurückgeführt, d.h. $(01110)_2 - (00111)_2 = (01110)_2 + (-(00111))_2$.

$(-(00111))_2 = (11000)_2 + 1 = (11001)_2$

```
   01110
+  11001
--------
  100111
```

Ergibt $(01110)_2 - (00111)_2 = (00111)_2$.

Test (nicht Teil der Aufgabe).
$(01110)_2 = (14)_{10}$, $(00111)_2 = (7)_{10}$, $(14)_{10} - (7)_{10} = (7)_{10}$.

4 Algorithmen (1/2)

1. Behauptung. Die Anzahl der Teile der Länge 1 oder 2, in die ein Faden der positiven ganzzahligen Länge n zerschnitten werden kann, wird durch folgende rekursive Formel bestimmt.

$$T(1) = 1$$
$$T(2) = 2$$
$$T(n) = T(n-1) + T(n-2)$$

   a) Zeigen Sie mit vollständiger Induktion die Korrektheit der Behauptung. | (6 Punkte)

   b) Geben Sie in Pseudo-/Java-Code einen Algorithmus zur iterativen Berechung der Anzahl der Teile der Länge 1 oder 2, in die ein Faden der positiven ganzzahligen Länge n zerschnitten werden kann, an. (6 Punkte)

Lösung.

1. a) Induktionsanfang.
Zu zeigen, die Behauptung gilt für $n = 1$ und $n = 2$.

Ein Faden der Länge 1 kann nur in ein Teil der Länge 1 geteilt werden, $T(1) = 1$.
Ein Faden der Länge 2 kann einmal in zwei Teile der Länge 1 und einmal in ein Teil der Länge 2 geteilt werden, $T(2) = 2$.

Induktionsannahme.
Es gibt ein $n > 2$, sodass die Behauptung für alle Fäden der Länge kleiner gleich n gilt.

Induktionsschritt.
Zu zeigen, mit der Induktionsannahme, gilt die Behauptung auch für Fäden der Länge $n + 1$ mit $n > 2$.

Ein Faden der Länge $n + 1$ kann einmal in ein Teil der Länge 1 und alle Teilungen eines Fadens der Länge n geteilt werden und einmal in ein Teil der Länge 2 und alle Teilungen eines Fadenes der Länge $n - 1$ geteilt werden.

Das heißt $T(n + 1) = 1 \cdot T(n) + 1 \cdot T(n - 1)$, mit Induktionsannahme.

b)

```
1  public long teilungen (int n) {
2      if (n<=0) return 0; // Fehler
3      if (n==1) return 1;
4      if (n==2) return 2;
5      int i = 3;
6      long a = 2;
7      long b = 1;
8      while (i <= n) {
9          long next = a + b;
10         i++;
11         b = a;
12         a = next;
13     }
14     return a;
15 }
```

4 Algorithmen (2/2)

2. Betrachten sie den Java-Programmtext der Funktion `f`.

```
1  public boolean f(int x) {
2      boolean b = true;
3      if ( x <= 1 )          // vorher ( x < 1 )
4          b = false;
5      else if ( x > 1 ) {
6          int t = 2;
7          while ( t*t <= x ) {
8              if ( b && ((x%t) == 0) )
9                  b = false;
10             t++;
11         }
12     }
13     return b;
14 }
```

a) Beschreiben Sie in maximal zwei Sätzen die Funktion der Methode `f`. (4 Punkte)

Hinweis. Welche Eigenschaft der übergebenen Zahl kann aus dem Rückgabewert der Methode gefolgert werden.

b) Definieren Sie für die Schleife Vorbedingung, Nachbedingung und Schleifeninvariante. Beweisen Sie mit deren Hilfe die Korrektheit Ihrer Aussage über die Funktion der Methode `f`. (6 Punkte)

Hinweis. Berücksichtigen Sie auch die Sonderfälle.

Lösung.

2. a) Behauptung. Die Methode `f(x)` liefert *true*, wenn die übergebene Zahl `x` eine Primzahl ist; sonst liefert die Methode *false*.

   b) Vorbedingung. Es gilt `b = true` und keine natürliche Zahl größer gleich 2 und kleiner gleich $t-1$ teilt x ohne Rest.

   Schleifeninvariante. Gilt am Ende jedes Schleifedurchlaufs.
   $(t-1)^2 \leq x$. Genau dann, wenn x von keiner natürliche Zahl größer gleich 2 und kleiner gleich $t-1$ ohne Rest geteilt wird, gilt `b = true` sonst gilt `b = false`.

   Begründung. In der Schleife wird getestet ob x von t ohne Rest geteilt wird, wenn ja wird `b = false` gesetzt, dann wird t um 1 erhöht. D.h. für $t-1$ gilt die Invariante aufgrund dieses Tests und für alle Zahlen kleiner $t-1$ gilt die Invariante wegen der Vorbedingung (beim ersten Durchlauf) oder der Schleifeninvariante (vom vorherigen Durchlauf).

   Nachbedingung. Es gilt `b = true` genau dann, wenn x von keiner natürliche Zahl größer gleich 2 und kleiner gleich $s := (t-1)$ mit $s^2 \leq x$ ohne Rest geteilt wird.

   Beweis der Behauptung. Sonderfall 1. Für 1 (und alle negativen Zahlen) wird *false* zurückgeliefert.

   Sonderfall 2. Für die Primzahlen 2 und 3 wird die Schliefe nicht durchlaufen, aber `b` ist mit *true* initalisiert.

   Für alle $x > 3$ wird die Schleife mindestens einmal durchlaufen.

   Die Nachbedingung stellt sicher, dass x von keiner natürliche Zahl größer gleich 2 und kleiner x ohne Rest geteilt wird, d.h eine Primzahl ist. Denn gäbe es noch einen Teiler r mit $r^2 > x$, dann wäre $x = r \cdot u$ und $u = x/r < x/\sqrt{x} = \sqrt{x}$, somit wäre u ein Teiler von x mit $u^2 \leq x$ und das widerspricht der Nachbedingung.

5 Wachstum von Funktionen

1. Sei $f(n) \in \mathcal{O}(g(n))$ und $a > 0$.

   Beweisen Sie $f(n) \in \mathcal{O}(a \cdot g(n))$. (4 Punkte)

2. Die folgende Tabelle definiert die Primitiven *Zuweisung*, *Sequenz*, *Alternative* und *Schleife* zur Zerlegung von Methoden. Berechnungen und Vergleiche ohne Zuweisung bleiben unberücksichtigt. Weiterhin ordnet die Tabelle jeder Primitive P eine Kostenmaß $T(P)$ zu.

Primitive	*Beispiel für P*	*Kosten T(P)*
Zuweisung	`a = b + c;`	1
Sequenz	P_1; P_2; ⋮ P_m;	$T(P_1) + T(P_2) + \ldots + T(P_m)$

Alternative	`if (<bedingung>)`
	  P_1;
	`else`
	  P_2;
		$\max(T(P_1), T(P_2))$

Schleife	`for(i=0; i<k; i++)` P_1;	$k \cdot T(P_1)$

Hinweis. Die Rückgabe eines Funktionswertes (`return`) ist eine Zuweisung.

Betrachten Sie folgende Methode `armstrong` zum Test ob die übergebene Zahl eine Armstrong-Zahl ist.

```
1  public boolean armstrong(int x) {
2      int summe = 0;
3      int zahl = x;
4      if ( zahl > 0 ) {
5          do {
6              int ziffer = zahl % 10;
7              summe = summe + ziffer*ziffer*ziffer;
8              zahl = zahl/10;
9          } while (zahl > 0);
10     }
11     return (x == summe);
12 }
```

a) Geben sie, basierend auf obigem Kostenmaß, eine Laufzeitfunktion $T_a(x)$ für die Methode `armstrong` an. (6 Punkte)

b) Geben Sie, mit Herleitung, die asymptotische Laufzeit der Methode `armstrong` in $\mathcal{O}$-Notation an. (6 Punkte)

Lösung.

1. Es gilt

   $f(n) \in \mathcal{O}(g(n)),$ d.h. es gibt $c, n_0 > 0,$ so dass für $n \geq n_0$ gilt $0 \leq f(n) \leq c \cdot g(n)$

   Definiere $d := c/a$, dann folgt für alle $n_0 > 0$

   $$0 \leq f(n) \leq c \cdot g(n) = d\big(a \cdot g(n)\big)$$

   Das heißt $f(n) \in \mathcal{O}(a \cdot g(n))$.

2. a) Die Methode ist eine Sequenz von drei Zuweisungen und einer Alternative.

      Die Alternative hat nur einen `if`–Block, der aus einer Schleife besteht.

      Die Schleife enthät eine Sequenz von drei Zuweisungen und wird sooft durchlaufen, wie x Ziffern hat, d.h. $\lceil \log_{10}(x) \rceil$–mal. Daraus folgt $3\lceil \log_{10}(x) \rceil$ Kosten für die Schleife.

      D.h. $T_a(x) = 3 + 3\lceil \log_{10}(x) \rceil$.

   b) Wähle $x_0 = 10$ und $c = 9$ daraus folgt, für alle $x > x_0$ gilt

      $$0 < 3 + 3\lceil \log_{10}(x) \rceil \quad \text{weil } \lceil \log_{10}(x) \rceil \geq 1 \text{ mit } x \geq 1$$

      und

      $$\begin{aligned} 3 + 3\lceil \log_{10}(x) \rceil &\leq 3 + 3(\log_{10}(x) + 1) \\ &= 6 + 3\log_{10}(x) \\ &\leq 6\log_{10}(x) + 3\log_{10}(x) \quad \text{weil } \log_{10}(x) \geq 1 \text{ mit } x \geq 10 \\ &= 9\log_{10}(x) \\ &\leq c\log_{10}(x) \end{aligned}$$

      Daraus folgt $T_a(x) \in \mathcal{O}(\log_{10}(x))$.

6 Abstrakte Datentypen

Ihnen steht eine Klasse `CharStack`, die einen Kellerspeicher für Zeichen implementiert, mit folgenden Methoden zur Verfügung.

Methode	*Funktion*
`void create`	Konstruktor, erzeugt einen leeren Stapel
`boolean empty`	Liefert *true*, wenn der Stapel leer ist, sonst *false*.
`void push(char ch)`	Legt das Zeichen `ch` oben auf den Stapel.
`char pop()`	Liefert das oberste Zeichen zurück und entfernt es vom Stapel, ist der Stapel leer wird `null` zurückgeliefert.

Schreiben Sie in Pseudo-/Java-Code eine Methode, die, mit Hilfe der Klasse `CharStack`, einen übergebenen String darauf testet, ob er ein Palindrom ist. (10 Punkte)

Hinweis. Ein Palindrom ist ein Wort bzw. eine Zeichenkette, die vorwärts und rückwärtsgelesen, genau gleich ist. Beispiele. `LAGERREGAL` oder `p#?& &!& &?#p`.

Lösung.

```
1   public boolean palindrom( String s ) {
2
3       Stack stack = new Stack();
4       // oder
5       Stack stack = Stack.create();
6
7       int half = s.length / 2;
8       int plus = s.length % 2;
9
10      for ( int i = 0; i < half; i++ ) {
11          stack.push(s.charAt(i));
12      }
13
14      boolean p = true;
15      for ( int i = half + plus; p && i < args[0].length; i++) {
16          if ( stack.pop() != s.charAt(i) )
17              p = false;
18      }
19
20      return p;
21  }
```

7 Java

Betrachten Sie folgendes Java-Programmfragment.

```
1  for ( i=0; i < n; i++) {
2      if ( a[i] == x)
3          break;
4      else
5          a[i] = 0;
6  }
```

Schreiben Sie das Java-Programmfragment so um, dass anstelle der `for`– eine `do-while`–Schleife und keine `break`–Anweisung benutzt wird. (4 Punkte)

Hinweis. Auch ein leeres Feld ($n = 0$) muß berücksichtigt werden.

Lösung.

1.
```
1  int i = 0;
2  do {
3       if ( (i < n) && (a[i] != x) )
4            a[i] = 0;
5       i++;
6  } while ( (i < n) && (a[i] != x) )
```

Anhang. Exkurs, Äquivalenzumformung

Der ursprüngliche Hinweis zu Aufgabe 2.2 lautete: Beginnen Sie, indem Sie $\wedge\neg x$ auf beide Seiten der Gleichung anwenden.

Anschließend sollte, durch Anwendung der bekannten Gesetze, jede Seite der Gleichung auf 0 zurückgeführt werden, sodass sich die Gleichung $0 = 0$ ergibt.

Diese Verfahren wird *Äquivalenzumformung* einer Gleichungen genannte. Kann eine Gleichung durch Äquivalenzumformungen auf eine korrekt Gleichung (z.B. $0 = 0$) zurückgeführt werden, ist damit die Korrektheit der ursprünglichen Gleichung bewiesen.

Die Anwendung einer *bijektiven Funktion* auf beide Seiten einer Gleichung ist eine Äquivalenzumformung.

Ein Funktion $f : B \to B$ ist bijektiven, wenn es eine Umkehrfunktion $f^{-1} : B \to B$ gibt, für die gilt $f^{-1}(f(b)) = b$ und $f(f^{-1}(b)) = b$

Das Problem ist, dass die Anwendung von $\wedge\neg x$ auf eine beliebige Boolesche Funktion b (z.B. $b(x, y) = x \wedge (x \vee y)$), also $f(b) = b \wedge \neg x$ keine bijektiven Funktion und damit auch keine Äquivalenzumformung ist.

$\mathbb{B} = (\{0, 1\}, \vee, \wedge, \neg)$ ist die Boolesche Algebra. Das neutrale Element der $\vee$–Verknüpfung ist 0 ($x \vee 0 = x$) und das der $\wedge$–Verknüpfung ist 1 ($x \wedge 1 = x$). Aber es gibt weder für die $\vee$– noch für die $\wedge$–Verknüpfung für jedes Element $x \in \{0, 1\}$ eine Inverses-Element, d.h. ein Element, dass verknüpft mit dem ursprünglichen Element das neutrale Element ergibt. Es gibt für 1 keine Inverses unter der $\vee$–Verknüpfung, denn $1 \vee 0 = 1 \vee 1 = 1 \neq 0$. Genauso gibt es keine Inverses für 0 unter der $\wedge$–Verknüpfung, denn $0 \wedge 0 = 0 \wedge 1 = 0 \neq 1$.

Angenommen es gäbe für $f(b) = b \wedge \neg x$ eine Umkehrfunktion f^{-1} mit $f(f^{-1}(b)) = b$, dann würde aus $f(f^{-1}(1)) = 1$ folgen $f^{-1}(1) \wedge \neg x = 1$, d.h. bedeutet $f^{-1}(1)$ ist das Inverse zu $\neg x$ unter der $\wedge$–Verknüpfung und das ist eine Widerspruch.

Bijektive Funktion sind z.B. $g(b) = g^{-1}(b) = \neg b$ oder $h(b) = h^{-1}(b) = (\neg x \wedge b) \vee (x \wedge \neg b)$.

Mit h läßt sich das Absorptionsgesetz zeigen.

	$x \vee (x \wedge y) = x$	
$\Leftrightarrow$	$h\big(x \vee (x \wedge y)\big) = h\big(x\big)$	Anwendung von h
$\Leftrightarrow$	$h\big(x \vee (x \wedge y)\big) = (\neg x \wedge x) \vee (x \wedge \neg x)$	
$\Leftrightarrow$	$h\big(x \vee (x \wedge y)\big) = 0$	Komplementär-
$\Leftrightarrow$	$(\neg x \wedge (x \vee (x \wedge y))) \vee (x \wedge \neg(x \vee (x \wedge y))) = 0$	
$\Leftrightarrow$	$(\neg x \wedge (x \vee (x \wedge y))) \vee (x \wedge (\neg x \wedge \neg(x \wedge y))) = 0$	De-Morgan
$\Leftrightarrow$	$(\neg x \wedge (x \vee (x \wedge y))) \vee ((x \wedge \neg x) \wedge \neg(x \wedge y))) = 0$	Assoziativ-
$\Leftrightarrow$	$(\neg x \wedge (x \vee (x \wedge y))) \vee (0 \wedge \neg(x \wedge y))) = 0$	Komplementär-
$\Leftrightarrow$	$\neg x \wedge (x \vee (x \wedge y)) = 0$	Extremal-, Neutralitäts-
$\Leftrightarrow$	$(\neg x \wedge x) \vee (\neg x \wedge (x \wedge y)) = 0$	Distributiv-
$\Leftrightarrow$	$0 \vee ((\neg x \wedge x) \wedge y)) = 0$	Komplementär-, Assoziativ-
$\Leftrightarrow$	$0 = 0$	Komplementär-, Extremal-

Das ist aber unnötig kompliziert, die Fallunterscheidung ist deutlich effizienter.

(?) 1. 1. Vermischtes zur Einführung 2015

Markieren Sie alle zutreffenden Aussagen:

- Dem Inhalt einer Speicherzelle kann man nicht ansehen, ob sie eine Zahl oder einen Befehl speichert..
- Beim euklidischen Algorithmus wird dieser Block iteriert: *{ x = y; y = r; r = x % y; }*
- Beim Testen wird das Programm auf allen erlaubten Eingaben ausgeführt.
- Beim Pipelining werden Eingaben eines Prozesses zur Ausgabe eines anderen geleitet.
- Beim Laden wird die Befehlsfolge in aufeinanderfolgende Register kopiert.
- Editieren und Testen gehören zu jedem Programmentwicklungszyklus.
- *public static void main(String[] arks)* ist Kopf der *main* -Methode.

(?) 1. 1. Allgemeine Grundkenntnisse 2016

1. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) Top-Down-Entwurf besteht darin, Details erst grob, dann feingliedrig zu sortieren.
   - (B) Algorithmen sind endliche Vorschriften, die Rechenabläufe schrittweise präzise beschreiben.
   - (C) Compiler übersetzen Quellcode erst in Assembler, dann in Unix-Code.
   - (D) Das Java-Programm ist auf dem System lauffähig, wenn darauf der entsprechende Compiler installiert ist. (JVM muss installiert sein)
2. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) Bei der Entwicklung eines Programms muss nach jedem Compilieren editiert werden.
   - (B) Interaktive Programme können Daten von der Standard-Eingabe erhalten.
   - (C) Am CMD-Flag ist erkennbar, ob die Speicherzelle eine Zahl oder einen Befehl enthält.
   - (D) Eine Pipe funktioniert im wesentlichen wie eine Queue.
3. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) Durch Eingabeumleitung kann man gespeicherte Daten an ein interaktives Kommando leiten.
   - (B) Informationsverarbeitung kann man zerlegen in die Schritte Codierung, Datenverarbeitung und Decodierung.
   - (C) Mit 7 Hexadezimalziffern kann man mehr als 1 Milliarde Werte darstellen. 268 milionen
   - (D) Der Bruch 4/5=0.8 kann exakt als double-Wert dargestellt werden.
4. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) Ganzzahlarithmetik ist normalerweise schneller als Gleitkomma-Arithmetik.
   - (B) Strukturierte Programmierung ist die Programmierung mit strukturierten Daten.
   - (C) Rekursion benötigt mehr Speicher auf dem Stack als Iteration.
   - (D) Die JVM interpretiert jeden Bytecodebefehl und prüft ihn auf syntaktische Korrektheit.

(?) 1. 1. Allgemeine Grundkenntnisse (I1-ID: mq4g9vq04ih0)

1. Wählen Sie die richtige Antwortkombination aus (1P, bei einer Abweichung 0.5P, sonst 0P):
   - A) Eine Spezifikation ist eine präzise Beschreibung der erlaubten Ausgaben.
   - (B) Algorithmen beschreiben Handlungsvorschriften (z.B. zur Berechnung eines Werts).
   - (C) Programmentwicklung heißt (immer wieder testen, dann compilieren, dann editieren.
   - (D) Compiler laden Maschinencode zusammen mit Klassenbibliotheken in das Laufzeitsystem.
2. Geben Sie die richtige Antwortkombination an (1P, bei einer Abweichung 0.5P, sonst 0P):
   - (A) Der Befehlszähler im von Neumann-Prozessormodell speichert arithmetische Befehle.
   - (B) Interaktive Programme können auf Daten von der Standard-Eingabe reagieren. -
   - (C) Die Pipe der bash-Shell ist der Stack der noch nicht verarbeiteten Eingaben.
3. Geben Sie die richtige Antwortkombination an (1P, bei einer Abweichung 0.5P, sonst 0P):
   - (A) Durch Ausgabeumleitung kann man erzeugte Daten in einer Datei abspeichern.
   - (B) Decodieren bedeutet, eine Nachricht als Information zu interpretieren bzw. zu verstehen.
   - (C) Durch N Hexadezimalziffern werden 4mal soviel Informationen dargestellt wie mit N Binärziffern.
   - (D) Der ASCII-Standard wurde zur Lösung des Jahr-2000-Problems durch Unicode ersetzt.
3. Geben Sie die richtige Antwortkombination an (1P, bei einer Abweichung 0.5P, sonst 0P):
   - (A) Strukturierte Daten sind zerlegbar in kleinere Bestandteile.
   - (B) Gleitkomma-Arithmetik ist schneller als Ganzzahl-Arithmetik.
   - (C) Die JVM interpretiert Bytecodebefehle ohne ihre Syntax zu prüfen.
   - (D) Imperative Programmierung heißt, Daten durch Befehle zu ersetzen.

2. 1. Zweierpotenzen 2015

Ergänzen Sie untenstehende *main*-Methode so, dass der Aufruf *java Zweierpotenzen*
die Liste der Zweierpotenzen von *2^0 = 1* bis *2^n* zeilenweise ausgibt

```
public class Zweierpotenzen {
        public static void main (String[] args) {
        int n = Integer.parseInt(args[0]);
        int p = 1;
        for(int i = 0; i < n; i++){
                System.out.println(p);
                p = p*2;
        }
}
```

Beispiel *java Zweierpotenzen 3* gibt diese Liste aus:
1
2
4
8

Algorithmische Idee (bitte GENAU diese Variablennamen benutzen):
initialisiere:
- Kommandozeilenargument (interpretiert als int -Wert) in n speichern
- int-Variable p als Akkumulator initialisieren

iteriere so oft wie nötig:
- p ausgeben und geeignet aktualisieren

finalisiere: entfällt

Probeklausur 2019 Punktklasse

Ein Punkt der Ebene soll allein durch seine Koordinaten gegeben sein. Schreiben Sie eine Java-Klasse für *unveränderliche* Punkt-Objekte, die folgende API realisiert. Alle Variablen sind private zu deklarieren, Instanzvariablen auch final.
- public class Punkt
  - public Punkt(double x, double y)// Konstruktor
  - public static int anzahl() // liefert die Anzahl der erzeugten Punkte
  - public double getX() // liefert x-koordinate des Punktes
  - public double getY() // liefert y-koordinate des Punktes
  - public String toString()// liefert eine String-Darstellung: “Punkt(...,....)
    aus dem man die Koordinatenwerte ablesen kann

2. 1. Einige Java-Grundkenntnisse 2016

1. Ordnen Sie Begriffe zeitlich (0.25P für jede korrekte Position)
   Algorithmus, Compiler, Editor, Loesungsidee, Quelltext, Spezifikation, Testen, Zielcode

Spezifikation, LoesungsIdee, Algorithmus, Editor, Quelltext, Compiler, Zielcode, Testen

2. Füllen Sie die Lücken, so dass das Programm mit zwei beliebigen Zahlwerten A und B als Kommandozeilenparametern aufgerufen Folgendes leistet: Es wird die Summe der ganzen Zahlen ausgeben, die innerhalb des abgeschlossenen Intervalls [A,B] liegen. (0.25P für jede korrekte Angabe)

```
public class Intervallsumme {
        public static void main(String[] args) {
                A ;
                min = B  (args[0]);
                max = C (args[1]);
                int D ;
                while( E ){
                        F;
                        G;
                }
                H;
        }
}
```

A: double s = A, int s = A, double s = 0; int s = 0
B: StdIn.readInt, Integer.parseInt
C: StdIn.readInt, Integer.parseInt
D: i = min, n = min, t = min
E: min <= n, min <= s; n <= max, s <= max
F: n += s, s+=n, n=n+1, s = s+1
G: n++, s++, n+=s, s+=n
H: return n, return s, System.out.println(n), System.out.println(s)

2. Wieviele Bytes beansprucht eine Variable?

- int: 4
- char: 2
- float: *4*
- boolean: 1

1, 2,4,8,16 , nichts von denen.
(0.25P für jede richtige Antwort)

(?) 2. 1. Einige Java-Grundkenntnisse (l1-ID: ia4a8tq09ih0)

1. Von klein nach groß ordnen (0.25P für jede korrekte Angabe)
Was kann ineinander enthalten bzw. Teil voneinander sein?
Ordnen Sie von OBEN nach UNTEN die Begriffe vom "kleinsten" bis zum "größten", d.h. wenn X Teil von Y sein kann, so muss X weiter oben stehen als Y:

Anweisung; Ausdruck; Datei; Java-Archiv; Klasse; Klassenpfad; Literal; Methode
Literal
Ausdruck
Anweisung
Methode
Klasse
Datei
Java-Archiv
Klassenpfad

3. 1. Quadratzahlentabelle 2015

Die *int*-Variable *n* speichere die ganze Zahl *n*.
Die ersten *n* positiven Zahlen sollen zusammen mit ihren Quadraten jeweils auf einer Zeile ausgegeben werden.
Beispiel für *n = 4* :
1 : 1
2 : 4
3 : 9
4 : 16
Im Fall *n <= 0* soll nichts ausgegeben werden.

"Programmieren" Sie dies durch Auswahl der richtigen Codezeilen, und zwar einmal mit einer *for*-Schleife und einmal mit einer *do-while*-Schleife.

Achten Sie auf den Unterschied zwischen *System.out.print(..)* und *System.out.println(..)* .

- mit einer for-Schleife:

```
for (int i = 1; i < n+1; i++) {
        System.out.print(i + ": ");
        System.out.println( i*i);
}
```

- mit einer do-while-Schleife:

```
int i = 1;
do {
        System.out.print(i + ": ");
        System.out.println( i*i);
        i++;
} while ( i < n+1);
```

(?) 4. 2. Vermischtes zu elementaren Daten 2015

Markieren Sie **alle** zutreffenden Aussagen:

- Der ASCII-Standard legt die Anordnung der 256 Zeichen fest.
- Java-Code darf beliebige Unicode-Zeichen enthalten.
- *int*-Werte können verlustfrei in *float*-Variablen gespeichert werden.
- *5/4 == 1* hat den Wert *true*.
- *Integer.MAX_VALUE*++ verursacht keinen Ganzzahlüberlauf.
- Anstatt *if (b > 0 && a/b < 2)* .. schreibt man besser *if (a/b < 2 && b > 0)* ...
- Ein Gleitkomma-Überlauf ist kein Laufzeitfehler.

5. 2. Integer-Arithmetik 2015 2016

Sei *d* eine *int*-Variable mit dem Wert *Integer.MAX_VALUE* (also 2147483647). Welche Textausgabe erzeugen diese Anweisungen?:

1: System.out.println(-d);	*-2147483647*
2: System.out.println(d-1);	*2147483646*
*3: System.out.println(1+2*d);*	*-1*
*4: System.out.println(-(2*d));*	*2*
5: System.out.println(2147483646-d);	-1
6: System.out.println((-1)*d);	*-2147483647*
7: System.out.println(d+2);	*-2147483647*
8:System.out.println(d*2-1);	*-3*

6. 2. Verarbeitung elementarer Daten (I1-ID: 93rdw3918rg0) *2015*

In einer Klasse sind einige Methoden definiert:

```
public static void a() {
        char a = 'a';
        a++;
        int b = (int) a;
        b++;
        System.out.print( (char) ('A' + b - a));
}
public static boolean getTrue() {
        System.out.print("T");
        return true;
}
public static boolean getFalse() {
        System.out.print("F");
        return true;
}
public static void q(int i) {
        int j = 43;
        System.out.print("G");
        if ( i / j > 0) System.out.print("Q");
        else System.out.print("R");
}
public static void d() {
        double dd = Double.MIN_VALUE;
        dd *= 2;
        dd -= Double.MIN_VALUE;
        dd /= 2;
        System.out.print(dd);
}
```

Welche Ausgaben bewirken diese Aufrufe (in der *main*-Methode der gleichen Klasse)?

- *a();* — *B*
- *boolean val = getTrue() || getFalse();* — *T (wenn beim logischen oder das erste stimmt wird das zweite nicht mehr ausgefuehrt)*
- *val = getTrue() ^ getTrue();* — *TT (exlusives oder)*
- *System.out.print(val);* — false
- *q(42);* — *GR*
- *d();* — *0.0*

3. 2. Grundlegende Datentypen 2016

1. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) Das Leerzeichen ist ein druckbares Zeichen.
   - (B) iso8859-1 ist eine Teilmenge von ASCII.
   - (C) 'C'++ liefert das gleiche Ergebnis wie 'C'+1.
   - (D) "C" + 1 ist kein erlaubter Java-Ausdruck.

2. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) int ist ein Ganzzahldatentyp, char nicht.
   - (B) int x = Integer.MAX_VALUE + 1; ist eine erlaubte Anweisung.
   - (C) Der Wert von (3 < 7) && ((17 + 4) % (12 - 5)) != 0) ist false.
   - (D) Der Wert von (3 + 1/3 == 3 - 1/3) ist false.

Wählen Sie die richtige Antwortkombination aus (1P, bei einer Abweichung 0.5P, sonst 0P):
   - (A) Das Tabulatorzeichen ist ein Steuerzeichen.
   - (B) Unicode ist eine Teilmenge von iso8859-1.
   - (C) Der Wert von (7 % (5 - 2) == 0) || (7 % (5 + 2) == 0) ist true.
   - (D) Der Wert von (1000 + 11/12 == 1000 - 11/12) ist false.

3. Wählen Sie die zutreffenden Antworten aus:
   - (A) String s = "Hallo"; System.out.println(s.length);
     Ausgabe?

     (5,6,keine(wegen Syntaxfehler), keine (wegen logischem Fehler)) s.length() waere richtig

   - (B) System.out.println(10e-1f);
     Ausgabe?

     (1,1.0, keine(wegen Syntaxfehler), keine (wegen logischem Fehler)) 10 verschoben um -1 nach hinten

   - (C) 2 * (short) 0xCafe
     Typ des Ausdrucks?

     (short,int, undefiniert (wegen Syntaxfehler), undefiniert (wegen logischem Fehler))

   - (D) double y = Double.MIN_VALUE; System.out.println(y/2);
     Ausgabe?

     (0,0.0, keine(wegen Syntaxfehler), keine (wegen logischem Fehler))

   - (E) int x = Integer.MAX_VALUE, y = Integer.MAX_VALUE++, z = x+y;
     Wert von z?

     (-1,+1, undefiniert (wegen Syntaxfehler), undefiniert (wegen logischem Fehler))

   - (F) System.out.println((double) (1/2));
     Ausgabe?

   - (0,0.0, keine(wegen Syntaxfehler), keine (wegen logischem Fehler))

4. Wählen Sie unten die richtige Antwortkombination aus (1P für die richtige/0P für jede falsche Kombination):
   - (A) int[] z = {1}; System.out.println(z); produziert die Ausgabe 1.
   - (B) length ist ein private-Attribut von Feldern.
   - (C) Durch int[] a = {1, 2, 3}, b = new int[]; werden zwei neue Felder angelegt.
   - (D) Durch int[][][][] x = new char[32][32][32][16]; wird etwa 1Mb Heap-Speicher reserviert.(Stack) (1MB 1000000 B) 1048576
   - (E) int[] z = {1}; System.out.println(z.length()); produziert die Ausgabe 1.

7. 3. Rekursion 2015

Betrachten Sie folgende Klasse:

```
1:  class Myst {
2:          static void ery(int n) {
3:                  if (n == 0)
4:                          return;
5:                  ery( n-1);
6:                  A();
7:                  yre( n-1);
8:          }
9:          static void yre(int n) {
10:                 if (n == 0)
11:                         return;
12:                 ery( n-1);
13:                 B();
14:                 yre( n-1);
15:         }
16:         static void A() {
17:                 System.out.println("A");
18:         }
19:         static void B() {
20:                 System.out.println("B");
21:         }
22:         public static void main(String[] args) {
23:                 int n = Integer.parseInt(args[0]);
24:                 ery(n);
25:                 System.out.println();
26:         }
27:}
```

1. Geben Sie die ersten 8 Zeichen der Ausgabe zum Aufruf *java Myst 4* an:

- AABAABAA
- BBABBAAB
- ABAAABBA
- BABBBAAB
- AABAABBA
- BBABBAAB
- AABBAABA
- BBAABBAB
- AABABABB
- BBABABAA

2. Aus wie vielen Zeichen besteht der Ausgabestring beim Aufruf *java Myst 7* (ohne den abschließenden Zeilenwechsel durch Zeile 25)?

15, 16, 17, 49, 50, 51, 127, 128, 129, 255, 256, 257

3. Wird Zeile 14 durch *yre(n-2);* ersetzt, so führt beispielsweise der Aufruf *java Myst 4* zu einem Fehler.
Welche andere Zeile müsste korrigiert werden, um den Fehler zu beseitigen? Geben Sie nur die Nummer an:

Zeile 3, Zeile 4, Zeile 5, Zeile 6, Zeile 7, Zeile 10, Zeile 11, Zeile 12, Zeile 13,
(zu kleiner gleich)

4. Werde Zeile 12 durch *ery(n-2);* und zusätzlich Zeile 14 durch *yre(n-2)* ersetzt.
Geben Sie das maximale Argument n an, bei dem der Aufruf *java Myst n* nicht zu einem Fehler führt:

-3, -2, -1, 0, 1, 2, 3

7. 3. Statische Methoden (l1-ID: 0f1dmv607ih0)

```
public class StatischeMethoden {
        public static void q1(int n) {
                n = n * n;
        }
        public static void q2(int[] f, int i) {
                f[i] = f[i] * f[i];
        }
        public static void main(String[] args) {
                int n = 2;
                int[] f = { n, n, n};
                for (int i = 0; i < 3; i++) {
                        q1(n); q2(f,n);
                }
                System.out.println(n);
                System.out.println(f[n]);
        }
}
```

Wie lautet die erste Ausgabezeile?

2

Wie lautet die zweite Ausgabezeile?

256

8. 3. Rekursion (l1-ID: 4mqjzge16ih0) 2017

```
public static void strange(int n) {
        if ( n <= 2) return;
        System.out.print("(");
        strange(n-2);
        strange(n-3);
        System.out.print(")");
}
```

Welche Ausgabe bewirkt der Aufruf strange(7)? Tipp: Rekursionsbaum benutzen

((())())

9. 4. Felder: Fehler oder ...? 2015

Erzeugt untenstehende Klasse

- einen Compilezeitfehler?Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein,ansonsten einen Bindestrich:
- einen Laufzeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich:
- eine Ausgabe? Falls ja, so tragen Sie die Ausgabe ein, ansonsten einen Bindestrich: 0

```
1: class FehlerOderWie {
2:          public static void main(String[] args) {
3:                  int[] a = {2,3,5,7,11};
4:                  int[] b = new int[2];
5:                  b = a;
6:                  b[4] = -4;
7:                  if (a[4] >= 1) System.out.println(1);
8:                  else System.out.println(0);
9:          }
10: }
```

6. 4. Felder: Fehler oder...?

Erzeugt untenstehende Klasse

- einen Compilezeitfehler?Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich -
- einen Laufzeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich: -
- eine Ausgabe? Falls ja, so tragen Sie Ausgabe ein, ansonsten einen Bindestrich : false

```
1: class FehlerOderKeiner {
2:          public static void main(String[] args) {
```

```
3:                  int[] a = {2,3,5,7,11}; int[] b = { a[2], a[2]};
4:                  b = a;
5:                  b[1] = b[0];
6:                  System.out.println(a[0] != a[1]);
7:          }
8: }
```

11. 4. Iteration in einem Feld 2015

Tragen Sie die Werte der Feldkomponenten nach Ausführung der for-Anweisung ein.

```
int[] d = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
for (int i = 0; i < 10; i++)
        d[i] = d[9-d[9-d[9-i]]];
```

vorher 0 1 2 3 4 5 6 7 8 9
nachher 9 8 7 6 5 5 6 7 8 9

5. 4. Iteration in Feldern

```
int[] p = { 1, 4, 0, 3, 6, 2, 7, 5 };
int[] c = { 0, 0, 0, 0, 0, 0, 0, 0 };
int[] q = { 0, 0, 0, 0, 0, 0, 0, 0 };
for (int i = 0; i < 8; i++)
        c[p[i]] = c[p[i]] + 1;
for (int i = 0; i < 8; i++)
        q[p[i]] = i;
```

Tragen Sie die Werte der Feldkomponenten nach Ausführung des Java-Fragments ein: Index

index 0 1 2 3 4 5 6 7
c 1 1 1 1 1 1 1 1
q 2 0 5 3 1 7 4 6

5. 4. Iteration in Feldern (I1-ID: 435dkh80w5h0)

```
int[] p = { 1, 0, 7, 3, 5, 4, 6, 2 };
int[] c = { 0, 0, 0, 0, 0, 0, 0, 0 };
int[] q = { 0, 0, 0, 0, 0, 0, 0, 0 };
for (int i = 0; i < 8; i++)
        c[i] = p[p[i]];
for (int i = 0; i < 8; i++)
        q[p[i]] = i;
```

Tragen Sie die Werte der Feldkomponenten nach Ausführung des Java-Fragments ein:

Index 0 1 2 3 4 5 6 7
c 0 1 2 3 4 5 6 7
q 1 0 7 3 5 4 6 2

Aufgabe 3

Betrachten Sie folgende Klassen und beantworten Sie darunterstehenden Fragen ohne einen Computer zu Hilfe zu nehmen.

```
1: public class Katze {
2:    private String name;
3:    public Katze(String name) {
4:       this.name = name;
5:    }
6:    public String miezmiez() {
7:       return "Fauch!";
8:    }
9: }
1: public class Hauskatze extends Katze {
2:    private String name;
3:    public Hauskatze(String name) {
4:       super(name);
5:    }
6:    public String miezmiez() {
7:       return "Miau Miau";
```

```
8:    }
9: }
1: public class Tierfreund {
 2:    public static void main(String[] args) {
 3:       Hauskatze h = new Hauskatze("Schnurri");
 4:       Katze k = new Katze("Garfield");
 5:       System.out.println(((Katze)h).miezmiez());
 6:       System.out.println(((Hauskatze)k).miezmiez());
 7:       k = h;
 8:       System.out.println(((Hauskatze)k).miezmiez());
 9:    }
10: }
```

- In Tierfreund ist eine Zeile fehlerhaft. Welche Zeile ist es?
  Zeile 6
- Geben Sie an, welche Art Fehler die Zeile bewirkt (Compile- oder Laufzeitfehler).
  Laufzeit

Welche Ausgabe erzeugt java Tierfreund wenn die fehlerhafte Zeile gestrichen wird?

Miau Miau
Miau Miau

16. 5. OOP-Entwurf 2015

Die Klasse *Ente* erbt von der Klasse *Vogel*. In *Vogel* gibt es eine Instanzmethode *fliege(..)*, in Ente gibt es die Instanzmethode schnattern().
Die fliege-Methode von Ente-Objekten soll nun so angepasst werden, dass schnattern() immer unmittelbar am Anfang von fliege aufgerufen wird.
Wie ist das am besten zu realisieren?

```
public void fliege(Ziel z) {
        this.schnattern();
        // hier den Rumpf der fliege-Methode aus Vogel einfuegen
}
public void fliege(Ziel z) {
        this.schnattern();
        super.fliege(z);
}
public void fliege(Ziel z) {
        this.schnattern();
        this.fliege(z);
}
public void fliege(Ziel z) {
        this.schnattern();
        Vogel.fliege(z);
}
public void fliege(Ziel z) {
        this.schnattern();
}
```

17. 5. Einfach verkettete Listen 2015

x sei Knoten einer einfach verketten Liste (genauer: eine Referenz darauf). Wofür sorgt das folgende Codefragment?

```
t.next = x.next;
x.next = t;
```

- Knoten t wird direkt hinter Knoten x eingefügt.

(?) 11. 5. OOP-Entwurf

In einer Spiele-Software gibt es folgende Hierarchie von Java-Klassen, die die verschiedenen Sorten von Akteuren modelliert:

class Aktuer

class GuterKaempfer extends Aktuer implemnts Kaempfer
class BoeserKaempfer extends Aktuer
class GefaehrlicheKreatur extends Aktuer
class FreundlicheKreatur extends Aktuer

Die (hier nichtt gezeigte) Klasse Spiel benutzt die zugreifbaren Methoden obiger Klassen, um den Ablauf zu gestalten.

Wie wird folgende Zusatzanforderung korrekt modelliert? Alle Kaempfer-Objekte aber keine Kreatur-Objekte sollen mit einer Methode *benutzeWaffe()* ausgestattet werden.

- *benutzeWaffe()* in *Akteur* implementieren, in *GefaerlicheKreatur* und *FreundlicheKreatur* mit *private* kennzeichnen.
- Interface Kaempfer mit benutzeWaffe() definieren und durch GuterKaempfer und BoeserKaempfer implementieren lassen.
- Die Methode *benutzeWaffe()* nur in GuterKaempfer und in BoeserKaempfer wie benötigt implementieren.
  Weitere Klasse Kaempfer schreiben, die benutzeWaffe() implementiert, GuterKaempfer, BoeserKaempfer davon erben lassen.
- benutzeWaffe() mit leerem Rumpf in Akteur implementieren, in GuterKaempfer und BoeserKaempfer geeignet überschreiben.

15. 5. OOP-Verständnisfragen 2015 2016
Sei p eine Referenzvariable vom Typ X. Was kann alles der dynamische Typ von p sein? Wählen Sie aus den beiden Blöcken unten jeweils die zutreffenden Antworten aus (jeweils 1.5P für die richtige Kombination/0P für jede andere).

- (A) eine Erweiterungsklasse von X (falls X eine Klasse ist)
- (B) ein elementarer Datentyp (falls X die zugehörige Wrapper-Klasse ist)
- (C) eine Klasse, die X implementiert (falls X ein Interface ist)
- (D) eine Klasse, von der X direkt oder indirekt erbt (falls X eine Klasse ist)
- (E) ein Interface, das X als abstrakte Klasse erweitert (falls X eine Klasse ist)
- (F) eine Erweiterungsklasse einer Klasse, die X implementiert (falls X ein Interface ist)

12. 5. OOP-Verständnisfragen (I1-ID: c6j5qfl08ih0) 2017
Frage 1 Wodurch wird Kapselung bei der objektorientierten Programmierung erreicht?

Durch public/private/protected Instanzvariablen und public/private/protected Methoden.

Frage 2 Welche der Konzepte (A) Polymorphie, (B) Vererbung,(C) Kapselung, (D) Überladen kann man anhand folgender Situationsbeschreibung illustrieren?
Jedes Tier kann einen Laut von sich geben (ein Geräusch machen). Jeder Hund ist ein Tier und kann einen Laut von sich geben, nämlich: Wau Wau! Jede Katze ist ein Tier und kann einen Laut von sich geben, nämlich: Miau!

10. 5. Vermischtes zu Klassentypen 2015
1. Objekte verstehen:

```
public class Spieler {
        private int score;
        public Spieler( int s) {
                this.score = s;
        }
}
```

Zwei Spieler-Objekte werden instanziiert:
Spieler a = new Spieler(50), b = new Spieler(50);
Welchen Wert hat nun der Ausdruck a == b?

- true, false, das ist Compile-anhaengig, das ist speicherabhaengig, nichts von denen

2. Fremden Code deuten:

```
1:  public class FirstBpp extends hepno.users.Frame {
2:          private hepno.users.Wring wrags;
3:          public FirstBpp() {
```

```
4:                      wrags = new hepno.users.Wring();
5:          }
6:          public static void main(String[] args) {
7:                      FirstBpp app = new FirstBpp();
8:          }
9:  }
```

Was bewirkt Zeile 2?:

- auf hepno.users.Wring kann mit wrags zugegriffen werden
- wrags wird als Instazvariable vereinbart
- hepno.users.Wring wird als private-Objekt deklariert
- wrags instanziiert die Klasse hepno.users.Wring
- nichts von denen

3. Statische Methoden und Instanzmethoden:

```
public class Klasse {
        private final int zahl = 10;
        double a() {
                System.out.println(zahl);
                return c();
        }
        void b() {
                System.out.println(this);
        }
        double c() {
                double r = Math.random();
                System.out.println(r);
                return r;
        }
        void d() {
                a();
                a();
        }
        int e() {
                return zahl;
        }
}
```

Welche der obigen Methoden ist ihrer Natur nach eher eine statische Methode (und sollte daher mit static versehen werden)?

- a, b, c, d, e

4. Fehlersuche:

```
public class Test {
        private int testCount;
        public static int getCount(){
                return testCount;
        }
        public Test(){
                testCount++;
        }
}
```

Obiger Code verursacht einen Compilezeitfehler. Was ist das Problem?

- testCount wurde nicht initialisiert
- auf testCount wurde nicht zugegriffen
- getCount() ist statisch
- testCount ist private
- Test() hat keinen Ergebnistyp
- nichts von denen

5. Tracing:

[Hinweis: Untenstehender Code ist syntaktisch korrekt, denn auf Klassenvariablen kann

auch mit Hilfe von Objektreferenzen zugegriffen werden.]

```
public class Tset {
        static int tsetCount = 0;
        public Tset(){
                tsetCount++;
        }
        public static void main(String [] Args){
                // Hinweis: auf Klassenvariablen kann man auch
                Tset t1 = new Tset();
                System.out.print(t1.tsetCount + ", ");
                Tset t2 = new Tset();
                System.out.print(t1.tsetCount + " "+ t2.tsetCount + ", ");
                Tset t3 = new Tset();
                System.out.print(t1.tsetCount+ " "+ t2.tsetCount+ " "+ t3.tsetCount);
                System.out.println();
        }
}
```

Welche Ausgabe produziert der Code?

- 0, 0 0, 0 0 0
- 1, 1 1, 1 1 1
- 1, 2 2, 3 3 3
- 1, 2 3, 4 5 6
- nichts von denen

6. Begriffsverständnis:

Wählen Sie aus dem Menü die einzige Aussage aus, die NICHT korrekt ist.

- In einer Klasse darf es Methoden geben, die den gleichen Namen haben wie ein Konstruktior
- In einer Klasse darf es Methoden geben mit gleichem Namen und gleichen Ergebnistyben, aber unterschiedlich vielen Argumenten
- In einer Klasse darf es Methoden geben, die den gleichen Namen und gleiche Argumenttyplisten haben aber unterschiedliche Ergebnistypen.
- In einer Klasse darf es mehrere Konstruktoren mit den gleichen Namen geben

14. 5. Eine Bibliothek für Permutationen (I1-ID: 8834zrd08ih0) 2017

Definition

Permutationen entsprechen int-Feldern einer Länge n, die die Werte 0,1,...,n-1 speichern (in beliebiger Reihenfolge).
Das Quadrat der Permutation p, ist die Permutation q mit: q[i] ist gleich p[p[i]] für alle Indexwerte i..
Die Unordnung der Permutation p ist die Anzahl der Indexpaare (i,j) mit i < j und p[i] > p[j]. Zum Beispiel hat (2,0,1) Unordnung 2..

Aufgabe

Schreiben Sie eine Klasse Permutation mit den statischen Methoden

- quadrat(..) und unordnung(..), die aus jeder Clientklasse aufgerufen werden können und Folgendes leisten:
- 1. quadrat(p) ermittelt und liefert das Quadrat der Permutation p
- 2. unordnung(p) ermittelt und liefert die Unordnung von p.

Es ist jeweils gesichert, dass p wirklich eine Permutation ist - nicht separat prüfen!

```
public static int[] quadrat(int[] p){
        int[] q = new int[p.lenght];
        for(int i = 0; i < p.lenght; i++){
                q[i] = p[p[i]];
        }
        return q;
}

public static int unordnung(int[] p){
        int unordnung = 0;
        for(int i = 0; i < p.length; i++){
                for(int j = 0; j < p.length; j++){
```

```
                        if(i<j && p[i] > p[j]){
                                unordnung++;
                        }
                }
        }
        return unordnung;
}

public static boolean check(p){
        for(nt i = 0; i < p.lenght; i++){
                if(p[i] < 0 || p[i] >= p.lenght){
                        return false;
                }
        }
        int[] c = new int[p.lenght];
        for(int i = 0; i < p.lenght; i++){
                if(c[p[i]] == 1){
                        return false;
                }
                c[p[i]] = 1;
        }
        return true;
}

public static int[] invers(int[] p){
        int q = new int[p.length];
        for(int i = 0; i < p.lenght; i++){
                q[p[i]] = i;
        }
}
```

14. 5. Eine Bibliothek für Permutationen

Definition

1. Wir nennen ein int-Feld p der Länge n eine Permutation, wenn p[0], …, p[n-1] im Bereich liegen und jeder Wert genau einmal vorkommt.
2. Die Permutation q heißt Inverse der Permutation p, wenn für alle i der Wert von q[p[i]] gleich i ist.

Aufgabe

Schreiben Sie eine Klasse Permutation mit den statischen Methoden check(..) und invers(..), die aus jeder Clientklasse aufgerufen werden können und Folgendes leisten:

- check(p) überprüft, ob p eine Permutation ist und liefert das Ergebnis (Hinweis: z.B. zuerst in einer Schleife prüfen, ob die Bereichsbedingung erfüllt ist, dann mit zweiter Schleife in einem Hilfsfeld c die Vorkommen der Werte p[0], …, p[n-1] erfassen)
- invers(p) ermittelt und liefert die Inverse von p. (Dabei ist bereits gesichert, dass p wirklich eine Permutation ist - nicht separat prüfen! Hinweis: Feld anlegen und in Schleife die Komponenten so setzen, wie es Definition 2. vorschreibt.)

15. 5. Eine einfache Klasse (I1-ID: 3g3j2i70x5h0) 2017

Programmieren Sie eine einfache Klasse Wein mit der Angaben zu Weinflaschen abgespeichert werden können.
Jeder Wein zeichnet sich durch seine unveränderliche Sorte aus (z.B. "2001er Goldtropfen"), und durch seinen Preis (z.B. 13,50€).
Die folgenden Operationen sollen durch Methoden/Konstruktoren ermöglicht werden (wenn nicht anders angegeben: aufrufbar aus beliebigen Clientklassen):

- Erzeugen einer Wein-Instanz unter Angabe der Sorte
- Erzeugen einer Wein-Instanz unter Angabe der Sorte und des Preises
- Setzen des Preises einer Instanz (nur von Wein-Methoden aus),
- Abfragen des Preises einer Instanz
- Abfragen der Anzahl der erzeugten Instanzen und der Summe ihrer Preise.

VERWENDEN SIE FÜR DIE DATEN BITTE DIESE BEZEICHNER: sorte, preis, anzahl, gesamt Zum Einrücken bitte unbedingt die Tabulatortaste anstelle Leerzeichen benutzen (siehe unten).

```
public Wein{
	private  final String sorte;
	private int preis;
	private static int anzahl = 0;
	private static int gesamt = 0;

	public Wein(String sorte){
		this.sorte = sorte;
		anzahl++;
	}
	public Wein(String sorte, int preis){
		this.sorte = sorte;
		this.preis = preis;
		anzahl++;
		gesamt += preis;
	}
	private void setPreis(int preis){
		this.preis = preis;
	}
	public getPreis(){
		return preis;
	}
	public static int getAnzahl(){
		return anzahl;
	}
	public static int getGesamt(){
		return gesamt;
	}
}
```

(?) 15. 5. Eine einfache Klasse 2016

Programmieren Sie eine einfache Klasse Auto mit der Angaben zu Autos abgespeichert werden können.
Jedes Auto zeichnet sich durch seinen Typ aus (z.B. "Mercedes") und durch seinen Preis (z.B. 30000).
Die folgenden Operationen sollen durch Methoden/Konstruktoren ermöglicht werden (wenn nicht anders angegeben: aufrufbar aus beliebigen Clientklassen):

- Erzeugen einer Auto-Instanz unter Angabe des Typs, der Typ soll nach Erzeugen des Objekts nicht mehr änderbar sein
- Erzeugen einer Auto-Instanz unter Angabe des Typs und des Preises,
- Setzen des Preises einer Instanz (nur von Auto-Methoden aus),
- Abfragen des Preises einer Instanz
- Abfragen der Anzahl der erzeugten Instanzen und der Summe der Preise

VERWENDEN SIE FÜR DIE DATEN BITTE DIESE BEZEICHNER: *typ, preis, anzahl, gesamt*

(?) 18. 5. Eine einfache Klasse 2015

Programmieren Sie eine einfache Klasse *Buch* mit der Angaben zu Büchern abgespeichert werden können. Jedes Buch zeichnet sich durch seinen Titel aus, und durch seine Seitenzahl.
Die folgenden Operationen sollen möglich sein:

- Erzeugen einer *Buch*-Instanz unter Angabe des Titels
- Erzeugen einer *Buch*-Instanz unter Angabe des Titels und der Seitenzahl
- Setzen und Abfragen der Seitenzahl jeder einzelnen Instanz.
- Die Anzahl erzeugter Instanzen und die Summe ihrer Seitenzahlen soll durch Methodenaufrufe ermittelt werden können.

VERWENDEN SIE DABEI BITTE DIESE BEZEICHNER: *titel, seiten, anzahl, gesamt*
Außerdem soll die Klasse für einen ersten Test mit einer *main* -Methode ausgestattet sein, die folgende Aktionen auslöst:

- ein *Buch*-Objekt erzeugen nur unter Angabe des Titels *Lieblingsbuch* und die Referenz der Variablen *b* zuweisen

- anschließend die Seitenzahl dieses Buchobjekts auf *500* setzen
- schließlich mittels *b* die Angaben zu dem erzeugten Objekt ausgeben: *Lieblingsbuch, 500 S.*

(?) 14. 6. Vermischtes zu Algorithmen und Datenstrukturen 2015

Markieren Sie alle zutreffenden Aussagen:

- Binäre Suche erfordert O(n) Vergleiche auf **unsortierten** Feldern.
- Mit einem Stack kann man z.B. feststellen, ob ein Ausdruck korrekt geklammert ist.
- Die abstrakte Datenstruktur Queue lässt sich nicht mit einfach verketteten Listen realisieren.
- Quicksort ist auf vorsortierten Feldern besonders schnell.
- Mergesort benötigt auf allen Feldern der Länge 2^m immer di\e gleiche Anzahl an Vergleichen.
- Selectionsort benötigt auf allen Feldern der Länge 2^m immer die gleiche Anzahl an Vergleichen.
- Binäre Bäume der Tiefe n haben mindestens n - 1 und höchstens 2^n - 1 innere Knoten. (geht mit n+1 statt n)
- Felder sind nicht als Grundlage der abstrakten Datenstruktur Stack geeignet.

9. 6. Binäre Suche protokollieren (l1-ID: i5k29e615ih0) 2017

Betrachten Sie folgende Implementierung der binären Suche in einem aufsteigend sortierten Feld:

```
public static int binarysearch(int[] f, int q) {
int lo = 0, hi = f.length - 1;
while (lo<=hi) {
        int m=(hi+lo)/2;
        if (q<f[m]) {
                System.out.print("L");
                hi = m - 1;
        }
        else if ( q>f[m] ){
                        System.out.print("R");
                        lo=m+1;}
                else {
                System.out.print("!");
                return m;
                }

}
System.out.print("-");
return -1;
}
```

Hier ein Ausschnitt aus dem Unit-Test.

```
int[] g = { 1, 1, 3, 5,    6, 7,   8, 9 ,   9, 9, 10, 11, 11, 12, 13, 15};
int s = g[4];
System.out.println(binarysearch(g,s));
```

Welche Ausgabe wird erzeugt?

LRL!4

11. 6. Einen Stack kopieren (l1-ID: kw3g7f70y6h0) 2017

Betrachten Sie folgende API von Stacks zum Speichern von *int*-Werten (alle Methoden/Konstruktoren *public)*:

public class Stack

- *Stack()* erzeugt einen neuen Stack
- boolean empty() testet, ob dieser Stack leer ist
- void push(int i) speichert i in diesem Stack
- int pop() in diesem Stack zuletzt gespeicherter Wert wird geliefert und entfernt

Stellen Sie die Zeilen für den Rumpf einer weiteren Instanzmethode

```
public Stack copy() {
        Stack h = new Stack();
        Stack c = new Stack();
        while (!this.empty()) {
                h.push( this.pop()); }
        while (!h.empty()) {
                int i = h.pop();
                this.push(i);
                c.push(i);}
        return c;
}
```

in richtiger Reihenfolge zusammen, so dass ihr Aufruf einen Stack mit gleichem Inhalt wie dieser liefert (d.h. es soll dann möglich sein, Original und Kopie unabhängig zu benutzen: Einspeichern beim Original ändert nicht den Zustand der Kopie usw.).

Idee: (1) alle Werte aus diesem Stack zunächst aus- und in einen neuen Stack h einspeichern, dann (2) von dort alle Werte aus- und sowohl in diesen Stack als auch in neuen Stack c einspeichern, schließlich Referenz der Kopie liefern.

(?) 13. 6. Analyse von Algorithmen 2016 (diese Aufgabe ist nicht Klausurrelevant denke ich)

1.Betrachten Sie folgenden Ausschnitt aus einem größeren Programm:

```
1: for (int i = 0; i < N; i++)
2:         for (int j = i+1; j < N; j++)
3:                 for (int k = j+1; k < N; k++)
4:                         verarbeite(daten);
```

Die Ausführungszahl *V(N)* von Zeile 4 bei Problemgröße *N* soll untersucht werden. Welcher der folgenden Ausdrücke ist am besten für die weitere asymptotische Analyse geeignet?

N(N-1)*(N-2), N*(N-1)*(N-2)/3 , N*(N-1)*(N-2)/6 , ~N^3, ~(n^3)/3 ~(N^3)/6, O(N^3), O((N^3)/3), O((N^3)/6)*

2. Eine Implementation wird zahlreichen Verdopplungstests unterzogen, mit zufällig generierten Eingabedaten. Die Ergebnisse — alle sehr ähnlich — sehen typischerweise so aus:

Problemgröße N:	*102*	*2048*	*4096*	*8192*	*16384*	*32768*	*65536*	*131072*	*62144*	*524288*	*1048576*	*2097152*	*4194304*	*8388608*
Zeit (s) :	*0.00*	*0.01*	*0.01*	*0.02*	*0.03*	*0.06*	*0.14*	*0.30*	*0.65*	*1.24*	*2.46*	*4.98*	*10.02*	*19.94*

Außerdem kann man nachweisen, dass die Ausführungszahl der Kernoperationen bei spezieller Struktur der Eingabe linear mit der Problemgröße wächst. Welche dieser Hypothesen über das Laufzeitverhalten erscheint dann am plausibelsten?

- *worst-case quadratisch und best-case logarithmisch*
- *average-case linear und best-case quadratisch*

- *best-case linear und average-case quadratisch*
- *average-case quadratisch und worst-case linear*
- *average-case linear und best-case linear*
- *worst-case quadratisch und average-case quadratisch*
- *best-case logarithmisch und worst-case logarithmisch*
- *average-case quadratisch und worst-case quadratische*

-- 3. Welche dieser Aussagen trifft NICHT zu?

- *Bubblesort ist quadratisch im worst case*
- *Insertion- und Selection-sort haben asymptotisch gleiche worst-case Laufzeit*
  Binaere Suche erfordert sortierte Eingabe
- *Quicksort hat auf sortierter Eingabe lineare Laufzeit*
- *Quicksort ist ein in-place Sortierverfahren*
- *Vergleichsanzahl bei Selection-sort hängt nur von Feldlänge ab*
- *vergleichsbasierte Sortierverfahren haben mindesten linearlogariithmisches worst-case-Verhalten*

12. 6. Stack und Queue in Aktion

1. Auf einem Stack werden in gemischter Reihenfolge 10 Push- und 10 Pop-Operationen ausgeführt. Es ist bekannt, dass die Push-Operationen die Zahlen 0, 1, 2, …, 9 in dieser Reihenfolge einspeichern und dass bei jedem Pop der ausgespeicherte Wert ausgegeben wird.
Welche Ausgabelisten sind möglich?
Wählen Sie aus den beiden Blöcken jeweils die zutreffende Antwort aus (jeweils 2P für die richtige Kombination/0P für jede andere).

- (A) 1 2 3 4 5 6 9 8 7 0
- (B) 0 4 6 5 3 8 1 7 2 9
- (C) 1 4 7 9 8 6 5 3 0 2
- (D) 2 1 4 3 6 5 8 7 9 0

2. Auf einer Queue werden in gemischter Reihenfolge 10 Enqueue- und 10 Dequeue-Operationen ausgeführt. Es ist bekannt, dass die Enqueue-Operationen die Zahlen 0, 1, 2, …, 9 in dieser Reihenfolge einspeichern und dass bei jedem Dequeue der ausgespeicherte Wert ausgegeben wird. Welche Ausgabelisten sind möglich? Wählen Sie aus den beiden Blöcken jeweils die zutreffende Antwort aus (jeweils 1P für die richtige Kombination/0P für jede andere).

- (E) 4 6 8 7 5 3 2 9 0 1
- (F) 2 5 6 7 4 8 9 3 1 0
- (G) 0 1 2 3 4 5 6 7 8 9
- (H) 4 3 2 1 0 5 6 7 8 9

(?) 9. 6. Queue aus zwei Stacks bauen 2016

Mit zwei Stacks kann man eine Queue implementieren:

- enqueue(...) entspricht Einspeichern in inbox
- dequeue() entspricht Ausspeichern aus outbox.

Unter Verwendung dieser API
public class Stack

- boolean empty() testet, ob dieser Stack leer ist
- void push(String s) speichert s in diesem Stack

- String pop() liefert zuletzt gespeicherten String aus diesem Stack und entfernt ihn

sähe die entsprechende Klasse z.B. so aus:

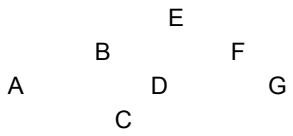
```
public class ZSQueue { // Zwei-Stack-Queue
        private Stack inbox, outbox;
        public ZSQueue() { ... }
        public boolean empty() { ... } // Leerheitstest
        public void enqueue(String s) { ... } // einspeichern
        public String dequeue() { // ausspeichern und liefern
                // Rumpf (Ihre Aufgabe)
        }
}
```

Stellen Sie die Zeilen für den Rumpf der Methode dequeue() richtiger Reihenfolge zusammen:

- ist die Queue leer, ist Ausspeichern nicht möglich!
- teste ob outbox leer ist und fülle in diesem Fall den kompletten Inhalt von inbox um in outbox, dann ein Element aus outbox ausspeichern und liefern.

13. 6. Binärer Suchbaum 2015

Es geht um binäre Suchbäume, deren Schlüssel Zeichen in alphabetischer Anordnung sind.



1. Werden die Schlüssel obigen binären Suchbaums in Inorder-, Preorder- oder Postorder-Reihenfolge ausgegeben, so erscheint ein Schlüssel immer an gleicher Position.
Welcher ist das?

Inorder ABCDEFG
preorder (erst wurzel dann linker dann rechter Teilbaum) EBADCFG
postorder(erst linker dann echter teilbaum und dann wurzel) ACDBGFE

F

2. Wählen Sie für die Schlüssel eine Einfügereihenfolge aus, bei der ein Binärbaum der Tiefe 2 entsteht:
DBFACEG

Info 1 - 2015 `// Viel Spass damit und lernt brav ;)`

1. 1. Vermischtes zur Einführung

Markieren Sie *alle* zutreffenden Aussagen:

- Top-Down-Entwurf besteht darin, Details erst grob, dann feingliedrig zu sortieren.
- Dem Inhalt einer Speicherzelle kann man nicht ansehen, ob sie eine Zahl oder einen Befehl speichert..
- Beim euklidischen Algorithmus wird dieser Block iteriert: `{ x = y; y = r; r = x % y; }`
- Beim Testen wird das Programm auf allen erlaubten Eingaben ausgeführt.
- Beim Pipelining werden Eingaben eines Prozesses zur Ausgabe eines anderen geleitet.
- Beim Laden wird die Befehlsfolge in aufeinanderfolgende Register kopiert.
- Editieren und Testen gehören zu jedem Programmentwicklungszyklus.
- `public static void main(String[] arks)` ist Kopf der `main`-Methode.

2. 1. Zweierpotenzen

Ergänzen Sie untenstehende `main`-Methode so, dass der Aufruf `java Zweierpotenzen` n
die Liste der Zweierpotenzen von $2^0 = 1$ bis 2^n zeilenweise ausgibt.

```
public class Zweierpotenzen {
    public static void main (String[] args) {
        // ---------------------------
        .....    // soviele Codezeilen wie nötig
        .....    // (angemessen EINGERÜCKT)
        ..
        // ---------------------------
   }
}
```

Beispiel `java Zweierpotenzen 3` gibt diese Liste aus:

```
1
2
4
8
```

Algorithmische Idee (bitte GENAU diese Variablennamen benutzen):

initialisiere:
- Kommandozeilenargument (interpretiert als `int`-Wert) in `n` speichern
- `int`-Variable `p` als Akkumulator initialisieren

iteriere so oft wie nötig:
- `p` ausgeben und geeignet aktualisieren

finalisiere: entfällt

3. 1. Quadratzahlentabelle

Die `int`-Variable `n` speichere die ganze Zahl n.

Die ersten n positiven Zahlen sollen zusammen mit ihren Quadraten jeweils auf einer Zeile

ausgegeben werden.

Beispiel für $n = 4$:

```
1 : 1
2 : 4
3 : 9
4 : 16
```

Im Fall $n \leq 0$ soll nichts ausgegeben werden.

"Programmieren" Sie dies durch Auswahl der richtigen Codezeilen, und zwar einmal mit einer `for`-Schleife und einmal mit einer `do-while`-Schleife.

Achten Sie auf den Unterschied zwischen `System.out.print(..)` und `System.out.println(..)`.

- mit einer for-Schleife:

```
for (int i = 1; i < n+1; i++) {
System.out.print(i + ": ");
System.out.println( i^2);
}
```

- mit einer do-while-Schleife:

```
int i = 1;
do {
System.out.print(i + ": ");
System.out.println( i^2);
i++;
} while ( i < n+1);
```

4. 2. Vermischtes zu elementaren Daten

Markieren Sie *alle* zutreffenden Aussagen:

- Der ASCII-Standard legt die Anordnung der 256 Zeichen fest.
- Zu jedem Datentyp gibt es ein Informationssystem.
- Java-Code darf beliebige Unicode-Zeichen enthalten.
- `int`-Werte können verlustfrei in `float`-Variablen gespeichert werden.
- `5/4 == 1` hat den Wert `true`.
- `Integer.MAX_VALUE++` verursacht keinen Ganzzahlüberlauf.
- Anstatt `if (b > 0 && a/b < 2) ..` schreibt man besser `if (a/b < 2 && b > 0) ...`
- Ein Gleitkomma-Überlauf ist kein Laufzeitfehler.

5. 2. Integer-Arithmetik

Sei `d` eine `int`-Variable mit dem Wert `Integer.MAX_VALUE` (also 2147483647). Welche Textausgabe erzeugen diese Anweisungen?:

```
1:  System.out.println(-d);
2:  System.out.println(d-1);          // dropdown menue mit loesungen
3:  System.out.println(1+2*d);
4:  System.out.println(-(2*d));
```

6. 2. Verarbeitung elementarer Daten (I1-ID: 93rdw3918rg0)

In einer Klasse sind einige Methoden definiert:

```
public static void a() {
    char a = 'a';
    a++;
    int b = (int) a;
    b++;
    System.out.print( (char) ('A' + b - a));
}
public static boolean getTrue() {
    System.out.print("T");
    return true;
}
public static boolean getFalse() {
    System.out.print("F");
    return true;
}
public static void q(int i) {
    int j = 43;
    System.out.print("G");
    if ( i / j > 0) System.out.print("Q");
    else System.out.print("R");
}
public static void d() {
    double dd = Double.MIN_VALUE;
    dd *= 2;
    dd -= Double.MIN_VALUE;
    dd /= 2;
    System.out.print(dd);
}
```

Welche Ausgaben bewirken diese Aufrufe (in der `main`-Methode der gleichen Klasse)?

```
a();



boolean val = getTrue() || getFalse();



val = getTrue() ^ getTrue();



System.out.print( val);



q(42);



d();
```

7. 3. Rekursion

Betrachten Sie folgende Klasse:

```
 1:  class Myst {
 2:      static void ery(int n) {
 3:          if (n == 0)
 4:              return;
 5:          ery( n-1);
 6:          A();
 7:          yre( n-1);
 8:      }
 9:      static void yre(int n) {
10:          if (n == 0)
11:              return;
12:          ery( n-1);
13:          B();
```

```
14:          yre( n-1);
15:      }
16:      static void A() {
17:          System.out.print("A");
18:      }
19:      static void B() {
20:          System.out.print("B");
21:      }
22:      public static void main(String[] args) {
23:          int n = Integer.parseInt(args[0]);
24:          ery(n);
25:          System.out.println();
26:      }
27:  }
```

1. Geben Sie die ersten 8 Zeichen der Ausgabe zum Aufruf `java Myst 4` an:

2. Aus wievielen Zeichen besteht der Ausgabestring beim Aufruf `java Myst 7` (*ohne* den abschließenden Zeilenwechsel durch Zeile 25)?

3. Wird Zeile 14 durch `yre( n-2);` ersetzt, so führt beispielsweise der Aufruf `java Myst 4` zu einem Fehler.
   Welche *andere* Zeile müsste korrigiert werden, um den Fehler zu beseitigen? Geben Sie nur die Nummer an:

4. Werde Zeile 12 durch `ery( n-2);` und zusätzlich Zeile 14 durch `yre( n-2)` ersetzt.
   Geben Sie das maximale Argument `n` an, bei dem der Aufruf `java Myst n` *nicht* zu einem Fehler führt:

8. 3. Grafik-Bibliothek benutzen

Die Klasse `public class StdDraw` bietet Methoden zum Zeichnen geometrischer Objekte:

- die quadratische Zeichenfläche (Seitenlänge 1) wird in einem Fenster mit dem Titel "File" angezeigt
- Koordinaten wie üblich (z.B. gehen die blauen Strecken oben alle durch den Punkt $(x, y) = (1, 0)$, die diagonal gegenüberliegende Quadratecke ist $(0, 1)$)
- gezeichnet wird mit der zuletzt gesetzen Stiftfarbe, wobei später Gezeichnetes Früheres (teilweise) überdecken kann

`void setPenColor(Color c)`	Stiftfarbe setzen
`BLACK`, `RED`, `BLUE`	in `StdDraw` definierte `Color`-Konstanten (schwarz, rot, blau)
`void line(double x0, double y0, double x1, double y1)`	Strecke zwischen Punkten `(x0,y0)` und `(x1,y1)`
`void square(double x, double y, double r)`	Quadrat: Mittelpunkt `(x,y)` u. Seitenlänge `2*r` (parallel zu Koordinatenachsen)
`void circle(double x, double y, double r)`	Kreis mit Mittelpunkt `(x,y)` und Radius `r`

Nachfolgend der Ausschnitt einer Methode einer Clientklasse von `StdDraw`.
Ergänzen Sie Zeilen so, dass der Aufruf `zeichneB(n)` (z.B. in der `main`-Methode dieser Klasse) eine Figur erzeugt aus `n` mal `n` Kreisen usw.
Genau so wie oben gezeigt. Achten Sie dabei auf gut verständlichen Code.

```
static void zeichneB(int n) {
```

```
                                // Ekelhafte Dropdown menues

    for (int i = 0; i < n; i++)                    StdDraw.circle(r*i,r*j,r/2);
      for (int j = 0; j < n; j++)                StdDraw.setPenColor(StdDraw.RED);
       StdDraw.setPenRadius(0.02);                  StdDraw.line(1.0,0.0,0,r+2*r*i);
         for (int i = 0; i < n; i++)              StdDraw.setPenColor(StdDraw.BLUE);
         double r = 1.0/(2*n);                     for (int j = 0; j < n; j++)
                                                    StdDraw.square(0.5,0.5,1);
}
```

9. 4. Felder: Fehler oder …?

Erzeugt untenstehende Klasse

- einen Compilezeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich:
- einen Laufzeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich:
- eine Ausgabe? Falls ja, so tragen Sie die Ausgabe ein, ansonsten einen Bindestrich:

```
 1:  class FehlerOderWie {
 2:      public static void main(String[] args) {
 3:          int[] a = {2,3,5,7,11};
 4:          int[] b = new int[2];
 5:          b = a;
 6:          b[4] = -4;
 7:          if (a[4] >= 1) System.out.println(1);
 8:          else System.out.println(0);
 9:      }
10:  }
```

10. 4. Vermischtes zu Klassentypen

1. Objekte verstehen:

   ```
   public class Spieler {
       private int score;
       public Spieler( int s) {
           this.score = s;
       }
   }
   ```

   Zwei `Spieler`-Objekte werden instanziiert:

   ```
   Spieler a = new Spieler(50), b = new Spieler(50);
   ```

   Welchen Wert hat nun der Ausdruck a == b?

2. Fremden Code deuten:

   ```
   1:  public class FirstBpp extends hepno.users.Frame {
   2:      private hepno.users.Wring wrags;
   3:      public FirstBpp() {
   4:          wrags = new hepno.users.Wring();
   5:      }
   6:      public static void main(String[] args) {
   7:          FirstBpp app = new FirstBpp();
   8:      }
   9:  }
   ```

   Was bewirkt Zeile 2?:
   `hepno.users.Wring wird als private-Objekt deklariert`

3. Statische Methoden und Instanzmethoden:

```
public class Klasse {
    private final int zahl = 10;
    double a() {
        System.out.println(zahl);
        return c();
    }
    void b() {
        System.out.println(this);
    }
    double c() {
        double r = Math.random();
        System.out.println(r);
        return r;
    }
    void d() {
        a();
        a();
    }
    int e() {
        return zahl;
    }
}
```

Welche der obigen Methoden ist ihrer Natur nach eher eine statische Methode (und sollte daher mit `static` versehen werden)?

`c`

4. Fehlersuche:

```
public class Test {
    private int testCount;
    public static int getCount(){
        return testCount;
    }
    public Test(){
        testCount++;
    }
}
```

Obiger Code verursacht einen Compilezeitfehler. Was ist das Problem?

`getCount() ist statisch`

5. Tracing:
[Hinweis: Untenstehender Code ist syntaktisch korrekt, denn auf Klassenvariablen kann auch mit Hilfe von Objektreferenzen zugegriffen werden.]

```
public class Tset {
    static int tsetCount = 0;
    public Tset(){
        tsetCount++;
    }
    public static void main(String [] Args){
        // Hinweis: auf Klassenvariablen kann man auch
        tset t1 = new tset();
        System.out.print(t1.tsetCount + ", ");
        tset t2 = new tset();
        System.out.print(t1.tsetCount + " "+ t2.tsetCount + ", ");
        tset t3 = new tset();
        System.out.print(t1.tsetCount+ " "+ t2.tsetCount+ " "+ t3.tsetCount);
        System.out.println();
    }
}
```

Welche Ausgabe produziert der Code?

6. Begriffsverständnis: Wählen Sie aus dem Menü die einzige Aussage aus, die NICHT korrekt ist.

In einer Klasse darf es Methoden geben, die den gleichen Namen und gleiche Argumenttyplisten haben aber unterschiedliche Ergebnistypen.

11. 4. Iteration in einem Feld

Tragen Sie die Werte der Feldkomponenten nach Ausführung der `for`-Anweisung ein.

```
int[] d = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9};

for (int i = 0; i < 10; i++)
    d[i] = d[9-d[9-d[9-i]]];
```

vorher	0	1	2	3	4	5	6	7	8	9
nachher	9	8	7	6	5	5	6	7	8	9

12. 6. Stapelarbeit

Auf einem Stack werden *in gemischter Reihenfolge* 10 Push- und 10 Pop-Operationen ausgeführt.

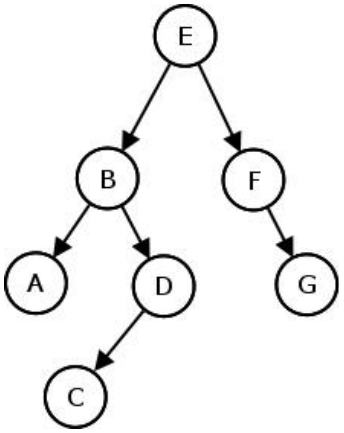
Es ist bekannt, dass die Push-Operationen die Zahlen 0, 1, 2, …, 9 in dieser Reihenfolge einspeichern und dass bei jedem Pop der ausgespeicherte Wert ausgegeben wird.

Markieren Sie alle Ausgabelisten, die dabei entstehen können (das können 0, 1 oder mehrere sein):

1 4 7 9 8 6 5 3 0 2
4 3 2 1 0 9 8 7 6 5
2 5 6 7 4 8 9 3 1 0
4 6 8 7 5 3 2 9 0 1

13. 6. Binärer Suchbaum

Es geht um binäre Suchbäume, deren Schlüssel Zeichen in alphabetischer Anordnung sind.



1. Werden die Schlüssel obigen binären Suchbaums in Inorder-, Preorder- oder Postorder-Reihenfolge ausgegeben, so erscheint ein Schlüssel immer an gleicher Position.
   Welcher ist das?

2. Wählen Sie für die Schlüssel eine Einfügereihenfolge aus, bei der ein Binärbaum der Tiefe 2 entsteht:

14. 6. Vermischtes zu Algorithmen und Datenstrukturen

Markieren Sie *alle* zutreffenden Aussagen:

- Binäre Suche erfordert $O(n)$ Vergleiche auf *unsortierten* Feldern.
- Mit einem Stack kann man z.B. feststellen, ob ein Ausdruck korrekt geklammert ist.
- Die abstrakte Datenstruktur Queue lässt sich nicht mit einfach verketteten Listen realisieren.
- Quicksort ist auf vorsortierten Feldern besonders schnell.
- Mergesort benötigt auf allen Feldern der Länge 2^m immer die gleiche Anzahl an Vergleichen.
- Selectionsort benötigt auf allen Feldern der Länge 2^m immer die gleiche Anzahl an Vergleichen.
- Binäre Bäume der Tiefe n haben mindestens $n-1$ und höchstens 2^n-1 *innere Knoten*.
- Felder sind nicht als Grundlage der abstrakten Datenstruktur Stack geeignet.

15. 5. Verständnisfragen zu OOP

Sei p eine Referenzvariable vom Typ A. Was kann alles der dynamische Typ von p sein?

Markieren Sie *alle* Möglichkeiten, die Java erlaubt:

- eine Erweiterungsklasse einer Klasse, die A implementiert (falls A ein Interface ist)
- ein Interface, dass A als abstrakte Klasse erweitert (falls A eine Klasse ist)
- eine Klasse, von der A abgeleitet ist (falls A eine Klasse ist)
- eine Erweiterungsklasse von A (falls A eine Klasse ist)
- ein elementarer Datentyp (falls A die zugehörige Wrapper-Klasse ist)
- eine Klasse, die A implementiert (falls A ein Interface ist)

16. 5. OOP-Entwurf

Die Klasse `Ente` erbt von der Klasse `Vogel`. In `Vogel` gibt es eine Instanzmethode `fliege(..)`, in `Ente` gibt es die Instanzmethode `schattern()`.

Die `fliege`-Methode von `Ente`-Objekten soll nun so angepasst werden, dass `schnattern()` immer unmittelbar am Anfang von `fliege` aufgerufen wird.

Wie ist das am besten zu realisieren?

○
```
public void fliege(Ziel z) {
    this.schnattern();
    // hier den Rumpf der fliege-Methode aus Vogel einfuegen
}
```

○
```
public void fliege(Ziel z) {
    this.schnattern();
    super.fliege(z);
}
```

```
public void fliege(Ziel z) {
    this.schnattern();
    this.fliege(z);
}
```

```
public void fliege(Ziel z) {
    this.schnattern();
    Vogel.fliege(z);
}
```

```
public void fliege(Ziel z) {
    this.schnattern();
}
```

17. 5. Einfach verkettete Listen

`x` sei Knoten einer einfach verketten Liste (genauer: eine Referenz darauf). Wofür sorgt das folgende Codefragment?

```
t.next = x.next;
x.next = t;
```

Knoten t wird direkt hinter Knoten x eingefügt.

18. 5. Eine einfache Klasse

Programmieren Sie eine einfache Klasse `Buch` mit der Angaben zu Büchern abgespeichert werden können.
Jedes Buch zeichnet sich durch seinen Titel aus, und durch seine Seitenzahl.
Die folgenden Operationen sollen möglich sein:

- Erzeugen einer `Buch`-Instanz unter Angabe des Titels
- Erzeugen einer `Buch`-Instanz unter Angabe des Titels und der Seitenzahl
- Setzen und Abfragen der Seitenzahl jeder einzelnen Instanz.
- Die Anzahl erzeugter Instanzen und die Summe ihrer Seitenzahlen soll durch Methodenaufrufe ermittelt werden können.

VERWENDEN SIE DABEI BITTE DIESE BEZEICHNER:
`titel`, `seiten`, `anzahl`, `gesamt`

Außerdem soll die Klasse für einen ersten Test mit einer `main`-Methode ausgestattet sein, die folgende Aktionen auslöst:

- ein `Buch`-Objekt erzeugen nur unter Angabe des Titels `Lieblingsbuch` und die Referenz der Variablen `b` zuweisen
- anschließend die Seitenzahl dieses Buchobjekts auf `500` setzen
- schließlich mittels `b` die Angaben zu dem erzeugten Objekt ausgeben:

```
Lieblingsbuch, 500 S.
```

Klausur Informatik I
Sommersemester 2012
Prof. Dr. C. Damm
13. April 2012
von 08:00+x bis 09:30+x Uhr

Vorname:

Nachname:

Matrikelnummer:

Es sind **keine Hilfsmittel** (Skript, Taschenrechner etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten **Blätter** für Ihre Antworten. Schreiben Sie mit blauem/schwarzem Kugelschreiber, Füller etc.. Bleistift ist nicht erlaubt. **Es wird nur das gewertet, was ohne Mühe lesbar ist**. Algorithmen sind in Java-Code zu notieren, sofern keine Alternativen genannt sind.

Viel Erfolg!

	Max. Punkte	Erreichte Punkte
Aufgabe 1 (Elementare Daten)	21	
Aufgabe 2 (Programmierparadigmen I)	16	
Aufgabe 3 (Strukturierte Daten)	17	
Aufgabe 4 (Programmierparadigmen II)	19	
Aufgabe 5 (Algorithmen und Datenstrukturen)	17	
Summe	90	

Note:

Name: MatNr.:

Aufgabe 1 (Elementare Daten [21 Punkte])

1. [3 Punkte] Übersetzen Sie `0xBCBD` in Oktalcodierung.

2. [6 Punkte] Streichen Sie in dem Fragment die Zeile(n) durch, die der Compiler nicht akzeptiert. Schreiben Sie für die übrigen die Textausgabe in den Zeilenkommentar. Anbei ein Ausschnitt aus der ASCII-Tabelle.

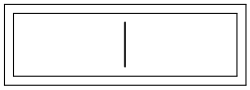
Dez	Hex	Okt	
96	0x60	140	`
97	0x61	141	a
98	0x62	142	b
99	0x63	143	c
100	0x64	144	d
101	0x65	145	e
102	0x66	146	f

```
char c = 'd';System.out.println(c); //
System.out.println('c'+1);          //
System.out.println(c); c = c-1;     //
System.out.println((char)('c'-1));  //
```

3. [6 Punkte] `a` und `b` seien `boolean-Werte`. Weisen Sie mittels Wahrheitswertetafel nach, dass der Ausdruck `(!(a || b) || (a && b)) && (a || b) && !(a && b)` den Wert `false` hat (bitte `true`/`false` mit 1/0 abkürzen — NICHT mit T/F oder t/f!!).

4. [6 Punkte] Sei `a` eine `int`-Variable mit dem Wert 2 und `b` eine `int`-Variable mit dem Wert 8. Welche Textausgabe erzeugen die vier Anweisungen, die die Ausdrücke `a/b` oder `b/a` enthalten.

```
System.out.println(b/a);
System.out.println(1.0 * a/b );
System.out.println((double) a/b );
System.out.println((double) ( a/b ));
```



Name: MatNr.:

Aufgabe 2 (Programmierparadigmen I [16 Punkte])

1. [3 Punkte] Der Ausdruck `(1 / 2) * (0.3 * 4f)` hat 6 Teilausdrücke (ohne den Ausdruck selbst). Geben Sie diese an und schreiben jeweils den Typ dahinter.

2. [3 Punkte]

   (A) Ergänzen Sie den Methodenkopf.

```
public static ..... cube(int i) {
  System.out.println(i);
  i = i*i*i;}
```

   (B) Wieviele Textausgaben produziert das folgende Fragment?

```
int i = 0;
while (i < 10) {
  System.out.println(i);
  cube(i);
  i++;}
```

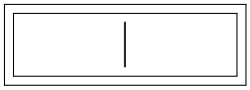
3. [4 Punkte] Welche Textzeile erzeugt der Aufruf `side(5);` ?

```
public static void side(int n){
    if (n <=1)
        return;
    if ( n % 2 == 1) {
        side(n-1);
        System.out.print(n);}
    else {
        System.out.print(n);
        side(n-1); }
}
```

4. [6 Punkte] (A) Kennzeichnen Sie mit `+` die Klassenelemente, die in den angegebenen Klassen zugreifbar sind. Kennzeichnen Sie mit `-` die Klassenelemente, die in den angegebenen Klassen nicht zugreifbar sind.

Zugriff aus ..	`protected` ⟨*Klassenel.*⟩	`private` ⟨*Klassenel.*⟩
.. gleicher Klasse		
.. anderer Klasse, gleiches Paket		
.. Unterklasse, anderes Paket		
.. Nicht-Unterklasse, anderes Paket		

   (B) Mit welchem Schlüsselwort werden Klassenmethoden definiert? Mit welchem Schlüsselwort werden symbolische Konstanten definiert?



Name: MatNr.:

Aufgabe 3 (Strukturierte Daten [17 Punkte])

1. [4 Punkte] Erzeugt der untenstehende Ausschnitt aus einer Methode
   (A) einen Compilezeitfehler? Falls ja, was ist das Problem?
   (B) einen Laufzeitfehler? Falls ja, was ist das Problem?
   (C) eine Textausgabe? Falls ja, so geben Sie die Ausgabe an.
   (Jede Frage mit Ja oder Nein beantworten, bei 'Ja' Fehler bzw. Ausgabe nennen)

```
int[] a = {2,3,5,7,11}, b = new int[2];
b = a;
b[4] = -4;
if (a[4] > 0) System.out.println("positiv");
if (a[4] < 0) System.out.println("negativ");
```

2. [5 Punkte] Eine Methode `transpose` zur Matrixtransposition (Bsp. $\begin{pmatrix} a & b & c \\ d & e & f \end{pmatrix} \mapsto \begin{pmatrix} a & d \\ b & e \\ c & f \end{pmatrix}$) nimmt als Argument ein zweidimensionales `int`-Feld `m`, das die Matrix speichert und liefert auch ein zweidimensionales `int`-Feld zurück.
   (A) Ergänzen Sie den Methodenkopf:

```
public static .............. transpose( ..............)
```

   (B) Geben Sie einen Ausdruck an für die Zeilenzahl der Matrix `m`:

```
int zeilen = ...............;
```

   (C) Geben Sie einen Ausdruck an für die Spaltenzahl der Matrix `m`:

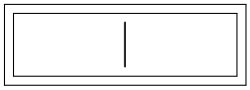
```
int spalten = ...............;
```

3. [5 Punkte] Welche Textausgabe erzeugt das folgende Codefragment?

```
int[] d = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
for (int i = 0; i < 10; i++)
  if ( i%2 == 0) d[i] = d[d[i]];
  else d[i] = d[d[9 - i]];
for (int i = 0; i < 10; i++) System.out.print(d[i]);
System.out.println();
```

4. [3 Punkte] Ein `Zeit`-Objekt speichert eine Tageszeitangabe in Stunden und Minuten: Ist $0 \leq$ `hh` $\leq 23, 0 \leq$ `mm` ≤ 59, so liefert `jetzt = new Zeit(hh,mm);` die Referenz auf ein `Zeit`-Objekt, dass der Zeitangabe `hh:mm` entspricht. Die Methode `in(int h, int m)` soll „die Uhr vorstellen“ um `h` Stunden und `m` Minuten (`h,m` ≥ 0, z.B. 12:30 um 1:20 vorgestellt ergibt 13:50). Korrigieren Sie die logischen Fehler im Methodenrumpf:

```
public class Zeit {
 int hh, mm;
 public Zeit(int hh,int mm){ ... } // Implementation nicht gezeigt
 public Zeit in(int h, int m) {// LOGISCHE FEHLER IN FOLGENDER ZEILE:
   this.hh = this.hh + h % 24; this.mm = this.mm + m % 60;
   }
}
```



Name: MatNr.:

Aufgabe 4 (Programmierparadigmen II [19 Punkte])

1. [13 Punkte] Schreiben Sie eine Klasse `Konto` zur Verwaltung von Bankkonten. Ein Konto hat eine Nummer (Bezeichner `nummer`, Typ `long`) und einen in Cent angegebenen Kontostand (Bezeichner `saldo`, Typ `int`).
   Die Daten der Klasse sollen vollständig gekapselt sein (Geheimnisprinzip) und folgende Operationen sollen unterstützt werden:

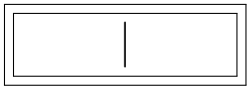
   - Erzeugen einer Instanz der Klasse unter Angabe der Nummer.
   - Erzeugen einer Instanz der Klasse unter Angabe von Nummer und Kontostand.
   - Setzen und Abfragen des Kontostands jedes einzelnen Kontos.
   - Ermitteln der Anzahl erzeugter `Konto`-Objekte und der Summe der Kontostände.

   |

2. [6 Punkte] Schreiben Sie zu der oben spezifizierten Klasse eine Erweiterungsklasse `Sparkonto` mit vollständig gekapselten Daten (Geheimnisprinzip). Objekte der Klasse haben zusätzlich einen Zinssatz (Bezeichner `zins`, Typ `double`). Folgende Operationen sollen unterstützt werden:

   - Erzeugen einer Instanz der Klasse unter Angabe von Nummer und Zinssatz.
   - Ändern des Zinssatzes.
   - Ermitteln des Zinsertrags (= Zinssatz $\cdot$ Saldo) einer Instanz.

   |



Name: MatNr.:

Aufgabe 5 (Algorithmen und Datenstrukturen [17 Punkte])

1. [3 Punkte] In `int[] f = {4, 6, 13, 18, 23, 100, 143, 899};` soll binär gesucht werden. Geben Sie in der Reihenfolge des Auftretens an, mit welchen Werten der Suchwert `q=13` dabei verglichen wird.

```
static int binarysearch(int[] f, int q) {
  int a = 0, e = f.length-1, m;
  while ( a <= e) {
    m = (a+e)/2;
    if ( f[m] == q) return m;
    if (q < f[m])
      e = m-1;
    else a = m+1;}
  return -1;}
```

2. [3 Punkte] Zeichnen Sie den binären Suchbaum, der beim Einfügen von 7, 9, 4, 6, 8, 5, 2, 1, 3 (in dieser Reihenfolge) entsteht.

3. [6 Punkte] Ein Sortieralgorithmus:

```
public static void sort(int[] feld) {
    int pos = 0, tmp, n = feld.length;
    for (int i = 0; i < n; i++) {
        pos = i;
        for (int j = i+1; j < n; j++)
            if ( feld[j] < feld[pos])
                pos = j;
        tmp = feld[i]; feld[i] = feld[pos]; feld[pos] = tmp;
    }
}
```

Ergänzen Sie die Feldinhalte nach Iterationen der äußeren `for`-Schleife:

Startbelegung von `feld`	6	1	9	8	3	2	4	5	7
nach 1. Iteration									
nach 2. Iteration									
nach 3. Iteration									

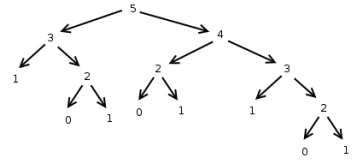
4. [5 Punkte] Betrachten Sie folgende zweifach rekursive Methode:

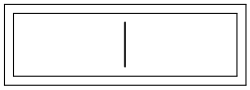
```
public static String mystery(String a) {
  int m = a.length();
  if (m <= 1) return a;
  return mystery(a.substring(m/2,m)) + mystery(a.substring(0,m/2));}
```

(A) Zeichnen Sie den Aufrufbaum zum Aufruf `mystery("abcde");`.Für einen Aufruf mit einem String der Länge n geben Sie in $\mathcal{O}$-Notation an: (B) die Tiefe des Aufrufbaums, (C) die Zeitkomplexität der Methode.

Hinweise: 1. `s.substring(i,j)` liefert, grob gesagt, den Teilstring von `s` zwischen Positionen `i` und `j-1` (Zählung beginnt jeweils bei 0, z.B. `"Aloha".substring(1,4)` liefert `"loh"`).

2. Die Knoten des Aufrufbaums sind mit den Parametern der entstehenden rekursiven Aufrufe markiert. Söhne werden dem Quelltext entsprechend von links nach rechts angeordnet. Beispiel rechts: Aufrufbaum zur rekursiven Berechnung der Fibonacci-Zahl f_5.





Name: MatNr.:

Info 1 - 2015 `// Viel Spass damit und lernt brav ;)`

1. 1. Vermischtes zur Einführung

Markieren Sie *alle* zutreffenden Aussagen:

- Top-Down-Entwurf besteht darin, Details erst grob, dann feingliedrig zu sortieren.
- Dem Inhalt einer Speicherzelle kann man nicht ansehen, ob sie eine Zahl oder einen Befehl speichert..
- Beim euklidischen Algorithmus wird dieser Block iteriert: `{ x = y; y = r; r = x % y; }`
- Beim Testen wird das Programm auf allen erlaubten Eingaben ausgeführt.
- Beim Pipelining werden Eingaben eines Prozesses zur Ausgabe eines anderen geleitet.
- Beim Laden wird die Befehlsfolge in aufeinanderfolgende Register kopiert.
- Editieren und Testen gehören zu jedem Programmentwicklungszyklus.
- `public static void main(String[] arks)` ist Kopf der `main`-Methode.

2. 1. Zweierpotenzen

Ergänzen Sie untenstehende `main`-Methode so, dass der Aufruf `java Zweierpotenzen` n
die Liste der Zweierpotenzen von $2^0 = 1$ bis 2^n zeilenweise ausgibt.

```
public class Zweierpotenzen {
    public static void main (String[] args) {
        // ---------------------------
        .....    // soviele Codezeilen wie nötig
        .....    // (angemessen EINGERÜCKT)
        ..
        // ---------------------------
   }
}
```

Beispiel `java Zweierpotenzen 3` gibt diese Liste aus:

```
1
2
4
8
```

Algorithmische Idee (bitte GENAU diese Variablennamen benutzen):

initialisiere:
- Kommandozeilenargument (interpretiert als `int`-Wert) in `n` speichern
- `int`-Variable `p` als Akkumulator initialisieren

iteriere so oft wie nötig:
- `p` ausgeben und geeignet aktualisieren

finalisiere: entfällt

3. 1. Quadratzahlentabelle

Die `int`-Variable `n` speichere die ganze Zahl n.

Die ersten n positiven Zahlen sollen zusammen mit ihren Quadraten jeweils auf einer Zeile

ausgegeben werden.

Beispiel für $n = 4$:

```
1 : 1
2 : 4
3 : 9
4 : 16
```

Im Fall $n \leq 0$ soll nichts ausgegeben werden.

"Programmieren" Sie dies durch Auswahl der richtigen Codezeilen, und zwar einmal mit einer `for`-Schleife und einmal mit einer `do-while`-Schleife.

Achten Sie auf den Unterschied zwischen `System.out.print(..)` und `System.out.println(..)`.

- mit einer for-Schleife:

```
for (int i = 1; i < n+1; i++) {
System.out.print(i + ": ");
System.out.println( i^2);
}
```

- mit einer do-while-Schleife:

```
int i = 1;
do {
System.out.print(i + ": ");
System.out.println( i^2);
i++;
} while ( i < n+1);
```

4. 2. Vermischtes zu elementaren Daten

Markieren Sie *alle* zutreffenden Aussagen:

- Der ASCII-Standard legt die Anordnung der 256 Zeichen fest.
- Zu jedem Datentyp gibt es ein Informationssystem.
- Java-Code darf beliebige Unicode-Zeichen enthalten.
- `int`-Werte können verlustfrei in `float`-Variablen gespeichert werden.
- `5/4 == 1` hat den Wert `true`.
- `Integer.MAX_VALUE++` verursacht keinen Ganzzahlüberlauf.
- Anstatt `if (b > 0 && a/b < 2) ..` schreibt man besser `if (a/b < 2 && b > 0) ...`
- Ein Gleitkomma-Überlauf ist kein Laufzeitfehler.

5. 2. Integer-Arithmetik

Sei `d` eine `int`-Variable mit dem Wert `Integer.MAX_VALUE` (also 2147483647). Welche Textausgabe erzeugen diese Anweisungen?:

```
1:  System.out.println(-d);
2:  System.out.println(d-1);          // dropdown menue mit loesungen
3:  System.out.println(1+2*d);
4:  System.out.println(-(2*d));
```

6. 2. Verarbeitung elementarer Daten (I1-ID: 93rdw3918rg0)

In einer Klasse sind einige Methoden definiert:

```
public static void a() {
    char a = 'a';
    a++;
    int b = (int) a;
    b++;
    System.out.print( (char) ('A' + b - a));
}
public static boolean getTrue() {
    System.out.print("T");
    return true;
}
public static boolean getFalse() {
    System.out.print("F");
    return true;
}
public static void q(int i) {
    int j = 43;
    System.out.print("G");
    if ( i / j > 0) System.out.print("Q");
    else System.out.print("R");
}
public static void d() {
    double dd = Double.MIN_VALUE;
    dd *= 2;
    dd -= Double.MIN_VALUE;
    dd /= 2;
    System.out.print(dd);
}
```

Welche Ausgaben bewirken diese Aufrufe (in der `main`-Methode der gleichen Klasse)?

```
a();



boolean val = getTrue() || getFalse();



val = getTrue() ^ getTrue();



System.out.print( val);



q(42);



d();
```

7. 3. Rekursion

Betrachten Sie folgende Klasse:

```
 1:  class Myst {
 2:      static void ery(int n) {
 3:          if (n == 0)
 4:              return;
 5:          ery( n-1);
 6:          A();
 7:          yre( n-1);
 8:      }
 9:      static void yre(int n) {
10:          if (n == 0)
11:              return;
12:          ery( n-1);
13:          B();
```

```
14:          yre( n-1);
15:      }
16:      static void A() {
17:          System.out.print("A");
18:      }
19:      static void B() {
20:          System.out.print("B");
21:      }
22:      public static void main(String[] args) {
23:          int n = Integer.parseInt(args[0]);
24:          ery(n);
25:          System.out.println();
26:      }
27:  }
```

1. Geben Sie die ersten 8 Zeichen der Ausgabe zum Aufruf `java Myst 4` an:

2. Aus wievielen Zeichen besteht der Ausgabestring beim Aufruf `java Myst 7` (*ohne* den abschließenden Zeilenwechsel durch Zeile 25)?

3. Wird Zeile 14 durch `yre( n-2);` ersetzt, so führt beispielsweise der Aufruf `java Myst 4` zu einem Fehler.
   Welche *andere* Zeile müsste korrigiert werden, um den Fehler zu beseitigen? Geben Sie nur die Nummer an:

4. Werde Zeile 12 durch `ery( n-2);` und zusätzlich Zeile 14 durch `yre( n-2)` ersetzt.
   Geben Sie das maximale Argument `n` an, bei dem der Aufruf `java Myst n` *nicht* zu einem Fehler führt:

8. 3. Grafik-Bibliothek benutzen

Die Klasse `public class StdDraw` bietet Methoden zum Zeichnen geometrischer Objekte:

- die quadratische Zeichenfläche (Seitenlänge 1) wird in einem Fenster mit dem Titel "File" angezeigt
- Koordinaten wie üblich (z.B. gehen die blauen Strecken oben alle durch den Punkt $(x, y) = (1, 0)$, die diagonal gegenüberliegende Quadratecke ist $(0, 1)$)
- gezeichnet wird mit der zuletzt gesetzen Stiftfarbe, wobei später Gezeichnetes Früheres (teilweise) überdecken kann

`void setPenColor(Color c)`	Stiftfarbe setzen
`BLACK`, `RED`, `BLUE`	in `StdDraw` definierte `Color`-Konstanten (schwarz, rot, blau)
`void line(double x0, double y0, double x1, double y1)`	Strecke zwischen Punkten `(x0,y0)` und `(x1,y1)`
`void square(double x, double y, double r)`	Quadrat: Mittelpunkt `(x,y)` u. Seitenlänge `2*r` (parallel zu Koordinatenachsen)
`void circle(double x, double y, double r)`	Kreis mit Mittelpunkt `(x,y)` und Radius `r`

Nachfolgend der Ausschnitt einer Methode einer Clientklasse von `StdDraw`.
Ergänzen Sie Zeilen so, dass der Aufruf `zeichneB(n)` (z.B. in der `main`-Methode dieser Klasse) eine Figur erzeugt aus `n` mal `n` Kreisen usw.
Genau so wie oben gezeigt. Achten Sie dabei auf gut verständlichen Code.

```
static void zeichneB(int n) {
```

```
                                // Ekelhafte Dropdown menues

    for (int i = 0; i < n; i++)                    StdDraw.circle(r*i,r*j,r/2);
      for (int j = 0; j < n; j++)                StdDraw.setPenColor(StdDraw.RED);
       StdDraw.setPenRadius(0.02);                  StdDraw.line(1.0,0.0,0,r+2*r*i);
         for (int i = 0; i < n; i++)              StdDraw.setPenColor(StdDraw.BLUE);
         double r = 1.0/(2*n);                     for (int j = 0; j < n; j++)
                                                    StdDraw.square(0.5,0.5,1);
}
```

9. 4. Felder: Fehler oder …?

Erzeugt untenstehende Klasse

- einen Compilezeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich:
- einen Laufzeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich:
- eine Ausgabe? Falls ja, so tragen Sie die Ausgabe ein, ansonsten einen Bindestrich:

```
 1:  class FehlerOderWie {
 2:      public static void main(String[] args) {
 3:          int[] a = {2,3,5,7,11};
 4:          int[] b = new int[2];
 5:          b = a;
 6:          b[4] = -4;
 7:          if (a[4] >= 1) System.out.println(1);
 8:          else System.out.println(0);
 9:      }
10:  }
```

10. 4. Vermischtes zu Klassentypen

1. Objekte verstehen:

   ```
   public class Spieler {
       private int score;
       public Spieler( int s) {
           this.score = s;
       }
   }
   ```

   Zwei `Spieler`-Objekte werden instanziiert:

   ```
   Spieler a = new Spieler(50), b = new Spieler(50);
   ```

   Welchen Wert hat nun der Ausdruck a == b?

2. Fremden Code deuten:

   ```
   1:  public class FirstBpp extends hepno.users.Frame {
   2:      private hepno.users.Wring wrags;
   3:      public FirstBpp() {
   4:          wrags = new hepno.users.Wring();
   5:      }
   6:      public static void main(String[] args) {
   7:          FirstBpp app = new FirstBpp();
   8:      }
   9:  }
   ```

   Was bewirkt Zeile 2?:
   `hepno.users.Wring wird als private-Objekt deklariert`

3. Statische Methoden und Instanzmethoden:

```
public class Klasse {
    private final int zahl = 10;
    double a() {
        System.out.println(zahl);
        return c();
    }
    void b() {
        System.out.println(this);
    }
    double c() {
        double r = Math.random();
        System.out.println(r);
        return r;
    }
    void d() {
        a();
        a();
    }
    int e() {
        return zahl;
    }
}
```

Welche der obigen Methoden ist ihrer Natur nach eher eine statische Methode (und sollte daher mit `static` versehen werden)?

`c`

4. Fehlersuche:

```
public class Test {
    private int testCount;
    public static int getCount(){
        return testCount;
    }
    public Test(){
        testCount++;
    }
}
```

Obiger Code verursacht einen Compilezeitfehler. Was ist das Problem?

`getCount() ist statisch`

5. Tracing:
[Hinweis: Untenstehender Code ist syntaktisch korrekt, denn auf Klassenvariablen kann auch mit Hilfe von Objektreferenzen zugegriffen werden.]

```
public class Tset {
    static int tsetCount = 0;
    public Tset(){
        tsetCount++;
    }
    public static void main(String [] Args){
        // Hinweis: auf Klassenvariablen kann man auch
        tset t1 = new tset();
        System.out.print(t1.tsetCount + ", ");
        tset t2 = new tset();
        System.out.print(t1.tsetCount + " "+ t2.tsetCount + ", ");
        tset t3 = new tset();
        System.out.print(t1.tsetCount+ " "+ t2.tsetCount+ " "+ t3.tsetCount);
        System.out.println();
    }
}
```

Welche Ausgabe produziert der Code?

6. Begriffsverständnis: Wählen Sie aus dem Menü die einzige Aussage aus, die NICHT korrekt ist.

In einer Klasse darf es Methoden geben, die den gleichen Namen und gleiche Argumenttyplisten haben aber unterschiedliche Ergebnistypen.

11. 4. Iteration in einem Feld

Tragen Sie die Werte der Feldkomponenten nach Ausführung der `for`-Anweisung ein.

```
int[] d = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9};

for (int i = 0; i < 10; i++)
    d[i] = d[9-d[9-d[9-i]]];
```

vorher	0	1	2	3	4	5	6	7	8	9
nachher	9	8	7	6	5	5	6	7	8	9

12. 6. Stapelarbeit

Auf einem Stack werden *in gemischter Reihenfolge* 10 Push- und 10 Pop-Operationen ausgeführt.

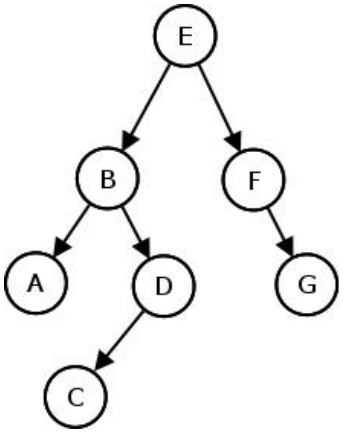
Es ist bekannt, dass die Push-Operationen die Zahlen 0, 1, 2, …, 9 in dieser Reihenfolge einspeichern und dass bei jedem Pop der ausgespeicherte Wert ausgegeben wird.

Markieren Sie alle Ausgabelisten, die dabei entstehen können (das können 0, 1 oder mehrere sein):

1 4 7 9 8 6 5 3 0 2
4 3 2 1 0 9 8 7 6 5
2 5 6 7 4 8 9 3 1 0
4 6 8 7 5 3 2 9 0 1

13. 6. Binärer Suchbaum

Es geht um binäre Suchbäume, deren Schlüssel Zeichen in alphabetischer Anordnung sind.



1. Werden die Schlüssel obigen binären Suchbaums in Inorder-, Preorder- oder Postorder-Reihenfolge ausgegeben, so erscheint ein Schlüssel immer an gleicher Position.
   Welcher ist das?

2. Wählen Sie für die Schlüssel eine Einfügereihenfolge aus, bei der ein Binärbaum der Tiefe 2 entsteht:

14. 6. Vermischtes zu Algorithmen und Datenstrukturen

Markieren Sie *alle* zutreffenden Aussagen:

- Binäre Suche erfordert $O(n)$ Vergleiche auf *unsortierten* Feldern.
- Mit einem Stack kann man z.B. feststellen, ob ein Ausdruck korrekt geklammert ist.
- Die abstrakte Datenstruktur Queue lässt sich nicht mit einfach verketteten Listen realisieren.
- Quicksort ist auf vorsortierten Feldern besonders schnell.
- Mergesort benötigt auf allen Feldern der Länge 2^m immer die gleiche Anzahl an Vergleichen.
- Selectionsort benötigt auf allen Feldern der Länge 2^m immer die gleiche Anzahl an Vergleichen.
- Binäre Bäume der Tiefe n haben mindestens $n-1$ und höchstens 2^n-1 *innere Knoten*.
- Felder sind nicht als Grundlage der abstrakten Datenstruktur Stack geeignet.

15. 5. Verständnisfragen zu OOP

Sei p eine Referenzvariable vom Typ A. Was kann alles der dynamische Typ von p sein?

Markieren Sie *alle* Möglichkeiten, die Java erlaubt:

- eine Erweiterungsklasse einer Klasse, die A implementiert (falls A ein Interface ist)
- ein Interface, dass A als abstrakte Klasse erweitert (falls A eine Klasse ist)
- eine Klasse, von der A abgeleitet ist (falls A eine Klasse ist)
- eine Erweiterungsklasse von A (falls A eine Klasse ist)
- ein elementarer Datentyp (falls A die zugehörige Wrapper-Klasse ist)
- eine Klasse, die A implementiert (falls A ein Interface ist)

16. 5. OOP-Entwurf

Die Klasse `Ente` erbt von der Klasse `Vogel`. In `Vogel` gibt es eine Instanzmethode `fliege(..)`, in `Ente` gibt es die Instanzmethode `schattern()`.

Die `fliege`-Methode von `Ente`-Objekten soll nun so angepasst werden, dass `schnattern()` immer unmittelbar am Anfang von `fliege` aufgerufen wird.

Wie ist das am besten zu realisieren?

○
```
public void fliege(Ziel z) {
    this.schnattern();
    // hier den Rumpf der fliege-Methode aus Vogel einfuegen
}
```

○
```
public void fliege(Ziel z) {
    this.schnattern();
    super.fliege(z);
}
```

```
public void fliege(Ziel z) {
    this.schnattern();
    this.fliege(z);
}
```

```
public void fliege(Ziel z) {
    this.schnattern();
    Vogel.fliege(z);
}
```

```
public void fliege(Ziel z) {
    this.schnattern();
}
```

17. 5. Einfach verkettete Listen

`x` sei Knoten einer einfach verketten Liste (genauer: eine Referenz darauf). Wofür sorgt das folgende Codefragment?

```
t.next = x.next;
x.next = t;
```

Knoten t wird direkt hinter Knoten x eingefügt.

18. 5. Eine einfache Klasse

Programmieren Sie eine einfache Klasse `Buch` mit der Angaben zu Büchern abgespeichert werden können.
Jedes Buch zeichnet sich durch seinen Titel aus, und durch seine Seitenzahl.
Die folgenden Operationen sollen möglich sein:

- Erzeugen einer `Buch`-Instanz unter Angabe des Titels
- Erzeugen einer `Buch`-Instanz unter Angabe des Titels und der Seitenzahl
- Setzen und Abfragen der Seitenzahl jeder einzelnen Instanz.
- Die Anzahl erzeugter Instanzen und die Summe ihrer Seitenzahlen soll durch Methodenaufrufe ermittelt werden können.

VERWENDEN SIE DABEI BITTE DIESE BEZEICHNER:
`titel`, `seiten`, `anzahl`, `gesamt`

Außerdem soll die Klasse für einen ersten Test mit einer `main`-Methode ausgestattet sein, die folgende Aktionen auslöst:

- ein `Buch`-Objekt erzeugen nur unter Angabe des Titels `Lieblingsbuch` und die Referenz der Variablen `b` zuweisen
- anschließend die Seitenzahl dieses Buchobjekts auf `500` setzen
- schließlich mittels `b` die Angaben zu dem erzeugten Objekt ausgeben:

```
Lieblingsbuch, 500 S.
```

PDF-XChange Click to buy NOW! www.docu-track.com

Liste der Antworten

1. 1. Vermischtes zur Einführung (I1-ID: bmhkprd07rg0)

Markieren Sie *alle* zutreffenden Aussagen:

- ☐ Top-Down-Entwurf besteht darin, Details erst grob, dann feingliedrig zu sortieren.
- ☑ Dem Inhalt einer Speicherzelle kann man nicht ansehen, ob sie eine Zahl oder einen Befehl speichert..
- ☑ Beim euklidischen Algorithmus wird dieser Block iteriert: `{ x = y; y = r; r = x % y; }`
- ☐ Beim Testen wird das Programm auf allen erlaubten Eingaben ausgeführt.
- ☐ Beim Pipelining werden Eingaben eines Prozesses zur Ausgabe eines anderen geleitet.
- ☐ Beim Laden wird die Befehlsfolge in aufeinanderfolgende Register kopiert.
- ☑ Editieren und Testen gehören zu jedem Programmentwicklungszyklus.
- ☑ `public static void main(String[] arks)` ist Kopf der `main`-Methode.

2. 1. Zweierpotenzen (I1-ID: g6f948r07rg0)

Ergänzen Sie untenstehende `main`-Methode so, dass der Aufruf `java Zweierpotenzen` n die Liste der Zweierpotenzen von $2^0 = 1$ bis 2^n zeilenweise ausgibt.

```
public class Zweierpotenzen {
    public static void main (String[] args) {
        // ----------------------------
        .....    // soviele Codezeilen wie nötig
        .....    // (angemessen EINGERÜCKT)
        ..
        // ----------------------------
   }
}
```

Beispiel `java Zweierpotenzen 3` gibt diese Liste aus:

```
1
2
4
8
```

Algorithmische Idee (bitte GENAU diese Variablennamen benutzen):

initialisiere:
- Kommandozeilenargument (interpretiert als `int`-Wert) in `n` speichern
- `int`-Variable `p` als Akkumulator initialisieren

iteriere so oft wie nötig:
- `p` ausgeben und geeignet aktualisieren

finalisiere: entfällt

```
1  int n = Integer.parseInt(args[0]);
2  for (int p = 0; p<=n;p++) {
3      System.out.println(2^p);
4  }
```

0 Zeichen zugelassen, Anzahl der eingegebenen Zeichen: **111**

3. 1. Quadratzahlentabelle (I1-ID: qiheowf07rg0)

Die `int`-Variable `n` speichere die ganze Zahl n.

Die ersten n positiven Zahlen sollen zusammen mit ihren Quadraten jeweils auf einer Zeile

PDF-XChange Click to buy NOW! www.docu-track.com

ausgegeben werden.

Beispiel für $n = 4$:

```
1 : 1
2 : 4
3 : 9
4 : 16
```

Im Fall $n \leq 0$ soll nichts ausgegeben werden.

"Programmieren" Sie dies durch Auswahl der richtigen Codezeilen, und zwar einmal mit einer `for`-Schleife und einmal mit einer `do-while`-Schleife.

Achten Sie auf den Unterschied zwischen `System.out.print(..)` und `System.out.println(..)`.

- mit einer for-Schleife:

```
for (int i = 1; i < n+1; i++) {
System.out.print(i + ": ");
System.out.println( i^2);
}
```

- mit einer do-while-Schleife:

```
int i = 1;
do {
System.out.print(i + ": ");
System.out.println( i^2);
i++;
} while ( i < n+1);
```

4. 2. Vermischtes zu elementaren Daten (I1-ID: g60dcvw08rg0)

Markieren Sie *alle* zutreffenden Aussagen:

- ☐ Der ASCII-Standard legt die Anordnung der 256 Zeichen fest.
- ☑ Zu jedem Datentyp gibt es ein Informationssystem.
- ☑ Java-Code darf beliebige Unicode-Zeichen enthalten.
- ☐ `int`-Werte können verlustfrei in `float`-Variablen gespeichert werden.
- ☑ `5/4 == 1` hat den Wert `true`.
- ☑ `Integer.MAX_VALUE++` verursacht keinen Ganzzahlüberlauf.
- ☐ Anstatt `if (b > 0 && a/b < 2) ..` schreibt man besser `if (a/b < 2 && b > 0) ...`
- ☑ Ein Gleitkomma-Überlauf ist kein Laufzeitfehler.

5. 2. Integer-Arithmetik (I1-ID: 5wa3il81gdg0)

Sei `d` eine `int`-Variable mit dem Wert `Integer.MAX_VALUE` (also 2147483647). Welche Textausgabe erzeugen diese Anweisungen?:

```
1:  System.out.println(-d);      // -2147483647
2:  System.out.println(d-1);     // 2147483646
3:  System.out.println(1+2*d);   // 0
4:  System.out.println(-(2*d)); // 1
```

6. 2. Verarbeitung elementarer Daten (I1-ID: 93rdw3918rg0)

In einer Klasse sind einige Methoden definiert:

PDF-XChange Click to buy NOW! www.docu-track.com

```
public static void a() {
    char a = 'a';
    a++;
    int b = (int) a;
    b++;
    System.out.print( (char) ('A' + b - a));
}
public static boolean getTrue() {
    System.out.print("T");
    return true;
}
public static boolean getFalse() {
    System.out.print("F");
    return true;
}
public static void q(int i) {
    int j = 43;
    System.out.print("G");
    if ( i / j > 0) System.out.print("Q");
    else System.out.print("R");
}
public static void d() {
    double dd = Double.MIN_VALUE;
    dd *= 2;
    dd -= Double.MIN_VALUE;
    dd /= 2;
    System.out.print(dd);
}
```

Welche Ausgaben bewirken diese Aufrufe (in der `main`-Methode der gleichen Klasse)?

```
a();
```

B

```
boolean val = getTrue() || getFalse();
```

T

```
val = getTrue() ^ getTrue();
```

TT

```
System.out.print( val);
```

false

```
q(42);
```

GR

```
d();
```

0.0

7. 3. Rekursion (I1-ID: ggj8m830erg0)

Betrachten Sie folgende Klasse:

```
 1:  class Myst {
 2:      static void ery(int n) {
 3:          if (n == 0)
 4:              return;
 5:          ery( n-1);
 6:          A();
 7:          yre( n-1);
 8:      }
 9:      static void yre(int n) {
10:          if (n == 0)
11:              return;
12:          ery( n-1);
13:          B();
```

PDF-XChange Click to buy NOW! www.docu-track.com

```
14:          yre( n-1);
15:      }
16:      static void A() {
17:          System.out.print("A");
18:      }
19:      static void B() {
20:          System.out.print("B");
21:      }
22:      public static void main(String[] args) {
23:          int n = Integer.parseInt(args[0]);
24:          ery(n);
25:          System.out.println();
26:      }
27:  }
```

1. Geben Sie die ersten 8 Zeichen der Ausgabe zum Aufruf `java Myst 4` an:
   AAAAAABA
2. Aus wievielen Zeichen besteht der Ausgabestring beim Aufruf `java Myst 7` (*ohne* den abschließenden Zeilenwechsel durch Zeile 25)?
   64
3. Wird Zeile 14 durch `yre( n-2);` ersetzt, so führt beispielsweise der Aufruf `java Myst 4` zu einem Fehler.
   Welche *andere* Zeile müsste korrigiert werden, um den Fehler zu beseitigen? Geben Sie nur die Nummer an:
   10
4. Werde Zeile 12 durch `ery( n-2);` und zusätzlich Zeile 14 durch `yre( n-2)` ersetzt.
   Geben Sie das maximale Argument `n` an, bei dem der Aufruf `java Myst n` *nicht* zu einem Fehler führt:
   1

8. 3. Grafik-Bibliothek benutzen (I1-ID: uuejhvx0grg0)

Die Klasse `public class StdDraw` bietet Methoden zum Zeichnen geometrischer Objekte:

- die quadratische Zeichenfläche (Seitenlänge 1) wird in einem Fenster mit dem Titel "File" angezeigt
- Koordinaten wie üblich (z.B. gehen die blauen Strecken oben alle durch den Punkt $(x, y) = (1, 0)$, die diagonal gegenüberliegende Quadratecke ist $(0, 1)$)
- gezeichnet wird mit der zuletzt gesetzen Stiftfarbe, wobei später Gezeichnetes Früheres (teilweise) überdecken kann

`void setPenColor(Color c)`	Stiftfarbe setzen
`BLACK, RED, BLUE`	in `StdDraw` definierte `Color`-Konstanten (schwarz, rot, blau)
`void line(double x0, double y0, double x1, double y1)`	Strecke zwischen Punkten `(x0,y0)` und `(x1,y1)`
`void square(double x, double y, double r)`	Quadrat: Mittelpunkt `(x,y)` u. Seitenlänge `2*r` (parallel zu Koordinatenachsen)
`void circle(double x, double y, double r)`	Kreis mit Mittelpunkt `(x,y)` und Radius `r`

Nachfolgend der Ausschnitt einer Methode einer Clientklasse von `StdDraw`.
Ergänzen Sie Zeilen so, dass der Aufruf `zeichneB(n)` (z.B. in der `main`-Methode dieser Klasse) eine Figur erzeugt aus `n` mal `n` Kreisen usw.
Genau so wie oben gezeigt. Achten Sie dabei auf gut verständlichen Code.

```
static void zeichneB(int n) {
```

PDF-XChange Click to buy NOW! www.docu-track.com

```
        StdDraw.setPenRadius(0.02);

        StdDraw.square(0.5,0.5,1);
        StdDraw.setPenColor(StdDraw.BLUE);
        double r = 1.0/(2*n);
        for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
        StdDraw.line(1.0,0.0,0,r+2*r*i);
        StdDraw.setPenColor(StdDraw.RED);
        for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
        StdDraw.circle(r*i,r*j,r/2);

        // Ende der Methode
    }
```

9. 4. Felder: Fehler oder ...? (I1-ID: xb2c64a1gdg0)

Erzeugt untenstehende Klasse

- einen Compilezeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-): -
- einen Laufzeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-): -
- eine Ausgabe? Falls ja, so tragen Sie die Ausgabe ein, ansonsten einen Bindestrich (-): `1`

```
 1:  class FehlerOderWie {
 2:      public static void main(String[] args) {
 3:          int[] a = {2,3,5,7,11};
 4:          int[] b = new int[2];
 5:          b = a;
 6:          b[4] = -4;
 7:          if (a[4] >= 1) System.out.println(1);
 8:          else System.out.println(0);
 9:      }
10:  }
```

10. 4. Vermischtes zu Klassentypen (I1-ID: t5k8w9h0frg0)

1. Objekte verstehen:

   ```
   public class Spieler {
       private int score;
       public Spieler( int s) {
           this.score = s;
       }
   }
   ```

   Zwei `Spieler`-Objekte werden instanziiert:

   ```
   Spieler a = new Spieler(50), b = new Spieler(50);
   ```

   Welchen Wert hat nun der Ausdruck a == b?
   `nichts von denen`

2. Fremden Code deuten:

   ```
   1:  public class FirstBpp extends hepno.users.Frame {
   2:      private hepno.users.Wring wrags;
   3:      public FirstBpp() {
   4:          wrags = new hepno.users.Wring();
   5:      }
   6:      public static void main(String[] args) {
   7:          FirstBpp app = new FirstBpp();
   8:      }
   9:  }
   ```

   Was bewirkt Zeile 2?:
   `hepno.users.Wring wird als private-Objekt deklariert`

PDF-XChange Click to buy NOW! www.docu-track.com

3. Statische Methoden und Instanzmethoden:

```
public class Klasse {
    private final int zahl = 10;
    double a() {
        System.out.println(zahl);
        return c();
    }
    void b() {
        System.out.println(this);
    }
    double c() {
        double r = Math.random();
        System.out.println(r);
        return r;
    }
    void d() {
        a();
        a();
    }
    int e() {
        return zahl;
    }
}
```

Welche der obigen Methoden ist ihrer Natur nach eher eine statische Methode (und sollte daher mit `static` versehen werden)?

d

4. Fehlersuche:

```
public class Test {
    private int testCount;
    public static int getCount(){
        return testCount;
    }
    public Test(){
        testCount++;
    }
}
```

Obiger Code verursacht einen Compilezeitfehler. Was ist das Problem?

getCount() ist statisch

5. Tracing:
[Hinweis: Untenstehender Code ist syntaktisch korrekt, denn auf Klassenvariablen kann auch mit Hilfe von Objektreferenzen zugegriffen werden.]

```
public class Tset {
    static int tsetCount = 0;
    public Tset(){
        tsetCount++;
    }
    public static void main(String [] Args){
        // Hinweis: auf Klassenvariablen kann man auch
        tset t1 = new tset();
        System.out.print(t1.tsetCount + ", ");
        tset t2 = new tset();
        System.out.print(t1.tsetCount + " "+ t2.tsetCount + ", ");
        tset t3 = new tset();
        System.out.print(t1.tsetCount+ " "+ t2.tsetCount+ " "+ t3.tsetCount);
        System.out.println();
    }
}
```

Welche Ausgabe produziert der Code?

1, 2 2, 3 3 3

6. Begriffsverständnis: Wählen Sie aus dem Menü die einzige Aussage aus, die NICHT korrekt ist.

PDF-XChange Click to buy NOW! www.docu-track.com

In einer Klasse darf es Methoden geben, die den gleichen Namen und gleic
Argumenttyplisten haben aber unterschiedliche Ergebnistypen.

11. 4. Iteration in einem Feld (I1-ID: pb562w80frg0)

Tragen Sie die Werte der Feldkomponenten nach Ausführung der `for`-Anweisung ein.

```
int[] d = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9};

for (int i = 0; i < 10; i++)
    d[i] = d[9-d[9-d[9-i]]];
```

vorher	0	1	2	3	4	5	6	7	8	9
nachher	9	8	7	6	5	5	6	7	8	9

12. 6. Stapelarbeit (I1-ID: 30t3atp0krg0)

Auf einem Stack werden *in gemischter Reihenfolge* 10 Push- und 10 Pop-Operationen ausgeführt.

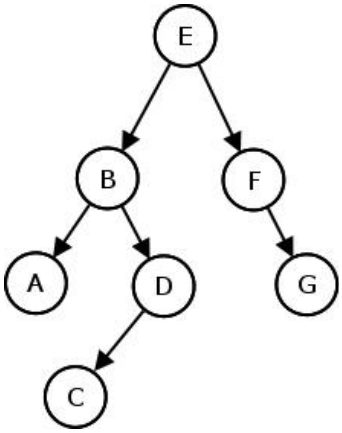
Es ist bekannt, dass die Push-Operationen die Zahlen 0, 1, 2, …, 9 in dieser Reihenfolge einspeichern und dass bei jedem Pop der ausgespeicherte Wert ausgegeben wird.

Markieren Sie alle Ausgabelisten, die dabei entstehen können (das können 0, 1 oder mehrere sein):

- ☐ 1 4 7 9 8 6 5 3 0 2
- ☑ 4 3 2 1 0 9 8 7 6 5
- ☑ 2 5 6 7 4 8 9 3 1 0
- ☐ 4 6 8 7 5 3 2 9 0 1

13. 6. Binärer Suchbaum (I1-ID: 5ym3lvv0krg0)

Es geht um binäre Suchbäume, deren Schlüssel Zeichen in alphabetischer Anordnung sind.



1. Werden die Schlüssel obigen binären Suchbaums in Inorder-, Preorder- oder Postorder-Reihenfolge ausgegeben, so erscheint ein Schlüssel immer an gleicher Position. Welcher ist das?
   D
2. Wählen Sie für die Schlüssel eine Einfügereihenfolge aus, bei der ein Binärbaum der Tiefe 2 entsteht:
   DFEGABC

14. 6. Vermischtes zu Algorithmen und Datenstrukturen (I1-ID: 5it4euk0jrg0)

Markieren Sie *alle* zutreffenden Aussagen:

PDF-XChange Click to buy NOW! www.docu-track.com

- [ ] Binäre Suche erfordert $O(n)$ Vergleiche auf *unsortierten* Feldern.
- [x] Mit einem Stack kann man z.B. feststellen, ob ein Ausdruck korrekt geklammert ist.
- [ ] Die abstrakte Datenstruktur Queue lässt sich nicht mit einfach verketteten Listen realisieren.
- [ ] Quicksort ist auf vorsortierten Feldern besonders schnell.
- [x] Mergesort benötigt auf allen Feldern der Länge 2^m immer die gleiche Anzahl an Vergleichen.
- [x] Selectionsort benötigt auf allen Feldern der Länge 2^m immer die gleiche Anzahl an Vergleichen.
- [x] Binäre Bäume der Tiefe n haben mindestens $n-1$ und höchstens 2^n-1 *innere Knoten*.
- [x] Felder sind nicht als Grundlage der abstrakten Datenstruktur Stack geeignet.

15. 5. Verständnisfragen zu OOP (I1-ID: pnp7plg0jrg0)

Sei p eine Referenzvariable vom Typ A. Was kann alles der dynamische Typ von p sein?

Markieren Sie *alle* Möglichkeiten, die Java erlaubt:

- [ ] eine Erweiterungsklasse einer Klasse, die A implementiert (falls A ein Interface ist)
- [ ] ein Interface, dass A als abstrakte Klasse erweitert (falls A eine Klasse ist)
- [ ] eine Klasse, von der A abgeleitet ist (falls A eine Klasse ist)
- [x] eine Erweiterungsklasse von A (falls A eine Klasse ist)
- [x] ein elementarer Datentyp (falls A die zugehörige Wrapper-Klasse ist)
- [ ] eine Klasse, die A implementiert (falls A ein Interface ist)

16. 5. OOP-Entwurf (I1-ID: e7oe5ki0jrg0)

Die Klasse `Ente` erbt von der Klasse `Vogel`. In `Vogel` gibt es eine Instanzmethode `fliege(..)`, in `Ente` gibt es die Instanzmethode `schattern()`.

Die `fliege`-Methode von `Ente`-Objekten soll nun so angepasst werden, dass `schnattern()` immer unmittelbar am Anfang von `fliege` aufgerufen wird.

Wie ist das am besten zu realisieren?

○
```
public void fliege(Ziel z) {
    this.schnattern();
    // hier den Rumpf der fliege-Methode aus Vogel einfuegen
}
```

◉
```
public void fliege(Ziel z) {
    this.schnattern();
    super.fliege(z);
}
```

○
```
public void fliege(Ziel z) {
    this.schnattern();
    this.fliege(z);
}
```

PDF-XChange Click to buy NOW! www.docu-track.com

```
public void fliege(Ziel z) {
    this.schnattern();
    Vogel.fliege(z);
}
```

```
public void fliege(Ziel z) {
    this.schnattern();
}
```

17. 5. Einfach verkettete Listen (II-ID: u161ebq0krg0)

`x` sei Knoten einer einfach verketten Liste (genauer: eine Referenz darauf). Wofür sorgt das folgende Codefragment?

```
t.next = x.next;
x.next = t;
```

Knoten t wird unmittelbar hinter Knoten x eingefügt.

18. 5. Eine einfache Klasse (I1-ID: ng8cmb70qrg0)

Programmieren Sie eine einfache Klasse `Buch` mit der Angaben zu Büchern abgespeichert werden können.
Jedes Buch zeichnet sich durch seinen Titel aus, und durch seine Seitenzahl.
Die folgenden Operationen sollen möglich sein:

- Erzeugen einer `Buch`-Instanz unter Angabe des Titels
- Erzeugen einer `Buch`-Instanz unter Angabe des Titels und der Seitenzahl
- Setzen und Abfragen der Seitenzahl jeder einzelnen Instanz.
- Die Anzahl erzeugter Instanzen und die Summe ihrer Seitenzahlen soll durch Methodenaufrufe ermittelt werden können.

VERWENDEN SIE DABEI BITTE DIESE BEZEICHNER:
`titel`, `seiten`, `anzahl`, `gesamt`

Außerdem soll die Klasse für einen ersten Test mit einer `main`-Methode ausgestattet sein, die folgende Aktionen auslöst:

- ein `Buch`-Objekt erzeugen nur unter Angabe des Titels `Lieblingsbuch` und die Referenz der Variablen `b` zuweisen
- anschließend die Seitenzahl dieses Buchobjekts auf `500` setzen
- schließlich mittels `b` die Angaben zu dem erzeugten Objekt ausgeben:

```
Lieblingsbuch, 500 S.
```

```
1  class Buch {
2      private static int anzahl = 0;
3      private static int gesamt = 0;
4      private String titel;
5      private String seiten;
6      Buch(String titel) {
7          this.titel = titel;
8          anzahl++;
       }
```

PDF-XChange Click to buy NOW! www.docu-track.com

```
    Buch(String titel, int seiten) {
        this.titel = titel;
        this.seiten = seiten;
        anzahl++;
        gesamt += seiten;
    }
    public int getSeiten() {
        return seiten;
    }
    public void setSeiten(int seiten) {
        this.seiten = seiten;
        gesamt += seiten;
    }
    static int getAnzahl() {
        return anzahl;
    }
    static int getGesamt() {
        return gesamt;
    }
    String toString() {
        return titel + ", " + seiten + " S.";
    }
    public static void main(String[] args) {
        Buch b = new Buch("Lieblingsbuch");
        b.setSeiten(500);
        System.out.printf("%s", b.toString());
    }
}
```

0 Zeichen zugelassen, Anzahl der eingegebenen Zeichen: **858**

Info 1 - 2015 `// Viel Spass damit und lernt brav ;)`

1. 1. Vermischtes zur Einführung

Markieren Sie *alle* zutreffenden Aussagen:

- Top-Down-Entwurf besteht darin, Details erst grob, dann feingliedrig zu sortieren.
- Dem Inhalt einer Speicherzelle kann man nicht ansehen, ob sie eine Zahl oder einen Befehl speichert..
- Beim euklidischen Algorithmus wird dieser Block iteriert: `{ x = y; y = r; r = x % y; }`
- Beim Testen wird das Programm auf allen erlaubten Eingaben ausgeführt.
- Beim Pipelining werden Eingaben eines Prozesses zur Ausgabe eines anderen geleitet.
- Beim Laden wird die Befehlsfolge in aufeinanderfolgende Register kopiert.
- Editieren und Testen gehören zu jedem Programmentwicklungszyklus.
- `public static void main(String[] arks)` ist Kopf der `main`-Methode.

2. 1. Zweierpotenzen

Ergänzen Sie untenstehende `main`-Methode so, dass der Aufruf `java Zweierpotenzen` n
die Liste der Zweierpotenzen von $2^0 = 1$ bis 2^n zeilenweise ausgibt.

```
public class Zweierpotenzen {
    public static void main (String[] args) {
        // ---------------------------
        .....    // soviele Codezeilen wie nötig
        .....    // (angemessen EINGERÜCKT)
        ..
        // ---------------------------
   }
}
```

Beispiel `java Zweierpotenzen 3` gibt diese Liste aus:

```
1
2
4
8
```

Algorithmische Idee (bitte GENAU diese Variablennamen benutzen):

initialisiere:
- Kommandozeilenargument (interpretiert als `int`-Wert) in `n` speichern
- `int`-Variable `p` als Akkumulator initialisieren

iteriere so oft wie nötig:
- `p` ausgeben und geeignet aktualisieren

finalisiere: entfällt

3. 1. Quadratzahlentabelle

Die `int`-Variable `n` speichere die ganze Zahl n.

Die ersten n positiven Zahlen sollen zusammen mit ihren Quadraten jeweils auf einer Zeile

ausgegeben werden.

Beispiel für $n = 4$:

```
1 : 1
2 : 4
3 : 9
4 : 16
```

Im Fall $n \leq 0$ soll nichts ausgegeben werden.

"Programmieren" Sie dies durch Auswahl der richtigen Codezeilen, und zwar einmal mit einer `for`-Schleife und einmal mit einer `do-while`-Schleife.

Achten Sie auf den Unterschied zwischen `System.out.print(..)` und `System.out.println(..)`.

- mit einer for-Schleife:

```
for (int i = 1; i < n+1; i++) {
System.out.print(i + ": ");
System.out.println( i^2);
}
```

- mit einer do-while-Schleife:

```
int i = 1;
do {
System.out.print(i + ": ");
System.out.println( i^2);
i++;
} while ( i < n+1);
```

4. 2. Vermischtes zu elementaren Daten

Markieren Sie *alle* zutreffenden Aussagen:

- Der ASCII-Standard legt die Anordnung der 256 Zeichen fest.
- Zu jedem Datentyp gibt es ein Informationssystem.
- Java-Code darf beliebige Unicode-Zeichen enthalten.
- `int`-Werte können verlustfrei in `float`-Variablen gespeichert werden.
- `5/4 == 1` hat den Wert `true`.
- `Integer.MAX_VALUE++` verursacht keinen Ganzzahlüberlauf.
- Anstatt `if (b > 0 && a/b < 2) ..` schreibt man besser `if (a/b < 2 && b > 0) ...`
- Ein Gleitkomma-Überlauf ist kein Laufzeitfehler.

5. 2. Integer-Arithmetik

Sei `d` eine `int`-Variable mit dem Wert `Integer.MAX_VALUE` (also 2147483647). Welche Textausgabe erzeugen diese Anweisungen?:

```
1:  System.out.println(-d);
2:  System.out.println(d-1);          // dropdown menue mit loesungen
3:  System.out.println(1+2*d);
4:  System.out.println(-(2*d));
```

6. 2. Verarbeitung elementarer Daten (I1-ID: 93rdw3918rg0)

In einer Klasse sind einige Methoden definiert:

```
public static void a() {
    char a = 'a';
    a++;
    int b = (int) a;
    b++;
    System.out.print( (char) ('A' + b - a));
}
public static boolean getTrue() {
    System.out.print("T");
    return true;
}
public static boolean getFalse() {
    System.out.print("F");
    return true;
}
public static void q(int i) {
    int j = 43;
    System.out.print("G");
    if ( i / j > 0) System.out.print("Q");
    else System.out.print("R");
}
public static void d() {
    double dd = Double.MIN_VALUE;
    dd *= 2;
    dd -= Double.MIN_VALUE;
    dd /= 2;
    System.out.print(dd);
}
```

Welche Ausgaben bewirken diese Aufrufe (in der `main`-Methode der gleichen Klasse)?

```
a();



boolean val = getTrue() || getFalse();



val = getTrue() ^ getTrue();



System.out.print( val);



q(42);



d();
```

7. 3. Rekursion

Betrachten Sie folgende Klasse:

```
 1:  class Myst {
 2:      static void ery(int n) {
 3:          if (n == 0)
 4:              return;
 5:          ery( n-1);
 6:          A();
 7:          yre( n-1);
 8:      }
 9:      static void yre(int n) {
10:          if (n == 0)
11:              return;
12:          ery( n-1);
13:          B();
```

```
14:          yre( n-1);
15:      }
16:      static void A() {
17:          System.out.print("A");
18:      }
19:      static void B() {
20:          System.out.print("B");
21:      }
22:      public static void main(String[] args) {
23:          int n = Integer.parseInt(args[0]);
24:          ery(n);
25:          System.out.println();
26:      }
27:  }
```

1. Geben Sie die ersten 8 Zeichen der Ausgabe zum Aufruf `java Myst 4` an:

2. Aus wievielen Zeichen besteht der Ausgabestring beim Aufruf `java Myst 7` (*ohne* den abschließenden Zeilenwechsel durch Zeile 25)?

3. Wird Zeile 14 durch `yre( n-2);` ersetzt, so führt beispielsweise der Aufruf `java Myst 4` zu einem Fehler.
   Welche *andere* Zeile müsste korrigiert werden, um den Fehler zu beseitigen? Geben Sie nur die Nummer an:

4. Werde Zeile 12 durch `ery( n-2);` und zusätzlich Zeile 14 durch `yre( n-2)` ersetzt.
   Geben Sie das maximale Argument `n` an, bei dem der Aufruf `java Myst n` *nicht* zu einem Fehler führt:

8. 3. Grafik-Bibliothek benutzen

Die Klasse `public class StdDraw` bietet Methoden zum Zeichnen geometrischer Objekte:

- die quadratische Zeichenfläche (Seitenlänge 1) wird in einem Fenster mit dem Titel "File" angezeigt
- Koordinaten wie üblich (z.B. gehen die blauen Strecken oben alle durch den Punkt $(x, y) = (1, 0)$, die diagonal gegenüberliegende Quadratecke ist $(0, 1)$)
- gezeichnet wird mit der zuletzt gesetzen Stiftfarbe, wobei später Gezeichnetes Früheres (teilweise) überdecken kann

`void setPenColor(Color c)`	Stiftfarbe setzen
`BLACK`, `RED`, `BLUE`	in `StdDraw` definierte `Color`-Konstanten (schwarz, rot, blau)
`void line(double x0, double y0, double x1, double y1)`	Strecke zwischen Punkten `(x0,y0)` und `(x1,y1)`
`void square(double x, double y, double r)`	Quadrat: Mittelpunkt `(x,y)` u. Seitenlänge `2*r` (parallel zu Koordinatenachsen)
`void circle(double x, double y, double r)`	Kreis mit Mittelpunkt `(x,y)` und Radius `r`

Nachfolgend der Ausschnitt einer Methode einer Clientklasse von `StdDraw`.
Ergänzen Sie Zeilen so, dass der Aufruf `zeichneB(n)` (z.B. in der `main`-Methode dieser Klasse) eine Figur erzeugt aus `n` mal `n` Kreisen usw.
Genau so wie oben gezeigt. Achten Sie dabei auf gut verständlichen Code.

```
static void zeichneB(int n) {
```

```
                                // Ekelhafte Dropdown menues

    for (int i = 0; i < n; i++)                    StdDraw.circle(r*i,r*j,r/2);
      for (int j = 0; j < n; j++)                StdDraw.setPenColor(StdDraw.RED);
       StdDraw.setPenRadius(0.02);                  StdDraw.line(1.0,0.0,0,r+2*r*i);
         for (int i = 0; i < n; i++)              StdDraw.setPenColor(StdDraw.BLUE);
         double r = 1.0/(2*n);                     for (int j = 0; j < n; j++)
                                                    StdDraw.square(0.5,0.5,1);
}
```

9. 4. Felder: Fehler oder …?

Erzeugt untenstehende Klasse

- einen Compilezeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich:
- einen Laufzeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich:
- eine Ausgabe? Falls ja, so tragen Sie die Ausgabe ein, ansonsten einen Bindestrich:

```
 1:  class FehlerOderWie {
 2:      public static void main(String[] args) {
 3:          int[] a = {2,3,5,7,11};
 4:          int[] b = new int[2];
 5:          b = a;
 6:          b[4] = -4;
 7:          if (a[4] >= 1) System.out.println(1);
 8:          else System.out.println(0);
 9:      }
10:  }
```

10. 4. Vermischtes zu Klassentypen

1. Objekte verstehen:

   ```
   public class Spieler {
       private int score;
       public Spieler( int s) {
           this.score = s;
       }
   }
   ```

   Zwei `Spieler`-Objekte werden instanziiert:

   ```
   Spieler a = new Spieler(50), b = new Spieler(50);
   ```

   Welchen Wert hat nun der Ausdruck a == b?

2. Fremden Code deuten:

   ```
   1:  public class FirstBpp extends hepno.users.Frame {
   2:      private hepno.users.Wring wrags;
   3:      public FirstBpp() {
   4:          wrags = new hepno.users.Wring();
   5:      }
   6:      public static void main(String[] args) {
   7:          FirstBpp app = new FirstBpp();
   8:      }
   9:  }
   ```

   Was bewirkt Zeile 2?:
   `hepno.users.Wring wird als private-Objekt deklariert`

3. Statische Methoden und Instanzmethoden:

```
public class Klasse {
    private final int zahl = 10;
    double a() {
        System.out.println(zahl);
        return c();
    }
    void b() {
        System.out.println(this);
    }
    double c() {
        double r = Math.random();
        System.out.println(r);
        return r;
    }
    void d() {
        a();
        a();
    }
    int e() {
        return zahl;
    }
}
```

Welche der obigen Methoden ist ihrer Natur nach eher eine statische Methode (und sollte daher mit `static` versehen werden)?

`c`

4. Fehlersuche:

```
public class Test {
    private int testCount;
    public static int getCount(){
        return testCount;
    }
    public Test(){
        testCount++;
    }
}
```

Obiger Code verursacht einen Compilezeitfehler. Was ist das Problem?

`getCount() ist statisch`

5. Tracing:
[Hinweis: Untenstehender Code ist syntaktisch korrekt, denn auf Klassenvariablen kann auch mit Hilfe von Objektreferenzen zugegriffen werden.]

```
public class Tset {
    static int tsetCount = 0;
    public Tset(){
        tsetCount++;
    }
    public static void main(String [] Args){
        // Hinweis: auf Klassenvariablen kann man auch
        tset t1 = new tset();
        System.out.print(t1.tsetCount + ", ");
        tset t2 = new tset();
        System.out.print(t1.tsetCount + " "+ t2.tsetCount + ", ");
        tset t3 = new tset();
        System.out.print(t1.tsetCount+ " "+ t2.tsetCount+ " "+ t3.tsetCount);
        System.out.println();
    }
}
```

Welche Ausgabe produziert der Code?

6. Begriffsverständnis: Wählen Sie aus dem Menü die einzige Aussage aus, die NICHT korrekt ist.

In einer Klasse darf es Methoden geben, die den gleichen Namen und gleiche Argumenttyplisten haben aber unterschiedliche Ergebnistypen.

11. 4. Iteration in einem Feld

Tragen Sie die Werte der Feldkomponenten nach Ausführung der `for`-Anweisung ein.

```
int[] d = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9};

for (int i = 0; i < 10; i++)
    d[i] = d[9-d[9-d[9-i]]];
```

vorher	0	1	2	3	4	5	6	7	8	9
nachher	9	8	7	6	5	5	6	7	8	9

12. 6. Stapelarbeit

Auf einem Stack werden *in gemischter Reihenfolge* 10 Push- und 10 Pop-Operationen ausgeführt.

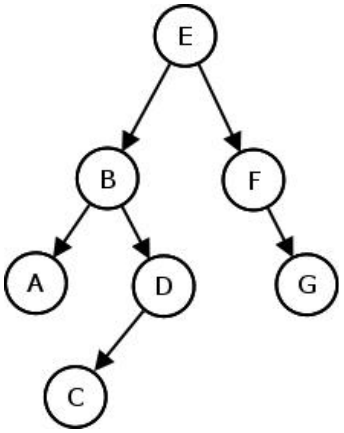
Es ist bekannt, dass die Push-Operationen die Zahlen 0, 1, 2, …, 9 in dieser Reihenfolge einspeichern und dass bei jedem Pop der ausgespeicherte Wert ausgegeben wird.

Markieren Sie alle Ausgabelisten, die dabei entstehen können (das können 0, 1 oder mehrere sein):

1 4 7 9 8 6 5 3 0 2
4 3 2 1 0 9 8 7 6 5
2 5 6 7 4 8 9 3 1 0
4 6 8 7 5 3 2 9 0 1

13. 6. Binärer Suchbaum

Es geht um binäre Suchbäume, deren Schlüssel Zeichen in alphabetischer Anordnung sind.



1. Werden die Schlüssel obigen binären Suchbaums in Inorder-, Preorder- oder Postorder-Reihenfolge ausgegeben, so erscheint ein Schlüssel immer an gleicher Position.
   Welcher ist das?

2. Wählen Sie für die Schlüssel eine Einfügereihenfolge aus, bei der ein Binärbaum der Tiefe 2 entsteht:

14. 6. Vermischtes zu Algorithmen und Datenstrukturen

Markieren Sie *alle* zutreffenden Aussagen:

- Binäre Suche erfordert $O(n)$ Vergleiche auf *unsortierten* Feldern.
- Mit einem Stack kann man z.B. feststellen, ob ein Ausdruck korrekt geklammert ist.
- Die abstrakte Datenstruktur Queue lässt sich nicht mit einfach verketteten Listen realisieren.
- Quicksort ist auf vorsortierten Feldern besonders schnell.
- Mergesort benötigt auf allen Feldern der Länge 2^m immer die gleiche Anzahl an Vergleichen.
- Selectionsort benötigt auf allen Feldern der Länge 2^m immer die gleiche Anzahl an Vergleichen.
- Binäre Bäume der Tiefe n haben mindestens $n-1$ und höchstens 2^n-1 *innere Knoten*.
- Felder sind nicht als Grundlage der abstrakten Datenstruktur Stack geeignet.

15. 5. Verständnisfragen zu OOP

Sei p eine Referenzvariable vom Typ A. Was kann alles der dynamische Typ von p sein?

Markieren Sie *alle* Möglichkeiten, die Java erlaubt:

- eine Erweiterungsklasse einer Klasse, die A implementiert (falls A ein Interface ist)
- ein Interface, dass A als abstrakte Klasse erweitert (falls A eine Klasse ist)
- eine Klasse, von der A abgeleitet ist (falls A eine Klasse ist)
- eine Erweiterungsklasse von A (falls A eine Klasse ist)
- ein elementarer Datentyp (falls A die zugehörige Wrapper-Klasse ist)
- eine Klasse, die A implementiert (falls A ein Interface ist)

16. 5. OOP-Entwurf

Die Klasse `Ente` erbt von der Klasse `Vogel`. In `Vogel` gibt es eine Instanzmethode `fliege(..)`, in `Ente` gibt es die Instanzmethode `schattern()`.

Die `fliege`-Methode von `Ente`-Objekten soll nun so angepasst werden, dass `schnattern()` immer unmittelbar am Anfang von `fliege` aufgerufen wird.

Wie ist das am besten zu realisieren?

○
```
public void fliege(Ziel z) {
    this.schnattern();
    // hier den Rumpf der fliege-Methode aus Vogel einfuegen
}
```

○
```
public void fliege(Ziel z) {
    this.schnattern();
    super.fliege(z);
}
```

```
public void fliege(Ziel z) {
    this.schnattern();
    this.fliege(z);
}
```

```
public void fliege(Ziel z) {
    this.schnattern();
    Vogel.fliege(z);
}
```

```
public void fliege(Ziel z) {
    this.schnattern();
}
```

17. 5. Einfach verkettete Listen

`x` sei Knoten einer einfach verketten Liste (genauer: eine Referenz darauf). Wofür sorgt das folgende Codefragment?

```
t.next = x.next;
x.next = t;
```

Knoten t wird direkt hinter Knoten x eingefügt.

18. 5. Eine einfache Klasse

Programmieren Sie eine einfache Klasse `Buch` mit der Angaben zu Büchern abgespeichert werden können.
Jedes Buch zeichnet sich durch seinen Titel aus, und durch seine Seitenzahl.
Die folgenden Operationen sollen möglich sein:

- Erzeugen einer `Buch`-Instanz unter Angabe des Titels
- Erzeugen einer `Buch`-Instanz unter Angabe des Titels und der Seitenzahl
- Setzen und Abfragen der Seitenzahl jeder einzelnen Instanz.
- Die Anzahl erzeugter Instanzen und die Summe ihrer Seitenzahlen soll durch Methodenaufrufe ermittelt werden können.

VERWENDEN SIE DABEI BITTE DIESE BEZEICHNER:
`titel`, `seiten`, `anzahl`, `gesamt`

Außerdem soll die Klasse für einen ersten Test mit einer `main`-Methode ausgestattet sein, die folgende Aktionen auslöst:

- ein `Buch`-Objekt erzeugen nur unter Angabe des Titels `Lieblingsbuch` und die Referenz der Variablen `b` zuweisen
- anschließend die Seitenzahl dieses Buchobjekts auf `500` setzen
- schließlich mittels `b` die Angaben zu dem erzeugten Objekt ausgeben:

```
Lieblingsbuch, 500 S.
```

Info I Klausur

Aufgabe 1 (15P) 3P pro Aufgabenteil Allgemeine Fragen

1. Nennen sie die Voraussetzung zur Anwendbarkeit der binären Suche auf Feldern. Geben sie in O-Notation die worst-case-Komplexität an (Feldlänge n)
2. Was bedeutet partielle Korrektheit eines Algorithmus in Bezug auf die Problemspezifikation (V,N)
3. Nennen und erläutern sie das Prinzip , nach dem bei Java die Parameterübergabe bei Methodenaufrufen erfolgt
4. Welche Elemente einer Java Klassendefinition werden durch das Schlüsselwort static gekennzeichnet? Was bedeutet private?
5. Beschreiben sie das teile und herrsche-Prinzip und nennen sie einen Beipielalgorithmus dafür.

Aufgabe 2 (15P) Elementare und strukturierte Daten

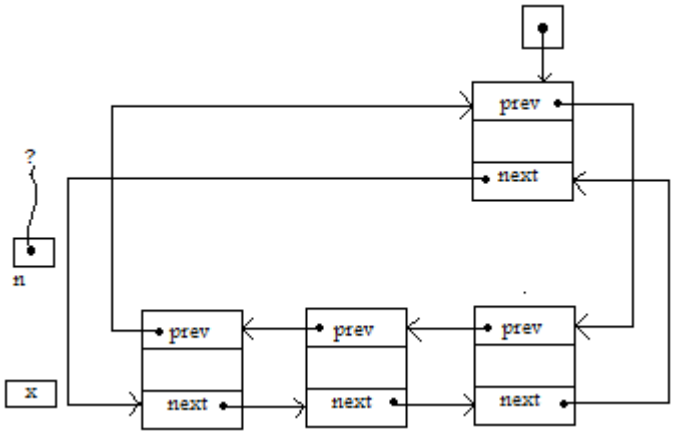
1. Stellen sie den Dezimalbruch 18,625 binär dar (5P)
2. Sei $b_{N-1}\, b_{N-2} \ldots b_1\, b_0 \in \{0,1\}^N$ die N-Bit Zweierkomplementdarstellung der ganzen Zahl z mit $-2^N < z 2^N$. Wie erhält man daraus die N-Bit Zweierkomplementdarstellung von –z? (2P)
3. Wieviele Bytes werden durch folgenden Konstruktoraufruf in Java angefordert? Es genügt eine Zweierpotenz 2^n sowie eine kurze Begründung als Antwort
   int [][][][] feld: new int [16][16][16][16] (3P)
4. Die main Methode soll einen Countdown erzeugen, d.h. nacheinander (gemeint untereinander) „4, 3, 2, 1, Start!“ ausgeben, ist aber fehlerhaft. Beschreiben sie den Fehler und geben sie an, an welcher Stelle er auf dem Weg vom Programmcode bis zur Ausführung auftritt. Korrigieren sie durch zusätzliche Programmzeilen (nachfolgend angegeben, Einfügestelle kennzeichnen)

```
Public static void main (String [] argv){
        Int [] xi
        For (int i=0; i<4; ++i)
                System.out.println (x[i]);
        System.out.println(„Start!“);
}
```

(5P)

Aufgabe 3 (15P) Datenstrukturen und 00-Konzepte

1. Ein Stack für int-Werte kann konkret mittels doppelt-verketteter Liste mit Ankerknoten realisiert werden. Dazu eine Zeichnung und Quellcode Ausschnitte aus Knoten und Listenklasse:




```
class Int Node{
Int wert;
IntNode next;
IntNode prev;
public IntNode (int x, IntNode next, IntNode prev){
        wert=x;
        this.prev=prev;
        this.next=next; }
}
```

```
class  Int Liste {
IntNode list;
…
void prepend (int x) //Knoten „links einfügen“//
…
void append (int x) //Knoten „rechts“ einfügen//
…
void insert (int x) //vor ersten Knoten mit Wert <=x einspeichern//
…
int remove Head() //Knoten „links“ entfernen (davon Wert liefern)//
…
int remove Tail //Knoten „rechts“ entfernen (davon Wert liefern)//
…
}
```

Nennen sie ein Methodenpaar, das Stack-artiges „Ein-„ bzw. „Auspeichern“ realisiert.
Programmieren sie davon die Einspeicher-Methode in Java. (6P)

2. Auf eine anfangs leere Queue werden Operationen angewendet. Tragen sie den Zustand nach den angegebenen Aufrufen so ein, dass das Ende der Queue jeweils ganz links steht (erste Zeile ist komplett vorgegeben).

   enqueue(1); enqueue(2); enqueue(8); 8 2 1
   enqueue(5); enqueue(3); dequeue();
   enqueue(1); dequeue(); dequeue();
   dequeue();enqueue(7); enqueue(1); (4P)

3. Welche Zeile im folgenden Java Code verursacht einen Compilezeit Fehler? Wenn diese Zeile gestrichen wird, welche Ausgabe erzeugt der Programmaufruf java AB? Begründen sie ihre Antworten.

```
1 class A {void hallo() {System.out.print(„A“);}}
2 class B extends A{string s=“b“; void hallo(){System.out.print(„B“);}}
3 class AB{
4   public static void main (String [] argv){
5           B y=new B();
6           A x=Y;
8           x.hallo();
9           y.hallo();
10  }
11}
```

(5P)

Aufgabe 4 (15P) Sortieren und suchen

1. Sortieren sie folgende Zahlenfolge aufsteigend mit Quicksort. Dazu das jeweils letzte Element der aktuellen Teilfolge als Pivot markieren (z.B. durch einkreisen), ein Schritt des Verfahrens durchführen und das Ergebniss ins die nächste Zeile der Tabelle eintragen. Es müssen nicht alle Zeilen der Tabelle genutzt werden.

4	8	6	3	2	7	1	9	5

(7P)

2. Zeichnen sie den binären Suchbaum S, der durch einfügen von 15, 5, 16, 3, 12, 10, 13, 6, 7 (in dieser Reihenfolge) entsteht. Zeichnen sie dann den binären Suchbaum, der aus S durch löschen der 5 entsteht. (6P)
3. Wahr oder falsch? : „bei Preorder Traversierung werden die Knoten eines Binärbaumes in umgekehrter Reihenfolge abgearbeitet wie bei der postorder Traversierung.“ Begründen sie oder geben sie ein kleines Gegenbeispiel. (Gegenbeispiel mit Angabe und Benennung der Reihenfolge) (2P)

Aufgabe 5 (15P) Algorithmenanalyse

1. Seien m und n natürliche Zahlen. Bestimmen Sie für folgende Programmierteile die genaue Anzahl von tuWas() Aufrufen. Der Rechenweg ist Teil der Lösung. Geben sie anschließend in O-Notation eine möglichst gute obere Schranke für diese Anzahl an.

```
//Teil A
for (int i=0; i<m; ++i)
        for(int j=i; j<=0; --j)
                tuWas();
//Teil B
for (int i=1; i<m;  ++i)
        for (int j=0; j<=n; ++j)
                tuWas();
```
(6P)
2. Gegeben sei eine sortierte Folge der Länge n und der Suchschlüssel q. „Terräre Suche“ hat folgende Rekursionsidee:
   - ist n=1, so teste, ob q gleich dem einzigen Folgeglied (bei Erfolg liefere dessen Index, sonst 1)
   - Anderenfalls teile in 3 Teilfolgen T_0, T_1, T_2 annähernd gleicher Länge , bestimme durch Vergleich mit dem jeweils letzten Element der 2 Teilfolgen T_0, T_1 in welcher der Suchschlüssel sein müsste und suche in dieser rekursiv weiter.

   Die Anzahl der Vergleiche mit q sei C(n) im schlechtesten Fall. Stellen sie eine Rekursionsgleichung für C(n) und die Randbedingung C(1)=? auf. (3P)

3. Sei n eine beliebige Zweierpotenz. Beweisen sie, dass für $T(n)=2n^2 \log_2 n+n^2$ die Rekursionsgleichung $T(n)=4T(n/2)+2n^2$ und die Randbedingung T(1)=1 gilt. (6P)

Aufgabe 6 (15+2P) Iteration und Rekursion

1. Die Folge g_n ist rekursiv definiert durch $g_0=0$, $g_1=1$, $g_n=g_{n-1}+2g_{n-2}$
   Programmieren sie eine rekursive Methode int g(int n), die zu Argumentwert n das Folgeglied g_n zurückliefert. (4P)
2. Zeichnen sie den Aufrufbaum zur rekursiven Berechnung von g_5. Stellen sie damit eine Rekursionsgleichung auf für die Anzahl a_n der Aufrufe, die die rekursive Berechnung von g_n erfordert. (5P)
3. Beschreiben sie informal eine Idee zur iterativen Berechnung von g_n und programmieren sie diese. (6P)
4. Beschreiben sie informal eine weitere iterative Berechnung von g_n (2P)